SYSTEM VERILOG PROGRAMMING IN VLSI EXPERT

Run Command

vcs half_adder.v -kdb -lca -debug_access+all -full64

[b23n_sarif_727@prac3 lab1.size_of_data]$ vcs -sverilog data_size.v -kdb -lca -debug_access+all -full64 && ./simv

./simv -verdi

verdi -cov -covdir simv.vdb/

vcs -sverilog compare.sv -ntb_opts uvm -kdb -lca -debug_access+all -full64

`include "uvm_macros.svh"

import uvm_pkg::*;

Dirct simulation use above command----imp

cd /data/pdk/pdk_cc

cd hspice/

vi reference40_models.lib

vcs mod_16load_counter.v mod16_load_testbench.v -kdb -lca -debug_access+all -full64 && ./simv

https://support.google.com/a/answer/1371417?hl=en#Match-Word-or-Phrase-in-a-List

1.0 Design Basic Logic Gates by using dataflow
2.0 2.0 Design 4x1 mux by using behavioural level modelling
3.0 3.0 Design decoder 2x4 by using behavioural level modelling
4.0 Design RTL encoder 4x2 by using behavioural level modelling
5.0 Design RTL 1x4 Demux by using behavioural level modelling

Description

Logic gates are the fundamental components of digital circuits. They are used to create combinational and sequential logic that forms the basis of all digital electronics -from simple circuit like calculators to complex devices like microprocessors and FPGAs. Each logic gate takes binary input (either 0 or 1) and produces a binary output based on a specific logic function.

There are seven basic types of logic gates:

- **AND Gate:** Produces an output of 1 only when both inputs are 1.
- **OR Gate:** Produces an output of 1 if any one of the inputs are 1.
- **NOT Gate:** Produces the opposite of an input.
- **NAND Gate:** Inverse of AND, output is 0 only when both inputs are 1.
- **NOR Gate:** Inverse of OR, output is 1 only when both inputs are 0.
- **XOR Gate:** Output is 1 when inputs are different.
- **XNOR Gate:** Output is 1 when inputs are same.

These gates are implemented using transistor in hardware, and RTL (Register Transfer Level) design, I have modelled this behaviour using **System Verilog -HDL.**

1.0 Design Basic Logic Gates by using dataflow

Design code

```
module Logic_gates (and_out, or_out, not_out, nand_out, nor_out, xor_out, xnor_out, a, b);

input logic a, b;

output logic and_out, or_out, not_out, nand_out, nor_out, xor_out, xnor_out;


assign and_out = a & b;

assign or_out = a | b;

assign not_out = ~a;

assign nand_out = ~ (a & b);

assign nor_out = ~ (a | b);

assign xor_out = a ^ b;
```

```
assign xnor_out = ~ (a ^ b);

endmodule
```

Testbench

```
module logic_gates_tb();

logic a,b;

logic and_out, or_out, not_out, nand_out, nor_out, xor_out, xnor_out;


Logic_gates
dut(.a(a),.b(b),.and_out(and_out),.or_out(or_out),.not_out(not_out),.nand_out(nand_out),.
nor_out(nor_out),.xor_out(xor_out),.xnor_out(xnor_out));


initial begin

    $monitor("for a=%b b=%b and output and_out=%b or_out=%b not_out=%b
nand_out=%b nor_out=%b xor_out=%b xnor_out=%b",a,b,and_out, or_out, not_out,
nand_out, nor_out, xor_out, xnor_out);

    a=0; b=0;

#10 a=0; b=1;

#10 a=1; b=0;

#10 a=1; b=1;

#10 $finish();

end


initial begin

    $fsdbDumpfile("dump.fsdb");

    $fsdbDumpvars(0,logic_gates_tb);

end

endmodule
```

Run command

vcs -sverilog logic_gates.sv logic_gates_tb.sv -kdb -lca -debug_access+all -full64

Simulation Result

Output

for a=0 b=0 and output and_out=0 or_out=0 not_out=1 nand_out=1 nor_out=1 xor_out=0 xnor_out=1

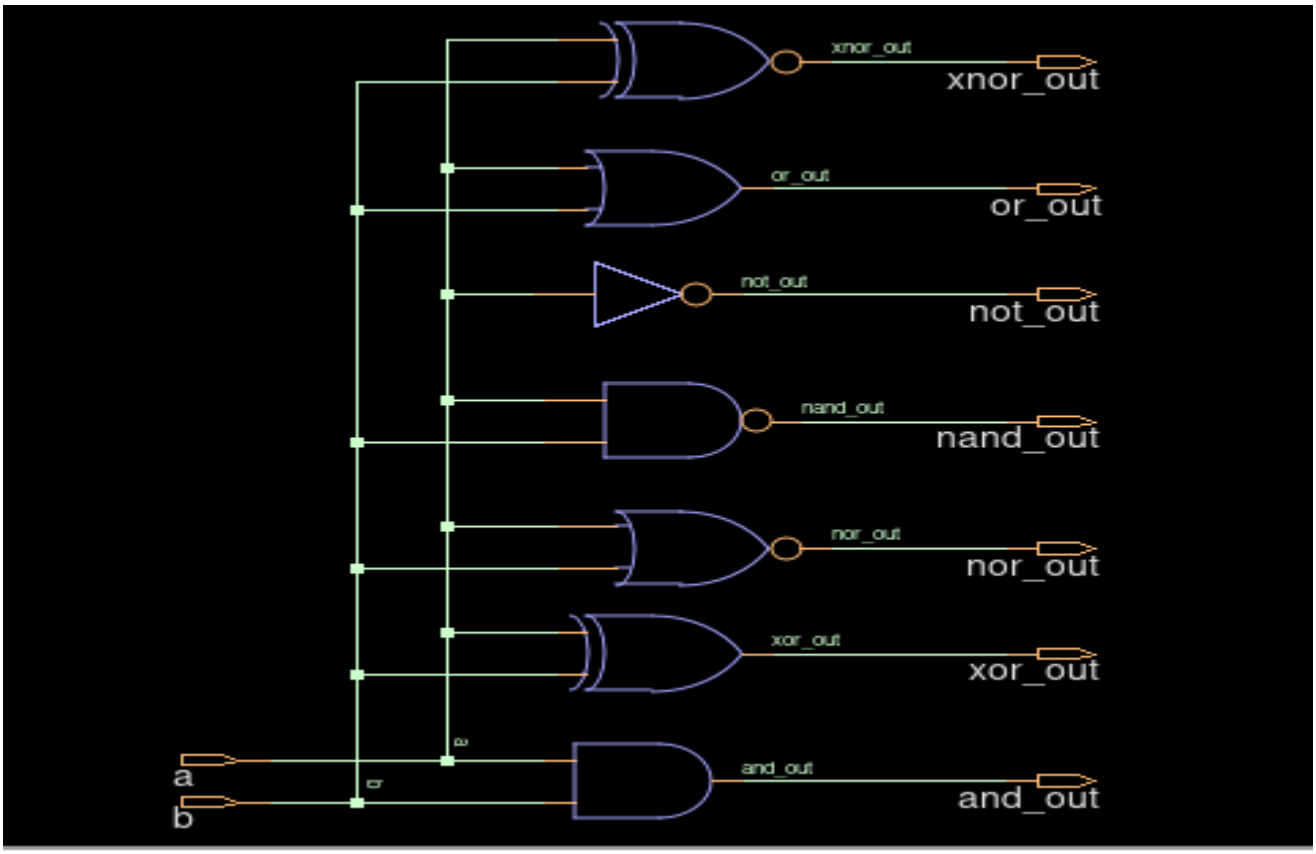
for a=0 b=1 and output and_out=0 or_out=1 not_out=1 nand_out=1 nor_out=0 xor_out=1 xnor_out=0

for a=1 b=0 and output and_out=0 or_out=1 not_out=0 nand_out=1 nor_out=0 xor_out=1 xnor_out=0

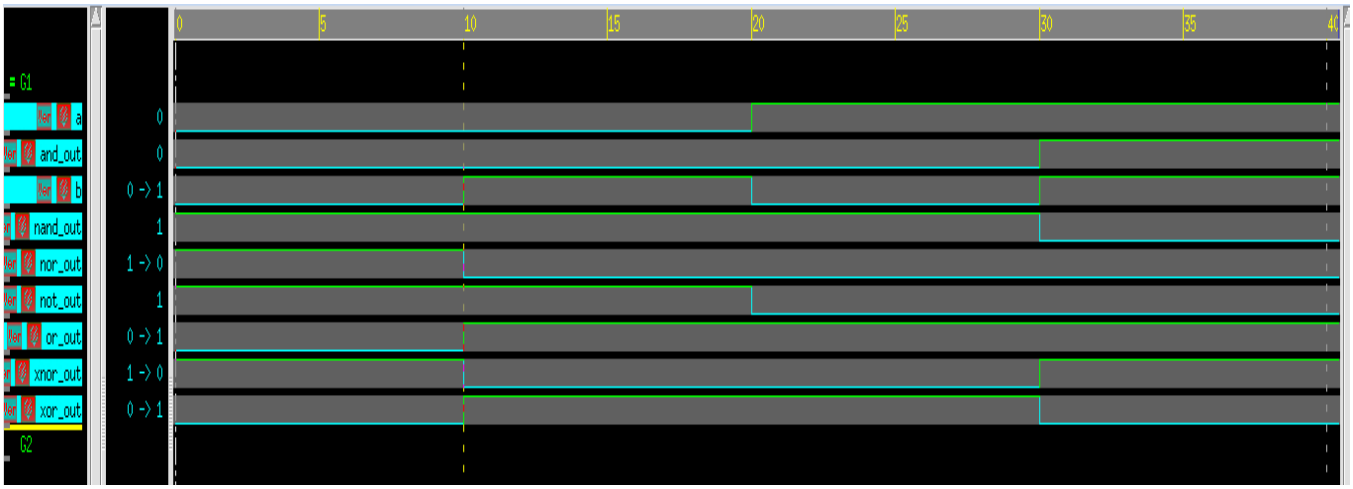
for a=1 b=1 and output and_out=1 or_out=1 not_out=0 nand_out=0 nor_out=0 xor_out=0 xnor_out=1

$finish at simulation time 40

Schematic Design



waveform



2.0 Design 4x1 mux by using behavioural level modelling

Design code

```
module mux_4x1 (out,in,sel);
input logic [3:0]in;
input logic [1:0]sel;
output logic out;

always_comb begin
    case(sel)
        2'b00:out=in[0];
        2'b01:out=in[1];
        2'b10:out=in[2];
        2'b11:out=in[3];
        default:out=1'b0;
    endcase
end
endmodule
```

Testbench

```
module mux_4x1_tb();
```

```
logic [3:0]in;
logic [1:0]sel;
logic out;

mux_4x1 dut(.in(in),.sel(sel),.out(out));

initial begin
        $display("for sel in => out");
        in=4'b1010;
sel=2'b00; #10;$display(" %b %b => %b",sel,in,out);
sel=2'b01; #10;$display(" %b %b => %b",sel,in,out);
sel=2'b10; #10;$display(" %b %b => %b",sel,in,out);
sel=2'b11; #10;$display(" %b %b => %b",sel,in,out);
#10 $finish();
end

initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,mux_4x1_tb);
end
endmodule
```

Simulation Result

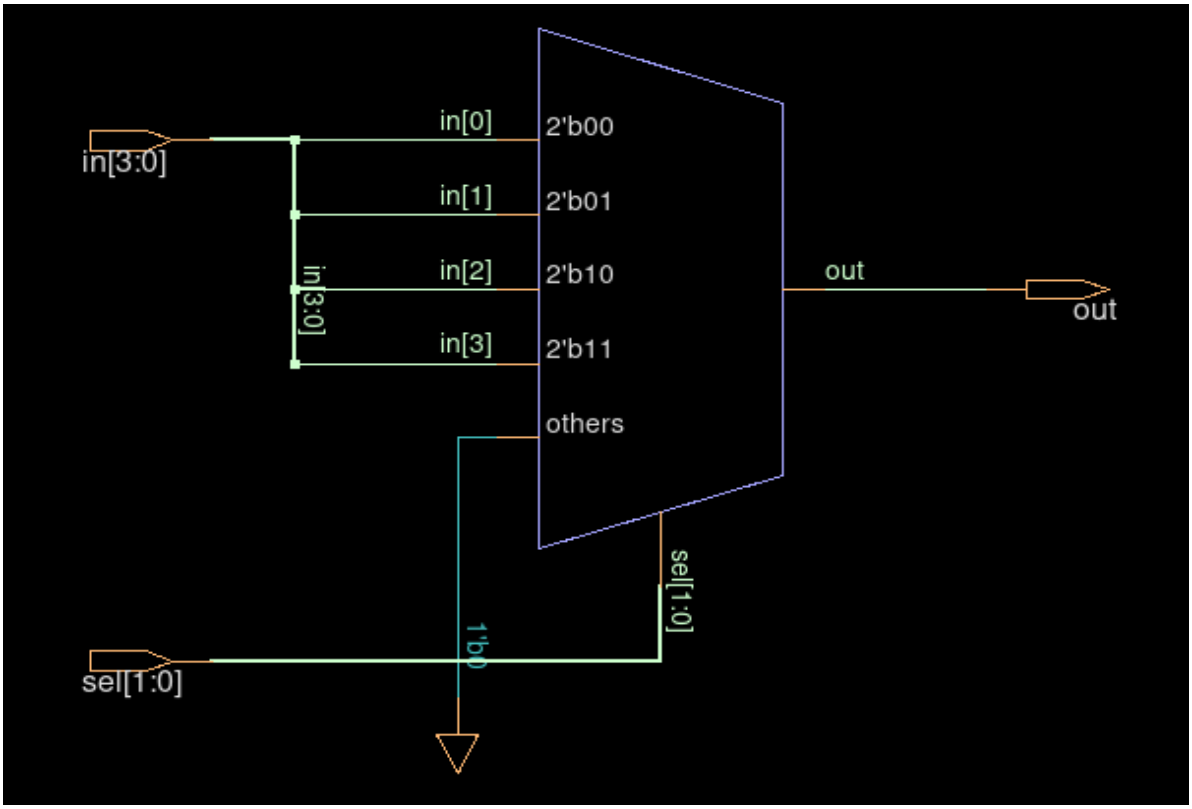
Output

```
00 1010 => 0
 01 1010 => 1
 10 1010 => 0
```

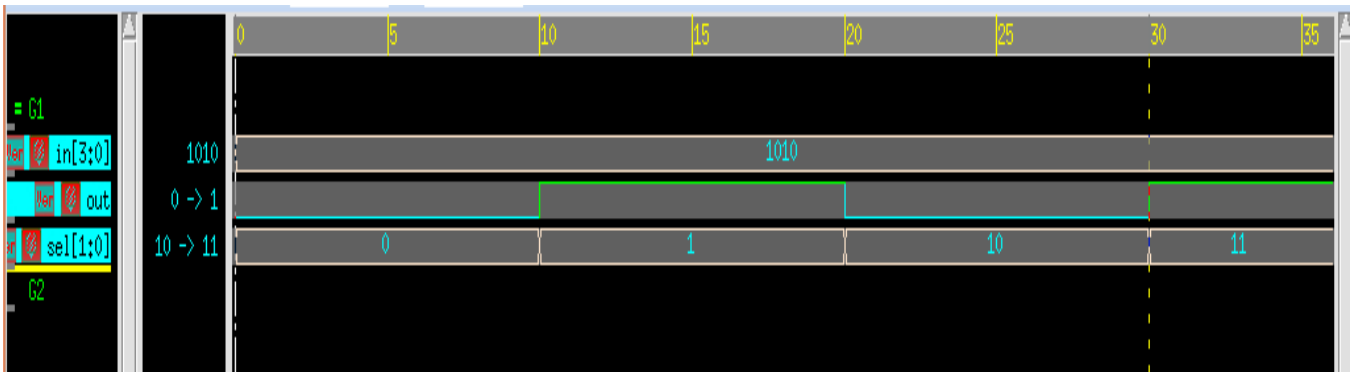
11 1010 => 1

$finish at simulation time 50

Schematic Design



waveform



3.0 Design RTL decoder 2x4 by using behavioural level modelling

Design code

```
module decoder_2x4 (out,in);
input logic [1:0]in;
output logic [3:0]out;

always_comb begin
	case(in)
		2'b00:out=4'b0001;
		2'b01:out=4'b0010;
		2'b10:out=4'b0100;
		2'b11:out=4'b1000;
		default:out=4'b0000;
	endcase
end
endmodule
```

Testbench

```
module decoder_2x4_tb();
logic [1:0]in;
logic [3:0]out;

decoder_2x4 dut(.in(in),.out(out));

initial begin
	$display("for  in => out");
	in=2'b00;#10;$display("%b => %b",in,out);
in=2'b01; #10;$display("%b => %b",in,out);
in=2'b10; #10;$display("%b => %b",in,out);
in=2'b11; #10;$display("%b => %b",in,out);
```

```
#10 $finish();
end

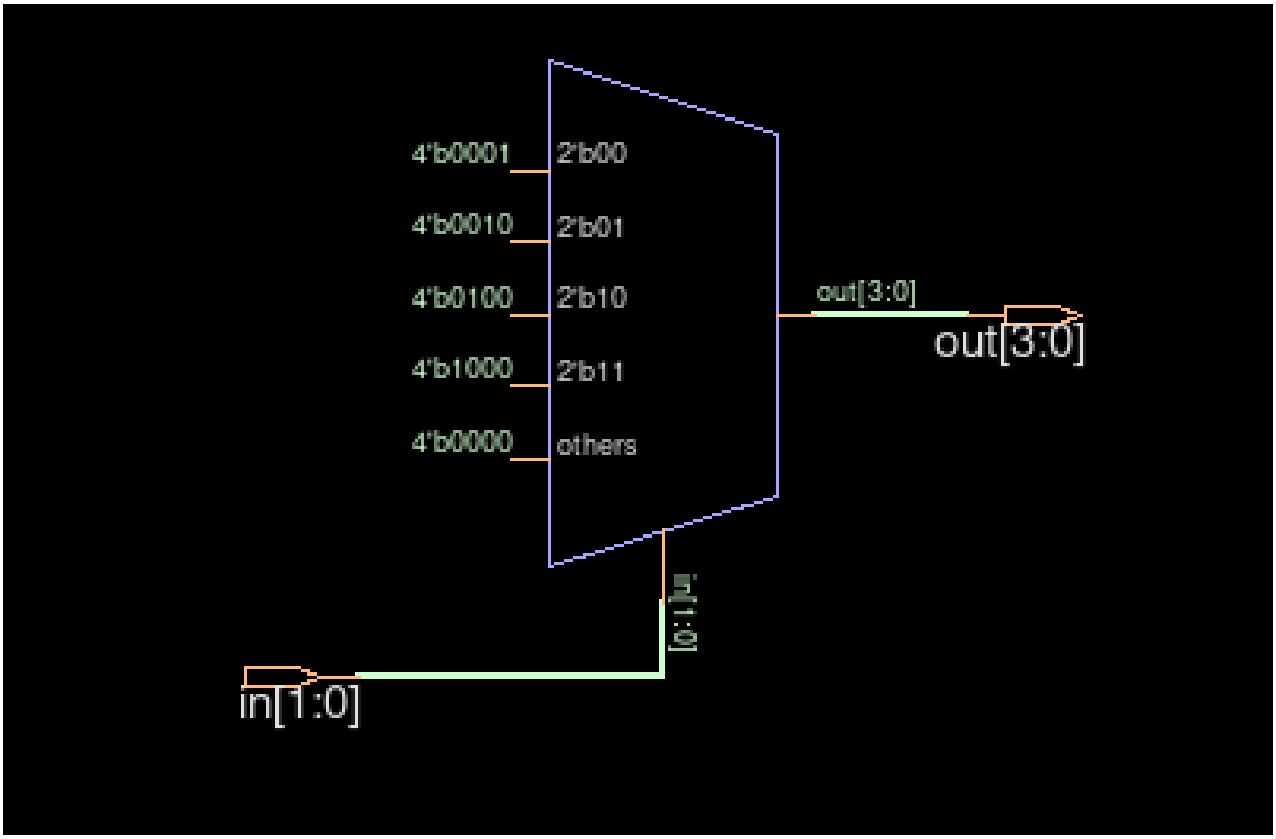
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,decoder_2x4_tb);
end
endmodule
```

Simulation Result

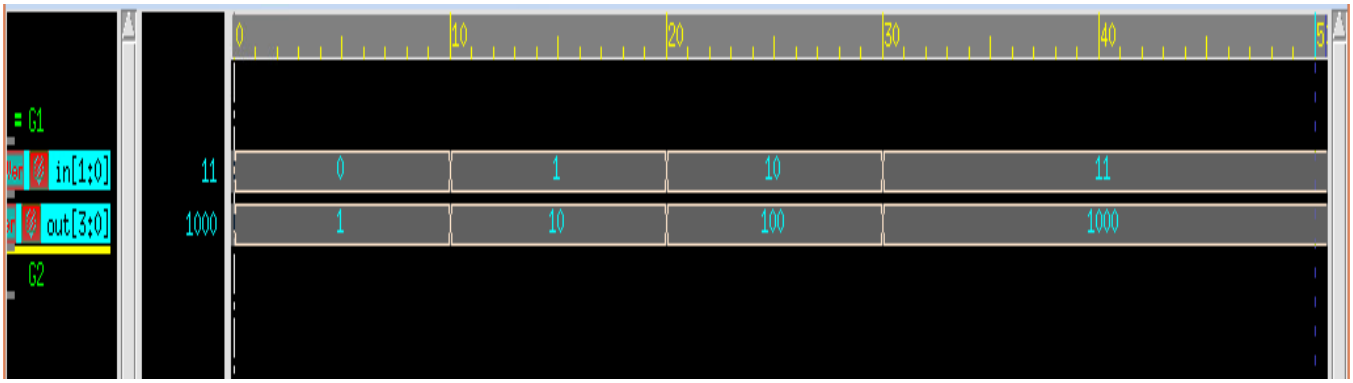
Output

```
00 => 0001
01 => 0010
10 => 0100
11 => 1000
$finish at simulation time                50
```

Schematic Design



waveform



4.0 Design RTL encoder 4x2 by using behavioural level modelling

Design code

module encoder_4x2 (out,in);

input logic [3:0]in;

output logic [1:0]out;

```
always_comb begin
	case(in)
		4'b0001:out=2'b00;
		4'b0010:out=2'b01;
		4'b0100:out=2'b10;
		4'b1000:out=2'b11;
		default:out=2'b00;
	endcase
end
endmodule
```

Testbench

```
module encoder_4x2_tb();
logic [3:0]in;
logic [1:0]out;

encoder_4x2 dut(.in(in),.out(out));

initial begin
	$display("for  in => out");
	in=4'b0001;#10;$display("%b => %b",in,out);
in=4'b0010; #10;$display("%b => %b",in,out);
in=4'b0100; #10;$display("%b => %b",in,out);
in=4'b1000; #10;$display("%b => %b",in,out);
#10 $finish();
end
```

```
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,encoder_4x2_tb);
end
endmodule
```

Simulation Result

Output

0001 => 00

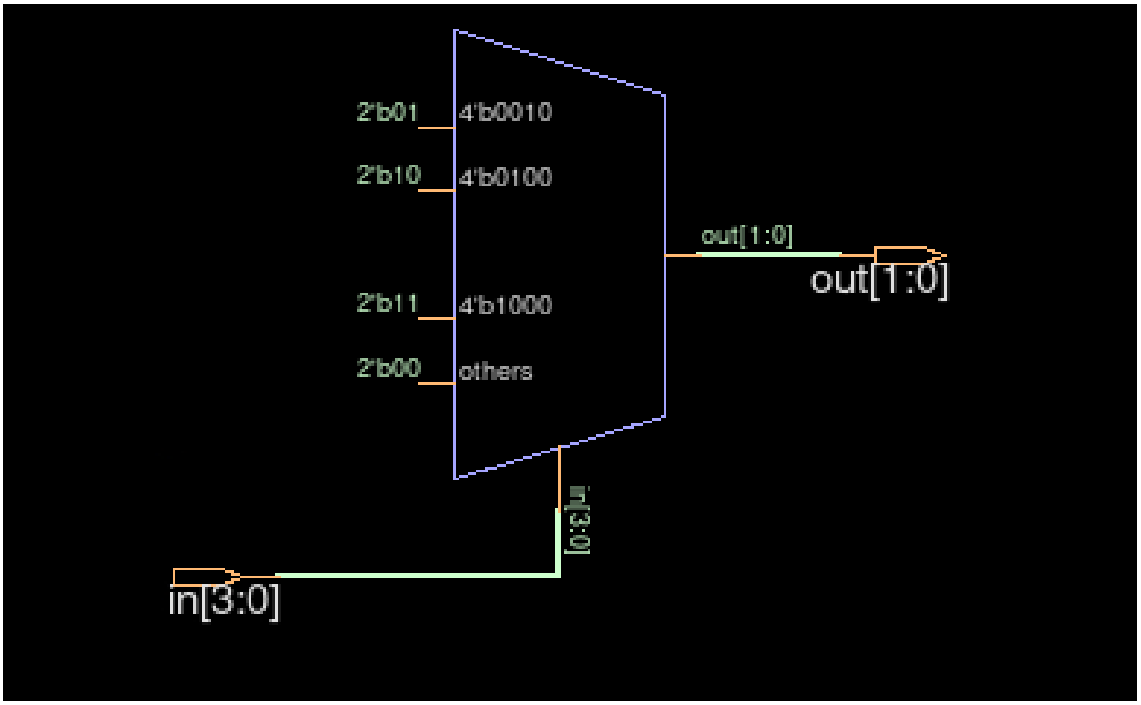
0010 => 01

0100 => 10

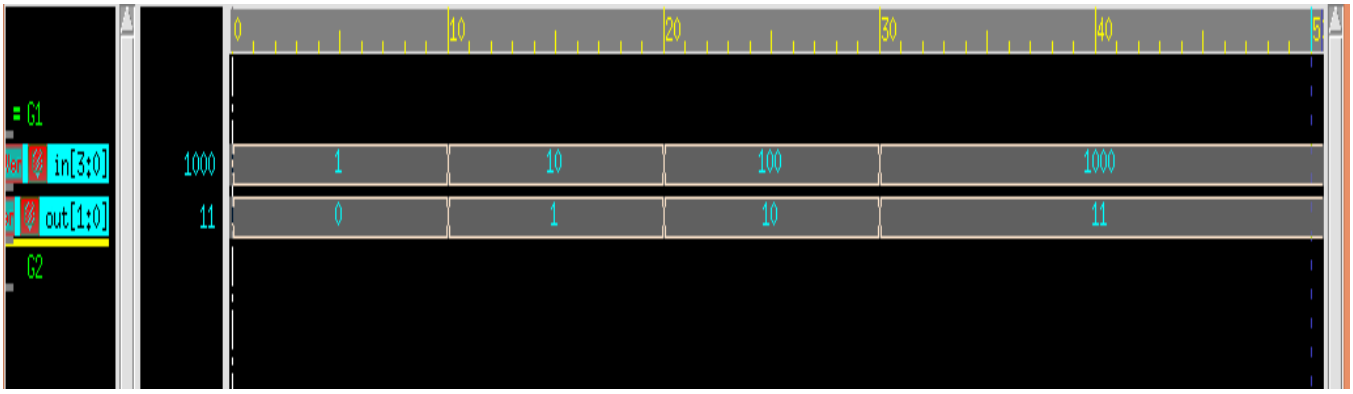
1000 => 11

$finish at simulation time 50

Schematic Design



waveform



5.0 Design RTL 1x4 Demux by using behavioural level modelling

Design code

```
module demux_1x4 (out,sel,in);
input logic in;
input logic [1:0]sel;
output logic [3:0]out;

always_comb begin
    out=4'b0000;
    case(sel)
        2'b00:out[0]=in;
        2'b01:out[1]=in;
        2'b10:out[2]=in;
        2'b11:out[3]=in;
        default:out=4'b0000;
    endcase
end
endmodule
```

Testbench

```
module demux_1x4_tb();
logic in;
logic [1:0]sel;
logic [3:0]out;

demux_1x4 dut(.in(in),.sel(sel),.out(out));

initial begin
	$display("for sel in => out");

	in=1;sel=2'b00;#10;$display("%b %b => %b",in,sel,out);
sel=2'b01; #10;$display("%b %b=> %b",in,sel,out);
sel=2'b10; #10;$display("%b %b=> %b",in,sel,out);
sel=2'b11; #10;$display("%b %b=> %b",in,sel,out);
#10 $finish();
end

initial begin
	$fsdbDumpfile("dump.fsdb");
	$fsdbDumpvars(0,demux_1x4_tb);
end
endmodule
```

Simulation Result

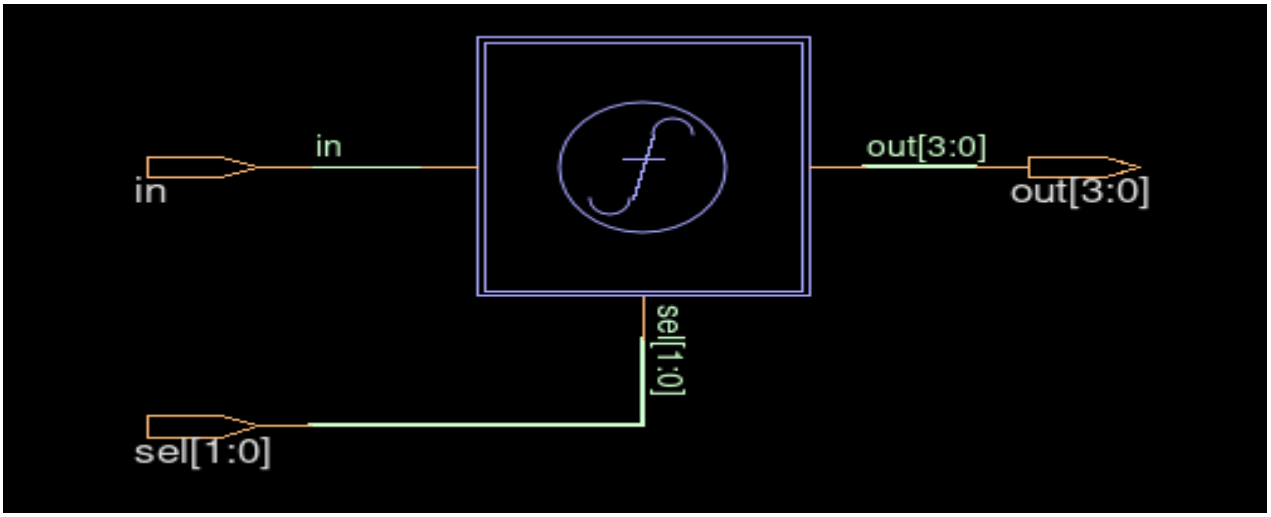
Output

```
1 00 => 0001
1 01=> 0010
1 10=> 0100
```

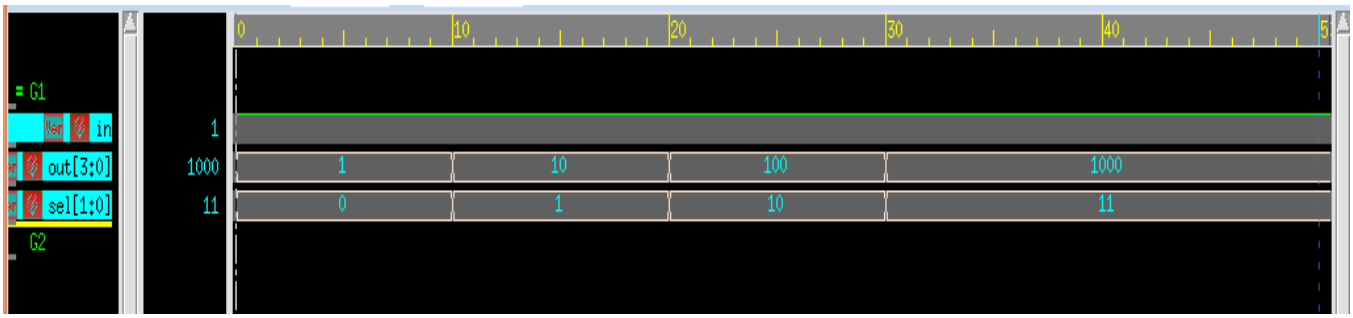
1 11=> 1000

$finish at simulation time 50

Schematic Design



waveform



System Verilog is a industry standard language for performing a verification of a complex design. Understanding of classes, methods, randomization, IPC, interfaces, testbench architecture, building a classes.

1.0 IDE: - Xilinx Vivado, EDA tool

How to use system Verilog code in vivado tool.

2.0 Global signals: - Ports could be divided in three categories. (1) global signal (2) Data signal (3) Control signal. We can take the help of SV generate the random stimulas. Generation of clock.

3.0 Data types-> Variables and array: - Different types of array present which are present in SV.

4.0 Verification+Fundamentals+Archtecture: - In SV testbenches, the type of test have you applied to the DUT and along with that what are the individual component are present in the architecture and fundamental data type is ready.

5.0 Fundamentals of classes and methods: -

6.0 Randomization: - We have covered all the fundamentals of randomization including different options, we have to create random value. Along with the randomization, the different type of constraint, methods, quickly reach to different feature.

7.0 Interprocess communication: - this allows to send the data between classes and once capable of sending the data between the classes then last thins is that to apply data to design that is with the help of interface.

8.0 Interface: - start apply stimulus which are generated by at different classes to the design and start collecting the response.

Design code & Testbench: -

System Verilog Testbench for Randomized Transaction Class: -

```
//

class transaction;
        randc bit [7:0] data;


        task display();
                $display("Time=%0t | Value of Data : %0d",$time, this.data);
        endtask
endclass


module tb;
transaction t;
reg [7:0] data;


initial begin
```

```
        t = new();
        for(int i = 0; i<10; i++) begin
                t.randomize();
                data = t.data;
                t.display();
                #10;
        end
        #10 $finish();
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0, tb);
end
endmodule
```

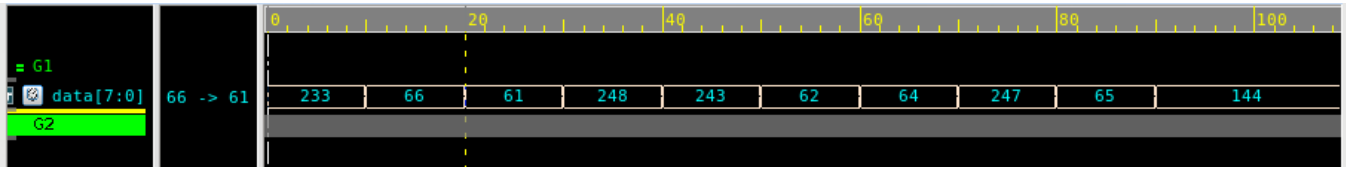
Simulation Result

Output

```
Time=0 | Value of Data : 233
Time=10 | Value of Data : 66
Time=20 | Value of Data : 61
Time=30 | Value of Data : 248
Time=40 | Value of Data : 243
Time=50 | Value of Data : 62
Time=60 | Value of Data : 64
Time=70 | Value of Data : 247
Time=80 | Value of Data : 65
Time=90 | Value of Data : 144
$finish called from file "example.sv", line 23.
```

$finish at simulation time 110

V C S S i m u l a t i o n R e p o r t



03 types of signals which are present in the design: -

- **Global signals-** reset and clock
- **Control signal –** control the operation of the design (en,wr_en,rd_en).
- **Data signals –** read data and write data

```
//
module clk_tb();
reg clk;
reg rst;
reg [3:0]temp;

initial begin
        $monitor("Time=%0t| temp=%0d| clk=%0b|rst=%0b",$time,temp,clk,rst);
        clk=1'b0;
        rst=1'b0;
end

initial begin
        rst=1'b0;
        #60;
        rst=1'b1;
end

initial begin
```

```
        temp=4'b0100;
        #10;
        temp=4'b1100;
        #10;
        temp=4'b0011;
        #10;
#50 $finish;

end
endmodule

/*
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,clk_tb);
end
*/
//endmodule
```

Simulation Result

Output

```
  Oct 23 00:39 2025
Time=0| temp=4| clk=0|rst=0
Time=10| temp=12| clk=0|rst=0
Time=20| temp=3| clk=0|rst=0
Time=60| temp=3| clk=0|rst=1
$finish called from file "example.sv", line 26.
$finish at simulation time                   80
```

V C S S i m u l a t i o n R e p o r t

- Testbench module do **not have any input and output ports.**
- Initial block start execution at **T=0 ns**
- always statement: **---single line**
- always begin ……end **---for multiple line**

```
module tb();
reg a=0;
initial a=1;

initial begin
    $monitor("Time=%0t| a=%0b ",$time,a);
    a=1'b1;
    #10;
    a=1'b0;

#20 $finish;
end
endmodule
```

Simulation Result

Output

```
Time=0| a=1
Time=10| a=0
$finish called from file "example.sv", line 13.
$finish at simulation time                   30
```

- Always_ff, always_comb, always_latch
- For combinational circuit- alwaus_comb (a,b,c,d,e,f)

- For sequential circuit- always_ff (posedge clk)

Uasge of always clock:

always #5 clk=~clk; //T=10ns

always #10 clk=~clk; // T=20ns

```
//
module multiple_clk_generate();
reg clk;
reg rst;
reg clk50;
reg clk25 = 0;

initial begin
        clk=1'b0;
        rst=1'b0;
        clk50=1'b0;
end
always #5 clk=~clk;         //100MHz
always #10 clk50=~clk50;    //50MHz
always #20 clk25=~clk25;    //25MHz

initial begin
        $monitor("Time=%0t| clk=%0b|clk25=%0b|clk50=%0b",$time,clk,clk25,clk50);

#100 $finish;
end
endmodule
```

Simulation Result

Output

Time=0| clk=0|clk25=0|clk50=0

Time=5| clk=1|clk25=0|clk50=0

Time=10| clk=0|clk25=0|clk50=1

Time=15| clk=1|clk25=0|clk50=1

Time=20| clk=0|clk25=1|clk50=0

Time=25| clk=1|clk25=1|clk50=0

Time=30| clk=0|clk25=1|clk50=1

Time=35| clk=1|clk25=1|clk50=1

Time=40| clk=0|clk25=0|clk50=0

Time=45| clk=1|clk25=0|clk50=0

Time=50| clk=0|clk25=0|clk50=1

Time=55| clk=1|clk25=0|clk50=1

Time=60| clk=0|clk25=1|clk50=0

Time=65| clk=1|clk25=1|clk50=0

Time=70| clk=0|clk25=1|clk50=1

Time=75| clk=1|clk25=1|clk50=1

Time=80| clk=0|clk25=0|clk50=0

Time=85| clk=1|clk25=0|clk50=0

Time=90| clk=0|clk25=0|clk50=1

Time=95| clk=1|clk25=0|clk50=1

$finish called from file "example.sv", line 21.

$finish at simulation time 100

V C S S i m u l a t i o n R e p o r t

Design code & Testbench: -

module aligned_all_clock();

```
reg clk;
//reg rst;
reg clk50;
reg clk25 = 0;

initial begin
	clk=1'b0;
	//rst=1'b0;
	clk50=1'b0;
end

always #5 clk=~clk;
always begin
	#5;
	clk50=1;
	#10;
	clk50=0;
	#5;
end

always begin
	#5;
	clk25=1;
	#20;
	clk25=0;
	#15;
end
```

```
initial begin
        $monitor("Time=%0t| clk=%0b|clk25=%0b|clk50=%0b",$time,clk,clk25,clk50);

#150 $finish;

end
endmodule

/*
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,multiple_clk_generate);
end
*/
//endmodule
```

Simulation Result

Output

```
Time=0| clk=0|clk25=0|clk50=0
Time=5| clk=1|clk25=1|clk50=1
Time=10| clk=0|clk25=1|clk50=1
Time=15| clk=1|clk25=1|clk50=0
Time=20| clk=0|clk25=1|clk50=0
Time=25| clk=1|clk25=0|clk50=1
Time=30| clk=0|clk25=0|clk50=1
Time=35| clk=1|clk25=0|clk50=0
Time=40| clk=0|clk25=0|clk50=0
```

Time=45| clk=1|clk25=1|clk50=1

Time=50| clk=0|clk25=1|clk50=1

Time=55| clk=1|clk25=1|clk50=0

Time=60| clk=0|clk25=1|clk50=0

Time=65| clk=1|clk25=0|clk50=1

Time=70| clk=0|clk25=0|clk50=1

Time=75| clk=1|clk25=0|clk50=0

Time=80| clk=0|clk25=0|clk50=0

Time=85| clk=1|clk25=1|clk50=1

Time=90| clk=0|clk25=1|clk50=1

Time=95| clk=1|clk25=1|clk50=0

Time=100| clk=0|clk25=1|clk50=0

Time=105| clk=1|clk25=0|clk50=1

Time=110| clk=0|clk25=0|clk50=1

Time=115| clk=1|clk25=0|clk50=0

Time=120| clk=0|clk25=0|clk50=0

Time=125| clk=1|clk25=1|clk50=1

Time=130| clk=0|clk25=1|clk50=1

Time=135| clk=1|clk25=1|clk50=0

Time=140| clk=0|clk25=1|clk50=0

Time=145| clk=1|clk25=0|clk50=1

$finish called from file "example.sv", line 34.

$finish at simulation time 150

V C S S i m u l a t i o n R e p o r t

Time scale directive-

- `timescale =time unit/ time precision = precision digits= precision value
- 10^0 Precision digits =0, **10^3 Precision digits =3**

Design code & Testbench: -

Generate multiple clocks in single code and analysed the impact of time unit and precision unit

```
`timescale 1ns/1ps
module GEN_clock_time_Precision();
reg clk_16 = 0;
reg clk_8 = 0;

always #31.25 clk_16=~clk_16;
always #62.50 clk_8=~clk_8;

initial begin
    $monitor("Time=%0.2f| clk_16=%0b|clk_8=%0b",$realtime,clk_16,clk_8);

#300 $finish;
end
endmodule
```

Simulation Result

Output

```
Time=0.00| clk_16=0|clk_8=0
Time=31.25| clk_16=1|clk_8=0
Time=62.50| clk_16=0|clk_8=1
Time=93.75| clk_16=1|clk_8=1
Time=125.00| clk_16=0|clk_8=0
Time=156.25| clk_16=1|clk_8=0
```

Time=187.50| clk_16=0|clk_8=1

Time=218.75| clk_16=1|clk_8=1

Time=250.00| clk_16=0|clk_8=0

Time=281.25| clk_16=1|clk_8=0

$finish called from file "example.sv", line 16.

$finish at simulation time 300000

V C S S i m u l a t i o n R e p o r t

Understanding the parameters for generating the clock: -

Duty cycle = T_ON/T = T_ON / (T_OFF+T_ON)

Periods T=1/ f

Real periods =ns

Real T_ON=period * (D.C.)

Real T_OFF=Period -T_ON

clock generates with different phase

```
module gen_clock_with_phase();
reg clk =0;
reg clk50 =0;

always #5 clk=~clk;  //100MHTZ

real phase=10;
real ton= 5;
real toff=5;

initial begin
```

```
        #phase;
while(1) begin
        clk50 = 1;
        #ton ;
        clk50 = 0;
        #toff ;
end
end

initial begin
        $monitor("Time=%0t| clk=%0b|clk50=%0b",$time,clk,clk50);

        #200 $finish;
end
endmodule
```

Simulation Result

Output

```
Time=0| clk=0|clk50=0
Time=5| clk=1|clk50=0
Time=10| clk=0|clk50=1
Time=15| clk=1|clk50=0
Time=20| clk=0|clk50=1
Time=25| clk=1|clk50=0
Time=30| clk=0|clk50=1
Time=35| clk=1|clk50=0
Time=40| clk=0|clk50=1
Time=45| clk=1|clk50=0
```

Time=50| clk=0|clk50=1

Time=55| clk=1|clk50=0

Time=60| clk=0|clk50=1

Time=65| clk=1|clk50=0

Time=70| clk=0|clk50=1

Time=75| clk=1|clk50=0

Time=80| clk=0|clk50=1

Time=85| clk=1|clk50=0

Time=90| clk=0|clk50=1

Time=95| clk=1|clk50=0

Time=100| clk=0|clk50=1

Time=105| clk=1|clk50=0

Time=110| clk=0|clk50=1

Time=115| clk=1|clk50=0

Time=120| clk=0|clk50=1

Time=125| clk=1|clk50=0

Time=130| clk=0|clk50=1

Time=135| clk=1|clk50=0

Time=140| clk=0|clk50=1

Time=145| clk=1|clk50=0

Time=150| clk=0|clk50=1

Time=155| clk=1|clk50=0

Time=160| clk=0|clk50=1

Time=165| clk=1|clk50=0

Time=170| clk=0|clk50=1

Time=175| clk=1|clk50=0

Time=180| clk=0|clk50=1

```
Time=185| clk=1|clk50=0
Time=190| clk=0|clk50=1
Time=195| clk=1|clk50=0
$finish called from file "example.sv", line 26.
$finish at simulation time                 200
         V C S   S i m u l a t i o n   R e p o r t
```

To clock generate with different phase using task function by using the multiple parameter:

```
module clk_gen_phase_taskfxn();
reg clk =0;
reg clk50 =0;
always #5 clk=~clk;  //100MHTZ

task clkgen(input real phase, input real ton, input real toff);
        #phase;
while(1) begin
        clk50 = 1;
        #ton ;
        clk50 = 0;
        #toff ;
end
endtask

initial begin
        clkgen(7, 5, 5);
end
```

```
initial begin
        $monitor("Time=%0t| clk=%0b|clk50=%0b",$time,clk,clk50);
        #100 $finish;
end
endmodule
```

Simulation Result

Output

```
Time=0| clk=0|clk50=0
Time=5| clk=1|clk50=0
Time=7| clk=1|clk50=1
Time=10| clk=0|clk50=1
Time=12| clk=0|clk50=0
Time=15| clk=1|clk50=0
Time=17| clk=1|clk50=1
Time=20| clk=0|clk50=1
Time=22| clk=0|clk50=0
Time=25| clk=1|clk50=0
Time=27| clk=1|clk50=1
Time=30| clk=0|clk50=1
Time=32| clk=0|clk50=0
Time=35| clk=1|clk50=0
Time=37| clk=1|clk50=1
Time=40| clk=0|clk50=1
Time=42| clk=0|clk50=0
Time=45| clk=1|clk50=0
Time=47| clk=1|clk50=1
```

Time=50| clk=0|clk50=1

Time=52| clk=0|clk50=0

Time=55| clk=1|clk50=0

Time=57| clk=1|clk50=1

Time=60| clk=0|clk50=1

Time=62| clk=0|clk50=0

Time=65| clk=1|clk50=0

Time=67| clk=1|clk50=1

Time=70| clk=0|clk50=1

Time=72| clk=0|clk50=0

Time=75| clk=1|clk50=0

Time=77| clk=1|clk50=1

Time=80| clk=0|clk50=1

Time=82| clk=0|clk50=0

Time=85| clk=1|clk50=0

Time=87| clk=1|clk50=1

Time=90| clk=0|clk50=1

Time=92| clk=0|clk50=0

Time=95| clk=1|clk50=0

Time=97| clk=1|clk50=1

$finish called from file "example.sv", line 24.

$finish at simulation time 100

V C S S i m u l a t i o n R e p o r t

clock generate using all parameter

module gen_clk_using_all_parameter( );

reg clk =0;

reg clk50 =0;

```
always #5 clk=~clk;  //100MHTZ

task calc(input real freq_hz, input real duty_cycle,input real phase , output real pout,
output real ton, output real toff);

        pout=phase;
        ton =(1.0 / freq_hz) * duty_cycle * 1000_000_000;
        toff= (1000_000_000 / freq_hz)-ton ;
endtask

task clkgen(input real phase, input real ton, input real toff);
        #phase;
while(1) begin
        clk50 = 1;
        #ton ;
       clk50 = 0;
       #toff ;
end
endtask

real phase;
real ton;
real toff;

initial begin
calc(100_000_000 , 0.5 , 7 , phase, ton , toff);
clkgen(phase, ton , toff);
end
```

```
initial begin
        $monitor("Time=%0t|
clk=%0b|clk50=%0b|phase=%0d|ton=%0d|toff=%0d",$time,clk,clk50,phase,ton,toff);


        #100 $finish;
end
endmodule
```

Simulation Result

Output

```
Time=0| clk=0|clk50=0|phase=7|ton=5|toff=5
Time=5| clk=1|clk50=0|phase=7|ton=5|toff=5
Time=7| clk=1|clk50=1|phase=7|ton=5|toff=5
Time=10| clk=0|clk50=1|phase=7|ton=5|toff=5
Time=12| clk=0|clk50=0|phase=7|ton=5|toff=5
Time=15| clk=1|clk50=0|phase=7|ton=5|toff=5
Time=17| clk=1|clk50=1|phase=7|ton=5|toff=5
Time=20| clk=0|clk50=1|phase=7|ton=5|toff=5
Time=22| clk=0|clk50=0|phase=7|ton=5|toff=5
Time=25| clk=1|clk50=0|phase=7|ton=5|toff=5
Time=27| clk=1|clk50=1|phase=7|ton=5|toff=5
Time=30| clk=0|clk50=1|phase=7|ton=5|toff=5
Time=32| clk=0|clk50=0|phase=7|ton=5|toff=5
Time=35| clk=1|clk50=0|phase=7|ton=5|toff=5
Time=37| clk=1|clk50=1|phase=7|ton=5|toff=5
Time=40| clk=0|clk50=1|phase=7|ton=5|toff=5
Time=42| clk=0|clk50=0|phase=7|ton=5|toff=5
```

Time=45| clk=1|clk50=0|phase=7|ton=5|toff=5

Time=47| clk=1|clk50=1|phase=7|ton=5|toff=5

Time=50| clk=0|clk50=1|phase=7|ton=5|toff=5

Time=52| clk=0|clk50=0|phase=7|ton=5|toff=5

Time=55| clk=1|clk50=0|phase=7|ton=5|toff=5

Time=57| clk=1|clk50=1|phase=7|ton=5|toff=5

Time=60| clk=0|clk50=1|phase=7|ton=5|toff=5

Time=62| clk=0|clk50=0|phase=7|ton=5|toff=5

Time=65| clk=1|clk50=0|phase=7|ton=5|toff=5

Time=67| clk=1|clk50=1|phase=7|ton=5|toff=5

Time=70| clk=0|clk50=1|phase=7|ton=5|toff=5

Time=72| clk=0|clk50=0|phase=7|ton=5|toff=5

Time=75| clk=1|clk50=0|phase=7|ton=5|toff=5

Time=77| clk=1|clk50=1|phase=7|ton=5|toff=5

Time=80| clk=0|clk50=1|phase=7|ton=5|toff=5

Time=82| clk=0|clk50=0|phase=7|ton=5|toff=5

Time=85| clk=1|clk50=0|phase=7|ton=5|toff=5

Time=87| clk=1|clk50=1|phase=7|ton=5|toff=5

Time=90| clk=0|clk50=1|phase=7|ton=5|toff=5

Time=92| clk=0|clk50=0|phase=7|ton=5|toff=5

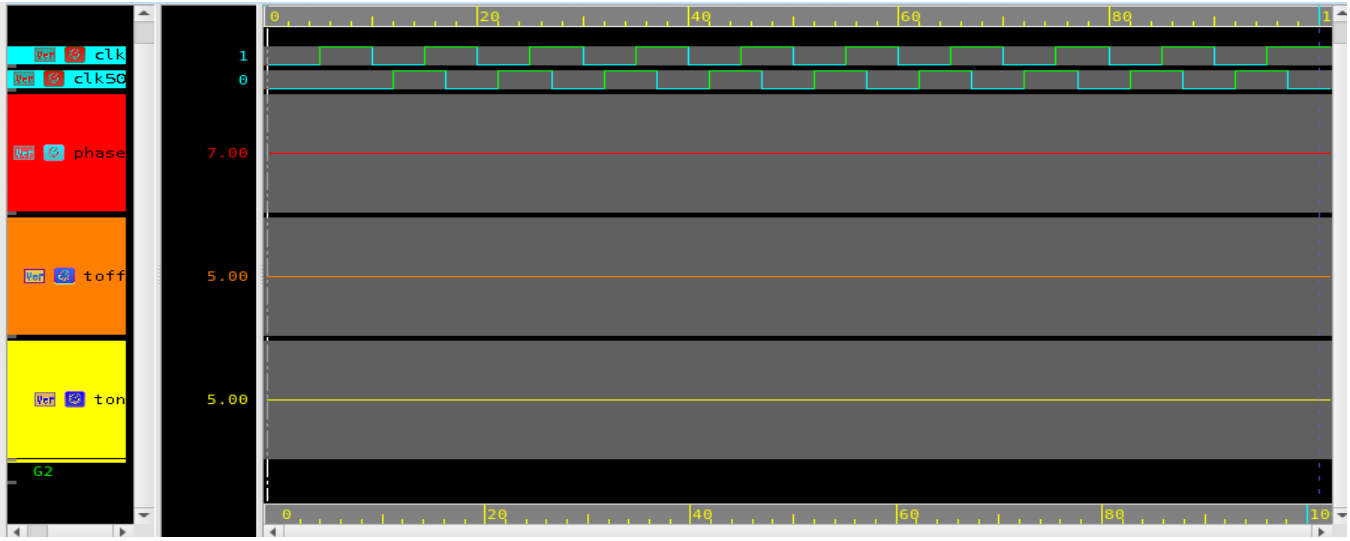
Time=95| clk=1|clk50=0|phase=7|ton=5|toff=5

Time=97| clk=1|clk50=1|phase=7|ton=5|toff=5

$finish called from file "example.sv", line 36.

$finish at simulation time 100

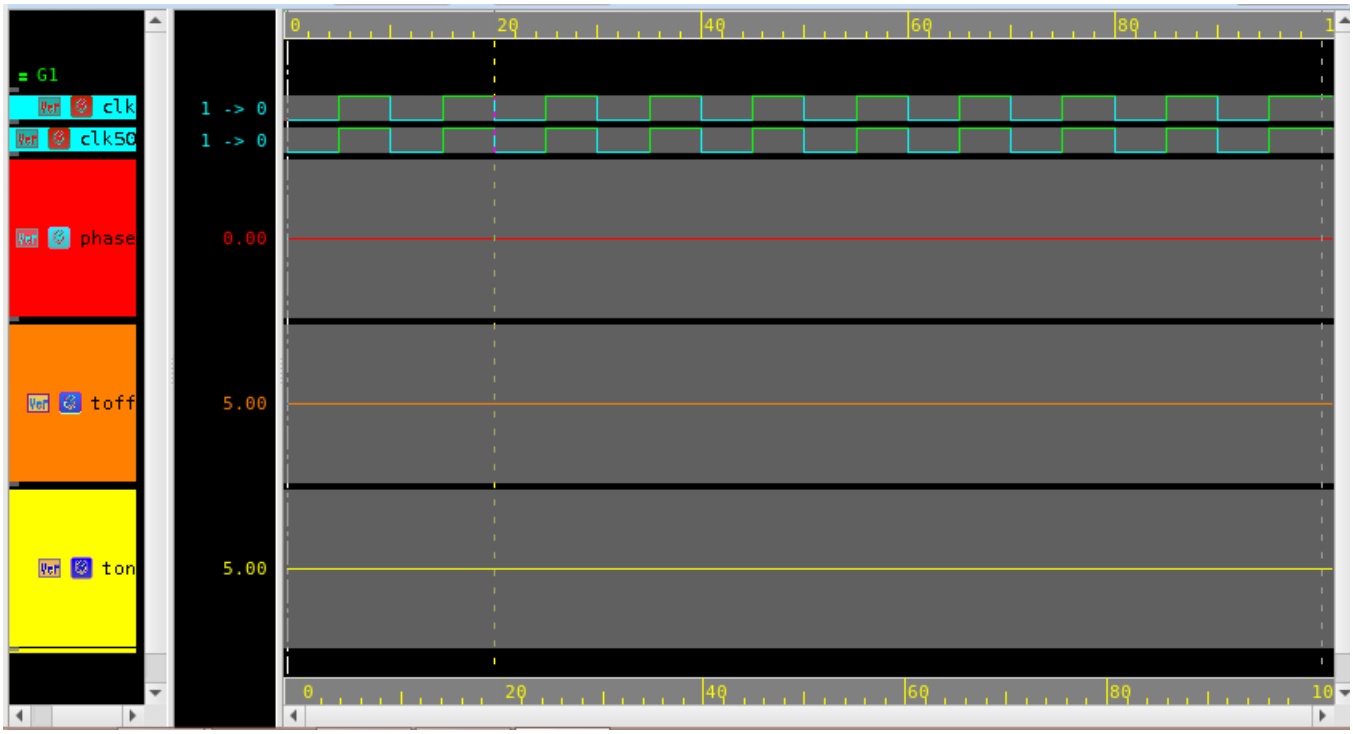
V C S S i m u l a t i o n R e p o r t



Method-2

Time=0| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=5| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=10| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=15| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=20| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=25| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=30| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=35| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=40| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=45| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=50| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=55| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=60| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=65| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=70| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=75| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=80| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=85| clk=1|clk50=1|phase=0|ton=5|toff=5
Time=90| clk=0|clk50=0|phase=0|ton=5|toff=5
Time=95| clk=1|clk50=1|phase=0|ton=5|toff=5
$finish called from file "example.sv", line 37.
$finish at simulation time 100
V C S S i m u l a t i o n R e p o r t

Waveform



Design code

Testbench

Run command

Simulation Result

Output

Schematic Design

waveform

System Verilog

Class-1

What is the System Verilog?

System Verilog (SV) is a hardware description and hardware verification language (HDVL). It is an extension of Verilog (the HDL used for RTL design), adding new features for:

1. RTL design modelling (like Verilog, but with more powerful constructs).

2. Verification (advanced testbenches, constrained-random stimulus, functional coverage, assertions).

In short: **Verilog = design language**, while **System Verilog = design + verification language.**

- Verilog (IEEE 1364) was great for design, but weak in verification.
- Testbenches became very large and hard to maintain for complex SoCs.
- System Verilog (IEEE 1800 standard) was introduced to unify design + verification in one language.
- Key Features of System Verilog-

1. Design Features (synthesizable)

Used in RTL coding (like Verilog, but richer):

logic (replaces `reg`/`wire`)

`always_comb`, `always_ff`, `always_latch` (better than just `always`)

Enumerated types (`enum`)

Structures and unions

Interfaces (to simplify module connections)

Packed/unpacked arrays

2. Verification Features (non-synthesizable)-

Used in Testbenches: -

- Object-oriented programming (OOP): classes, inheritance, polymorphism.
- Randomization (`rand`, `constraint`) for constrained-random test generation.
- Functional coverage (`cover group`).
- Assertions (`assert property`) → check design correctness during simulation

- Mailboxes, semaphores (for inter-process communication)

Industry Usage-

Design engineers use SV for RTL coding (instead of plain Verilog).

Verification engineers use SV for advanced testbenches along with UVM (Universal Verification Methodology).

In simple words: -

System Verilog = Verilog + verification features. (HVL)

It is the modern standard for both design and verification in the semiconductor industry.

……

System Verilog is an extended version of language of Verilog.

UVM is a methodology, not a language but it is based on SV.

ASIC/DV Engineer- Verilog+ SV+ UVM+ Protocals

Verilog supports static properties only (everything is determined at compile/elaboration time). But System Verilog (SV) supports dynamic + static both properties (like dynamic objects, constraints, randomization).

Coverage- Measure the how much functionality/ design of code have been tested.

Verification= cover case/ total case and we obtain the result like 75%, 90% and 100%.

If I found coverage verification 50% then we need to again check the remaining 50% cases → may to pass the 30%, 40%.

Check, more than 90% or 95% verification is okay which is near around 100%.

There are two methods for the verification –

1.0 manually apply the input signal all cases
2.0 Randomization technique

Data Types: -

In System Verilog, the logic data type is the most commonly used 4-state data type. It was introduced to replace `reg` (from Verilog) and to unify usage.

Logic → reg/ wire

Key points about `logic`:

4-state type → can take values: 0, 1, X (unknown), Z (high-impedance).

- Can be used in procedural blocks (`always`, `initial`) and for continuous assignments (unlike Verilog `reg`).
- Eliminates the **confusion** between `wire` and `reg`. because 80% work +20% confuse.
- By default, **`logic` is scalar**, but can be declared as **vectors** (`logic [7:0] data; `).
- For multiple drivers (like tri-state buses), wire` is still needed, not `logic`.

Example: system Verilog

logic a; // single-bit 4-state

logic [7:0] bus; // 8-bit vector

- **Synthesis tool will decide automatically- reg/wire.**
- **Mostly, logic of hardware is reg.**

System Verilog provides better constructs for synthesis purposes (always_comb, always_ff, always_latch).

- Reg, time → default value is X, real default value → 0 and wire → Z
- 4-state data types basic values → 0, 1, x, z, default/initial value is X.
- 2-state data types basic value → 0, 1, default/initial value is 0.
- **int i;** default value of i is **32'b0** and it supports two state value (0,1).

Enumerated data type for design modelling in system Verilog-

In System Verilog, an enumerated data type (enum) is used in design modeling to give meaningful names to a **set of values** (instead of just **using numbers**). It **improves readability, maintainability, and debugging of the code.**

Syntax:

enum [datatype] {list_of_names} enum_variable;

- `datatype` → underlying representation (default = `int`, but can be `bit`, **`logic`**, etc.).
- `list_of_names` → symbolic names for values.
- `enum_variable` → variable of that enumerated type.

#Example (Design Modeling):

```
typedef enum logic [1:0] {
    IDLE = 2'b00,
    READ = 2'b01,
    WRITE = 2'b10,
    DONE = 2'b11
} state_t;
state_t current_state, next_state;
```

- Here `state_t` is an **enumerated type.**
- Instead of using raw numbers like 00,01,10,11we use direct name in the code, the FSM states are modelled as IDLE, READ, WRITE, DONE.

Benefits in design modeling:

- Clear **readability** → avoids magic numbers.
- Automatic value assignment (if not specified, **default starts from 0, increments by 1**).
- Debug-friendly → waveform viewers (like Verdi) show names (e.g., IDLE instead of `2'b00`).

☑ So, in design modeling, enumerated data types are commonly used for **FSM state representation** and other symbolic constants.

- enum is a data type in system Verilog.
- User-defined types that can be defined once and used thought the design-- **typedef** use for user-friendly only.

- Supporting construct for user-defined data types- **typedef**
- Calling by name and it back works number.
- Improve the readability of the code only.

✓ Package to share declarations amongst several modules:
- package……. endpackage.
- Package is like an instantiation.

Verilog has strict data type rules-

- Variables (**registers**) are assigned values in **procedural blocks.**
- **Nets are driven by continuous assignment**, module output or primitive instances.

These rules lead to following connectivity characteristics: -

- Default data type of input is of wire(net) type. It can be changed to reg type inside the design as per the law of port mapping.
- Default data type of output port is of wire (net) type. But it can be changed to other data type. Example: reg (abstract) type if it is to be assigned by behavioral blocks (initial and always).
- While instantiating the module,
- **Input ports** can be connected to **reg or wire** both type outside the module.
- **Output /inout ports** will be connected to **wire type only** outside the module.

2-State Data Type-

- The Verilog set consists of four basic values (0, 1, X, Z).
- System Verilog adds 2-state value types based on bit.
- Values 0 & 1 only.
- Direct replacement for reg, logic or integer.
- Greater efficiency at higher-abstraction level module (RTL).
- Reset value is ‘b0.
- 2-state variables initialize to 0.
- 4-state variables initialize to X.

4-state to 2-state-

0	0
1	1
x	0
z	0

Literal value Assignment-

<size>`<value> ------- Verilog suntax

`<value> --------System Verilog syntax

Example:

reg [5:0] databus;

databus = 6'hF; //001111

databus = 6'h0; //000000

databus = 6'h1; //000001

databus = 6'hZ; //ZZZZZZ

databus = 6'hX; //XXXXXX

databus = 4'hX; //00XXXX

reg [5:0] databus;

databus = `1; //111111

databus = `0; //000000

databus = `X; //XXXXXX

databus = `Z; //ZZZZZZ

- Verilog -2001 extends to size of expression.
- New system Verilog single bit literal syntax and fill whole target with that single value.

SV added new escaped character- all these types of character will improve beauty of output readability in better way for present and print on the terminal screen.

- "\V" // for verticalalignment
- "\a" // for alignment

- “\f” // form feed
- “\x02” //hexadecimal number
- “\”” // double quote

1.0 lab1 Checking the literal value of assignment Verilog and system Verilog syntax.

Design code

```
// literal value assignment check the Verilog and system Verilog syntax
module test;
reg [5:0] databus;

initial begin
    $monitor("final output result databus=%b ",databus);
    databus=6'hF;
#5 databus=6'h0;
#5 databus=6'h1;
#5 databus=6'hZ;
#5 databus=6'hX;
#5 databus=4'hX;

#5 databus='1;
#5 databus='0;
#5 databus='X;
#5 databus='Z;
end
endmodule
```

Simulation Result

Output

final output result databus=001111

final output result databus=000000

final output result databus=000001

final output result databus=zzzzzz

final output result databus=xxxxxx

final output result databus=00xxxx

final output result databus=111111

final output result databus=000000

final output result databus=xxxxxx

final output result databus=zzzzzz

V C S S i m u l a t i o n R e p o r t

Time: 45

1.1 Checking the literal value of assignment Verilog and system Verilog syntax.

In the above code change only one line of code- Vertical alignment on the output screen

initial begin

$monitor("final output result databus=%b \v",databus);

Simulation Result

Output

final output result databus=001111

final output result databus=000000

final output result databus=000001

final output result databus=zzzzzz

final output result databus=xxxxxx

final output result databus=00xxxx

final output result databus=111111

final output result databus=000000

final output result databus=xxxxxx

final output result databus=zzzzzz

1.2 Checking the literal value of assignment Verilog and system Verilog syntax.

In the above code change only one line of code- it will align the output only.

initial begin

$monitor("final output result databus=%b \a",databus);

Simulation Result

Output

final output result databus=001111

final output result databus=000000

final output result databus=000001

final output result databus=zzzzzz

final output result databus=xxxxxx

final output result databus=00xxxx

final output result databus=111111

final output result databus=000000

final output result databus=xxxxxx

final output result databus=zzzzzz

1.3 Checking the literal value of assignment Verilog and system Verilog syntax.

In the above code change only one line of code- it will align the output only.

initial begin

$monitor("final output result databus=%b \x02",databus);

Simulation Result

Output

final output result databus=001111

final output result databus=000000

final output result databus=000001

final output result databus=zzzzzz

final output result databus=xxxxxx

final output result databus=00xxxx

final output result databus=111111

final output result databus=000000

final output result databus=xxxxxx

final output result databus=zzzzzz

Time Literals-

System Verilog allows units when specifying time literals.

- Integer or fixed-point value.
- Time-unit (fs, ps, ns, us, ms, s step)
- 1 step is one time unit precision unit .i.e 1 clock cycles.

- No space between the value and unit.

Literal is interpreted as a realtime value scaled to current time unit and rounded to the current precision.

Example-

module vlsi_expert();

timeunit 1ns;

timeprecision 10ps;

endmodule

In System Verilog, a time literal is a constant value that represents a time duration, written as a number + time unit suffix.

Syntax

<integer_or_real> <time_unit>

- `s` → seconds
- `ms` → milliseconds
- `us` → microseconds
- `ns` → nanoseconds
- `ps` → picoseconds
- `fs` → femtoseconds

Examples

#10ns // delay of 10 nanoseconds

time t1 = 5us;

time t2 = 2.5ms;

Key points

- Time literals are scaled according to the `timeunit` / `timeprecision` declarations in your module/file.

Example:

timeunit 1ns / 1ps;

```
initial begin
  #5us; // 5 microseconds = 5000 ns delay
  #10ps; // 10 picoseconds delay
end
```

* `time` is a built-in 64-bit integer type in SV that can store time values.

1. `#10` → Delay in simulation time units.

- If you **don't specify a unit**, the delay is interpreted in terms of the **current `timeunit`.**

Example:

```
timeunit 1ns / 1ps;
initial begin
  #10;  // delay = 10 ns (because timeunit = 1ns)
end
```

If you change the `timeunit`:

```
timeunit 1ps / 1ps;
initial begin
  #10; // delay = 10 ps (now, because timeunit = 1ps)
end
```

- So, `#10` depends on the `timeunit` declaration.

2. `#10ns` → Delay in explicit time literal

- A time literal includes its own unit suffix (`ns`, `ps`, etc.).
- It gets converted automatically to the current `timeunit`.

Example:

```
timeunit 1ps / 1ps;
initial begin
  #10ns;   // delay = 10 ns = 10000 ps
end
```

- Even though `timeunit` is 1ps, the `10ns` is scaled into that unit correctly.

☑ Key Difference

* `#10` → **depends on the current `timeunit`.**

* `#10ns` → absolute duration**, independent of `timeunit`**, automatically scaled.

Time unit and time precision-

- Time unit/time-precision declared in a design unit is only visible only in that design unit and nested units
- **Must be the first statement.**

2.0 Checking the Time scale literals by using System Verilog syntax.

Design code

```
// literal value assignment using different time scale
`timescale 1ns/1ps
module test;
reg [5:0] databus;

initial begin
    $monitor ("Time=%0t final output result databus=%b \a", $realtime, databus);
    databus=6'hF;
#5ns databus=6'h0;
#5ns databus=6'h1;
#5ps databus=6'hZ;
#5ps databus=6'hX;
```

```
#5ns databus=4'hX;


#5 databus='1;
#5 databus='0;
#5 databus='X;
#5 databus='Z;
end
endmodule
```

Simulation Result

Output

```
Time=0 final output result databus=001111
Time=5000 final output result databus=000000
Time=10000 final output result databus=000001
Time=10005 final output result databus=zzzzzz
Time=10010 final output result databus=xxxxxx
Time=15010 final output result databus=00xxxx
Time=20010 final output result databus=111111
Time=25010 final output result databus=000000
Time=30010 final output result databus=xxxxxx
Time=35010 final output result databus=zzzzzz
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 35010 ps

`$realtime` → returns the **current simulation time** as a real number (with fractional precision).

`$real` → is just a data type (not a system function) used to declare real-valued variables.

👉 **In short:**

`$realtime` = gives simulation time (value).

`real` = data type to store decimal numbers.

‘logic’ Replaces ‘reg OR wire’

The **Verilog reg data** is potentially **confusing.**

- Can synthesizable to net or register depending upon use.

System Verilog has a new logic data type.

- var logic (variable of logic type) replaces reg.
- var keyword is optional.

Example:

module clock;

logic [15:0]b_reg;

logic [7:0] mem8x32 [31:0];

Example:

module [3:0] dout;

always_ff @(posedge clk or posedge rst)

if (rst)

dout <= 4’b0000;

else if (cnt)

dout <=dout +1;

Logic	Reg or wire
Default to X	reg Default to X. wire defaults to Z.
Can be driven using gate level data flow and behavioral modelling.	reg is only derived from behavioral modelling.
Single driven.	Wire is multiple driven.
Can be used as a reg and wire both.	Reg and wire can’t be replaced each other, both have separate scope.

3.0 Checking the Time scale units by using System Verilog syntax.

Design code

```
module testmodule;
timeunit 1ns;
timeprecision 1ps;
reg a, sel, y, z, clk;

initial begin
    #20ns sel =0;
    $display("Time=%0t ns : sel=%0b",$realtime,sel);
    #5.18ns sel = 1;
    $display("Time=%0t ns : sel=%0b",$realtime,sel);
    y = #1step z;
    $display("Time=%0t ns : y=%0b (after #1step delay)",$realtime,y);
end

always @(posedge clk)begin
    #5.1ns a <= a + 1;
    $display("Time=%0t ns :a=%0b(after clk posedge +5.1ns)",$realtime,a);
end
endmodule
```

Simulation Result

Output

```
Time=20000 ns : sel=0
```

Time=25180 ns : sel=1

Time=25181 ns : y=x (after #1step delay)

V C S S i m u l a t i o n R e p o r t

Time: 25181 ps

4.0 Checking the Time unit and timeprecisions-1ns/100ps =10

Design code

```
module testbench;
timeunit 1ns;
timeprecision 100ps;
reg status;

initial begin
    $monitor ("Time=%0f, ns status=%b", $realtime, status);
    #10ns status=1'b1;
    #10 status= 1'b0;
end
endmodule
```

Simulation Result

Output

Time=0.000000,ns status=x

Time=10.000000,ns status=1

Time=20.000000,ns status=0

V C S S i m u l a t i o n R e p o r t

Time: 20000 ps

Advantage of Logic data type-

1.0 Simplifies the port declarations

In Verilog-

- Inputs were declared as wire by default.
- Output needed to be declared as reg if driven procedurally.

In System Verilog-

- **logic** can be used for **all port directions**(input,output,inout), simplifying the declarations.

```
module example (
input wire clk,
input wire a,
output reg b);
always @(posedge clk)
b <= a;
endmodule
```

```
module example (
input wire clk,
input wire a,
output reg b);
always_ff @(posedge clk)
b <= a;
endmodule
```

2.0 Support Enhanced Synthesis Constructs-

System Verilog provides better constructs for synthesis (**always_ff, always_comb,always_latch)**, and logic works seamless with these.

// **help for creating the synthesizable practical hardware circuit**

```
logic clk,reset,d,q;
```

```
//sequential logic
Always_ff  @(posedge clk or posedge reset) begin
If (reset)
q <=0;
else
q <=d;
end
endmodule
```

3.0 Supports Explicit intent for multi-driver Scenario-

When you explicit declare a signal as “wire” , you are indicating it can have multiple drivers. **The simulators resolve conflicts between these drivers based on logical rules.**

```
wire bus;                /// if logic would have been used here then, complier would have thrown error.
logic en1,en2;
logic data1,data2;
// Tri-state drivers    //  what if en1 and en2 both are enabled??
assign bus = (en1) ? data1 : 1’bZ;  // High-impedance if en1 is not enabled
assign bus = (en2) ? data2 : 1’bZ;  // High-impedance if en2 is not enabled
```

The advantage is that if use logic data type, then, you cannot have multiple drivers for a variable by mistake, and if you wanted to have it then explicity use wire data type (but in Verilog if reg would be there then you must have received unexpected result rather than error.

Data Type “STRING”-

In SV, size of string data type is dynamically allocated at run time.

- Verilog does not have direct string data type.

Str1 ==str2	equality
Str1 != str2	inequality
< , <=, >=, >	comparision
{str1, str2,str3}	concatenation
Str [l]	Accessing index element

By default string is empty.

If the length of string is empty N,then the elements od string are numbered from 0 to N-1.

String s1 = “VLSI Expert”;

String s2 = { “vyom sharma”, “ “,s1);

bit [31:0] b=128;

string s3 = b ;

Predefined method of string-

len() – to calculate the length of string.

putc() – replace the value at given index.

toupper()- convert the string into uppercase.

tolower()- convert the string into lowercase.

Substr()- Extract a Sub-string between two indices.

getc()- retrives a character at a specific index.

compare()- compares two string (returns 0 if equal, negative if less, positive if greater).

atio()- convert a string to an integer.

itoa()- convert an integer to an string.

strcpy()- Copies one string into another.

strcat()- concatenates two string.

find()- Finds the first occurrence of a string.

icompare()- Case-insensivitive string comparision.

trim()- Removes the leading and trailing whitespace.

split()- Split a string into parts based on a delimeter.

5.0 Use of common string method.

Design code-

module test;

```
string my_length= "sarif_ahmad";
initial begin
    $display("length of string: %0d",my_length.len());
    $display("upper_case of string: %0s",my_length.toupper());
    $display("lower_case of string: %0s",my_length.tolower());
    $display("extract the string between indexes: %0s",my_length.substr(0,4));
    my_length.putc(5,"P");
    $display("string after putc: %0s",my_length);

end
endmodule
```

Simulation Result

Output

```
 length of string: 11
upper_case of string: SARIF_AHMAD
lower_case of string: sarif_ahmad
extract the string between indexes: sarif
string after putc: sarifPahmad
```

6.0 Predefined method of string.

Design code-

```
module string_method;
string num_str="12345";
int num=6789;
bit [7:0]b =8'b10101010;
string q[$];
```

```
initial begin
        string s1="Hello,SystemVerilog!";
        string s2="SystemVerilog";
        string s3="    Trim me    ";
        string s4="Comma,Seperated,Value";
        string s5="Hello";
        string s6;
        string s7;

        $display("original s1: '%s'",s1);
        $display("length of s1: %0d",s1.len());
        s1.putc(7,"P");
        $display("s1 after putc: '%s'",s1);
        $display("character at position 7 in s1: '%s'",s1.getc(7));
        $display("s1 compare to s2: %0d", s1.compare(s2));
        $display("s1 compare to s5: %0d", s1.compare(s5));
        $display("s1 to uppercase: %s", s1.toupper());
        $display("s1 to lowercase: %s", s1.tolower());
        $display("substring of s1 (index 7 to 14):'%s'", s1.substr(7,14));
        $display("string '%s' to integer: %0d",num_str,num_str.atoi());

        s6.itoa(num);
        $display("integer to %0d to string: '%s'",num,s6);
        s6.itoa(b);
        $display("bit vector to string: '%s'",b,s6);
        s7=s2;
        $display("s6 after strcpy: '%s'",s7);
```

```
        s7= {s7," is powerful"};
        $display("s6 after strcat : '%s'", s7);
//      $display("Find 'system' in s1: %0d",s1.find("system"));
//      $display("Find 'system' in s1 starting at index 8: %0d", s1.find("system",8));
        //$display("find 'VHDL' in s1: %0d",s1.find("VHDL"));
        //string mixed= "systemverilog";
        //$display("compare 'systemverilog' with 'systemverilog':
%0d",mixed.compare("systemverilog"));
        //$display("case-insensitive comparision: %0d",mixed.icompare("systemverilog"));
//      $display("original s3 : '%s'",s3);
//      $display("s3 after trim : '%s'", s3.trim());
//      string q[$];
//      s4.split(",", q);
//      $display("splitting s4 by ','result in: ");
//      foreach (q[i]) $display(" q[%0d]: '%s'",i,q[i]);
end
endmodule
```

Simulation Result

Output

```
original s1: 'Hello,SystemVerilog!'
length of s1: 20
s1 after putc: 'Hello,SPstemVerilog!'
character at position 7 in s1: 'P'
s1 compare to s2: -11
s1 compare to s5: 1
s1 to uppercase: HELLO,SPSTEMVERILOG!
s1 to lowercase: hello,spstemverilog!
```

substring of s1 (index 7 to 14):'PstemVer'

s1 to uppercase: HELLO,SPSTEMVERILOG!

s1 to lowercase: hello,spstemverilog!

substring of s1 (index 7 to 14):'PstemVer'

s6 after strcpy: 'SystemVerilog'

s6 after strcat : 'SystemVerilog is powerful'

Class-2

Struct types-

In System Verilog, a `struct` is a composite data type that groups together variables of different types under one name (similar to `struct` in C/C++). It is useful for organizing related signals or data into a single package.

- Syntax

```
typedef struct {
   data_type   field1;
   data_type   field2;
   ...
} struct_name;
```

▧ Example 1: **Unpacked struct**

```
module tb;
  typedef struct {
    int   id;
    string name;
    real  score;
  } student_t;
```

```
  student_t s1, s2;

  initial begin
    s1.id    = 101;
    s1.name  = "Sarif";
    s1.score = 92.5;

    s2 = '{id:102, name:"Ahmad", score:88.0}; // aggregate assignment

    $display("Student 1 -> id=%0d, name=%s, score=%0f", s1.id, s1.name, s1.score);
    $display("Student 2 -> id=%0d, name=%s, score=%0f", s2.id, s2.name, s2.score);
  end
endmodule
```

Simulation Output

```
Student 1 -> id=101, name=Sarif, score=92.500000
Student 2 -> id=102, name=Ahmad, score=88.000000
```

Example 2: **Packed struct**

A packed struct is useful when you want bit-level control (e.g., **for RTL signals**).

```
module tb;
  typedef struct packed {
    logic [3:0] opcode;
    logic [7:0] src;
    logic [7:0] dest;
  } instr_t;
```

```
  instr_t instr;

  initial begin
    instr = '{opcode:4'b1010, src:8'h12, dest:8'hAB};
    $display("instr as bits: %b", instr);   // prints all bits
    $display("opcode=%b src=%h dest=%h", instr.opcode, instr.src, instr.dest);
  end
endmodule
```

▧ Example 3: **Struct inside arrays**

```
module tb;
  typedef struct {
    int roll_no;
    string name;
  } student_t;

  student_t class[3]; // array of structs

  initial begin
    class[0] = '{1, "Ali"};
    class[1] = '{2, "Rahul"};
    class[2] = '{3, "Meena"};

    foreach (class[i]) begin
      $display("Roll=%0d, Name=%s", class[i].roll_no, class[i].name);
    end
  end
endmodule
```

Key Points

- Packed struct = bit-accurate, synthesizable, behaves like a `logic` vector.
- Unpacked struct = more flexible (can hold strings, reals, dynamic arrays, etc.), mainly for testbenches.
- **Fields can be accessed using the dot (`.`) operator.**
- You can use `typedef` to define reusable struct types.

****

Struct- collection of different data-static

- Array of different data type
- Synthesizable
- Subset of class- use class for TB

Struct {bit [7:0] address;

Logic [7:0] payload [63:0];

Enum {good,error} pkt_type;

Logic [7:0] parity ; } packet;

Packect.address = 8'd25;

Packet.pkt_type =error;

Packed struct-

By default, struct is unpacked element

Struct can be created as packed by using packed keyword

- Packed structure store all elements as contiguous bits
- First member of the structure is left-most field of the vector.

Packed struct {

Logic valid ;

Logic [7:0] tag;

Logic [31:0] data;

} data_word ;

Packed struct-

The member of a packed structure can be referenced by either the name of the member or by part select of the vector.

- Data_word.tag =8'hf0;
- Data_word [39:32]=8'hf0;

Operations on the packed structures

- Assigning values to a packed structure
- Data_word = `{1,8'hff, 1024};

Logical operation that can be performed on vectors can be performed on packed structures.

Casting in SV-

In system Verilog, casting means converting one data type into another data type.

- Sized Casting: To change the size of a value.
- Type casting: To convert a variable to a specific data type.
- Enumerated Casting: To convert raw values to enum types.

1.0 Sized Casting-

- **To change the size of a value.**

When you use 'followed by a size, it ensures that the value is **adjusted to the specified size.**

Examples:

Logic [3:0] a;

Logic [7:0] b=8'hFF;

a= 4'(b); // cast the 8-bit value of b to a 4-bit value.

4'(b) means "interpret b as a 4-bit value."

If b=8'b11111111 (8'FF), then a becomes 4'b1111 (**the lower 4 bits**).

2.0 Type casting

- **To convert a variable to a specific data type.**

You can cast a variable to a specific data type using this syntax.

Syntax:

<data_type>`(<value>)

Example:

Logic [7:0] byte_val;

int int_val;

int_val = int `(byte_val); //**cast the 8-bit byte_val to an int (32 bits by default)**

int`(byte_val) converts the byte_val from an 8-bit logic vector to a 32-bit integer.

3.0 Enumerated Costing-

- **To convert raw values to enum types.**

This is a particularly useful for converting a value to an enum type. In the example, the ‘ is used before the enum type name to cast the value that enum.

Example:

typedef enum logic [1:0] {RED, YELLOW, GREEN, } state_t ;

logic [1:0] raw_value = 2’b0;

state_t current_state;

current_state = state_t`(raw_value); //Cast raw_value to state_t enum

- State_t`(raw_value) interprets raw_value (2’b10) as a member of the state_t enum.
- If raw_value is invalid (ex_ 2’b11 in this case), the behaviour depends on the simulator and whether the value is defined the enum.

User Defined Data Type

- “typedef” is the keyword that is responsible for creating a user defined data type.
- Whatever definition/logic has been created by use, if it has “typedef” written before it then that definition/ logic becomes the data type.

Example-

7.0 Predefined method of string.

- state_t → data type (L → R index)
- typedef → variable → act as a data type convert

Design code-

```
module test;
typedef struct {bit [7:0] r,g,b;} pixel_st;

initial begin

pixel_st Samsung_pixel;
pixel_st Sony_pixel;

Samsung_pixel.r = 8'b10101101;
Sony_pixel = '{8'd38, 8'd23, 8'd44};

$display("print the value of
r=%d,g=%d,b=%d",Samsung_pixel.r,Samsung_pixel.g,Samsung_pixel.b);
$display("print the value of
r=%d,g=%d,b=%d",Sony_pixel.r,Sony_pixel.g,Sony_pixel.b);
end
endmodule
```

Simulation Result

Output

```
print the value of r=173, g= 0, b= 0
print the value of r= 38, g= 23, b= 44
        V C S  S i m u l a t i o n  R e p o r t
```

Enumeration-

enum [enum_base_type] {

enum_label,

enum_label

} enum_name_identifier;

Note: -

enum_base_type: **Default is int.**

- enumeration types defined a set of named values. It contains a list of constant names and one or more variables.
- Name of enum labels must be unique.

Enumeration types-

- A type with user defined values-
- Label must be unique in the current scope.
- Declared with enum keyword.
- Enumerated types are strongly typed.
- Should only be directly assigned:
- Labels by default increment from 0 in declaration order.
  // Anonymous enum type:
  enum {idle, setup,access} state;
- // typed enum type
  typedef enum {idle, setup,access} state;
  sate next,current;

enum {idle, setup, access} state;

// idle =0

// setup =1

// access =2

Encoded Enumerated Types:

You can define values one hot coding etc.

- enum {idle, setup, access} state;
- // idle=0

- // setup =1
- // access=2

- enum {idle=1, setup=2, access=4} state;

- enum bit [2:0] {idle,setup,access}state;
- // idle =3'b000;
- // setup= 3'b001;
- //access =3'b011;

- //typed enum type

typedef enum {

idle =3'b000,

setup=3'b001,

access=3'b011} state;

state next, current;

state <=onehot0_t'(3'b001);

Enumerated Type value Name sequence-

- User can use a range notation for simple value name sequences.
- User name can mixed and un-ranged values.

typedef enum {s[2]} state;

typedef enum { s0, s1} state; **//similar both**

typedef enum {K0, s[2], p0} state;

typedef enum {K0, s0,s1,p0} state; **//similar both**

Enumerated Type Initialization-

- The default initial value of an enum variable is the default initial value of its base type:

- For 2-state type (default int type -0)
- For 4-state type (default type X)

Design code-

```
module test;
enum bit [2:0] {BIT[4]} var_b1;
initial begin
    $display("value of %b", var_b1);
    $display("name %s",var_b1.name());
end

enum logic [2:0] {LOGVAL[4]} var_b2 ;
initial begin
    $display("value of %b", var_b2);
    $display("name %s ", var_b2.name());
end

enum logic [2:0] {LOGX=3'bxxx, LOG0=3'b000} var_b3 ;
initial begin
    $display("value of %b", var_b3);
    $display("name %s ", var_b3.name());
end
endmodule
```

Simulation Result

Output

```
value of 000
name BIT0
```

value of xxx

name

value of xxx

name LOGX

Enumerated Type Access Methods-

System Verilog methods enable iteration over enumeration values.

Method	Description
first()	Returns first value
last()	Returns last value
next(n)	Returns next Nth value from current
prev(n)	Returns previous Nth value from current
num()	Returns number of values
name()	Returns string equivalent of value

```
typedef enum {
   idle, start, done
} state_t;
state_t st, st = st.first();

forever begin
   $display("%s = %d", st.name(), st);
   if (st == st.last())
      break;
   st = st.next();
end
```

```
module enum_datatype;
    enum {red, green, blue, yellow, white, black} colors;
    initial begin
        colors = colors.first;
        for(int i = 0; i < 6; i++) begin
            $display("colors: value of %0s is = %0d",
                    colors.name(), colors);
            colors = colors.next();
        end
    end
endmodule
```

Simulation Result

Output

```
idle  = 0
start = 1
done  = 2
colors: value of red is = 0
colors: value of green is = 1
colors: value of blue is = 2
colors: value of yellow is = 3
colors: value of white is = 4
colors: value of black is = 5
```

Constant and Parameters-

Constants are those types of variables whose value **can not be changed at run time.**

In Verilog, parameter is given as constant types. To study about parameter, refer the Verilog notes and slides.

- Parameter is elaboration time constant which can be **overridden in elaboration time using "defparam" keyword.**
- Localparam is also elaboration time constant but it can not be changed throughout the simulation.
- Specparam is also elaboration time constant which can be overridden in elaboration time using SDF files.

In System Verilog, new type is given which is "**const**" along with parameter type.

"const" type is resolved at elaboration time same as parameter.

Global constant can have expression with hierarchical path as given in right side code.

Parameters-

```
module parameterized_module  # (parameter int DATA_WIDTH = 8) (
   input  logic [DATA_WIDTH-1:0] data_in,
   output logic [DATA_WIDTH-1:0] data_out
);
   assign data_out = data_in;
endmodule
```

Here this parameter can be overridden during instantiation from testbench like: -

```
parameterized_module #(.DATA_WIDTH(16)) mycall (
   .data_in(data_in_16),
   .data_out(data_out_16)
);
```

Constants-

```
module example;
   const int MAX_VALUE = 1024;
   initial begin
      $display("Max value: %0d", MAX_VALUE);
      $display("Threshold: %0d", MAX_VALUE / 2);
   end
```

endmodule

- Now the value of MAX_VALUE has been fixed.

Constant vs parameters-

Feature	Constant	Parameter
Scope	Local to the procedural block (e.g., `always`, `initial`, functions)	**Global** to the module or instance
Modifiability	Cannot be overridden (fixed in its block)	Can be **overridden** during instantiation
Use Case	Used for fixed values inside blocks or functions	Used for module-wide, configurable values
Runtime vs Compile-time	**Set at runtime (in the simulation)**	**Set at compile-time (before simulation**)
Synthesis	| Not typically used for synthesis or module configuration	Often used to define hardware characteristics like memory sizes
Example Use	**Constants within `always` / `initial` blocks**	**Width of buses, I/O sizes, depth, etc**.

Operators-

Operator in Verilog-

- Arithmetic, Bitwise, Logical, Reduction, Equality, Relational, Shift, Concatenation, Replication, Conditional

New operator in system Verilog-

- =, +=, -=,*=, /=, %=, >>=, <<=, &=, |=, ^=, ~^=, ~|=, ~^|=

Pre increment and post increment-

- ++i, i++

Pre decrement and post decrement-

- --i, i --

Wildcard case equality / inequality

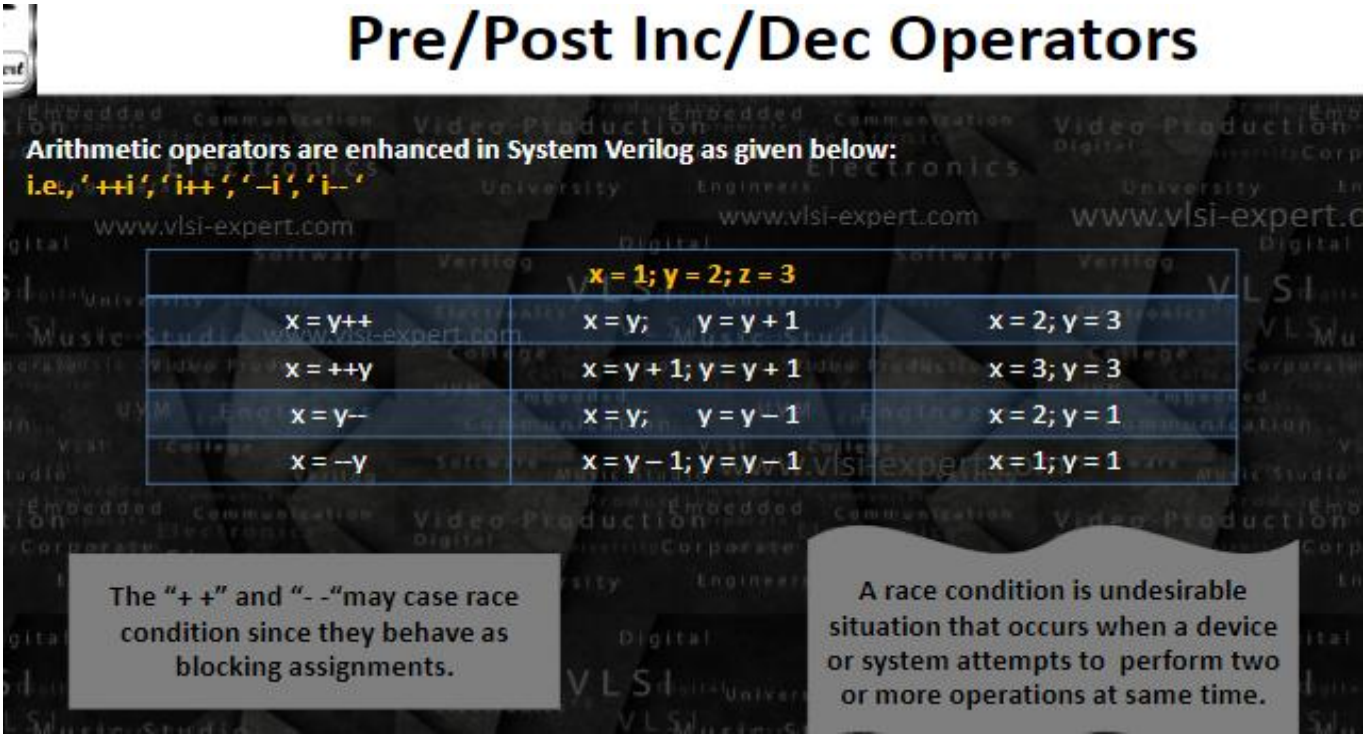
- ==?, !=?

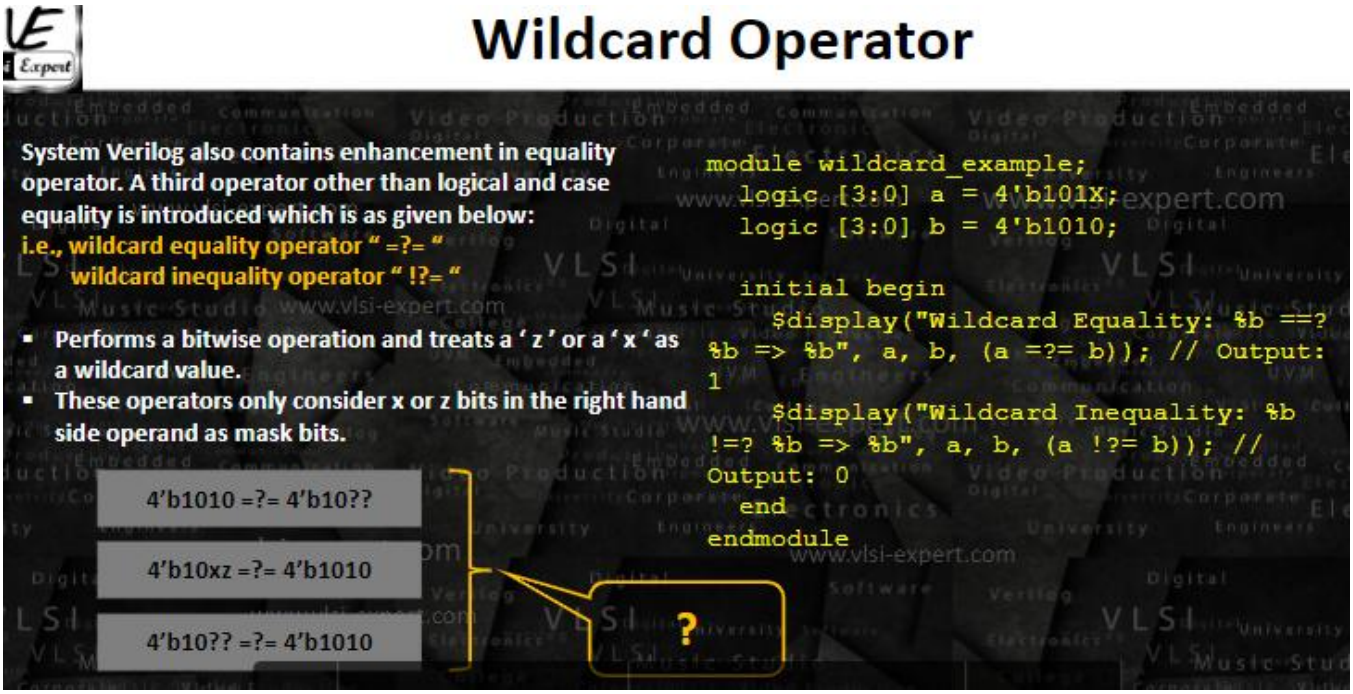
Inside operator

- a inside {b,c}

Assignment in Assignment operator-

Assignment operator is one which is used to assign the RHS to LHS.





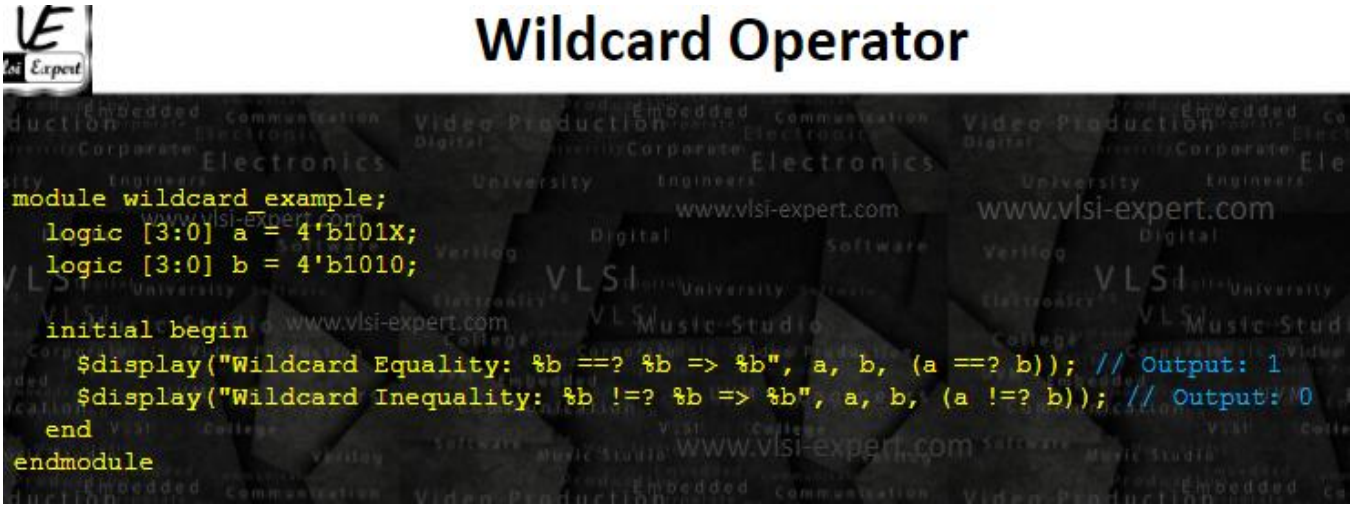
Wildcard Equality: 101x =?= 1010 => 1

Wildcard Inequality: 101x !=? 1010 => x

V C S S i m u l a t i o n R e p o r t

Explanation:

Expression	Meaning	Result	Why?
a ==? b	wildcard only in b	x	b has no X, so normal comparison; x vs 0 → x
a =?= b	wildcard allowed in both	1	x matches 0, everything matches
a !=? b	wildcard only in b	x	normal compare gives x



- **The above output is showing incorrect result.**

```
module wildcard_example;
  logic [3:0] a = 4'b101X;
  logic [3:0] b = 4'b1010;

  initial begin
    $display("Wildcard Equality: %b ==? %b => %b", a, b, (a ==? b)); //
    $display("Wildcard Inequality: %b !=? %b => %b", a, b, (a !=? b)); //
  end
endmodule
```

Simulation Output:

Wildcard Equality: 101x ==? 1010 => x

Wildcard Inequality: 101x !=? 1010 => x

V C S S i m u l a t i o n R e p o r t

Explanation

SystemVerilog provides wildcard equality ==? and wildcard inequality !=? operators.

Rule:

- Only X/Z on the RIGHT-HAND SIDE are treated as wildcards (don't care bits).
- X/Z on the LEFT-HAND SIDE are NOT wildcards—they behave like real unknown values.

Analyze the Code

We have:

a = 101X

b = 1010

Let's compare bit by bit:

Bit No	a	b	RHS Wildcard?	Comparison
3	1	1	No	Match
2	0	0	No	Match
1	1	1	No	Match
0	X	0	No	X compared with 0 = X (unknown)

FINAL SIMPLE SUMMARY

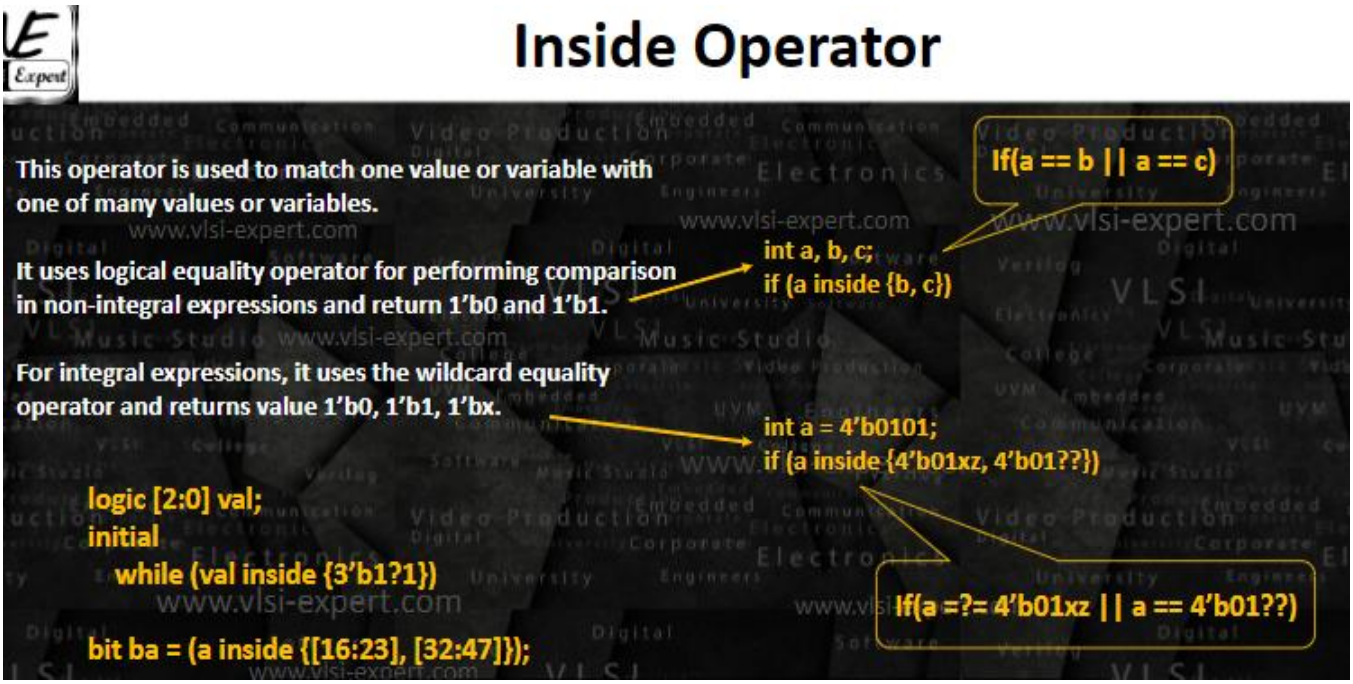
- `==?` = wildcard equality
- Only X/Z on RHS act as wildcards
- LHS X/Z are not wildcards

☑Wildcard Rule Reminder (Very Important)

👉 ONLY the RHS (`right-hand side`) X/Z act as wildcards

👉 LHS X/Z are treated as real unknown values

👉 RHS `?` = wildcard = can match 0/1/X/Z



- The inside operator checks whether a value belongs to a set or a range.

(value inside {list_or_range});

(a inside {1, 3, 5, 7})

(a inside {[10:20]}) → **Range**

If (a=7 || a=4 || a=6 || a=9 || a=11) can be represent in other form-

Simplest form of representation-

a inside {7,4,6,9,11}

.....

Enumeration

module enum_example;

```
// Define an enum type
typedef enum {RED, GREEN, BLUE} color_t;

color_t c;   // declare variable of enum type
```

```
  initial begin
    c = GREEN;  // assign enum value
    $display("Color = %0d", c);      // prints the numeric value
    $display("Color Name = %s", c.name()); // prints text name
  end
endmodule
```

Explanation in Simple Way

typedef enum {RED, GREEN, BLUE} color_t;

→ Creates an enumeration with 3 values: RED=0, GREEN=1, BLUE=2

→ color_t becomes a new datatype

- color_t c;

→ Variable `c` of enum type

- c = GREEN;

→ Assign enum value

- c.name()

→ Returns the string name: “GREEN”

Simulation Output

Color = 1

Color Name = GREEN

Enumeration example without using typedef:

Enum Example (No typedef)

```
module enum_example;
  enum {RED, GREEN, BLUE} color;   // enum variable declared directly

  initial begin
    color = BLUE;   // assign an enum value

    $display("Color = %0d", color);         // numeric value
    $display("Color Name = %s", color.name()); // string name
  end
endmodule
```

Simulation Output

```
Color = 2
Color Name = BLUE
```

inside operator

The inside operator checks whether a value belongs to a set or a range.

Think of it like:

> "Is this value present in this list or in this range?"

General Syntax

```
(value inside {list_or_range});
```

It returns:

1 (true) → if value matches any item in the set

0 (false) → if it does not match

X → if value contains X/Z

Example 1: Checking inside a list

```
int a = 5;

if (a inside {1, 3, 5, 7})
   $display("Matched");
else
   $display("Not Matched");
```

Output:

Matched

Because 5 is present in {1,3,5,7}.

Example 2: Checking inside a range

```
int a = 12;

if (a inside {[10:20]})
   $display("Inside Range");
else
   $display("Outside Range");
```

Output:

Inside Range

Because 12 is between 10 and 20.

Example 3: Multiple ranges

```
int a = 25;

if (a inside {[1:10], [20:30]})
   $display("Inside");
```

```
else
    $display("Outside");
```

Output:

Inside

Because 25 is in the range [20:30].

Example 4: Using inside with enums

```
typedef enum {IDLE, READ, WRITE, ERROR} state_t;
state_t s = READ;

if (s inside {READ, WRITE})
    $display("Valid Transfer State");
```

Output:

Valid Transfer State

Example 5: X or Z inside check

If the value contains X/Z → result becomes X.

```
logic [2:0] a = 3'b1x0;
$display("%b", a inside {3'b100, 3'b1x0});
```

✓ Output:

x

Because `a` contains an `x`.

Summary Table

Expression	Meaning
`a inside {1,2,3}`	True if a is 1 or 2 or 3
`a inside {[5:10]}`	True if 5 ≤ a ≤ 10
`a inside {1,[20:25],40}`	Matches list + range
X/Z inside set	Returns X

☑ Now Evaluate Each Expression

1.0 `4'b1010 ==? 4'b10??`

Simulation Output:

Wildcard Equality: 1010 ==? 10zz => 1

V C S S i m u l a t i o n R e p o r t

Time: 0

1010 ==? 10??

Compare bit-by-bit:

Bit	LHS	RHS	RHS wildcard?	Match?
3	1	1	No	✓
2	0	0	No	✓
1	1	?	Yes	✓ (don’t care)
0	0	?	Yes	✓ (don’t care)

👉 All bits match (because `??` on RHS ignore bits)

2.0 `4'b10xz ==? 4'b1010`

Simulation Output:

Wildcard Equality: 10xz ==? 1010 => x

V C S S i m u l a t i o n R e p o r t

Time: 0

10x z ==?1010

Compare bit-by-bit:

Bit	LHS	RHS	RHS wildcard?	Result
3	1	1	No	✓
2	0	0	No	✓
1	X	1	No	❌ → X vs 1 gives X (unknown)
0	Z	0	No	❌ → Z vs 0 gives X (unknown)

Wildcard equality result becomes X, and since RHS has no wildcards, X → 0.

3.0 `4'b10?? ==? 4'b1010`

Simulation Output:

Wildcard Equality: 10zz ==? 1010 => x

V C S S i m u l a t i o n R e p o r t

Time: 0

Here, RHS has no wildcard.

LHS has `??` but LHS ? is treated as X (not wildcard).

10?? ==?1010

Compare:

Bit	LHS	RHS	RHS wildcard?	Result

| 3 | 1 | 1 | No | ✓ |

| 2 | 0 | 0 | No | ✓ |

| 1 | ? (X) | 1 | No | X |

| 0 | ? (X) | 0 | No | X |

X comparisons → result becomes X.

Wildcards NOT allowed on RHS → Treat X as FALSE.

☑ FINAL ANSWER: RESULTS SUMMARY

Expression	Result
`4'b1010 ==? 4'b10??`	1 (TRUE)
`4'b10xz ==? 4'b1010`	0 (FALSE)
`4'b10?? ==? 4'b1010`	0 (FALSE)

- **Treats 'x' and 'z' on the RHS as don't cares:**

  If the right-hand operand contains 'x' or 'z' at a particular bit position, that bit position is ignored during the comparison. The value on the left-hand side at that position can be anything (0, 1, x, or z) and it will still be considered a match for that specific bit.
- **'x' and 'z' on the LHS are not wildcards:**

  If the left-hand operand contains 'x' or 'z', these values are treated as specific values and must match the corresponding value on the right-hand side (unless the right-hand side is 'x' or 'z' and thus acts as a wildcard).
- **Result can still be 'x':**

  If the left-hand operand contains an 'x' or 'z' at a bit position where the right-hand operand has a specific 0 or 1 value, the result of the wildcard equality operation can still be 'x' (unknown).

Wildcard operator

SystemVerilog provides two wildcard comparison operators:

✓ `==?` → wildcard equality

✓ `!=?` → wildcard inequality

They allow you to compare values while ignoring certain bits (don't care bits = X or Z).

⭐ Why used?

- To match patterns
- To ignore some bits
- To check only specific bits of a signal
- Used often in case statements and packet comparisons

⭐ Rule

In wildcard comparison:

* `?` means "don't care"

* If the bit in the compare value is X or Z, that bit is ignored

Example:

`4'b1X0Z`

Here X and Z bits are ignored.

⭐ Very Simple Example

```
module wildcard_example;
  logic [3:0] a;
  initial begin
    a = 4'b1010;

    if (a ==? 4'b1X1X)   // X means "ignore"
      $display("Matched!");
    else
```

```
    $display("Not Matched");
  end
endmodule
```

Output

Matched!

Example2 (Ignore LSBs)

```
logic [7:0] data = 8'hA5;   // 1010_0101

if (data ==? 8'b1010_XXXX)
  $display("Upper 4 bits match!");
```

`XXXX` means ignore lower 4 bits.

Upper bits = `1010` → match

LSB bits don’t matter.

Example 3 (wildcard inequality)

```
logic [3:0] val = 4'b1100;

if (val !=? 4'b11XX)
  $display("Not equal");
else
  $display("Equal ignoring last two bits");
```

`4'b11XX` → last two bits ignored

`1100` matches pattern → Equal

Output:

Equal ignoring last two bits

Wildcard Inequality: 1100 !=? 11xx => 0

V C S S i m u l a t i o n R e p o r t

Time: 0

Simple Summary (3 lines)

1. 0 `==?` and `!=?` allow comparing values while ignoring X/Z bits.

2. 0 X or Z in the pattern means “don’t care”.

3. 0 Useful for matching only important bits in a signal.

wildcard equality operator (==?) behaves.

☑ RULE 1:

RHS X/Z = Don’t Care

If the right-hand side contains `x` or `z`, **those bits are ignored during comparison**.

✓ Example 1 – RHS has don’t-cares

```
module ex1;
  logic [3:0] a;

  initial begin
    a = 4'b1011;
    if (a ==? 4'b1x0z)     // x,z on RHS → ignore these bits
      $display("MATCH (Rule 1)");
    else
      $display("NOT MATCH");
  end
endmodule
```

Simulation Output:

Wildcard Equality: 1011 ==? 1x0z => 0

V C S S i m u l a t i o n R e p o r t

Time: 0

So result = NOT MATCH.

Example2:

a = 4'b1001;

if (a ==? 4'b1x0z)

a = 1 0 0 1

RHS = 1 X 0 Z

- All specific bits match → MATCH.
- **Rule 1 proven:** RHS X/Z = ignored.

☑ RULE 2:

LHS X/Z are NOT wildcards

If the left-hand side has `x` or `z`, they are treated as real values, NOT don't-cares.

Example 2 – LHS has X, RHS has real value

```
module ex2;
  logic [3:0] a;

  initial begin
    a = 4'b10x1;          // LHS has X

    if (a ==? 4'b1011)    // RHS has no X/Z
```

```
    $display("MATCH");
  else
    $display("NOT MATCH (Rule 2)");
 end
endmodule
```

Therefore → NOT MATCH

✓ Rule 2 proven: LHS X/Z are “real unknowns,” not wildcards.

☑ RULE 3:

Result can still be X

If LHS has X or Z and RHS has a specific 0/1, comparison may produce X (unknown).

Example 3 – Comparison yields X

```
module ex3;
 logic [3:0] a;
 logic match;

 initial begin
  a = 4'b1x01;
  match = (a ==? 4'b1101);   // store the result
  $display("Comparison Result = %b", match);
 end
endmodule
```

Output:

Comparison Result = x

✓ Rule 3 proven: wildcard operator can still give X if LHS contains X/Z and RHS has specific bit.

FINAL SUMMARY

Rule	Meaning	Example Result
1. RHS X/Z are don’t-cares	RHS X/Z ignored	Match if other bits match
2. LHS X/Z are NOT wildcards	LHS X must match RHS	Often leads to X or mismatch
3. Result can be X	Unknown bits on LHS compared with known RHS → X	match result becomes X

a = 4'b1010; b = 4'b10??; (a =?= b)

Simulation Result:

Wildcard Equality: 1010 =?= 10zz => 1

V C S S i m u l a t i o n R e p o r t

Time: 0

a = 4'b10xz; b = 4'b1010; (a =?= b)

Simulation Result:

Wildcard Equality: 10xz =?= 1010 => 1

V C S S i m u l a t i o n R e p o r t

Time: 0

a = 4'b10xz; b = 4'b1010; (a =?= b)

Simulation Result:

Wildcard Equality: 10zz =?= 1010 => 1

V C S S i m u l a t i o n R e p o r t

Time: 0

inside operator examples used in constraints

1. Basic `inside` in constraint

```
class pkt;
  rand int size;

  constraint c1 {
    size inside {4, 8, 16};
  }
endclass
```

Meaning:

`size` can take only one of these values: 4, 8, 16

2. Range inside constraint

```
class pkt;
  rand int addr;

  constraint c1 {
    addr inside {[100:200]};
  }
endclass
```

Meaning:

`addr` can be any number from 100 to 200.

3. Multiple ranges + values

```
class pkt;
  rand int opcode;
```

```
  constraint c1 {
    opcode inside {0, 1, [10:15], 20};
  }
endclass
```

Possible outputs:

`0, 1, 10, 11, 12, 13, 14, 15, 20`

4. Inside with enum

```
typedef enum {IDLE, READ, WRITE, ERROR} state_t;

class ctrl;
  rand state_t s;

  constraint c1 {
    s inside {READ, WRITE};
  }
endclass
```

Meaning:

`State` will be only READ or WRITE.

5. Inside with arrays

```
class abc;
  rand int x;
  int allowed[3] = '{5, 10, 20};
```

```
  constraint c1 {
    x inside {allowed};
  }
endclass
```

Meaning:

`x` can take 5 or 10 or 20 (elements of the array).

6. `inside` + `unique`

```
class my_class;
  rand int a, b;

  constraint c1 {
    a inside {[1:5]};
    b inside {[1:5]};
    unique {a, b};  // both must be different
  }
endclass
```

Meaning:

Both must be between 1–5 but not equal.

7. Using `inside` to avoid illegal values

```
constraint c1 {
  !(mode inside {3, 7, 9}); // illegal values
}
```

Meaning:

`mode` should not be 3, 7, 9.

Final Summary (1 line)

The `inside` operator restricts random variables to take values only from a specific list or range.

Session-4 Loops in SV: -

begin..end, for, while, do-while, foreach, forever, repeat loop, break and continue, iff event control, always, always_ff, always_comb, always_latch,

The loop is useful to read/ update an array content, execute a few statements multiple times based on a certain condition.

Named end and join: -

- System Verilog allows a matching name to be specified with the block end.
- Allowed for the following code blocks:

1. module...endmodule
2. begin…end
3. task…endtask
4. function…endfunction
5. fork…join (all)
6. primitive…endprimitive
7. interface…endinterface

In System Verilog, you can **give a name** to a `fork...join` block (or even `begin...end` block). That name helps you:

- **Make the block easier to read**,

- Control or disable the block from outside,
- **Avoid confusion when multiple blocks exist.**

Basic Syntax

fork: **block_name**

  // parallel statements

join

or for sequential blocks:

begin: **block_name**

  // sequential statements

end

- Here, `block_name` is just a **label** for the block.

Example 1 — Named `begin...end` Block

module named_begin_example;

initial begin : **start_block**

  int a = 5;

  int b = 10;

  $display("Sum = %0d", a + b);

end

endmodule

Output:

Sum = 15

Example 2 — Named `fork...join`

module named_fork_example;

initial begin

```
    fork: parallel_block
        $display("Task 1 started at time %0t", $time);
        #5 $display("Task 2 started at time %0t", $time);
        #10 $display("Task 3 started at time %0t", $time);
    join
    $display("All tasks in fork...join completed at time %0t", $time);
end
endmodule
```

Output:

Task 1 started at time 0

Task 2 started at time 5

Task 3 started at time 10

All tasks in fork...join completed at time 10

Example 3 — Disabling a Named Block

We can stop (disable) a named block using the **`disable`** keyword.

```
module disable_example;
initial begin
    fork: my_block
        begin
            #5 $display ("Task A running...");
            #5 $display ("Task A completed.");
        end

        begin
            #3 $display ("Disabling the block now!");
            disable my_block; // ❌ stops all tasks in this fork
        end
```

```
    join
    $display ("End of simulation");
end
endmodule
```

Output:

Disabling the block now!

End of simulation

Summary

Concept	**Keyword**	**Meaning**
Named block	`begin: name ... end` or `fork: name ... join`	Gives a label to a block
Purpose	To make code readable, allow control or disable	
Disable	`disable block_name;`	Stops all activity in that block immediately

for loop enhancements

- System Verilog provides two for loop enhancements.
- We can declare loop variable within the for statement

Verilog:

// loop variable declared outside the loop

```
int i;
for (i=0; i<10; i=i+1)
```

System Verilog:

// loop variable declared inside the loop

```
for (int i=0; i<10; i=i+1/i++)
```

- We can use multiple initial and step assignments:
- For (int i=0, int j=0; i*j < 250; i=i+1, j=j+1)
- First loop- i=0, j=0

- Second loop- i=1, j=1

A for loop is used to **repeat a block** of statements a **fixed number of times**.

It has **three** parts inside parentheses:

- for (initialization; condition; update)
- Initialization → sets the starting value
- Condition → loop runs while this is true
- Update → changes the variable after each iteration

Basic Syntax

```
for (int i = 0; i < 5; i++) begin
   // statements to repeat
end
```

This means:

- Start with `i = 0`
- Run the loop while `i < 5`
- After each iteration, increase `i` by 1

Example 1 — Simple Loop

```
module for_loop_example;
   initial begin
      for (int i = 0; i < 5; i++) begin
         $display ("Value of i = %0d", i);
      end
   end
endmodule
```

Output:

```
Value of i = 0
Value of i = 1
Value of i = 2
```

Value of i = 3

Value of i = 4

Example 2 — Using `for` to Initialize an Array

```
module for_array_example;
   int arr[5];

   initial begin
      // Assign values to array using for loop
      for (int i = 0; i < 5; i++) begin
         arr[i] = i * 2;
      end

      // Display array elements
      for (int i = 0; i < 5; i++) begin
         $display("arr[%0d] = %0d", i, arr[i]);
      end
   end
endmodule
```

Output:

arr[0] = 0

arr[1] = 2

arr[2] = 4

arr[3] = 6

arr[4] = 8

Example 3 — Nested `for` Loop

We can place one `for` loop inside another (useful for matrices).

```
module nested_for_example; // Fill 3x3 matrix
    int matrix[3][3];


    initial begin
        for (int i = 0; i < 3; i++) begin
            for (int j = 0; j < 3; j++) begin
                matrix[i][j] = i + j;
                $display("matrix[%0d][%0d] = %0d", i, j, matrix[i][j]);
            end
        end
    end
endmodule
```

Output:

```
matrix[0][0] = 0
matrix[0][1] = 1
matrix[0][2] = 2
matrix[1][0] = 1
matrix[1][1] = 2
matrix[1][2] = 3
matrix[2][0] = 2
matrix[2][1] = 3
matrix[2][2] = 4
```

Summary

Part	**Description**	**Example**
Initialization	Start variable	`int i = 0`

| **Condition** | When to stop | `i < 5` |

| **Update** | Step change | `i++` |

while loop

A while loop is a control flow statement that executes statements repeatedly if the condition holds true else loop terminates.

Syntax:

while(<condition>) begin

...

end

Design Code

```
module while_example;

int count;


initial begin

while(count<10) begin

    $display("Value of count = %0d", count);

    count++;

end

end

endmodule
```

Simulation Output:

```
Value of count = 0

Value of count = 1

Value of count = 2

Value of count = 3

Value of count = 4
```

Value of count = 5

Value of count = 6

Value of count = 7

Value of count = 8

Value of count = 9

do…while loop

A do while loop is a control flow statement that executes statements at least once and then the condition is checked. If the condition holds true, statements execute repeatedly else the loop terminates.

Syntax:

do begin

...// statements

end

while(<condition>);

Design Code

```
module do_while_example;
int count;
initial begin
    do begin
        $display("Value of count = %0d", count);
        count++;
    end
while(count<10);
end
endmodule
```

Simulation Output:

Value of count = 0

Value of count = 1

Value of count = 2

Value of count = 3

Value of count = 4

Value of count = 5

Value of count = 6

Value of count = 7

Value of count = 8

Value of count = 9

- Statements block executes at least once.
- Condition is checked after statements execute.
- Makes certain loop functions easier to create.

```
do
    count=count+1;
while (enable);
```

- If enable false on loop entry, count incremented.

```
while (enable)
            count=count +1;
```

- If enable false on loop entry, count not incremented.

A do…while loop is similar to a `while` loop,

but the condition is checked at the end** —

so the loop runs at least once, no matter what.

Example 2 --Condition False Initially

```
module do_while_once_example;
    int i = 10;

    initial begin
        do begin
            $display("This will print even if condition is false");
        end while (i < 5);
    end
endmodule
```

Output:

This will print even if condition is false

Example 3 — Using `do…while` for Countdown

```
module countdown_example;
    int count = 3;

    initial begin
        do begin
            $display("Countdown: %0d", count);
            count--;
        end while (count > 0);
        $display("Blast Off!");
    end
endmodule
```

Output:

Countdown: 3

Countdown: 2

Countdown: 1

Blast Off!

Summary

Feature	**Description**
Loop type	Post-test loop (condition checked at end)
Runs at least once?	☑ Yes
Syntax	`do begin … end while (condition);`
Use case	When you want the body to run before checking the condition

foreach loop

- It iterates over the element of an array.
- Loop variables characteristics:
- Does not have to be declared.
- Read only
- Only visible inside the loop
- Use multiple loop variables for multidimensions arrays
- Equivalent to nested loops
- Useful for initializing and array processing.

The foreach loop in System Verilog iterates over the array element. Unlike for loop, foreach loop **does not require initialization, condition, or update value.**

Syntax:

foreach (variable[iterator]) begin

...

end

➢

int intarr [7:0];

…….

```
foreach (intarr[i])
intarr [i]= 7-i;
```

-

```
for (int i=7; i>=0; i=i-1)
intarr [i]= 7-i;
```

-

```
int arr2d [7:0] [2:0];
foreach (arr2d [k,1])
arr2d [k] [1] =k*1;
```

Design Code:

//foreach loop in the single dimensions (1D) array

```
module foreach_example;
int array[5] = '{100, 200, 300, 400, 500};

initial begin
    foreach(array[i])begin
        $display("array[%0d] = %0d", i, array[i]);
    end
end
endmodule
```

Simulation Output:

```
array[0] = 100
array[1] = 200
array[2] = 300
array[3] = 400
array[4] = 500
```

The loop automatically goes through all indices of the array.

Design Code:

```
// foreach loop in two-dimensional (2D) array
module foreach_example;
int array[6][2] = '{'{1,100},'{2,200},'{3,300},'{4,400},'{5,500},'{6,600}};

initial begin
    foreach (array[i,j]) begin
        $display("array[%0d][%0d] = %0d", i,j, array[i][j]);
    end
end
endmodule
```

Simulation Output:

```
array[0][0] = 1
array[0][1] = 100
array[1][0] = 2
array[1][1] = 200
array[2][0] = 3
array[2][1] = 300
array[3][0] = 4
array[3][1] = 400
array[4][0] = 5
array[4][1] = 500
array[5][0] = 6
array[5][1] = 600
```

- foreach (array[i, j]) automatically loops through both dimensions (`i` and `j`).
- You don’t need to write nested loops.

The `foreach` loop is a special loop in System Verilog used to iterate over arrays (packed, unpacked, associative, or dynamic).

- It automatically handles array indices, so you don’t have to write `for (i = 0; i < size; i++)` manually.

Syntax

```
foreach (array_name[index])
   // statements
```

For multidimensional arrays:

```
foreach (array_name[i, j])
   // statements
```

Example 3 — Associative Array

```
module foreach_example3;
   int marks[string] = '{"Ram": 85, "Shyam": 90, "Ravi": 78};

   initial begin
      foreach (marks[name]) begin
         $display("%s scored %0d", name, marks[name]);
      end
   end
endmodule
```

Output:

```
Ram scored 85
Shyam scored 90
```

Ravi scored 78

Summary Table

Feature	**Description**
Purpose	To iterate through array elements easily
Works with	Fixed-size, dynamic, associative, and multidimensional arrays
Advantage	No need to know array size; auto index handling
Syntax	`foreach (array[index])` or `foreach (array[i, j])`

forever loop

As the name suggests, a **forever loop runs indefinitely**. To terminate the loop, a break statement can be used.

Syntax:

```
forever begin
...
end
```

Example with $finish

```
module forever_example;
  int count;
  initial begin
    forever begin
      $display("Value of count = %0d", count);
      count++;
      #5;
    end
```

```
  end


  initial begin
    #30;
    $finish;
  end
endmodule
```

Output:

```
Value of count = 0
Value of count = 1
Value of count = 2
Value of count = 3
Value of count = 4
Value of count = 5
```

- $finish called from file "testbench.sv".
- $finish at simulation time 30

Example with a break statement

The break statement is used in the below example to terminate the loop based on the condition. The break statement will be discussed under the **break** and **continue** section.

```
module forever_example;
  int count;
  initial begin
    forever begin
      $display("Value of count = %0d", count);
      count++;
      if(count == 10) break;
    end
  end
```

endmodule

Output:

Value of count = 0

Value of count = 1

Value of count = 2

Value of count = 3

Value of count = 4

Value of count = 5

Value of count = 6

Value of count = 7

Value of count = 8

Value of count = 9

Difference between always and forever block

Both always and forever block the same effect. The always block is a procedural block and it cannot be placed inside other procedural blocks. Also, always block can not be used inside the class.

Syntax for always block:

// For multiple statements in always block

always begin

...

end

// For single statement

always <single statement>

Examples of always block

module always_example;

```
  int count;
  always begin
    $display("Value of count = %0d", count);
    count++;
    #5;
  end

  initial begin
    #30;
    $finish;
  end
endmodule
```

output

```
Value of count = 0
Value of count = 1
Value of count = 2
Value of count = 3
Value of count = 4
Value of count = 5
$finish at simulation time 30
```

A always block inside the another procedural block

A compilation error is expected when always block is used inside another procedural block. In such a case, a forever block can be used.

```
module always_example;
  int count;
  initial begin
```

```
   always begin // can not use inside other procedural block
     $display("Value of count = %0d", count);
     count++;
     #5;
   end
  end

  initial begin
    #30;
    $finish;
  end
endmodule
```

output

output will show the error -

always begin **// can not use inside other procedural block.**

A always block inside the class

A compilation **error is expected** when always block is used **inside a class.**

```
class transaction;
  int count;

  task inc_cnt();
    always begin
      $display("Value of count = %0d", count);
      count++;
      #5;
```

```
    end
  endtask
endclass

module always_example;
  transaction tr;
  initial begin
    tr = new();
    tr.inc_cnt();
  end

  initial begin
    #100;
    $finish;
  end
endmodule
```

output:

It will show the **error** due to **always** construct is illegal.

forever loop inside a class

Now, **replace** always block with forever block.

```
class transaction;
  int count;

  task inc_cnt();
    forever begin
```

```
      $display("Value of count = %0d", count);
      count++;
      #5;
    end
  endtask
endclass

module always_example;
  transaction tr;
  initial begin
    tr = new();
    tr.inc_cnt();
  end

  initial begin
    #100;
    $finish;
  end
endmodule
```

output:

```
Value of count = 0
Value of count = 1
Value of count = 2
Value of count = 3
Value of count = 4
Value of count = 5
Value of count = 6
```

Value of count = 7

Value of count = 8

Value of count = 9

Value of count = 10

Value of count = 11

Value of count = 12

Value of count = 13

Value of count = 14

Value of count = 15

Value of count = 16

Value of count = 17

Value of count = 18

Value of count = 19

forever loop

The forever loop runs continuously without stopping —

it repeats the statements infinite times until the simulation is stopped manually (like using `$finish` or control signals).

- It's often used in testbenches to generate continuous clock signals or repetitive behavior.

Syntax

```
forever begin
   // statements
end
```

or single-line form:

forever statement;

Example 1 — Basic Forever Loop

```
module forever_example1;
    int i = 0;

    initial begin
        forever begin
            $display("Iteration: %0d", i);
            i++;
            #5; // wait 5 time units
            if (i == 5) $finish; // stop simulation after 5 iterations
        end
    end
endmodule
```

Output:

Iteration: 0

Iteration: 1

Iteration: 2

Iteration: 3

Iteration: 4

Example 2 — Clock Generation (Most Common Use)

```
module clock_gen;
    logic clk = 0;

    initial begin
```

```
    forever #5 clk = ~clk;  // toggle clock every 5 time units
  end

  initial begin
    #50 $finish; // stop simulation after 50-time units
  end
endmodule
```

Output:

```
Time=0  clk=0
Time=5  clk=1
Time=10 clk=0
Time=15 clk=1
```

Example 3 — Using `disable` to Stop a Forever Loop

```
module forever_disable;
  initial begin: loop_block
    forever begin
      $display ("Running...");
      #10;
    end
  end

  initial begin
    #35 disable loop_block; // stops the forever loop
    $display("Loop stopped!");
    #5 $finish;
  end
```

endmodule

Output:

Running...

Running...

Running...

Loop stopped!

Summary Table

Feature	**Description**
Purpose	To repeat a block of code infinitely
Stops when	`$finish`, `disable`, or control signal used
Common use	Clock generation, stimulus generation
Syntax	`forever begin ... end` or `forever statement;

repeat loop

A repeat loop is used to execute statements a given **number of times** (**fixed number of times**).

Syntax:

```
repeat(<number>) begin   // <number> can be variable or fixed value
  ...
end
```

repeat loop example

```
module repeat_example;
  int array[5] = '{100, 200, 300, 400, 500};
```

```
  int i;
  initial begin
    repeat ($size(array)) begin
      $display("array[%0d] = %0d", i, array[i]);
      i++;
    end

    repeat(3)
      $display("VLSI Verify");
  end
endmodule
```

output

```
array[0] = 100
array[1] = 200
array[2] = 300
array[3] = 400
array[4] = 500
VLSI Verify
VLSI Verify
VLSI Verify
```

repeat loop

The **repeat** loop is used to run a block of code **a fixed number of times**.

Unlike `forever` (which **runs infinitely**), `repeat` stops **automatically after the given count.**

Syntax

```
repeat (number_of_times) begin
```

```
    // statements
end
```

- The number inside parentheses must be an integer or an expression that evaluates to a number.

Example 1 — Basic Repeat Loop

```
module repeat_example1;
    initial begin
        repeat (5) begin
            $display("Hello SystemVerilog!");
        end
    end
endmodule
```

Output:

```
Hello SystemVerilog!
Hello SystemVerilog!
Hello SystemVerilog!
Hello SystemVerilog!
Hello SystemVerilog!
```

Example 2 — Repeat with Delay

```
module repeat_example2;
    int i = 0;

    initial begin
        repeat (3) begin
            #10;  // wait 10 time units
```

```
            $display("Time = %0t, i = %0d", $time, i);
            i++;
        end
    end
endmodule
```

Output:

```
Time = 10, i = 0
Time = 20, i = 1
Time = 30, i = 2
```

Example 3 — Repeat for Clock Generation

```
module repeat_example3;
    logic clk = 0;

    initial begin
        repeat (6) begin
            #5 clk = ~clk;  // toggle every 5 time units
            $display("Time=%0t, clk=%0b", $time, clk);
        end
    end
endmodule
```

Output:

```
Time=5, clk=1
Time=10, clk=0
Time=15, clk=1
Time=20, clk=0
```

Time=25, clk=1

Time=30, clk=0

Summary Table

Feature	**Description**
Purpose	To repeat a block a fixed number of times
Syntax	repeat (N) begin ... end
Stops automatically?	☑ **Yes, after N iterations**
Common uses	Fixed number of delays, loops, or signal toggles

break and continue statements

The break and continue keywords are used to **control the loop flow**. Both break and continue keywords can be used in all supported loops (while, do while, forever, for, foreach, repeat).

break keyword

The break keyword is used to **terminate the loop** prematurely. Generally, based on certain conditions the loop is terminated.

Continue keyword

The continue keyword is used to **jump the next iteration immediately without executing** statements after the continue keyword.

break and continue example

When a break keyword is used, the loop **immediately is terminated** after printing array elements having an index from 0 to 5.

When a continue keyword is used, the loop is **skipped without printing** the array element having an index 6.

module break_continue_example;

```
  int array[10];

  initial begin
    // Update array
    for (int i = 0; i < $size(array); i++) begin
      array[i] = i*i;
    end

    // Break keyword
    for (int i = 0; i < $size(array); i++) begin
      if(i == 6) break;
      $display("array[%0d] = %0d", i, array[i]);
    end
    $display("------------------------");

    // Contimue keyword
    for (int i = 0; i < $size(array); i++) begin
      if(i == 6) continue;
      $display("array[%0d] = %0d", i, array[i]);
    end
    $display("------------------------");
  end
endmodule
```

output

array[0] = 0

array[1] = 1

array[2] = 4

array[3] = 9

array[4] = 16

array[5] = 25

---

array[0] = 0

array[1] = 1

array[2] = 4

array[3] = 9

array[4] = 16

array[5] = 25

array[7] = 49

array[8] = 64

array[9] = 81

Both are loop control statements that help you control how a loop behaves.

Statement	**Meaning**
break	Immediately exits the loop (stops the loop completely).
continue	Skips the remaining code in the current iteration and goes to the next iteration.

Syntax

break; // **exit loop**

continue; // **skip to next iteration**

These can be used inside any looping construct:

- for, while, do...while, foreach, repeat

Example 1 — Using `break`

```
module break_example;
   initial begin
      int i;
      for (i = 0; i < 10; i++) begin
         if (i == 5) begin
            $display("Breaking at i = %0d", i);
            break;  // exit loop
         end
         $display("i = %0d", i);
      end
      $display("Loop ended");
   end
endmodule
```

Output:

```
i = 0
i = 1
i = 2
i = 3
i = 4
Breaking at i = 5
```

Loop ended

Example 2 — Using `continue`

```
module continue_example;
    initial begin
        int i;
        for (i = 0; i < 6; i++) begin
            if (i == 3)
                continue;  // skip this iteration
            $display("i = %0d", i);
        end
    end
endmodule
```

Output:

```
i = 0
i = 1
i = 2
i = 4
i = 5
```

Example 3 — Using Both Together

```
module break_continue_example;
    initial begin
        int i;
        for (i = 0; i < 10; i++) begin
            if (i == 3)
```

```
            continue;  // skip 3
        if (i == 7)
            break;     // stop completely
        $display("i = %0d", i);
      end
   end
endmodule
```

Output:

```
i = 0
i = 1
i = 2
i = 4
i = 5
i = 6
```

Summary Table

Keyword	**Meaning**	**Effect**
break	**Exits loop** immediately	Stops loop completely
continue	**Skips rest of current** iteration	Goes to next iteration
Works in	`for`, `while`, `repeat`, `foreach`	All loop types
Common Use	To stop loop early or skip certain values	Control flow in testbench

- SV adds the break and continue keywords for loop constructs.

break

- Jumps to end of loop
- Usually under conditionally control

Continue

- Jumps to a next iteration for the loop
- Usually under conditionally control

Examples

for (int i=0; i <=7; i=i+1) **begin**

data (data[6:0],data[7]);

if (data[7]) **break;**

end

for (int i=0; i<=7; i=i+1) begin

if (data[i]) continue:

count = count+1;

end

iff Event Control

The iff keyword qualifies a **procedural event control.**

The event expressions triggers only if the **iff condition is true.**

Examples

Always @(a iff enable==1)

y <=a;

always @(a)

if (enable)

y <=a;

- block executes when a changes and enable is 1
- not when enables changes

iff in Event Control

- iff` stands for "if and only if".
- It is used with event controls (like `@`) to conditionally trigger an event.
- The event **happens only if the condition after `iff` is true** at that time.

Syntax:

@(event_expression iff condition)

- The statement inside the `@()` block executes only when:

1. The event occurs, and

2. The condition is true.

Simple Example:

```
module iff_example;
  logic clk, enable;

  // Generate clock
  initial begin
    clk = 0;
    forever #5 clk = ~clk;
  end

  // Toggle enable signal
  initial begin
    enable = 0;
    #12 enable = 1; // enable becomes 1 after 12-time units
    #30 enable = 0; // disable again
  end
```

```
  // Event control using iff
  always @(posedge clk iff enable)
    $display("Clock edge detected at time=%0t when enable=1", $time);
endmodule
```

Output:

Clock edge detected at time=15 when enable=1

Clock edge detected at time=25 when enable=1

Clock edge detected at time=35 when enable=1

Key Points:

- `iff` is used only with event controls (`@`, `posedge`, `negedge`).
- It is not the same as a normal `if` condition.
- Helps in conditional triggering without writing extra `if` statements inside blocks.

Procedural block enhancement

- An **always** block can be **synthesized** to the combinational circuits, latches, or clocked circuits.
- The always block sometimes lead to **ambiguity errors** because of assigning single variable within multiple always block.
- Ambiguity leads to **wrong synthesis results**. Such ambiguity can cause problems in your design & Designer error can lead to **incorrect logic.**
- Ambiguity error is prevented by special procedural blocks. Ex-
- **always_comb, always_latch, always_ff**
- Because single varible can not be assigned by multiple special procedural blocks.

Register

```
always @( posedge clk or posedge rst)
if (rst==1)
op1 <=8'h00;
```

else

op1 <= data_in

Latch

Always @(sel or a)

If (sel)

Op <=a;

Combinational circuits

Always @(sel or a or b)

If (sel)

Op3 <=a;

else

op3 <=b;

Latch inferred?

How the latch is inferred in Design?

- Latch is 1-bit storage element. There is no problem with this.
- But this is **non-synchronized circuit with the system clock** (main clock of the chip) which is very extraordinary to handle it in the design at run time.
- Because of non-synchronization of the feature, we need to keep an eye on the design (difficult) while creating combinational circuits.
- While creating the combinational circuits from “always” block, if output is undefined for any of inputs in this design, then latch will automatically be inferred in the design.
- The dangerous thing is, we do not get even warning about it, **neither in simulator nor in the synthesis.**
- This is why **always_comb** is introduced in SV to avoid such challenges.

Latch

always @(sel or a)

If (sel)

Op2 <=a;

Combinational Circuit

always (a or b or sel)

if (sel==1)

op3 =a;

else

op3=b;

- Latch can be inferred in alwaus_comb also but we get warning here in synthesis.

Three ways to look while removing latch infer issue:

- In the if/else ladder, latch is inferred if else is missing.

Latch

always @(sel or a)

if (sel)

op <=a;

- In the case statement, latch is inferred if expression is not matching with any condition. To remove inferred latch, default condition is recommended.

Latch

always @(sel or a)

case (sel)

0: op2 <=a;

endcase

- In the sensitivity list, if any input is not available then latch is inferred.

Latch

always @(sel or a)

```
case (sel)
0: op2 <=a;
1: op2 <=b;
default: op2 <=a;
endcase
```

- **Sometimes,** if expression has 2-bits variables, which means there will be 4 possible combinations to match with the conditions inside it.
- If all 4 conditions are present but default statement is not given.

```
module mux2x1 (sel,op2,a,b,c,d);
input [1:0] sel;
input a,b,c,d;
output op2;
always @(sel or a or b or c or d)
case (sel)
0: op2 <=a;
1: op2 <=b;
2: op2 <=c;
3: op2 <=d;
endcase
endmodule
```

Output:

- output op2 is not declared as reg;
- incorrect module name, because of 4-input, this is actually 4x1 mux.
- This is a combinational circuit, so we should use blocking assignments.
- Choose select 2'b00, 2'b01, 2'b10,2'b11
- Output will be depended on the selection line and input value.

always_comb

SV adds a specialized procedural block for modelling combinational logic:

```
always_comb
if (sel ==1)
op =a;
else
op =b;
```

- Variable assigned in an always_comb can not be assigned by another procedure.
- Can not contain further blocking timing or event control.
- Procedure first executions at time 0 without waiting for trigger
- After all initial and always blocks have executed.
- Ensures outputs are consist with inputs.
- Tools may issue warning if the block does not infer combinational logic.

```
always_comb
if (sel==1)
op =a;
else
op =b;
```

X // error

```
//comb1
always_comb
if (sel==1)
op =a;
else
op =b;
//comb2
always_comb
if (en==1)
op =q;
```

```
else
op =p;
// we can not use two combinational block
```

always vs always_comb

```
module control(clock,resetn);
input clock,resetn;
enum {wait, load,store}state, next_state;

always @(posedge clock or negedge resetn)
if (!resetn)
state <= wait;
else
state <= next_state;

always @(state)
case (state)
wait: next_state=load;
load:next_state=store;
store:next_state=wait;
endcase
endmodule
```

Solution:

- At the simulation time 0, both state,next_state variable by default points to first label of enum, WAIT
- Since both variables have the same state value, there is no change when a posedge clk or negedege resetn occurs.

- Because the next_state value is same as the state value.
- As a result, always@(state) does not get triggered and next_state variable does not change.

```
module control(clock,resetn);

input clock,resetn;

enum {wait, load,store}state, next_state;


always @(posedge clock or negedge resetn)

if (!resetn)

state <= wait;

else

state <= next_state;


always_comb

case (state)

wait: next_state=load;

load:next_state=store;

store:next_state=wait;

endcase

endmodule
```

Solution

- always_comb will automatically the infer sensitivity list.
- Since always_comb block executes once at simulation time 0 after all the procedural blocks have been activated.
- This means that at time 0, next_state variable will be updated to value LOAD to reflect the value of state variable.
- Now, when the posedge clk or negedge resetn occurs,state will change to next_state value (which is different from state variable).

- As a result, always_comb will again get triggered and next_state variable will get changed again.
- The locking problem that we saw stands resolved with always_comb block.

always vs always_comb

always_comb	always
It automatically triggers at least once at 0 simulation time	It waits for change to the signals present in sensitivity list.
It does not require sensitivity list.	It requires sensitivity list.
As per given code, it is sensitive to the change within the content of a called function “decode”.	Whereas, it is sensitive only to change to its arguments in the sensitivity list.i.e data
As per given code, variables on the LHS of assignment within always_comb includes the variables from the contents of a called function, shall not be written by any other processes.	Whereas, it permits the multiple processes to write to the same varible.
It shall not include those statements that block or having blocking timing or event control or fork_join.	Whereas, it includes them all.

Code:

```
always @(*) begin
a1 =data <<1;
b1=decode();
end
always_comb begin
a2 = data <<1;
b2 =decode();
end
function decode();
case(sel)
2’b01: decode= d | e;
```

2’b10: decode=d & e;

default: decode=c;

endcase

endmodule

- Procedural blocks are used to describe sequential behavior in System Verilog — code that executes step by step.
- **initial**
- **always**

System Verilog enhances these blocks from Verilog to make them clearer and safer for modelling hardware.

Types of Procedural Blocks (Enhancements)

1. always_comb

- Used for combinational logic.
- Automatically adds all right-hand-side signals to the sensitivity list.
- Prevents latches by enforcing full assignment.

Example:

1.0 always_comb begin

y = a & b; // **combinational logic**

end

2. always_ff`

Used for sequential logic (flip-flops).

Ensures that only one clock or reset controls the block.

Example:

always_ff @(posedge clk or negedge rst_n) begin

if (!rst_n)

```
    q <= 0;
  else
    q <= d;
end
```

3. always_latch

Used to model latches.

Compiler checks that output changes only when enable is active.

Example:

```
always_latch begin
  if (en)
    q <= d;
end
```

Why These Enhancements?

- Clear intent (combinational, sequential, or latch).
- Reduces simulation/synthesis mismatches.
- Easier to debug and verify.

In short:

Block Type	**Purpose**	**Example Trigger**
always_comb	Combinational	@(*)
always_ff	Flip-flop (Seq.)	@(posedge clk)
always_latch	Latch behavior	Conditional

Syntax:

```
always @(sensitivity_list)
begin
```

// code

end

- You manually specify what signals trigger the block.
- If you forget a signal in the sensitivity list → simulation mismatch or synthesis issues may occur.

Example (traditional Verilog):

```
module mux_always(input logic a, b, sel, output logic y);
  always @(a or b or sel) begin
    if (sel)
      y = b;
    else
      y = a;
  end
endmodule
```

Problem:

If you forget to include one signal (e.g., `sel`) in the sensitivity list,

the output `y` may not update correctly in simulation — leading to bugs.

Syntax:

always_comb begin

// code

end

- Automatically includes all right-hand-side signals in the sensitivity list.
- Used only for combinational logic.
- The compiler checks for:
- Missing assignments (to avoid latch inference)
- Use of illegal timing controls (like `@` or `#`)
- Multiple drivers on the same signal

Example:

```
module mux_always_comb(input logic a, b, sel, output logic y);
  always_comb begin
    if (sel)
      y = b;
    else
      y = a;
  end
endmodule
```

- No need to write sensitivity list
- System Verilog automatically adds (a, b, sel) internally.

Key Differences Summary

Feature	`always`	`always_comb`
Sensitivity list	Manual (`@(a or b or sel)`)	**Automatic**
For combinational logic	☑ Yes, but manual	☑ Yes, preferred
For sequential logic	☑ Yes (with @(posedge clk))	❌ No
Detects missing assignments	❌ No	☑ Yes
Safer (avoids latches)	❌	☑
Checks for timing controls	❌	☑ Compiler error if used

- ➢ always → you must manually specify everything (old way).
- ➢ always_comb → System Verilog does it for you **automatically** and safely (modern way).
- **In short:**
- Use always_comb for combinational logic,
- Use always_ff` for sequential logic (flip-flops),
- Avoid plain `always` unless you really need it.

always_latch

SV adds a special procedural block for modelling latched logic:

- Variables assigned in an always_latch can not be assigned by another procedure.
- Can not contain further block timing or event control.
- Procedures first executes that time 0 without waiting for triggering.
- Tools may issue warnings if the the block does not infer latched logic.

Syntax:

always_latch

If (sel==1)

Op =a;

always_latch

- always_latch is a System Verilog procedural block used to model level-sensitive latch behavior.
- It is similar to always @(*), but intentionally describes a latch, not combinational logic.

Why do we need it?

In normal `always @(*)`, if you forget to assign a signal in all conditions,

- System Verilog assumes you didn't mean to create a latch — and it may issue a warning.
- But in `always_latch`, you are explicitly saying:
- "Yes, I want a latch here. Don't warn me about it."
- So it helps avoid confusion between combinational and latch behavior.

What is a latch?

- A latch is a storage element that:
- Remembers its value when the enable signal is 0 (inactive).
- Updates its output when the enable signal is 1 (active).

Example 1 — Simple D Latch

module d_latch(input logic d, en, output logic q);

```
  always_latch begin
    if (en)
      q = d;     // when enable = 1 → update q
    // when enable = 0 → retain previous q
  end
endmodule
```

Explanation:

always_latch automatically creates level-sensitive storage.

When `en` = 1 → latch is transparent, `q` follows `d`.

When `en` = 0 → latch holds last value.

Example 2 — Equivalent using `always @(*)`

```
module d_latch(input logic d, en, output logic q);
  always @(*) begin
    if (en)
      q = d;   // no else part → synthesizer infers a latch
  end
endmodule
```

Problem:

If you forget `else`, synthesis will infer a latch,

but **tools may give warnings**, because they think it might be unintentional.

Solution: use `always_latch` to say “I really mean to infer a latch”.

Example 3 — 4-bit latch

```
module latch4(input logic [3:0] d, input logic en, output logic [3:0] q);
  always_latch begin
```

```
    if (en)
      q = d;
  end
endmodule
```

When `en=1` → output `q=d`

When `en=0` → output holds previous `q`

Key Points Summary

Feature	Description
Purpose	Used to model level-sensitive latches
Sensitivity	Automatically sensitive to all signals read inside
Tool behavior	Ensures synthesis infers a latch
Typical mistake	Using `always @(*)` without `else` unintentionally infers latch
Best practice	Use `always_latch` when latch behavior is intentional

Rivisons:

- Use `always_comb` → **for pure combinational logic.**
- Use `always_ff` → **for clocked sequential logic (flip-flops)**
- Use `always_latch` → **for level-sensitive latch**

always_ff

SV adds a special procedural block for modelling registered logic:

- Variables assigned in always_ff can not be assigned by another procedure.
- Vontains one and only one event control.
- Can not contain any block timing.
- Tools may issue warning if block does not infer registered logic.

Syntax:

Always_ff (posedge clk or posedge rst)

If (rst)

Op =1’b1;

else

Op= in1;

- always_ff` stands for always flip-flop.
- It is used to describe edge-triggered sequential logic — like D flip-flops, registers, or counters.
- It runs on clock edges (posedge or negedge).

Why use `always_ff`?

It makes your intent clear to the synthesis tool — that you are describing flip-flop behavior, not combinational logic.

- It avoids accidental mixing of blocking (`=`) and non-blocking (`<=`) assignments.
- It helps catch errors — for example, if you accidentally try to use a signal assigned in another procedural block.

Example 1 — Simple D Flip-Flop

```
module d_ff(input logic clk, d, reset, output logic q);
  always_ff @(posedge clk or posedge reset) begin
    if (reset)
      q <= 0;     // reset output to 0
    else
      q <= d;     // store data on rising edge
  end
endmodule
```

Example 2 — 4-bit Counter

```
module counter (input logic clk, reset, output logic [3:0] count);
```

```
  always_ff @(posedge clk or posedge reset) begin
    if (reset)
      count <= 4'b0000; // reset counter
    else
      count <= count + 1; // increment on each clock edge
  end
endmodule
```

Key Rules of `always_ff`

Rule	Description
Sensitivity	Must contain only one event control (like posedge clk or negedge clk)
Assignment	Use non-blocking assignment (<=)
Use case	Sequential logic (flip-flops, registers)
Error check	Simulator warns if multiple signals drive same variable

Difference Summary

Block	Purpose	Trigger	Typical Use
always_comb	Combinational logic	Automatic (all inputs)	ALUs, multiplexers
always_latch	Level-sensitive latch	Automatic (with enable)	Enable-controlled latch
always_ff`	Sequential logic	Edge-triggered (clock/reset)	Flip-flops, registers, counters

Simple Analogy:

- always_comb → **logic gates (instant output)**
- always_latch → **latch (holds when disabled)**
- always_ff → **flip-flop (changes only on clock edge)**

Class-5 Array, Struct, Union

Arrays

System Verilog allows arrays to be declared as packed and unpacked arrays.

- The **unpacked** dimensions are those you declare after the name.
- Any type - even user defined.
- Implementation may store element of unpacked arrays non-contiguously.

The **packed** dimensions are those you declare before the name.

- bit, logic, reg types only.
- Implementation must store elements of packed arrays contiguously.

int inta [2:0] [1:0]; → unpacked

typedef enum {S[4]} seq_t;

seq_t seq5 [4:0]; → unpacked

logic [7:0] var_1 [3:0]; → packed – unpacked

Arrays Types

There are three types of arrays in system Verilog

Array	Definition	Declaration
Fixed size	It is also called as static array. Its size is defined at time of declaration.	Packed and unpacked
Dynamic Array	Its size id defined at run time.	Unpacked only
Associative Array	Its size is depended upon the numbers of accessed elements.	Unpacked only

In memory – R X C (Row and Column) store the data.

Packed = Row, Unpacked = Column

Accessing element of array

```
module acces();
logic [31:0]source[5],dest[5];
int i,j;

initial begin
	for (int i=0; i<5;i++)
		source[i]=i;
	$display("received the data of source  %p",source);
	//#1;

	foreach(dest[j])
		dest[j]= source[j]*4;
	$display("received the data of dest %p",dest);
	#10 $finish();
end
endmodule
```

Simulation output:

```
received the data of source  '{'h0, 'h1, 'h2, 'h3, 'h4}
received the data of dest '{'h0, 'h4, 'h8, 'hc, 'h10}
$finish called from file "example.sv", line 17.
$finish at simulation time                   10
```

Accessing element of array

```
module acces();
int mda[3][3]='{'{0,1,2},'{3,4,5},'{6,7,8}};
```

```
initial begin
    foreach(mda[i,j])
    $display("mda[%0d][%0d]=%0d",i,j,mda[i][j]);
    #10 $finish();
end
endmodule
```

simulation Output:

```
mda[0][0]=0
mda[0][1]=1
mda[0][2]=2
mda[1][0]=3
mda[1][1]=4
mda[1][2]=5
mda[2][0]=6
mda[2][1]=7
mda[2][2]=8
$finish called from file "example.sv", line 10.
$finish at simulation time                   10
```

In System Verilog, **static arrays** are arrays **whose size is fixed** at compile time and **cannot be changed during runtime.** They are a fundamental type of array and can be categorized into two main types based on their memory allocation and representation:

- **Packed Arrays:**
- These are arrays where the elements are stored contiguously in memory, effectively treated as a single, large vector of bits.
- They are declared with the array dimension before the variable name.

- Packed arrays are commonly used for representing hardware structures like registers or buses, where the bit order and contiguity are important.
- Example: bit [7:0] data_bus; (a packed array of 8 bits)

• Unpacked Arrays:

- These are arrays where each element is treated as an independent variable, and their memory allocation is not necessarily contiguous.
- They are declared with the array dimension after the variable name.
- Unpacked arrays are more flexible and can be used to represent collections of data elements, similar to arrays in software programming languages.
- Example: int my_array[10]; (an unpacked array of 10 integers)

Fixes sizes- Dimensions array ←**Static array** → 1.0 packing → 1.0 packed array 2.0 unpacked array

Dynamic array:

- A dynamic array is an unpacked array whose **size is set or change at run time** not compile time.
- Can be allocated and resized during simulation.
- Declared with empty subscripts [ ].
- $size system function returns the size of fixed-array or dynamic array.
- The space for a dynamic array doesn't exist until array is explicitly created at run time, space is allocated when new [**number**] operator is called.

➔ **number** indicates the number of space/elements to be allocated.

data_type array_name [ ]; // **array declaration**

Array_name = new[ ]; // **this operator allocates memory**

array_name.delete(); // **delete the array**

Example:

```
module testbench;
  int dyn_arr[]; // Declare a dynamic array
```

```
  initial begin
    dyn_arr[0] = 10; // ERROR: No memory allocated yet
    // Allocating memory for 5 elements
    dyn_arr = new[5];

    // Assigning values
    foreach (dyn_arr[i])
      dyn_arr[i] = i * 10;

    // Accessing elements
    $display("dyn_arr[0] = %0d", dyn_arr[0]);

    // Resizing while keeping old values
    dyn_arr = new[10](dyn_arr);

    // Deleting the array
    dyn_arr.delete();
  end
endmodule
```

Shallow copy and deep copy:-

```
reg [7:0][3:0] r_arr [];   // Legal
reg [][3:0] r_arr [2:0];    // Illegal

int dyn [], d2 [];
initial
begin
```

```
    dyn = new [5];
    foreach (dyn [j])
        dyn [j] = j;

    d2 = dyn;       // Performing shallow copy
    d2[0] = 5;
    $display(dyn[0], d2[0]);

    dyn = new[20](dyn);
    dyn = new[100];

    dyn.delete();
end
```

// Add this For Deep copy

```
d2 = new[dyn.size()];   // Allocate new memory for d2
foreach (dyn[j])
    d2[j] = dyn[j];   // Copy each element manually
```

Shallow Copy: -

- Only copies the reference (handle) of the object.
- Both variables point to the same memory location.
- **Changing one will also change the other.**

Example:

```
class C;
  int x;
endclass
```

```
C a, b;
initial begin
  a = new;
  a.x = 10;

  b = a;  // shallow copy (same handle)
  b.x = 20;

  $display("a.x = %0d", a.x); // prints 20
end
```

Deep Copy

- Creates a new object and copies the contents (data).
- Two independent objects are created.
- **Changing one does not affect the other.**

Example:

```
class C;
  int x;
  function C copy (); // deep copy function
    copy = new;
    copy.x = this.x;
  endfunction
endclass

C a, b;
initial begin
  a = new;
  a.x = 10;
```

```
b = a.copy();   // deep copy (separate object)
b.x = 20;

$display("a.x = %0d", a.x); // prints 10
end
```

- Key Difference:
- Shallow copy → **same object, multiple handles.**
- Deep copy → **completely new object with same data.**

Task-

1.0 Define a dynamic array
2.0 Add 10 elements to the dynamic array and prints the contents.
3.0 Resize the array to add 5 more elements and print the updated contents.
4.0 Delete a few elements and print the final contents.

Design code-

```
module dynamic_array_test;
int dyn_arr[];

initial begin
    dyn_arr = new[10];

    foreach(dyn_arr[i]) begin
        dyn_arr[i] = i;
    end
  $display(" Print the first 10 Initial elements");

  foreach(dyn_arr[i])
```

```
        $display("dyn_arr[%0d] = %0d", i,dyn_arr[i]);

    dyn_arr = new[15](dyn_arr);
    for (int i = 10; i < 15; i++)begin
        dyn_arr[i] = (i)*5;
    end
    $display(" After Resizing to 15 elements and print the value");

    foreach(dyn_arr[i])
        $display("dyn_arr[%0d] = %0d", i, dyn_arr[i]);

    dyn_arr = new[10](dyn_arr);

    $display("after deleting the last 5 element print data");
    foreach(dyn_arr[i])
        $display("dyn_arr[%0d] = %0d", i, dyn_arr[i]);

    dyn_arr.delete();
    $display(" display data after deleting, size =%0d", dyn_arr.size());
    end
endmodule
```

Simulation Result

Output

Print the first 10 Initial elements

dyn_arr[0] = 0

dyn_arr[1] = 1

dyn_arr[2] = 2

dyn_arr[3] = 3

dyn_arr[4] = 4

dyn_arr[5] = 5

dyn_arr[6] = 6

dyn_arr[7] = 7

dyn_arr[8] = 8

dyn_arr[9] = 9

After Resizing to 15 elements and print the value

dyn_arr[0] = 0

dyn_arr[1] = 1

dyn_arr[2] = 2

dyn_arr[3] = 3

dyn_arr[4] = 4

dyn_arr[5] = 5

dyn_arr[6] = 6

dyn_arr[7] = 7

dyn_arr[8] = 8

dyn_arr[9] = 9

dyn_arr[10] = 50

dyn_arr[11] = 55

dyn_arr[12] = 60

dyn_arr[13] = 65

dyn_arr[14] = 70

after deleting the last 5 element print data

dyn_arr[0] = 0

dyn_arr[1] = 1

dyn_arr[2] = 2

dyn_arr[3] = 3

dyn_arr[4] = 4

dyn_arr[5] = 5

dyn_arr[6] = 6

dyn_arr[7] = 7

dyn_arr[8] = 8

dyn_arr[9] = 9

display data after deleting, size =0

V C S S i m u l a t i o n R e p o r t

QUESTION:

Create an array of size 5 element and copy in another array, increase 2 more element in array2 and print the data present at array2.

Design code-

```
module array_copy_test;

int array1[];

int array2[];


initial begin


array1 = new[5];

foreach (array1[i])begin

      array1[i] = (i) * 10;

end


array2 = new[array1.size()];

foreach (array1[i]) array2[i] = array1[i];
```

```
$display("print the data present in array1");

foreach (array1[i])
    $display("array1[%0d] = %0d", i, array1[i]);

$display("print array2 copied ");

foreach (array2[i])
    $display("array2[%0d] = %0d", i, array2[i]);

array2 = new[array2.size()+2](array2);
array2[array2.size()-2] = 50;
array2[array2.size()-1] = 60;

$display("Updated array2 after resizing to 7 elements");
foreach (array2[i])
    $display("array2[%0d] = %0d", i, array2[i]);
end
endmodule
```

Simulation Result

Output

```
y1[0] = 0
array1[1] = 10
array1[2] = 20
array1[3] = 30
array1[4] = 40
```

print array2 copied

array2[0] = 0

array2[1] = 10

array2[2] = 20

array2[3] = 30

array2[4] = 40

Updated array2 after resizing to 7 elements

array2[0] = 0

array2[1] = 10

array2[2] = 20

array2[3] = 30

array2[4] = 40

array2[5] = 50

array2[6] = 60

Method-2

```
module array_copy_resize;
int array1[], array2[];
initial begin
    array1 = new[5];
    foreach (array1[i]) array1[i] = (i) * 10;

    array2 = array1;
    $display(" print the Initial data of array1 ");
    foreach (array1[i])
$display("array1[%0d] = %0d", i, array1[i]);

    $display(" print the Initial array2 (copied from array1) ");
```

```
    foreach (array2[i])
$display("array2[%0d] = %0d", i, array2[i]);

    array2 = new[array2.size()+2](array2);
    array2[array2.size()-2] = 60;
    array2[array2.size()-1] = 70;

    $display("Updated array2 after resizing to 7 elements");
    foreach (array2[i])
$display("array2[%0d] = %0d", i, array2[i]);
  end
endmodule
```

Question:

Create an array of 6 element, delete the first 2 element and print the remaining elements.

Design code-

```
module dynamic_array_delete;
int arr[];
int arr2[];

initial begin
        arr = new[6];
        arr = '{10, 20, 30, 40, 50, 60};
        $display("Original Array");

        foreach (arr[i])
                $display("arr[%0d] = %0d", i, arr[i]);
```

```
        $display("print the Original Array of 6 elements");

        arr2 = new[arr.size()-2];
        for (int i = 0; i < arr2.size(); i++)begin
            arr2[i] = arr[i+2];
        end

        arr = arr2;
        $display("Array after deleting first 2 elements");
        foreach (arr[i])
            $display("arr[%0d] = %0d", i, arr[i]);
end
endmodule
```

Simulation Result

Output

```
arr[0] = 10
arr[1] = 20
arr[2] = 30
arr[3] = 40
arr[4] = 50
arr[5] = 60
print the Original Array of 6 elements
Array after deleting first 2 elements
arr[0] = 30
arr[1] = 40
arr[2] = 50
arr[3] = 60
```

Associative Array-

- Used for sparse memories
- Dynamically allocated, non-contiguous elements
- Accessed with integer, or string index, single dimension
- Great for sparse arrays with wide ranging index

```
logic [63:0] assoc [*];
logic [63:0] idx = 1;

initial
  repeat(64) begin
    assoc[idx] = idx;
    idx = idx << 1;
  end
```

Standard Array vs Associative Array--

- Standard Array → All memory allocated, even unused elements
- Associative Array → **Unused elements do not allocate memory.**

Associative Array Methods-

- **num()**: returns number of entries
- **delete(index)**: delete specific index
- **exists(index)**: returns 1 if element exists at specified index
- **first(var)**: assign the value of first index to variable
- **last(var)**: assign the value of last index to variable
- **next(var)**: assign the value of next index to variable
- **prev(var)**: assign the value of prev index to variable

Associative Array Example: -

```
module assoc_array;
```

```
    int arr[*], index;
    initial begin
        repeat(3) begin
            arr[index] = index * 2;
            index = index + 3;
        end
        $display("no of entries are = %0d", arr.num);
        for (index = 0; index < 7; index = index + 1) begin
            $display("arr[%0d] = %0d", index, arr[index]);
            arr.first(index);
            $display("First entry is %d : %0d", index, arr[index]);
            arr.last(index);
            $display("Last entry is %d : %0d", index, arr[index]);
        end
    end
endmodule
```

Associative Array Methods-

```
module associative_array_example;
    bit [7:0] array [int];
    int index;

    initial begin
        array[5]  = 2;
        array[10] = 4;
        array[7]  = 6;
        array[9]  = 8;
```

```
        array[20] = 10;
        array[13] = 12;
        // Print array elements
        foreach (array[i]) $display("array[%0d] = %0d", i, array[i]);

// Print array size and number of entries
$display("size = %0d, Number of entries = %0d of array",
          array.size(), array.num());
$display("---------------------------");

// exists method
if(array.exists(7)) $display("An element exists at index = 7");
else $display("An element doesn't exists at index = 7");

if(array.exists(8)) $display("An element exists at index = 8");
else $display("An element doesn't exists at index = 8");
$display("---------------------------");
// first, last, prev, next method
array.first(index);
$display("First index of array = %0d", index);

array.last(index);
$display("Last index of array = %0d", index);

index = 9;
array.prev(index);  // Previous index of 9
$display("Prev index of 9 is %0d", index);
```

```
index = 10;
array.next(index);  // Next index of 10
$display("Next index of 10 is %0d", index);

$display("--------------------------");
// Delete particular index
array.delete(7);
// Print array elements
$display("After deleting element having index 7");
foreach (array[i]) $display("array[%0d] = %0d", i, array[i]);
$display("--------------------------");

// Delete complete array
array.delete();
$display("size = %0d of array", array.size());
   end
endmodule
```

Array Methods-

System Verilog provides various kind of methods. They are-

- Array Querying function.
- Array Locator methods
- Array Ordering Methods
- Array Reduction Methods

Array Querying functions-

System Verilog provides some system functions to return information about an array.

- **$bits():** shall return total no. of bits in dimension.

- **$left():** shall return MSB of the dimension.
- **$right():** shall return LSB of the dimension.
- **$low():** shall return the minimum of MSB and LSB.
- **$high():** shall return the maximum of MSB and LSB.
- **$increment():** shall return -1 if $left < $right and 1 if $left >= $right.
- **$size():** shall return the number of elements in dimension which is equal to [$high-$low +1]
- **$dimension():** shall return the total number maximum of dimension in array.

Example:

bit [1:0] [7:0] a [3:0] [3:0];

$left (a, 1);

$left (a, 3);

$dimension (a);

$low (a, 2);

$high (a ,4);

$bits (a);

Array Locator Methods: -

- find (): return all elements
- find_index(): return all the indexes of array
- find_first(): return first element
- find_first_index(): returns index of first element.
- find_last(): returns last element.
- find_last_index (): returns index of last element.

Example:

int d [ ] = `{2,3,4,56,67,45,4};

tqueue [$] ;

initial begin

tqueue = d.find with (item > 3);

tqueue = d.find_last_index with (item == 4);

end

Array Ordering Methods-

Array ordering methods can reorder the elements of one-dimensional arrays or queues.

- **reverse ():** reverse all elements of packed or unpacked arrays.
- **sort ():** sort unpacked array in **ascending** order.
- **rsort():** sort unpacked array in **descending** order.
- **shuffle ():** randomize the order of elements in array.

Example:

int d [ ] = `{2,3,4,56,67,45,4};

initial begin

d.sort();

d.rsort();

d.reverse();

d.shuffle();

end

Array Ordering Methods-

int count, summation;

int da [ ] = {10,1,8,3,5,5};

count =da.sum with (item >7); // 2: [10,8]

summation =da.sum with (item==5); //2: [5,5]

// sorting methods

int da [ ] = { 10, 1,7,3,4,4};

da.reverse (); //{ 4, 4, 3, 7, 1 ,10}

da.sort () ; // { 1,3,4,4,,7,10}

da.rsort (); // { 10,7,4,4,3,1}

da.shuffle (); // { 10,4,3,4,1,7}

Array Reduction Methods- given the single value ans

These methods are used to reduce and unpacked array in single value.

- sum (); return the sum of all elements of array.
- product (): return the product of all elements of array.
- and (): Perform bitwise and gate across all elements.
- Or (): Perform bitwise or gate across all elements.
- xor (): Perform bitwise xor gate across all elements.

Example:

int d [ ] =`{2,3,4,56,67,45,4};

int sum, productt;

initial begin

sum = d.sum;

productt = d.product;

end

Queue

- can provide easy sorting and searching
- allocates extra space to quickly add extra elements.
- **Does not need new [ ] operator.**
- **Push and pop operations are efficient.**
- **Can add and remove elements from anywhere.**

Declaration:

Data_type queue_name [$] ;

int q1 [$];

bit q2 [$];

string q3 [$];

byte q4 [$];

initialization:

q1 = {0,3,5,8,4};

q3 = {“RED”, “BLUE”, “GREEN”};

Queue methods: -

size(): return the number of item in queue.

insert (): insert given item at specified index.

delete(): delete item at specified index.

push_front(): insert the given element at the front of queue.

push_back(): insert the given element at the end of queue.

pop_front(): removes and returns first element of queue.

pop_back(): removes and returns last element of queue.

Example:

```
int j=1;
int b[$] = {3,4};
int q[$] = {0,2,5};
initial begin
q.insert (1, j);
q.insert ( 3, b[0]);
q.insert (4, b[1]};
$display (q);
q.delete (1);
$display(q);
q.push_front (6);
j = q.pop_back;
q.push_back (8);
j= q.pop_front;
```

end

Queue Tips & Tricks:

```
int p[$];
int q[$] = {2,4,8};
int e, pos;
initial begin
e = q[0];       // e = 2
e= q[$];        // e = 8
q[0] =e;        // {8,4,8}
p = q;          // entire queue copy
q = {q , 6}     // insert 6 at the end
q = {e, q}      // insert 6 at the beginning
q = q[1 : $];   //delete the first element
q = q[0 : $-1]  // delete the last element
q = q { }       // delete all element
q = { q [0 : pos -1], e, q[pos +1 : $]};   //insert 'e' at position 'pos'
q = { q [0 : pos ], e, q[pos +1 : $]};   //insert 'e' after position 'pos'
end
```

Struct

Structure can be created using the **struct** statement.

- A structure is a **collection** of variables and/or constants that can be accessed separately or as a whole.
- Allows logical groups of signals to be combined together (example: control signals of a bus protocal).

Example:

Struct {

int a, b;

logic [7:0] opcode;

logic [23:0] address;

bit error; } instrunction_word;

- The entire collection can be referenced using the name of the structure-
- < structure_name>.<variable_name>

instrunction_word.address = 24'h F00000;

UNION

- Union is a **single storage** element having multiple representations.
- The members that compose a union, **all share same storage area**.

Example: -

union {

int a;

bit [3:0]b;

byte c;

} union_comp

int a;	byte c;	bit [3:0] b;

Array

- An array is a way to store multiple values of the same data type in a single variable name.
- Think of it like a box with multiple compartments.

Example:

If you want to store 5 numbers, instead of writing:

int a0, a1, a2, a3, a4;

You simply write:

int a[5];

- Types of Arrays in SystemVerilog

System Verilog has 4 main types of arrays:

1. Fixed-size arrays
2. Dynamic arrays
3. Associative arrays
4. Queues

Below is a simple explanation for each.

1.0 Fixed-size Array (Normal Array)

- Size is known at compile time.

Example:

```
int a[5];   // array of 5 integers
initial begin
   a[0] = 10;
   a[1] = 20;
   a[2] = 30;
   $display("a[0] = %0d, a[2] = %0d", a[0], a[2]);
end
```

2.0 Dynamic Array

Size is decided at runtime using new[].

Example:

```
int da[];


initial begin
   da = new[4];     // create 4 elements
   da[0] = 5;
   da[1] = 10;
   da[2] = 15;
   da[3] = 20;
   $display("da[3] = %0d", da[3]);
end
```

3.0 Associative Array

- **Index can be int, string, or any type.**
- **Used when you do not know how many elements will be used.**

Example:

Using string as index:

```
int marks[string];


initial begin
   marks["sarif"] = 90;
   marks["rahul"] = 85;

   $display("Sarif marks = %0d", marks["sarif"]);
```

end

4.0 Queue

A queue is like a dynamic array that grows and shrinks.

Operations:

push_front()

push_back()

pop_front()

pop_back()

Example:

int q[$]; // **queue**

initial begin

  q.push_back(10);

  q.push_back(20);

  q.push_front(5); // **front = 5**

  $display("Queue = %p", q);

  q.pop_front(); // **removes 5**

  $display("After pop = %p", q);

end

Summary Table

| Array Type | Size Known When? | Key Feature |

| ---------- | ---------- | ---------- |

Fixed array	Compile time	Simple, fixed size
Dynamic array	Runtime	Size can change using new []
Associative array	No fixed size	Indexed using int/string/etc
Queue	Runtime	Growth/shrink automatically

Access array elements

1.0 Accessing Elements of a 1D Array

A 1D array has only one index.

Example:

```
int a[5] = '{10, 20, 30, 40, 50};

initial begin
  $display(a[0]); // 10
  $display(a[2]); // 30
  $display(a[4]); // 50
end
```

👉 Use a[index]

2.0 Accessing Elements of a 2D Array

A 2D array needs two indexes: row and column.

Example:

```
int mda[2][3] = '{ '{1,2,3}, '{4,5,6} };

initial begin
```

```
  $display(mda[0][0]); // 1
  $display(mda[0][2]); // 3
  $display(mda[1][1]); // 5
end
```

👉 **Use mda[row][column]**

3.0 Accessing Elements Using `foreach` Loop

This is the easiest way to iterate over array elements.

1D Example:

```
int a[4] = '{5,10,15,20};

initial begin
  foreach(a[i])
    $display("a[%0d] = %0d", i, a[i]);
end
```

2D Example:

```
int m[2][2] = '{ '{1,2}, '{3,4} };
initial begin
  foreach(m[i,j])
    $display("m[%0d][%0d] = %0d", i, j, m[i][j]);
end
```

4.0 Accessing Elements of a Dynamic Array

Dynamic array size is decided at runtime.

```
int da[];

initial begin
  da = new[3];
  da[0] = 11;
  da[1] = 22;
  da[2] = 33;

  $display(da[1]); // 22
end
```

5.0 Accessing Elements of an Associative Array

Associative arrays use keys instead of numeric indexes.

#Using string as index:

```
int marks[string];

initial begin
  marks["sarif"] = 90;

  $display(marks["sarif"]); // 90
end
```

6. Accessing Elements of a Queue

Queue behaves like a dynamic list.

```
int q[$] = '{10,20,30};
```

```
initial begin
  $display(q[0]); // 10
  $display(q[$]); // last element = 30
end
```

Summary (Easy)

Type	How to Access
1D Array	a[i]
2D Array	a[row][column]
foreach loop	foreach(a[i,j])
Dynamic Array	da[i]
Associative Array	arr[key]
Queue	q[index]

Dynamic Arrays

A dynamic array is an array whose size can be decided at runtime (during simulation), not at compile time.

- Size can change using new[]
- Useful when you don't know the array size in advance
- Memory is allocated dynamically

Syntax

```
datatype array_name[];
```

Example:

```
int da []; // dynamic array
```

Example 1: Create a Dynamic Array and Assign Size

```
module dyn_ex;

  int da[];   // declare dynamic array
  initial begin
    da = new[4];   // allocating size = 4
    da[0] = 10;
    da[1] = 20;
    da[2] = 30;
    da[3] = 40;
    $display("da[2] = %0d", da[2]);  // prints 30
  end
endmodule
```

- da = new[4]; creates 4 elements
- You can access like normal array

Example 2: Changing Size of Dynamic Array

You can change the size anytime using `new[]`.

```
int da[];


initial begin
  da = new[3];     // size = 3
  da = new[6];     // size changed to 6
end
```

- When size changes → old data is lost unless you use new[size](old_array).

Example 3: Copy Old Data While Resizing

```
int da[];
```

```
initial begin
  da = new[3];
  da[0] = 1;
  da[1] = 2;
  da[2] = 3;

  da = new[5](da);   // copy old data, new size = 5

  $display("%p", da);
end
```

- New array size = 5
- Old values copied
- New elements initialized to 0
- Output:

```
'{1, 2, 3, 0, 0}
```

Example 4: Iterate Dynamic Array Using foreach

```
int da[];

initial begin
  da = new[4];
  da = '{10, 20, 30, 40};

  foreach (da[i])
    $display("da[%0d] = %0d", i, da[i]);
end
```

Difference Between Dynamic Array & Static Array

Feature	Static Array	Dynamic Array
Size	Fixed at compile time	Decided at runtime
Memory	Fixed	Allocated dynamically
Resize	❌ No	✓ Yes
Use Case	Fixed-size data	Unknown or changing size

Simple Real-Life Explanation

Imagine a bus:

- **Static array** = 40-seater bus (fixed seats)
- **Dynamic array** = seats increase or decrease depending on passengers

Associative Arrays

1.0 What is an Associative Array? **(key- value) concept remember in mind**

An associative array is an array that does not use numeric index (0,1,2...).

Instead, it uses a key, which can be:

- Int
- String
- Bit
- scalar type

👉 Best when you don't know array size and want to store data with meaningful keys.

2.0 Syntax

datatype array_name[index_type];

Examples:

int marks[string]; // **string index**

int regfile[int]; **// int index**

3. Simple Example — Using String Index

```
module aa_ex;
  int marks[string];  // associative array

  initial begin
    marks["sarif"] = 90;
    marks["rahul"] = 85;

    $display("Sarif marks = %0d", marks["sarif"]);
  end
endmodule
```

✓ Index is `"sarif"` (a name, not a number)

✓ No fixed size required

4. Example — Using int Index

```
int data[int];

initial begin
  data[10] = 100;
  data[25] = 250;

  $display("data[10] = %0d", data[10]);
end
```

- You only create elements when you write to them.

5. Associative Array Methods

System Verilog gives built-in methods to work with associative arrays.

1. num() → Number of elements

```
$display("count = %0d", marks.num());
```

2. exists(key) → Checks if key is present

```
if(marks.exists("sarif"))
  $display("Sarif entry exists");
```

3. delete(key) → Deletes one element

```
marks.delete("rahul");
```

delete() → deletes entire array

```
marks.delete();
```

4. first(key) → Get first key in the array

```
string k;
if( marks.first(k) )
  $display("First key = %s", k);
```

5. last(key) → Get last key

```
string k;
marks.last(k);
$display("Last key = %s", k);
```

6. next(key) → Get next key

```
string k = "sarif";
marks.next(k);
```

```
$display("Next key = %s", k);
```

7. prev(key) → Get previous key

```
string k = "rahul";
marks.prev(k);
$display("Previous key = %s", k);
```

6. Full Simple Example Including All Methods

```
module assoc_example;

  int marks[string];

  initial begin
    marks["sarif"] = 90;
    marks["rahul"] = 85;
    marks["john"]  = 70;

    // count
    $display("Total = %0d", marks.num());

    // exists
    if (marks.exists("rahul"))
      $display("rahul marks exist");

    // first & last
    string key;
    marks.first(key);
```

```
    $display("First key = %s", key);

    marks.last(key);
    $display("Last key = %s", key);

    // next
    key = "rahul";
    marks.next(key);
    $display("Next key after rahul = %s", key);

    // delete one element
    marks.delete("john");

    // delete all
    marks.delete();
  end
endmodule
```

Summary Table

Method	Meaning
num()	number of elements
exists(key)	checks if key exists
delete(key)	delete one element
delete()	delete entire array
first(key)	get first key
last(key)	get last key
next(key)	get next key

| prev(key) | get previous key |

Array Reduction Methods

These methods operate on packed or unpacked arrays and **reduce** them to **a single value**:

Method	Operation
sum()	Adds all elements
product()	Multiplies all elements
and()	Bitwise AND of all elements
or()	Bitwise OR of all elements
xor()	Bitwise XOR of all elements

```
module reduction_methods;
  int d[] = '{2, 3, 4, 56, 67, 45, 4};
  int sumn, productt;

  initial begin
    sumn     = d.sum();       // sum of array elements
    productt = d.product();   // product of all elements

    $display("Sum of array = %0d", sumn);
    $display("Product of array = %0d", productt);
  end
endmodule
```

Explanation in Simple Way

- d.sum()

Adds all elements

`2 + 3 + 4 + 56 + 67 + 45 + 4 = 181`

- d.product()

Multiplies all elements

2 * 3 * 4 * 56 * 67 * 45 * 4 = very large number

OUTPUT:

Sum of array = 181

Product of array = 29154240 // (value depends on exact calculation)

✓ `and()`

✓ `or()`

✓ `xor()`

reduction methods in SystemVerilog.

These methods reduce all array elements into one single value using bitwise operations.

1. and() — Bitwise AND of all elements

Rule:

All bits must be `1` to get `1`.

If any bit is `0`, result is `0`.

✓ Example:

```
module and_example;

  bit [3:0] a[] = '{4'b1111, 4'b1100, 4'b1010};
  bit [3:0] result;

  initial begin
    result = a.and();
```

```
    $display("AND result = %b", result);
  end
endmodule
```

✓ **Explanation**:

1111

AND 1100 = 1100

AND 1010 = 1000

Final: 1000

2. or() — Bitwise OR of all elements

Rule:

If ANY bit is `1`, result is `1`.

Only gets 0 when all bits are 0.

✓ **Example:**

```
module or_example;
  bit [3:0] a[] = '{4'b0001, 4'b0010, 4'b0100};
  bit [3:0] result;

  initial begin
    result = a.or();
    $display("OR result = %b", result);
  end
endmodule
```

✓ **Explanation:**

0001

OR 0010 = 0011

OR 0100 = 0111

Final: 0111

3. xor() — Bitwise XOR of all elements

Rule:

XOR = 1 when bits are different,

XOR = 0 when bits are same.

✓ Example:

```
module xor_example;
  bit [3:0] a[] = '{4'b1010, 4'b1100, 4'b0110};
  bit [3:0] result;

  initial begin
    result = a.xor();
    $display("XOR result = %b", result);
  end
endmodule
```

✓ Explanation:

1010

XOR 1100 = 0110

XOR 0110 = 0000

Final: 0000

SUMMARY TABLE

Method	Meaning	Example Result
sum()	Add all elements	2+3+4 = 9
product()	Multiply all elements	2*3*4 = 24
and()	Bitwise AND of all	1111 & 1100 = 1100
or()	Bitwise OR of all	0001 | 0010 = 0011
xor()	Bitwise XOR of all	1010 ^ 1100 = 0110

ALL Array Methods:

- Array Querying functions
- Array Locator methods
- Array Ordering methods
- Array Reduction methods

1.0 ARRAY QUERYING FUNCTIONS

These give information about the array.

1.1 size() → returns number of elements

```
int a[] = '{10,20,30};
$display(a.size());  // 3
```

1.2 num() → number of entries (for associative arrays)

```
int marks[string];
marks["a"]=10;
marks["b"]=20;
$display(marks.num());  // 2
```

1.3 exists(key) → checks if index exists (associative array)

```
if(marks.exists("a"))
  $display("Key exists!");
```

1.4 first(), last(), next(), prev() (associative array keys)

```
string k;
marks.first(k);   // first key
marks.last(k);    // last key
marks.next(k);    // next key
marks.prev(k);    // previous key
```

Summary of Querying Methods

Method	Use
size()	total elements
num()	number of entries
exists(key)	key present or not
first/last	get first/last key
next/prev	traverse keys

2. ARRAY LOCATOR METHODS

Used to search inside arrays using conditions.

Works only on dynamic arrays, queues, and associative arrays.

2.1 find → returns all matching elements

```
int a[] = '{1, 2, 3, 4, 5};

int res[$];
res = a.find with (item > 3);
$display("%p", res);   // '{4, 5}
```

2.2 find_index → returns indexes of matching elements

```
int idx[$];
idx = a.find_index with (item%2==0);  // even numbers
$display("%p", idx); // '{1,3}
```

2.3 find_first / find_last

```
int x;
x = a.find_first with (item > 2); // returns 3
x = a.find_last with (item < 5);  // returns 4
```

Summary of Locator Methods

Method	Returns
find	elements
find_index	indexes
find_first	first matched element
find_last	last matched element
find_first_index	first matched index
find_last_index	last matched index

3. ARRAY ORDERING METHODS

Used to sort or shuffle arrays.

3.1 sort() → sort ascending

```
int a[]='{5,3,9};

a.sort();
$display("%p", a); // '{3,5,9}
```

3.2 rsort() → sort descending

a.rsort();

$display("%p", a); // '{9,5,3}

3.3 sort() with key (for array of objects/structs)

a.sort with (item.id); // custom sorting

3.4 shuffle() → random permutation

a.shuffle();

$display("%p", a);

Summary of Ordering Methods

Method	Meaning
sort()	ascending
rsort()	descending
shuffle()	random order
sort with()	sort using a condition

4. ARRAY REDUCTION METHODS

Reduce entire array → single value.

Used for packed or unpacked arrays.

- sum()

int a[]='{1,2,3};

$display(a.sum()); // 6

- product()

$display(a.product()); // 1*2*3 = 6

- and(), or(), xor()

bit [3:0] b[]='{4'b1100,4'b1010};

$display(b.and()); // AND all
$display(b.or());   // OR all
$display(b.xor()); // XOR all

Summary of Reduction Methods

Method	Output
sum()	add all
product()	multiply all
and()	bitwise AND
or()	bitwise OR
xor()	bitwise XOR

FINAL SUPER SUMMARY (Very Easy)

Category	Methods	Use
Querying	size(), exists(), first()	Information
Locator	find, find_index, find_first	Searching
Ordering	sort, rsort, shuffle	Sorting
Reduction	sum, product, and, xor	Reduce to single value

Queues in System Verilog

A queue is like a dynamic array with automatic **push/pop**.

- Size can **grow and shrink** automatically
- You can **add or remove elements from front or back**

Syntax:

datatype queue_name[$];

2. Simple Example of Queue

```
module queue_example;
  int q[$];       // declare queue
  initial begin
    q = {1,2,3}; // initialize
    $display("Queue = %p", q);   // {1,2,3}
  end
endmodule
```

3. Queue Methods (VERY IMPORTANT)

SystemVerilog Queues support:

1.0 push_front()

2. 0 push_back()

3. 0 pop_front()

4. 0 pop_back()

5. 0 insert()

6. 0 delete()

7. 0 size()

Let’s explain one by one.

Method 1: push_front()

Adds an element at the beginning.

Example:

q.push_front(10);

Before: `{1,2,3}`

After: `{10,1,2,3}`

Method 2: push_back()

Adds an element at the end.

q.push_back(20);

Before: `{1,2,3}`

After: `{1,2,3,20}`

Method 3: pop_front()

Removes first element and returns it.

```
int x;
x = q.pop_front();
$display("Removed = %0d", x);
```

Method 4: pop_back()

Removes last element.

int y;

y = q.pop_back();

Method 5: insert (index, value)

Insert at a particular position.

q.insert(1, 50);

If queue = `{10,20,30}`

After insert at index 1 → `{10,50,20,30}`

Method 6: delete(index)

Delete at a specific index.

q.delete(2);

If queue = `{10,20,30,40}`

After delete index 2 → `{10,20,40}`

Delete full queue:

q.delete();

Method 7: size()

Returns the number of elements.

$display("Queue size = %0d", q.size());

FULL WORKING EXAMPLE (All methods)

```
module queue_methods;
  int q[$];

  initial begin
    q = {1,2,3};
    $display("Initial Queue = %p", q);

    q.push_front(10);
    q.push_back(20);
    $display("After push = %p", q);

    int a = q.pop_front();
    int b = q.pop_back();
    $display("pop_front = %0d, pop_back = %0d", a, b);
    $display("After pop = %p", q);

    q.insert(1, 99);
    $display("After insert = %p", q);

    q.delete(2);
    $display("After delete = %p", q);

    $display("Queue size = %0d", q.size());
  end
endmodule
```

FINAL SUMMARY (Very Easy)

Method	Meaning	Example
push_front(x)	Add at beginning	{x, ...}
push_back(x)	Add at end	{..., x}
pop_front()	Remove 1st element	returns element
pop_back()	Remove last element	returns element
insert(i,x)	Insert at index i	{..., x, ...}
delete(i)	Delete index i	removes element
size()	Number of items	q.size()

Difference between Dynamic Array, Queue, and Associative Array in SystemVerilog, with easy examples.

1. Dynamic Array

A Dynamic Array has a fixed size during runtime, but you can resize it manually using new().

Syntax:

```
int da[];     // dynamic array
```

✓ Key Points:

- Size is decided at runtime
- Size does NOT grow automatically
- You must use new(size) to resize

✓ Example:

```
int da[];

initial begin
  da = new[4];      // size becomes 4
  da = '{1,2,3,4};

  da = new[6];      // size becomes 6 (old data lost)
end
```

2. Queue

A Queue is a dynamic array with automatic growth/shrink.

✓ Syntax:

```
int q[$];
```

✓ Key Points:

- Automatically grows and shrinks
- You can easily push or pop
- Used for **FIFO**-like behavior

✓ Example:

```
int q[$];

q.push_back(10);
q.push_front(5);
q.pop_back();
q.pop_front();
```

3. Associative Array

Associative Array uses a key instead of index → like a dictionary.

- **Syntax:**

int aa[string]; **// string as key**

int bb[int]; **// integer as key**

✓ Key Points:

- No fixed size
- Index can be string, int, or any scalar type
- Very useful in UVM for storing objects by name

✓ Example:

int aa[string];

aa["id1"] = 10;

aa["id2"] = 20;

if (aa.exists("id1"))

  $display("Key exists");

FINAL COMPARISON TABLE

Feature	Dynamic Array	Queue	Associative Array
Size	Fixed after `new()`	Auto grow/shrink	No size required
Resize	Must use `new()`	Automatic push/pop	No resizing
Index Type	Only integer	Only integer	string/int/enum

| Use Case | Data of known size | FIFO / LIFO | Lookup table, maps |

| Example | `da = new[10];` | q.push_back(5); | `aa["name"]=10;` |

Simple Real-Life Example

Type	Real-Life Example
Dynamic Array	Fixed seats in a bus (capacity fixed after booking)
Queue	People standing in queue (first in, first out)
Associative Array	Phone contacts (key = name, value = number)

struct

A struct is a **collection of different data types grouped together under one name**.

- It is like a "mini-database"
- Stores related information in one variable
- Similar to a record in C or a table row

Simple Example: Student Data

```
typedef struct {
   int id;
   string name;
   int marks;
} student_t;


module test;
  student_t s1;


  initial begin
```

```
    s1.id = 101;
    s1.name = "Sarif";
    s1.marks = 90;
    $display("ID=%0d Name=%s Marks=%0d", s1.id, s1.name, s1.marks);
  end
endmodule
```

Output:

```
ID=101 Name=Sarif Marks=90
```

2. Why do we use struct?

Because it helps group related data:

- Better readability
- Easier handling of multiple variables
- Used in transactions (UVM), packets, protocols, etc.

3. Types of Struct

System Verilog gives 2 types:

A. Unpacked struct (default)

- Fields stored separately
- Easier to read
- Can mix different sizes

Example:

```
struct {
    bit [7:0] addr;
    bit [3:0] control;
```

```
    int data;
} pkt;
```

Access:

```
pkt.addr = 8'hAA;
pkt.control = 4'hF;
pkt.data = 32;
```

B. Packed struct

- Stored as one continuous bit-vector
- Used when you need a compact data packet
- All elements must be bit-based (logic, bit)

Example:

```
typedef struct packed {
    logic [7:0] address;
    logic [3:0] control;
} header_t;

header_t h1;
```

This struct becomes:

[7:0 address] [3:0 control] --> total 12 bits

4. Initializing a Struct

Method 1: Field-by-field

```
s1.id = 1;
s1.name = "A";
```

```
s1.marks = 80;
```

Method 2: List style

```
s1 = '{1, "A", 80};
```

Method 3: Named style (best)

```
s1 = '{ id:1, name:"A", marks:80 };
```

5. Array of Structs (Very Useful)

```
student_t classA[3];

initial begin
  classA[0] = '{101, "Ram", 80};
  classA[1] = '{102, "Shyam", 85};
  classA[2] = '{103, "Aman", 90};
  $display("%p", classA);
end
```

6. Struct Inside a Struct

```
typedef struct {
  int day;
  int month;
  int year;
} date_t;

typedef struct {
  string name;
```

```
  int id;
  date_t dob;
} employee_t;

employee_t emp;
```

7. Passing Struct to Function/Task

```
function void print(student_t s);
  $display("ID=%0d Name=%s", s.id, s.name);
endfunction
```

SIMPLE SUMMARY

Feature	**Unpacked struct**	**Packed struct**
Storage	Each field separate	Continuous bits
Type mix	Allowed	Only bit/logic
Usage	Normal data	Protocol packets / drivers
Example	{int, string}	{logic [7:0], bit [3:0]}

Difference between struct and union

STRUCT vs UNION (Very Simple Explanation)

Feature	**STRUCT**	**UNION**
Storage	Stores all members separately	Stores only one member at a time
Memory size	Sum of all members	Max size among members
Access	All fields can be used simultaneously	Only one field is valid at a time
Use case	Group related data	Save memory / reinterpret bits
Example	Student info (id, name, marks)	Same memory used as int or byte

1. STRUCT (Stores everything)

A struct stores all members in **separate memory.**

✓ Example:

```
typedef struct {
  int id;
  byte marks;
} student_t;
```

Memory layout:

id (4 bytes)

marks (1 byte)

Total = 5 bytes

You can use both:

```
s.id = 10;
s.marks = 90;
```

2. UNION (Shares same memory)

A union stores only one member at a time.

All members share same memory location.

✓ **Example**:

```
typedef union {
  int   a;
  byte  b;
} data_t;
```

Memory layout:

Max(a=4 bytes, b=1 byte) = 4 bytes

If you write:

d.a = 32'hAABBCCDD;

$display("b = %0h", d.b);

b will show the lower 8 bits of a.

This is called type reinterpretation.

Key Difference (Easy to Remember)

✓ STRUCT

- Think: Box with multiple compartments
- Each variable has its own space.

✓ UNION

Think: One compartment reused

You can store either `a` OR `b`, not both.

Example: STRUCT

```
typedef struct {
  int a;
  int b;
} st_t;

st_t s;
```

```
initial begin
  s.a = 10;
  s.b = 20;
  $display("a=%0d b=%0d", s.a, s.b); // Both valid
end
```

Example: UNION

```
typedef union {
  int a;
  byte b;
} un_t;

un_t u;
initial begin
  u.a = 32'h11223344;
  $display("b=%0h", u.b);  // prints 44 (lower byte)
end
```

Summary (One Line)

✓ **STRUCT →** **store multiple values together**

✓ **UNION →** **store one value in shared memory**

What is `s` in this code?

```
typedef struct {
  int a;
  int b;
} st_t;
```

st_t s;   // <--- THIS is the variable of type st_t

✓ `s` is a struct variable

✓ It is an instance (object) of the struct type `st_t`

- Why do we need `s`?

Because:

✓ `typedef struct { … } st_t;`

This line defines a new data type named `st_t`.

But it does NOT create variables.

✓ `st_t s;`

This line creates a variable `s` of type `st_t`.

Think of it like this:

* `typedef struct {...} st_t;` → **creates a template**

* `st_t s;` → **creates an actual object**

Without creating `s`, you cannot store or access data.

Real-Life Example

Imagine:

STRUCT = Blueprint of a house

```
typedef struct {
  int rooms;
  int floors;
} house_t;
```

VARIABLE (s) = Actual house built from blueprint

house_t h1; // **this is the real house**

- Why do we use `s` in your code?
- **Because we want to store values in `a` and `b`.**

s.a = 10;

s.b = 20;

Here:

✓ `s` is the container

✓ `a` and `b` are fields inside that container

What happens if we don't create `s`?

This will cause an error:

Error: Cannot access struct fields without creating a variable

Because you cannot use the struct until you have an instance.

Very Simple Summary

Term	Meaning
typedef struct ... st_t;	Creates a new struct data type
st_t s;	Creates a variable of that type
s.a, s.b	Access members of struct

Final One-Line Explanation

- s is the actual variable where data is stored.
- Without `s`, the struct has no memory.

Packed vs Unpacked struct using memory diagrams (ASCII diagrams).

This will help you visualize how data is stored in memory.

1. PACKED STRUCT → stored as one continuous bit vector

✓ All fields are stored next to each other

✓ No gaps

✓ Treated like one single packed bit array

✓ Only bit/logic types allowed

✓ Useful in: protocol packets, drivers, RTL

Example: Packed Struct

```
typedef struct packed {
  logic [7:0] addr;
  logic [3:0] command;
  logic       enable;
} pkt_t;

pkt_t p;
```

PACKED STRUCT MEMORY DIAGRAM

```
|<--------------  8 bits  ------------->|
|<-- 4 bits  -->|<-1 bit->|
+-----------------------------------------------+
| addr[7:0]     | cmd[3:0] | enable |   (total 13 bits)
+-----------------------------------------------+
```

bit positions:

[12:5] [4:1] [0]

👉 All fields are combined into ONE 13-bit vector

👉 Looks like: logic [12:0] p;

2. UNPACKED STRUCT → stored as separate variables

✓ Each field stored independently

✓ Memory not continuous

✓ Can mix any data type (int, string, class, etc.)

✓ Used in: testbenches, UVM objects

Example: Unpacked Struct

```
typedef struct {
  int    id;
  byte   age;
  string name;
} student_t;

student_t s;
```

UNPACKED STRUCT MEMORY DIAGRAM

```
+-----------------+       +----------+       +--------------+
|    s.id (32b)   |       | s.age(8b)|       | s.name(ptr)  |
+-----------------+       +----------+       +--------------+
(separate memory)         (separate)         (separate)
```

- Each field is stored at different memory locations
- Struct is like a folder containing separate files

⭐ SUPER SIMPLE COMPARISON (DIAGRAM FORM)

PACKED

ONE continuous block of bits

+--+

| field1 | field2 | field3 | field4 | ... |

+--+

UNPACKED

Separate memory locations

+---------+ +---------+ +---------+

| field1 | | field2 | | field3 |

+---------+ +---------+ +---------+

⭐ Final Summary Table

Feature	**Packed Struct**	**Unpacked Struct**
Memory layout	Continuous bits	Separate fields
Data type allowed	Only bit/logic	Any type (int, string, class)
Usage	RTL, packets	Testbench, UVM
Acts like	One big vector	Group of variables
Example	**`{addr, cmd, enable}`**	**`{id, age, name}`**

One-Line Summary

👉 Packed = compact, bit-level, continuous memory

👉 Unpacked = separate variables stored individually

UNION

A union is like a single memory box that can store different types of data, but only one at a time.

- All fields of the union share the same memory location.
- Changing one field will change the others, because they overlap in memory.

- Useful when you want multiple views of the same data.

1. Normal (Unpacked) UNION Example – Different Sizes Allowed

#Code

```
module test;
  union {
    int   a;      // 32-bit
    byte  b;      // 8-bit
  } u;

  initial begin
    u.a = 32'hAABBCCDD;
    $display("a = %h", u.a);
    $display("b = %h", u.b);   // Shows lowest 8 bits of a
  end
endmodule
```

Output (conceptually)

a = AABBCCDD

b = DD

🔍 Why?

- Both `a` and `b` use same memory.
- `b` reads the lower byte of `a`.

2. Packed UNION – All fields must be SAME size

A packed union is bit-accurate and all members must have same size.

#Example

```
module test;
```

```
  typedef union packed {
    logic [7:0] byte_view;
    logic [3:0] nibble_view[2];   // 4+4 bits = 8 bits
  } u_t;

  u_t u;

  initial begin
    u.byte_view = 8'hAB;   // AB = 10101011
    $display("byte = %h", u.byte_view);
    $display("nibble[0] = %h", u.nibble_view[0]); // lower 4 bits
    $display("nibble[1] = %h", u.nibble_view[1]); // upper 4 bits
  end
endmodule
```

📌 Output (conceptually)

```
byte = AB
nibble[0] = B   (1011)
nibble[1] = A   (1010)
```

🔍 Why?

Both views refer to same 8 bits.

☑ 3. Why do we use UNION? (Simple Reasons)

✓ 1. Multiple views of the same data

Useful in protocols:

- One field interpreted as 32-bit

- The same field also interpreted as 4 bytes

✓ 2. Save memory

Only one memory location used.

✓ 3. Data reinterpretation

Easy to convert between formats without copying data.

Key Differences: STRUCT vs UNION

Feature	STRUCT	UNION
Memory	Separate memory for each field	All fields share same memory
Size	Sum of all members	Size of largest member
Can store	All fields at same time	Only one field meaningfully at a time
Use case	Grouping data	Reinterpreting data

Small Real-Life Example (Very Simple)

Imagine a box of size 32 bits:

Union fields:

- int a (32 bits)
- byte b (8 bits)

They share the same box.

If you write:

u.a = 0x12345678

Then `u.b` reads the last 8 bits:

u.b = 0x78

Because they overlap.

Session-6: -

Task & Functions, Threads: fork..join, fork..join_any, fork..join_none, wait fork, disable fork

- function [return_type] function_name(input [type] arg1, input [type] arg2, ...);

// body (no time-consuming operations)

function_name = <expression>; // return value

endfunction

- **function int add(input int a, input int b);**
- **add = a + b; // Return sum**
- **endfunction**
-
- **module test;**
- **initial begin**
- **int result;**
- **result = add(5, 3);**
- **$display("Sum = %0d", result);**
- **end**
- **endmodule**

task task_name(input [type] arg1, output [type] arg2, inout [type] arg3);

// body (can contain delay, event control, or wait)

endtask

task display_with_delay(input string msg);

#5; // wait 5 time units

$display("Time=%0t, Message=%s", $time, msg);

endtask

```
module test;
   initial begin
      display_with_delay("Hello SystemVerilog!");
   end
endmodule
```

Function

Design Code: -

```
module my_module;
int i;

function sum (input a,b);
      int m,n;
      m=a;
      n=b;
      sum=m+n;
      return sum;
endfunction

initial begin
      i=sum(10,20);
      $display("i=%0d",i);
end
endmodule
```

Simulation Output: -

i=0

- you did not specify the return type of function.
- Function **default to a 1-bit return value**.
- Expression m+n evaluates 30 (5'b**11110** in binary)
- Only **LSB** is kept 0 return type & function result 1-bit value.

Design Code: -

```
module my_module;
int i;


function int sum (input a,b);
    int m,n;
    m=a;
    n=b;
    sum=m+n;
    return sum;
endfunction


initial begin
    i=sum(10,20);
    $display("i=%0d",i);
end
endmodule
```

Simulation Output: -

i=0

- you did not specify their data types (int, logic).
- Input takes **default to a 1-bit logic** because of not defining data type.
- Expression sum=m+n =0 +0 =0 (sum of LSB)

Design Code: -

```
module my_module;
int i;

function int sum (input int a,b);
    int m,n;
    m=a;
    n=b;
    sum=m+n;
    return sum;
endfunction

initial begin
    i=sum(10,20);
    $display("i=%0d",i);
end
endmodule
```

Simulation Output: -

i=30

- In case of return any value "**return"** keyword is used.
- In case of no return value "**void"** keyword is used with function.

```
module my_module;
int i;

function void sum (input int a, b);
    $display ("a=%0d, b=%0d", a, b);
```

```
endfunction

initial begin
        sum (10,20);
end
endmodule
```

Simulation Output: -

a=10, b=20

- Function can have input/output/inout type arguments.

Design Code: -

```
module m;
int i;

function int sum (input int a,b, output int c);
      c=a+b;
endfunction

initial begin
      sum(10,20,i);
      $display("i=%0d",i);
end
endmodule
```

Simulation Output: -

i=30

- Function can be with no arguments also.

Design Code: -

```
module m;
int i;

function void sum ();
    $display("print inside function");
endfunction

initial begin
    sum();
end
endmodule
```

Simulation Output: -

print inside function

Static and automatic functions: -

- Static in system Verilog that do not store the local copies of variables for each call.
- They **share the same memory across all invocations**.
- Defining using **static keyword or by default** if **not specified as automatic.**

- Each call to automatic gets its own copy of local variables, preventing the shared memory issue.

Static function

- By default, every function in System Verilog is static, unless you specify **automatic.**

- Static variables are **shared** among all function calls (**like a single copy**). So, if the function is called multiple times, all calls use the same storage for local variables -- they can overwrite each other's values.

Automatic function

- When a function is declared as **`automatic`**, each call to the function gets **its own copy of local variables.**
- It's like giving every function call its own workspace, so values don't interfere between calls.
- This is useful when functions are called concurrently (e.g., from multiple threads or processes).

Example:

(a) Static Function Example

```
module static_example;
  function int static_func(input int a);
    int temp;       // Static by default
    temp = temp + a;
    return temp;
  endfunction

  initial begin
    $display("Call 1: %0d", static_func(5));
    $display("Call 2: %0d", static_func(10));
  end
endmodule
```

Output:

Call 1: 5

Call 2: 15

- temp` is static, so it remembers its previous value.
- On first call: `temp = 0 + 5 = 5

- On second call: `temp = 5 + 10 = 15

(b) Automatic Function Example

```
module automatic_example;
  function automatic int auto_func(input int a);
    int temp;       // Each call gets its own copy(created different memory address)
    temp = temp + a;
    return temp;
  endfunction

  initial begin
    $display("Call 1: %0d", auto_func(5));
    $display("Call 2: %0d", auto_func(10));
  end
endmodule
```

Output:

Call 1: 5

Call 2: 10

- temp is automatic, so it doesn't remember previous calls.
- Each call starts with a fresh temp (default = 0).

Summary Table

Feature	Static Function	Automatic Function
Default type	**Yes (default)**	Must write `automatic`
Variable storage	Shared (one copy)	Separate copy per call
Suitable for	Simple, sequential functions	Reentrant/concurrent calls
Remembers old values?	☑ Yes	❌ No
Example use	Counters, accumulators	Recursive or parallel calls

- Always prefer `automatic` in testbenches or recursive functions, because they may be called in parallel.
- Keep `static` only when you want to preserve old data between calls.

Design Code: - Static function

```
module my_module;
//int i=10,j=20;

function int fact (int num);
    if (num==1)
        return (1);
    else
        return (num*fact(num-1));
endfunction

initial begin
    $display("factorial of number 6 is =%0d",fact(6));
end
endmodule
```

Simulation Output: -

factorial of number 6 is =1

- As recursion goes deeper, all numbers values **overwrite each-other.**
- When fact (1) return 1, the intermediate calculations are lost because of **same memory location.**

Design Code: -automatic function

```
module my_module;
//int i=10,j=20;
```

```
function automatic int fact (int num);
    if (num==1)
        return (1);
    else
        return (num*fact(num-1));
endfunction

initial begin
    $display("factorial of number 6 is =%0d",fact(6));
end
endmodule
```

Simulation Output: -

factorial of number 6 is =720

- Each recursive call gets its own copy of local variables.
- No overwritten happens.
- Recursions works correctly.
- Each call needs independent storage.

ref and const ref Keyword: -

ref Meaning:

ref = **pass by reference.** That means:

- When you pass a variable using `ref`, the function or task gets direct access to the original variable **(not a copy).**
- Any change inside the function **changes the original variable** in the calling scope.
- ref argument can propagate changes (modify) immediately (directly).

```
module ref_example;
```

```
task change_value (ref int x);
   x = x + 10;
endtask

initial begin
   int a = 5;
   $display("Before task: a = %0d", a);
   change_value(a);
   $display("After task : a = %0d", a);
end
endmodule
```

Output:

Before task: a = 5

After task : a = 15

- ref int x means `x` refers to the same memory as `a`. So, when the task changes `x`, **it's actually modifying a.**

const ref: Meaning:

const ref` = **pass by reference, but read-only** **(main -important things)**

That means:

- The function or task still accesses the original variable directly **(no copy).**
- But it **cannot modify the variable's value** inside the function/task.
- Useful when you want to save memory and time but still protect the variable from being changed.
- If user want the reference formal argument **as read only** then use of const ref is suggested as it **prevents** subroutines from modifying arguments.

```
module const_ref_example;
task display_value(const ref int x);
```

```
    $display("Inside task: x = %0d", x);
    // x = x + 10;  // ❌ ERROR: Cannot modify const ref variable
endtask

initial begin
    int a = 100;
    display_value(a);
    $display("After task: a = %0d", a);
end
endmodule
```

Output:

```
Inside task: x = 100
After task: a = 100
```

- **const ref int x` gives read-only access to `a`.**
- **The task can read but not change the variable.**

Comparison Table

Feature	`ref`	`const ref`
Data passing	By reference	By reference
Copy created?	❌ No	❌ No
Can modify variable?	☑ Yes	❌ No
Memory efficiency	☑ High	☑ High
Used for	When you need to modify the original	When you need read-only access

- **ref`:** When a task should **update values** (e.g., increment counters, modify data).
- **const ref`:** When passing large arrays or structs to a function **for reading only**, to **save memory and speed.**

Design Code: -

```
module m;
int i=10,j=20;

function int sum(ref int a,b);
    b=a+b;
endfunction

initial begin
    sum(i,j);
    $display("j =%0d",j);
end
endmodule
```

Simulation Output: -

j =30

Design Code: -

```
module m;
int i=10,j=20;

function int sum(const ref int a,const ref int b);
    b=a+b;
endfunction
initial begin
    sum(i,j);
    $display("j =%0d",j);
```

```
end

endmodule
```

Simulation Output: -

```
example.sv, 7

  'const' variable is either driven or connected to a non-const variable.

  Variable 'b' declared as 'const' cannot be used in this context

  Source info: b = (a + b);

1 warning

1 error
```

- We are passing as **const ref means read-only reference**- we can not change its value (that's why compiler is showing an error).
- Error: Cannot assign to variable 'b' because it is declared as **'const ref'**

ref (Pass by Reference):

- When an argument is declared with ref, the task or function receives a reference to the original variable.
- Any **modifications** made to the formal argument within the subroutine will **directly affect the actual argument** in the calling scope.
- This is useful when a subroutine needs **to update the value of a variable** in the calling scope.

const ref (Constant Reference):

- When an argument is declared with const ref, the task or function receives a reference to the original variable, but the formal argument is treated as read-only within the subroutine.
- Attempts to modify a const ref argument inside the subroutine will result in a compilation error.
- This provides a mechanism to ensure that the original variable's value is not accidentally modified by the subroutine while still avoiding the overhead of copying large data structures or objects.

Key Differences:

- **Modifiability:**

  ref allows modification of the original variable; const ref prevents modification.

- **Purpose:**

  ref is used when the subroutine needs to update the original variable; const ref is used when the subroutine only needs to read the original variable without modifying it, while still benefiting from pass-by-reference efficiency.

- **Error Handling:**
  Attempting to modify a const ref argument will result in a compiler error, providing design-time protection.

Design Code: -

```
module function_example;

function static increment_static();

        static int count_A;

        automatic int count_B;

        int count_C;


        count_A ++;

        count_B ++;

        count_C ++;

        $display("static: count_A=%0d,count_B=%0d,
count_C=%0d",count_A,count_B,count_C);

endfunction


function automatic increment_automatic();

        static int count_A;

       automatic int count_B;

       int count_C;
```

```
    count_A ++;
  count_B ++;
  count_C ++;
  $display("automatic: count_A=%0d,count_B=%0d,
count_C=%0d",count_A,count_B,count_C);
endfunction

function increment();
    static int count_A;
  automatic int count_B;
  int count_C;

  count_A ++;
  count_B ++;
  count_C ++;
  $display("Normal: count_A=%0d,count_B=%0d,
count_C=%0d",count_A,count_B,count_C);
endfunction

initial begin
    $display("calling static functions");
    increment_static();
    increment_static();
    increment_static();
    $display("\ncalling automatic functions");
    increment_automatic();
    increment_automatic();
    increment_automatic();
```

```
    $display("\ncalling Normal functions: without static/automatic keyword");
    increment();
    increment();
    increment();

    //accesing varibles using functions
    //count_A
    $display("\nstatic: count_A= %0d", increment_static.count_A);
    $display("\nautomatic: count_A= %0d", increment_automatic.count_A);
    $display("\nNormal: count_A= %0d", increment.count_A);

    //count_B : hierarchical refrences to automatic variable is not legal
    /*
    $display("\nstatic: count_B= %0d", increment_static.count_B);
  $display("\nautomatic: count_B= %0d", increment_automatic.count_B);
  $display("\nNormal: count_B= %0d", increment.count_B);
    */
    //count_C
    $display("\nstatic: count_C= %0d", increment_static.count_C);
  //$display("\nautomatic: count_C= %0d", increment_automatic.count_C);
    //illegal reference to automatic variable
  $display("\nNormal: count_C= %0d", increment.count_C);
end
endmodule
```

Simulation Output: -

```
calling static functions
static: count_A=1,count_B=1, count_C=1
```

static: count_A=2,count_B=1, count_C=2

static: count_A=3,count_B=1, count_C=3

calling automatic functions

automatic: count_A=1,count_B=1, count_C=1

automatic: count_A=2,count_B=1, count_C=1

automatic: count_A=3,count_B=1, count_C=1

calling Normal functions: without static/automatic keyword

Normal: count_A=1,count_B=1, count_C=1

Normal: count_A=2,count_B=1, count_C=2

Normal: count_A=3,count_B=1, count_C=3

static: count_A= 3

automatic: count_A= 3

Normal: count_A= 3

static: count_C= 3

Normal: count_C= 3

V C S S i m u l a t i o n R e p o r t

- Local variables default depends on the function.
- In static function (default/normal function), local variable without an explicit keyword is static.
- In an automatic function (default/normal function), local variable without an explicit keyword is automatic (a fresh copy of each call).

Task-Pass by value: -

A pass by value argument passing mechanism **does copy arguments locally** and operates on those variables. Any change in variables in the task will not be visible outside of the task.

“Pass by value” means: When you pass an **argument by value** to a task (or function):

- The task gets a copy of the variable.
- Any changes made inside the task **will not affect the original variable** outside the task.
- **Pass by value = copy passed, not the original.**

Syntax

```
task my_task(input int a);
  // 'a' is passed by value (default for 'input')
  ...
endtask
```

- The default for `input` arguments is pass by value.

Examples: -

```
module pass_by_value_example;
int x = 5;

task change_value (input int a);
   a = a + 10;
   $display("Inside task: a = %0d", a);
endtask

initial begin
   $display("Before task call: x = %0d", x);
   change_value(x);
   $display("After task call: x = %0d", x);
end
endmodule
```

Step	Statement	What happens	Output

| ---- | ------------------------ | ------------------------------------- | ------------------------- |
1	`x = 5;`	Initialize `x`	
2	`$display("Before...")`	Prints value of `x`	Before task call: x = 5
3	`change_value(x);`	A copy of `x` (5) is passed to `a`.	
4	Inside task: `a = a + 10;`	Now `a = 15`, but `x` still remains `5`.	Inside task: a = 15
5	After task	We display again — **`x` has not changed**	After task call: x = 5

Simulation Output

Before task call: x = 5

Inside task: a = 15

After task call: x = 5

Explanation:

The task received a copy of `x`.

Changing `a` does not affect the real `x`.

Key Point

Keyword	Meaning	Effect on caller variable
`input`	Pass by value	❌ No change outside
`output`	Pass by reference (output from task to caller)	☑ Updates caller variable after task ends
`inout`	Pass by reference (both ways)	☑ Changes visible both inside and outside

Quick Recap

- Default for `input` is pass by value.
- Task gets its own copy of the variable.
- Changing that copy does not modify the original.

Design Code: -

module task_example;

```
int a,b;
int out;

task tk_multply(int a,b, output int out);
    a=a*b;
    out=a;
endtask

initial begin
    a=5;
    b=6;
    tk_multply(a,b,out);
    $display("Task: out=%0d for a=%0d and b=%0d",out,a,b);
end
endmodule
```

Simulation Output: -

Task: out=30 for a=5 and b=6

Task-Pass by reference: -

Pass by Reference:

- Normally, when we pass a variable to a task, a copy of it is made (that's pass by value).
- But when we use `ref`, the **actual variable is passed** to the task — **not a copy.**
- So if the task changes the value, the original variable outside the task also changes.

Syntax

```
task task_name(ref data_type variable);
  // statements
```

endtask

- `ref` = reference. It means: “use the same variable, not a copy.”

Example

```
module pass_by_ref_example;
int x = 5;

task change_value (ref int a);
   a = a + 10;
   $display("Inside task: a = %0d", a);
endtask

initial begin
   $display("Before task call: x = %0d", x);
   change_value(x);
   $display("After task call: x = %0d", x);
end
endmodule
```

Step-by-Step Execution

1. `x = 5`
2. Task `change_value(x)` is called — but since it uses **`ref`,**
   `a` refers to the **same memory as `x`.**
3. Inside task: `a = a + 10;` → changes `x` to 15.
4. After the task ends, `x` is still 15 because both `a` and `x` are the same variable.

Simulation Output

```
Before task call: x = 5
Inside task: a = 15
```

After task call: x = 15

Key Point Summary

Keyword	Meaning	Copy or Reference?	Original Value Changes?
`input`	Pass by value	Copy	❌ No
`ref`	Pass by reference	Same variable	☑ Yes
`const ref`	Pass by reference (read-only)	Same variable	❌ No (can't modify)

A pass by reference argument passing mechanism does not copy arguments locally but reference to the **original arguments is passed.** This also means that any change in values for the arguments inside the subroutine will **affect the original values of the variables.**

Note: **pass by reference** does not work with static tasks in some compilers.

Design Code: -

```
module task_example;
int a,b;
int out;

task automatic tk_multply(ref int a,b, output int out);
    a=a*b;  // value of a is updated , it will not update the global variable a and b
    out=a;
endtask
initial begin
    a=5;
    b=6;
    tk_multply(a,b,out);
    $display("Task: out=%0d for a=%0d and b=%0d",out,a,b); // observe a updated
end
```

endmodule

Simulation Output: -

Task: out=30 for a=30 and b=6

Task-Pass by reference with const: -

const ref:

- ref` → **passes the actual variable** (**not a copy**).
- const → **means read-only** (you can use the variable inside the task, but can't modify it).

const ref` means:

- Pass the original variable (not a copy),
- but don't allow changing it inside the task.

Syntax

task task_name(const ref int variable);

// **can read 'variable' but cannot modify it**

endtask

Example

module pass_by_const_ref_example;

int x = 10;

task display_value(const ref int a);

// a = a + 5; // ❌ **This line would cause an error**

$display("Inside task: value of a = %0d", a);

endtask

```
initial begin
    $display("Before task call: x = %0d", x);
    display_value(x);
    $display("After task call: x = %0d", x);
end
endmodule
```

Step-by-Step Explanation

1. `x = 10`
2. Task `display_value(x)` is called
   → `a` refers to the same variable as `x`, but can't modify it because of `const`.
3. Inside task: you can read `a`, print it, or use it in expressions.
4. After the task, since we never modified `a`, `x` stays the same.

Simulation Output

Before task call: x = 10

Inside task: value of a = 10

After task call: x = 10

Key Points Summary

Keyword	Meaning	Can change variable inside task?	Original variable affected?
`input`	Pass by value	☑ Yes (inside task only — affects copy)	❌ No
`ref`	Pass by reference	☑ Yes	☑ Yes
`const ref`	Pass by reference (read-only)	❌ No	❌ No

- You want to avoid copying large data structures (like arrays or structs),
- But still prevent accidental modification of the original data.

A pass by reference argument along with “const” gives us advantage that it prevents value getting changed outside task just like in pass by value, but advantage is that it prevents unnecessary copy creation during the argument pass (that happen in pass by value).

Design Code: -

```
module task_example;

int a,b;

int out;


task automatic tk_multply(const ref int a,b, output int out);

    a=a*b; //expects compilation error when subroutine tries to modify argument value.

    out=a;

endtask

initial begin

    a=5;

    b=6;

    tk_multply(a,b,out);

    $display("Task: out=%0d for a=%0d and b=%0d",out,a,b);

end

endmodule
```

Simulation Output

```
example.sv, 7
  'const' variable is either driven or connected to a non-const variable.
  Variable 'a' declared as 'const' cannot be used in this context
  Source info: a = (a * b);
1 warning
```

1 error

FORK-JOIN

System Verilog supports concurrent execution of multiple processes using the fork...join construct.
When you want several blocks of code to run at the same time (like generating multiple stimuli, monitoring signals, or driving interfaces), you can use fork...join.

fork..join
It allows you to create parallel threads that execute simultaneously within a simulation.
Syntax:
fork
// parallel processes
join
Example:
initial begin
fork
$display("Process 1 start at %0t", $time);
#5 $display("Process 2 start at %0t", $time);
#10 $display("Process 3 start at %0t", $time);
join
$display("All processes finished at %0t", $time);
end
Output:
Process 1 start at 0
Process 2 start at 5
Process 3 start at 10
All processes finished at 10
In fork-join, the join waits until all the forked processes done. Then join processes will start.

fork- join

```
module fork_join();

initial begin

    $display($time,":starts_fork....join_examples");

    #10 $display($time,":starts_fork....join_examples_#10");

    fork
```

```
        $display($time,":Parallel_start");
        #50 $display($time,":Parallel_start_#50");
        #10 $display($time,":Parallel_start_#10");
    begin
        #30 $display($time,":sequentially_start_#30");
        #10 $display($time,":sequentially_start_#10");
    end
join
$display($time,": After_join");
#80 $display($time,":final after_#80");
end
endmodule
```

Simulation Output: -

```
 0:starts_fork....join_examples
10:starts_fork....join_examples_#10
10:Parallel_start
20:Parallel_start_#10
40:sequentially_start_#30
50:sequentially_start_#10
60:Parallel_start_#50
60: After_join
140:final after_#80
    V C S  S i m u l a t i o n  R e p o r t
Time: 140
```

fork- join_any

fork...join_any
Waits until any one of forked process finishes, then join threads proceeds.
Example:

```
initial begin
fork
#2 $display("Short Task");
#10 $display("Long Task");
join_any
$display("Exited after one task finished at %0t", $time);
end
```

Output:
Short Task
Exited after one task finished at 2
Long Task

Here the after finishes of one fork threads (short task) join thread proceeds. The join thread won't wait for all the forked threads completion. It's not supported in Verilog. When one process result is enough to continue (eg: waiting for first valid response).

```
module fork_join_any();
initial begin
    $display($time,":starts_fork....join_examples");
    #10 $display($time,":starts_fork....join_examples_#10");
    fork
        $display($time,":Parallel_start");
        #50 $display($time,":Parallel_start_#50");
        #10 $display($time,":Parallel_start_#10");
    begin
        #30 $display($time,":sequentially_start_#30");
        #10 $display($time,":sequentially_start_#10");
    end
join_any
$display($time,": After_join");
#80 $display($time,":final after_#80");
end
```

endmodule

Simulation Output: -

0:starts_fork....join_examples

10:starts_fork....join_examples_#10

10:Parallel_start

10: After_join

20:Parallel_start_#10

40:sequentially_start_#30

50:sequentially_start_#10

60:Parallel_start_#50

90:final after_#80

V C S S i m u l a t i o n R e p o r t

Time: 90

fork- join_none

Does not wait for any forked process to finish — joined process continues immediately.

Example:

```
initial begin
fork
#5 $display("Background Task 1");
#10 $display("Background Task 2");
join_none
$display("Parent process continues at %0t", $time);
#15 $display("End of simulation");
end
```

Output:

Parent process continues at 0
Background Task 1
Background Task 2
End of simulation

Here the join threads won’t wait for any forked threads. First join threads excecuted then forked threads will start. Forked processes run in the background.

```
module fork_join_none();
initial begin
    $display($time,":starts_fork....join_examples");
    #10 $display($time,":starts_fork....join_examples_#10");
    fork
        $display($time,":Parallel_start");
        #50 $display($time,":Parallel_start_#50");
        #10 $display($time,":Parallel_start_#10");
    begin
        #30 $display($time,":sequentially_start_#30");
        #10 $display($time,":sequentially_start_#10");
    end
join_none
$display($time,": After_join");
#80 $display($time,":final after_#80");
end
endmodule
```

Simulation Output: -

```
 0:starts_fork....join_examples
10:starts_fork....join_examples_#10
10: After_join
10:Parallel_start
20:Parallel_start_#10
40:sequentially_start_#30
50:sequentially_start_#10
```

60:Parallel_start_#50

90:final after_#80

V C S S i m u l a t i o n R e p o r t

Time: 90

wait fork

- SV provides a construct a **wait fork** that waits for the completion for the other process.
- The **wait fork** is used to ensure that all child process (processes created by child process) have completed their execution.

```
module fork_join_none();
initial begin
    $display($time,":starts_fork....join_examples");
    #10 $display($time,":starts_fork....join_examples_#10");
    fork
        $display($time,":Parallel_start");
        #50 $display($time,":Parallel_start_#50");
        #10 $display($time,":Parallel_start_#10");
    begin
        #30 $display($time,":sequentially_start_#30");
        #10 $display($time,":sequentially_start_#10");
    end
join_none
wait fork;
$display($time,": After_join");
#80 $display($time,":final after_#80");
end
```

endmodule

Simulation Output: -

0:starts_fork....join_examples

10:starts_fork....join_examples_#10

10:Parallel_start

20:Parallel_start_#10

40:sequentially_start_#30

50:sequentially_start_#10

60:Parallel_start_#50

60: After_join

140:final after_#80

V C S S i m u l a t i o n R e p o r t

Time: 140

disable fork

- SV provides a construct a **disable fork** that stops the execution of the process.

Sometimes, you might want to stop the remaining threads after one process finishes.

Example:

```
initial begin
fork
#10 $display("Task 1");
#20 $display("Task 2");
#30 $display("Task 3");
join_any
disable fork; // stops remaining forked processes
$display("Stopped other threads at %0t", $time);
end
```

Output:
Task 1
Stopped other threads at 10

It is very useful, when we need to stop unnecessary background threads once a condition is met in testbench.

```
module fork_join_none();
initial begin
	$display($time,":starts_fork....join_examples");
	#10 $display($time,":starts_fork....join_examples_#10");
	fork
		$display($time,":Parallel_start");
		#50 $display($time,":Parallel_start_#50");
		#10 $display($time,":Parallel_start_#10");
	begin
		#30 $display($time,":sequentially_start_#30");
		#10 $display($time,":sequentially_start_#10");
	end
join_none
disable fork;
$display($time,": After_join");
#80 $display($time,":final after_#80");
end
endmodule
```

Simulation Output: -

```
		0:starts_fork....join_examples
		10:starts_fork....join_examples_#10
		10: After_join
		90:final after_#80
	V C S  S i m u l a t i o n  R e p o r t
Time: 90
```

Session-7

Interface, Clocking Block, Program Block, Casting- Static & Dynamic

Interface: an **interface is a construct that bundles a collection of signals and other elements into a single entity** to simplify and standardize communication between **modules or between a design and a testbench.** It acts as a high-level connection point, allowing you to group many individual signals into one block, making it easier to pass them as a single port. This enhances **modularity, reusability, and code readability by abstracting the complexity of signal connections.**

Verilog based Port connection in SV: -

Port Connection in SV

```
module design(input A,B,C, output reg sum,carry);


always @(A or B or C) begin
        {carry,sum}=A+B+C;
end
endmodule
module testbench();
reg A,B,C;
wire sum,carry;


design dut(.A(A),.B(B),.C(C),.sum(sum),.carry(carry));


initial begin
        $monitor("for A=%0b B=%0b C=%0b and sum=%0b
carry=%0b",A,B,C,sum,carry);
```

```
    A=0; B=0; C=0;
#10 A=1; B=1; C=0;
#10 A=1; B=1; C=1;

#10 $finish();
end
endmodule
```

output simulation

for A=0 B=0 C=0 and sum=0 carry=0

for A=1 B=1 C=0 and sum=0 carry=1

for A=1 B=1 C=1 and sum=1 carry=1

$finish called from file "example.sv", line 21.

$finish at simulation time 30

- design dut(.A(A),.B(B),.C(C),.sum(sum),.carry(carry));

in above DUT –

sum - formal argument

(sum) – Actual argument

.C() – Do not want to connect this port

[.name] Connection

If actual and formal port argument are identical in name and size, then below coding is suitable.

In design formal ports- & in testbench actual ports – (are identical in name and size)

Resultant-

- design dut(.A(A),.B(B),.C(C),.sum(sum),.carry(carry));

If we do not want to connect C port

- design dut(.A,.B,.C(),.sum, .carry);

implicit [.*] dot-star

Automatically infers connction of all nets and ports od same name and size.

Type also should be compatible

- design dut(.*,.C()); // do not want to connect port C

Verilog based port connection

```
module mem_mod(
Input bit req,clk,start,
Input logic [1:0] mode;
Input logic [7:0] addr;
Input wire [7:0] data;
Output bit gnt);
……Body……..
endmodule: mem_mod

module cpu_mod(
input bit clk,gnt,
inout wire [7:0] data;
output bit req, start,
output logic [1:0] mode);
…….Body…….
endmodule: cpu_mod

module top;
logic req, gnt, start;
logic clk;
```

logic [1:0]mode;

logic [7:0] addr;

wire [7:0] data;

mem_mod mem (req,clk,start,mode,addr,data,gnt);

cpu_mod cpu(clk,gnt,data,req,start,addr,mode);

……Body……

endmodule: top

Verilog Port conventions are difficult(impractical) and tradious for large designs. (why)?

- Need to change the port list of each module.
- Need to change the port list of the connecting module.
- Need to change the name in the instantiation of the module.
- Need to change the name everywhere in the hierarchy.

Disadvantage of Verilog module port connections:

- Declarations must be duplicated in multiple modules.
- Communication protocols must be duplicated in several modules.
- Risk of mismatched declarations.
- A change in design specifications in multiple modules.

Memory → Instruction → Master processor → Test generator → Slave processor

Interface

SV provides the powerful features for Verilog port connection problems. It is called interface.

- An interface allows a number of signals to be grouped together and represented as a single port.
- The declarations of signals that make up the interface are contained in a single location.
- Each module that uses these signals then has a single port of the interface type, instead of many ports with the discreate signals.
- Aim of the interface is to encapsulate communication.

- Also provide-
- Directional information (modport)
- Timing information (clocking block)

Interface

- Encapsulation communication between different hardware blocks.
- Contain the connectivity, synchronization, and optionally, the functionality of the communication between two or more hardware blocks more like a class template.
- Connection design blocks and /or testbenches.
- Could be called bundle of wires or reg(s) if no functionality.
- Multiple signals that you declare over and over in every module or program grouped together into a single unit (put in central location), passed around the design hierarchy **reducing the possibility of misconnecting signals**.
- Adding a new signal made easy, you must need to **declare it once in the interface**, not in higher level modules, **reducing the error**.
- Ideal for design reuse, **prevents the code redundancy**.
- When **two blocks communicate** with a specified protocols using two or more signals, **use an interface**.
- Can be instantiated in a design module just like module instance.
- Design modules can not be instantiated in an interface (no design hierarchy allowed inside an interface).

Interface Syntax:

interface if_name[interface ports];

[parameter if any] // optional

Interface definition

endinterface

interface bus (input clk);

logic [31:0] addr;

logic [31:0] data;

logic en;

```
endinterface

interface bus #(parameter WIDTH=32) (input clk);
logic [WIDTH -1:0] addr;
logic [WIDTH -1:0] data;
logic en;
endinterface;
```

Example1:

```
// Full adder --Design
module f_add(input A,B,C, output reg sum,carry);

always @(A or B or C) begin
        {carry,sum}=A+B+C;
end
endmodule
//interface
interface intf();
        logic A,B,C;
        logic sum,carry;

        modport tb (input sum,carry,output A,B,C);
        modport dut(output sum,carry,input A,B,C);
endinterface
// Testbench
module tb_f_add();
intf inf();
```

```
f_add dut(.A(inf.A),.B(inf.B),.C(inf.C),.sum(inf.sum),.carry(inf.carry));


initial begin
        $monitor("for A=%0b B=%0b C=%0b and sum=%0b
carry=%0b",inf.A,inf.B,inf.C,inf.sum,inf.carry);


        inf.A=0; inf.B=0; inf.C=0;
#5 inf.A=0; inf.B=0; inf.C=1;
#5 inf.A=0; inf.B=1; inf.C=0;
#5 inf.A=0; inf.B=1; inf.C=1;
#5 inf.A=1; inf.B=0; inf.C=0;
#5 inf.A=1; inf.B=0; inf.C=1;
#5 inf.A=1; inf.B=1; inf.C=0;
#5 inf.A=1; inf.B=1; inf.C=1;


#10 $finish();
end
endmodule
```

Output:

```
for A=0 B=0 C=0 and sum=0 carry=0
for A=0 B=0 C=1 and sum=1 carry=0
for A=0 B=1 C=0 and sum=1 carry=0
for A=0 B=1 C=1 and sum=0 carry=1
for A=1 B=0 C=0 and sum=1 carry=0
for A=1 B=0 C=1 and sum=0 carry=1
```

for A=1 B=1 C=0 and sum=0 carry=1

for A=1 B=1 C=1 and sum=1 carry=1

$finish called from file "example.sv", line 35.

$finish at simulation time 45

V C S S i m u l a t i o n R e p o r t

Two modports define direction for signals for:

- TB→ testbench side (drive inputs, read outputs);
- Dut → dut side (reads inputs , drive outputs);
- intf inf(); → create interface instances.
- Modports actually module port define.

Example-2

// Design

```
module f_add(input A,B,C, output reg sum,carry);
always @(A or B or C) begin
        {carry,sum}=A+B+C;
end
endmodule
```

//interface

```
interface intf();
        logic A,B,C;
        logic sum,carry;

        modport tb (input sum,carry,output A,B,C);
        modport dut(output sum,carry,input A,B,C);
endinterface
```

//top module

```
module Top;
intf inf();
f_add dut(inf.A,inf.B,inf.C,inf.sum,inf.carry);
tb_f_add tb(inf);

endmodule
```

//Testbench

```
module tb_f_add(intf inf);

initial begin
        $monitor("for A=%0b B=%0b C=%0b and sum=%0b carry=%0b",inf.A,inf.B,inf.C,inf.sum,inf.carry);

        inf.A=0; inf.B=0; inf.C=0;
#5 inf.A=0; inf.B=0; inf.C=1;
#5 inf.A=0; inf.B=1; inf.C=0;
#5 inf.A=0; inf.B=1; inf.C=1;
#5 inf.A=1; inf.B=0; inf.C=0;
#5 inf.A=1; inf.B=0; inf.C=1;
#5 inf.A=1; inf.B=1; inf.C=0;
#5 inf.A=1; inf.B=1; inf.C=1;
#10 $finish();
end
endmodule
```

Output simulation

for A=0 B=0 C=0 and sum=0 carry=0

for A=0 B=0 C=1 and sum=1 carry=0

for A=0 B=1 C=0 and sum=1 carry=0

for A=0 B=1 C=1 and sum=0 carry=1

for A=1 B=0 C=0 and sum=1 carry=0

for A=1 B=0 C=1 and sum=0 carry=1

for A=1 B=1 C=0 and sum=0 carry=1

for A=1 B=1 C=1 and sum=1 carry=1

$finish called from file "example.sv", line 39.

$finish at simulation time 45

Program block

- **I**n SV, the test bench code is written in a program block (specific for testbench).
- **T**o avoid the race-around condition
- **U**sed for separating test and benches and design.
- Program variables are assigned using blocking assignments only.
- Using non-blocking assignments with program variables or blocking assignments with design (non-program) variables shall be an error.
- Always put the clock generators in module and never in program block as it may result in race around conditions.
- Implicit $finish when all initial blocks end in program.

Program Block vs Module

How is a Program block different from a Module?

Feature	Program Block	Module
Purpose	Used for writing testbenches	Used for designing synthesizable logic
Executive Order	Runs in the Reactive region	Runs in the active region
Race Condition handling	Avoid the race conditions with DUT	Can have race condition with testbench if not synchronized properly
Event sheduling	Controlled and safe	Can lead to unpredictable execution if not managed properly

Instantiation	Can not be instantiated inside another program	Can be instantiated inside another program
Synthesizable?	X No (simulation only)	Yes (used for hardware design)

Virtual interface

A Virtual Interface is **used as a pointer or handle for an actual interface** that allows to communicate dynamic objects with static modules.

In Verilog testbench, we know that our testbench is module and also DUT is module So, we can directly instantiate the interface in top module and we can connect with DUT and testbench and we know that module is static in nature and interface is also static in nature. So, we can use interface in modules.

But we come to system Verilog, we have to oops concept. We will use the class-based test-bench in the system Verilog. In order to use the interface in the class based testbench. **We know class is the dynamic but interface is static. So, we can't use interface directly for that we need the virtual keyword so by using virtual keyword we can use the interface inside the class.**

Interface is static and module is static so we can directly use the interface. But when we come to the class based testbench there we have to use virtual keyword in order to access the interface

- A virtual interface is a variable that represents an interface instance.
- A virtual interface must be initialized before it can be used, it has the value null before it is initialized.
- Virtual interface can not be used as a ports, interface items or as a members of unions.

In System Verilog, a virtual interface is a pointer or a variable that represents a connection to an actual interface instance. It allows a class to access signals and methods within a design's interface, enabling the use of object-oriented programming to connect and control hardware blocks.

- An interface represents signals that are used to connect design modules or testbench to the DUT and commonly known as a physical interface. The design and physical interface are static in nature. Hence, they can't be used as a directly.
- In modern testbench, randomized class objects are used and connect to the design dynamically. Hence, to bridge the gap between the static world of modules and the

dynamic world of objects, a **virtual interface** is used as a pointer or handle for an actual interface.

Feature of the virtual interface:

- ✓ The virtual interface must be pointed to the actual or physical interface. This is also known as virtual interface initialization.
- ✓ Before accessing or driving to the virtual interface, it must be initialized otherwise it will cause a fatal run-time error as it has a null value.
- ✓ The virtual interface can be passed to the functions and tasks as a argument.
- ✓ The dynamic object can be assigned to the virtual interface. Also, interface value is recorded in the dynamic object by accessing the virtual interface.
- ✓ They can be assigned to another actual interface having the same type using '=' operator.

Example:

```
//Mux 2x1
interface mult_if(input bit clk);
    logic en;
   logic[7:0]a,b;
   logic [15:0] out;
   logic ack;
endinterface

module mult_dut(
    input bit clk,
   input logic en,
   input logic [7:0] a,b,
   output logic [15:0] out,
   output logic ack);

always @(posedge clk) begin
   if (en) begin
```

```
            out <= a * b;
            ack <= 1;
        end
        else begin
            ack <= 0;
        end
end
endmodule

program test(mult_if inf);

        class driver;
       virtual mult_if inf;

        function new(virtual mult_if inf);
            this.inf = inf;
        endfunction

        task run();
            #5;
            inf.a = 'd5;
          inf.b = 'd6;
          inf.en = 1;
            @(posedge inf.ack)

            $display("%0t:a=%0d b=%0d out=%0d",$time,inf.a,inf.b,inf.out);
            inf.en=0;
```

```
            #20;
        inf.a = 'd20;
        inf.b = 'd7;
        inf.en = 1;
            @(posedge inf.ack)
        $display("%0t: a=%0d b=%0d out=%0d",$time,inf.a,inf.b,inf.out);

            inf.en=0;
        #20;
        inf.a = 'd10;
        inf.b = 'd4;
        inf.en = 1;
            @(posedge inf.ack)
        $display("%0t: a=%0d b=%0d out=%0d",$time,inf.a,inf.b,inf.out);
            inf.en=0;

        #10;
        $finish;
      endtask
endclass

initial begin
    driver drv = new(inf);
    drv.run();
end
```

```
endprogram

module tb;
bit clk = 0;
always #5 clk = ~clk;

mult_if inf(clk);

mult_dut dut(.clk(clk),.en(inf.en),.a(inf.a),.b(inf.b),.out(inf.out),.ack(inf.ack));

test t1(inf);
endmodule
```

Output:

```
15:a=5 b=6 out=30
45: a=20 b=7 out=140
75: a=10 b=4 out=40
$finish called from file "example.sv", line 63.
$finish at simulation time                   85
```

Explanation:

In the above code, the **ack** (acknowledge) signal is used for handshaking between the driver (testbench) and the DUT.

Simple Explanation

When the driver sends inputs (`a`, `b`, `en`) to the DUT, it must wait until the DUT finishes the calculation.

The DUT sets ack = 1 to indicate:

➞ "The output is ready. You can read it now."

Then the driver uses:

wait(inf.ack); →to wait until the DUT has finished the multiplication.

- Why ack is needed?
- Without ack, the driver doesn't know when the DUT output (`out`) is valid.
- Prevents reading output too early
- Synchronizes driver and DUT
- Ensures stable, correct output

In one line:

ack = Signal that tells the driver that the DUT has completed the operation and the output is valid.

Use of the constructor in the above code

The constructor:

```
function new(virtual mult_if inf);
   this.inf = inf;   // virtual interface assignment
endfunction
```

Use:

The constructor is used to pass the virtual interface from the testbench to the driver class.

- Why do we use it?

We use the constructor because a class cannot directly access hardware signals of an interface.

So, we give the interface **handle** (`inf`) to the class using the constructor.

This allows the class to access:

* `inf.a`

* `inf.b`

* `inf.en`

* `inf.ack`

* `inf.out`

- In one line:

The constructor connects the driver class to the DUT signals through a virtual interface.

- test t1(inf);

This line creates an object of the class `test` and passes the interface `inf` to it*.

- In one sentence:
- **It means: “Make a test object named t1 and connect it to interface inf.”**

So now the test class can drive signals and read signals through inf.

- You have an interface called simple_bus, and inside it you define signals and modports.
- A modport tells a module which signals are inputs and which are outputs with respect to that module.
- Different modules looking at the same interface, but each sees the signals in its own direction.”

Why do we need modports?

Because CPU and Memory modules use the same bus signals in opposite directions.

Example:

* CPU sends `req`

* Memory receives `req`

* Memory sends `gnt`

* CPU receives `gnt`

So we tell SystemVerilog:

📌 CPU modport → req is output, gnt is input

📌 Memory modport→ req is input, gnt is output

Explanation of the code

interface simple_bus(input bit clock);

logic req, gnt;

logic start, mode;

logic [7:0] addr;

wire [7:0] data;

- modport mem_mp

Used by memory module.

modport mem_mp (input req, start, mode, addr, output gnt, inout data);

Meaning:

Memory receives: req, start, mode, addr

Memory sends: gnt

Memory reads/writes: data

- modport cpu_mp

Used by CPU module.

modport cpu_mp (output req, start, mode, addr,input gnt,inout data);

Meaning:

- CPU sends: req, start, mode, addr
- CPU receives: gnt
- CPU reads/writes: data

Simple one-line explanation

Modports define who is driving and who is receiving each signal when two modules share the same interface.

We have:

- An interface → `simple_bus`
- Two modules → `cpu_mod` and `mem_mod`
- A top module that connects everything

1.0 Step-by-Step Explanation (Super Simple)

- Interface instance created in top module

simple_bus bus(clk);

This means:

- We create one common bus called `bus`
- CPU and Memory will share this bus

2.0 CPU module connects using cpu_mp modport

module cpu_mod(simple_bus.cpu_mp bus);

Meaning:

👉 CPU sees the bus through cpu_mp modport

👉 So CPU drives req, start, mode, addr

👉 CPU receives gnt

👉 Directions already defined by modport

3.0 Memory module connects using mem_mp modport

module mem_mod(simple_bus.mem_mp bus);

Meaning:

👉 Memory sees the bus through mem_mp modport

👉 Memory receives req, start, mode, addr

👉 Memory drives gnt

👉 Directions defined by modport

4.0 Top module connects both to same interface

mem_mod mem(bus);

cpu_mod cpu(bus);

Meaning:

👉 CPU and Memory both **connect** to the **same bus** interface instance

👉 But each uses its own modport direction

👉 So signals don’t conflict

One-Line Simple Explanation

- CPU and Memory use the same interface, but modports decide the signal directions, and the top module connects both to the same bus.

Virtual interface

A virtual interface is a pointer to a real interface.

- In testbench, classes cannot directly connect to signals/wires.
- But your DUT signals (clk, rst, data, etc.) are inside an interface, which is outside the class.
- Virtual interface = bridge between class-based TB and interface signals

Why do we use Virtual Interface?

Because classes cannot contain wires or connect directly to modules.

So virtual interface allows:

- Classes to drive DUT signals

- Classes to sample DUT signals
- Avoiding global variables
- Clean connection between TB and DUT
- Used heavily in UVM driver, monitor, sequencer

Simple Diagram

class driver ----- virtual interface -----> interface -----> DUT

☑ Step-by-Step Example

1 Create an interface

```
interface intf(input logic clk);
   logic rst;
   logic d;
   logic q;
endinterface
```

2 DUT – simple D flip-flop

```
module dff(input clk, rst, d, output logic q);
   always @(posedge clk) begin
      if (!rst)
         q <= 0;
      else
         q <= d;
   end
endmodule
```

3 Create a class driver (cannot directly connect signals)

```
class driver;
```

```
    virtual intf vif;      // virtual interface declaration

    function new(virtual intf vif);
        this.vif = vif;    // store the interface handle
    endfunction

    task run();
        $display("Driver: Driving signals");

        vif.rst = 1;
        vif.d   = 0;
        #10;

        vif.rst = 0;
        vif.d   = 1;
        #10;

        vif.d   = 0;
        #10;

        vif.d   = 1;
        #10;
    endtask
endclass
```

4 Top testbench – connect everything

```
module tb;
    logic clk = 0;
    always #5 clk = ~clk;
    intf inf(clk);

    dff dut(.clk(clk), .rst(inf.rst), .d(inf.d), .q(inf.q));

    driver drv;     // class object

    initial begin
        drv = new(inf);       // pass interface to class (IMPORTANT)
        drv.run();            // call task
        #100 $finish;
    end
endmodule
```

☑ What is happening internally?

- You created a normal interface → `intf inf(clk)`
- You created virtual interface variable in class → `virtual intf vif;`
- You passed the real interface to driver → `drv = new(inf)`

* Now class can do:

```
vif.rst = 1;
vif.d = 0;
```

which drives DUT signals.

Final Simple Explanation

| Term | Meaning |

| ------------------------------ | -- |

| interface | Actual signals used by DUT |

| virtual interface | Pointer/handle to interface used inside classes |

| Why needed? | **Classes cannot contain wires** → need virtual interface to drive DUT |

One-Line Definition (Remember for Interview)

Virtual interface is a handle that allows class-based testbench components (driver/monitor) to access interface signal.

Clocking Block

- Verilog primarily supported communication between different blocks through module ports.
- SV adds interface to encapsulate communication between different blocks thereby enabling users to easily change level of abstraction.
- An interface specifies signals or nets through which a test bench communicate with a DUT (device/design under test).
- An interface does not specify any timing disciplines, synchronization requirements, or clocking paradigms.
- SV adds the clocking block that identifies the clock signals, captures the timing and synchronization requirements of the blocks being modelled.
- Clocking block are key elements in cycle-based simulation.
- They assemble signals synchronous to a particular clock and make their timing explicit.
- The test can be defined in terms of cycles and transaction rather than signals or transition time.
- They separate the timing and synchronization details from structural, functional and procedural elements of a testbench.

The timing between the testbench and the design should be maintained.

- Ensuring driving and receiving synchronous signal at the proper time in relation to the clock.
- Driving a signal too late or sampling it too early can cause the test-bench to be off a cycle.

- To ensure proper sampling and signaling the testbench should be separate from the design.
- The **input should be just driven before the clock edge** and **output should be sampled just after the clock edge.**

Controlling timing for synchronous signals using clocking blocks:

- Clocking block are mainly used by test-benches.
- An interface may contain a clocking block to specify the timing of synchronous signals relative to the clock, may contain multiple clocking block.
- Any signal in the clocking block is driven or sampled synchronous, ensuring that the test-benches interacts with the signals at the right time.
- When you are using an interface with a clocking block.
- There is a 1-cycle delay from the DUT output to testbench input. A Virtual synchronizer added to the TB input.

Clocking blocks controlling timing of synchronous signals using clocking blocks

- When you are using an interface with a clocking block.
- No delay from TB output to DUT input i.e., the default used with a clocking block is –

Default input #1step output #0;

- Timing may be specified without any signals declarations as shown above.
- The timing for sampling and driving clocking block signal is implicit and relative to the clocking block's clock.
- This enables a set of key operations synchronous events, input sampling, synchronous driver to be written without explicitly using clocks or specified timing.

- Clocking block can only be declared inside a module, interface or program, cant be nested, usually used in interface, just for TB.
- Clocking block is both a declaration and an instances of itself i.e. there is no need to declare instances for a clocking block.
- The same signals – clock, input, inout, or outputs can appear in more than one clocking block.
- Clocking blocks that share the same clock, share the same synchronization event,in the same way as several latches can be controlled by the same clock.
- **Signal direction in the clocking block within the testbench are with respect to the testbench and not under the design under test.**

Question

```
//
module dff(input clk,rst,d, output reg q);
always @(posedge clk) begin
        if (!rst)
                q <=0;
        else
                q <=d;
end
endmodule

interface intf(input clk);
        logic rst,d;
        logic q;

        clocking cb @(posedge clk);
                default input #15ns output #20ns;
                input q;
                output rst,d;
        endclocking
endinterface

module tb_dff();
bit clk;
intf inf(clk);

dff dut(.clk(clk),.rst(inf.rst),.d(inf.d),.q(inf.q));
```

```
always #5 clk=~clk;


initial begin
        $monitor("for Time=%0t rst=%0b d=%0b q=%0b",$time,inf.rst,inf.d,inf.q);


        inf.cb.rst <=0; inf.cb.d <=0;
#35;
inf.cb.rst <= 1;
inf.cb.d <= 1;
#35;
inf.cb.d <= 0;
#35;
inf.cb.d <= 1;
#35;
#50 $finish();
end
endmodule
```

Simulation output:

```
for Time=0 rst=x d=x q=x
for Time=5 rst=0 d=0 q=x
for Time=15 rst=0 d=0 q=0
for Time=35 rst=1 d=1 q=0
for Time=45 rst=1 d=1 q=1
for Time=75 rst=1 d=0 q=1
for Time=85 rst=1 d=0 q=0
for Time=105 rst=1 d=1 q=0
```

for Time=115 rst=1 d=1 q=1

$finish called from file "example.sv", line 44.

$finish at simulation time 190

Interface

An interface in System Verilog is used to **group related signals (wires) together.**

It helps to simplify connections between modules, testbenches, and DUT (Design Under Test).

- Instead of connecting many wires one by one, you connect one interface — making your design clean, readable, and less error-prone.

Example Without Interface

```
module adder (input logic [3:0] a, b, output logic [4:0] sum);
  assign sum = a + b;
endmodule


module tb;
  logic [3:0] a, b;
  logic [4:0] sum;

  adder dut(.a(a), .b(b), .sum(sum));

  initial begin
    a = 4; b = 5;
    #5 $display("Sum = %0d", sum);
  end
endmodule
```

Here, we connect each signal manually (a, b, sum).

If there are 10+ signals, it becomes messy.

Same Example Using Interface

```
// Define an interface
interface adder_if;
  logic [3:0] a, b;
  logic [4:0] sum;
endinterface

// DUT uses interface
module adder(adder_if intf);
  assign intf.sum = intf.a + intf.b;
endmodule

// Testbench
module tb;
  adder_if intf();   // Create interface instance

  adder dut(intf);   // Connect DUT with interface

  initial begin
    intf.a = 4;
    intf.b = 5;
    #5 $display("for a=%0d b=%0d and Sum = %0d", intf.a, intf.b, intf.sum);
  end
```

endmodule

Output simulation

for a=4 b=5 and Sum = 9

V C S S i m u l a t i o n R e p o r t

Time: 5

Key Benefits

1. ☑ Groups multiple signals into one connection
2. ☑ Makes testbench connections simpler
3. ☑ Allows using clocking blocks, modports, etc.
4. ☑ Reduces wiring errors

In short:

- Interface = **bundle of signals** that simplifies communication between design and testbench.

Questions

Let’s write the D Flip-Flop example

Step 1: Define Interface

We’ll group all related signals (like `clk`, `d`, and `q`) into one interface.

```
interface dff_if;
  logic clk;
  logic d;
  logic q;
endinterface
```

- This interface now acts like a **bundle of wires** that carry the DFF signals.

Step 2: Design the D Flip-Flop (DUT)

```
module dff(dff_if intf);
  always_ff @(posedge intf.clk)
    intf.q <= intf.d;
endmodule
```

- **The flip-flop samples input `d` at the rising edge of clock (posedge clk) and stores it into output q.**

Step 3: Write the Testbench

```
module tb;
  dff_if intf();  // Create interface instance

  dff dut(intf);  // Connect DUT with interface

  // Generate clock
  initial begin
    intf.clk = 0;
    forever #5 intf.clk = ~intf.clk;  // 10ns clock period
  end

  // Apply test stimulus
  initial begin
    intf.d = 0;
    #10 intf.d = 1;   // Apply input = 1
    #10 intf.d = 0;   // Apply input = 0
    #10 intf.d = 1;   // Apply input = 1
    #20 $finish;
```

```
  end
  // Monitor signals
  initial
    $monitor("Time=%0t | D=%b | Q=%b", $time, intf.d, intf.q);
endmodule
```

How It Works

Time	D	Q
0ns	0	0
10ns	1	0 → 1 (on rising edge)
20ns	0	1 → 0
30ns	1	0 → 1

Advantages of Using Interface

- You connect the whole DFF (clk, d, q) using just one interface handle.
- Makes your code clean, modular, and easy to extend (for bigger testbenches).

Modport

A modports **defines the direction of signals** (input/output) for different modules that use the same interface.

In other words:

- An interface groups signals.
- A modport tells who can read or write those signals.

Example Without Modport

Let’s take an interface for a simple design:

```
interface simple_if;
  logic clk;
  logic [3:0] a, b;
  logic [4:0] sum;
endinterface
```

Now, both the DUT and Testbench can access and modify any signal directly.

- That can cause confusion (both trying to drive the same signal).

Example With Modports

Let’s control signal directions using modports-

```
interface adder_if;
  logic [3:0] a, b;
  logic [4:0] sum;

  // Define modports
  modport dut_mp (input a, b, output sum);
  modport tb_mp  (output a, b, input sum);
endinterface
```

- Design (DUT)

```
module adder(adder_if.dut_mp intf);
  assign intf.sum = intf.a + intf.b;
endmodule
```

Testbench

```
module tb;
```

```
  adder_if intf();   // Create interface instance

  adder dut(intf);   // Connect DUT using interface

  initial begin
    intf.a = 4;
    intf.b = 5;
    #5 $display("Sum = %0d", intf.sum);
  end
endmodule
```

Why Modports Are Useful

- They define signal directions (input/output/inout).
- They prevent accidental signal driving from the wrong side.
- They make testbench communication clear and structured.

In short:

- Interface = **bundle of signals**
- Modport = **defines which side (DUT/testbench) can read or write signals**

Clocking Block

A clocking block is used to **synchronize signal** sampling and driving with a clock edge.

It helps **avoid race conditions** between the **testbench and design (DUT).**

Why Do We Need It?

In simulation, both DUT and Testbench may drive or read signals at the same time — which **causes race conditions**.

- Clocking block ensures:
  - Inputs (from DUT) are sampled slightly after the clock edge.
  - Outputs (to DUT) are driven slightly before the next clock edge.
  - So, everything happens in an **orderly** and **timed** way.

Syntax

```
clocking <name> @(posedge clk);
  default input #1step output #2;
  input  signal1, signal2;
  output signal3;
endclocking
```

Simple Example

- Interface with Clocking Block

```
interface clk_if (input logic clk);
  logic [3:0] data;
  logic ready;

  // Define a clocking block synchronized to posedge clk
  clocking cb @(posedge clk);
    input ready;    // sample this signal after clock edge
    output data;    // drive this signal before clock edge
  endclocking
endinterface
```

Testbench

```
module tb;
  logic clk;
  clk_if intf(clk);

  // Clock generation
```

```
  initial clk = 0;
  always #5 clk = ~clk;

  initial begin
    intf.cb.data <= 4'b1010;  // driven via clocking block
    #10;
    $display("Ready = %0b", intf.cb.ready);  // sampled via clocking block
  end
endmodule
```

Key Points

Concept	Description
`clocking cb @(posedge clk);`	Defines the clock event for synchronization
`input`	Signals sampled from DUT
`output`	Signals driven to DUT
`#1step`, `#2`	Optional timing skew to avoid race conditions
`cb.signal`	Access signal through clocking block

In short:

- A clocking block is like a “time controller” inside an interface —
- it ensures signals are read and written at the right clock edge, **avoiding race conditions.**

Interface + Modport + Clocking Block

We’ll take a small 2×1 MUX design as an example to see how it all fits together

- **Step 1:** Interface (with modport + clocking block)

```
interface mux_if (input logic clk);
  logic [1:0] sel;
  logic a, b;
  logic y;

  // 🕒 Clocking block for synchronization
  clocking cb @(posedge clk);
    output sel, a, b;  // testbench drives these
    input y;           // testbench samples this
  endclocking

  // 🎯 Modports for direction control
  modport dut_mp (input sel, a, b, output y); // for DUT
  modport tb_mp  (clocking cb);               // for testbench
endinterface
```

- **Step 2:** DUT (Design Under Test)

```
module mux2x1 (mux_if.dut_mp intf);
  always_comb begin
    case (intf.sel)
      0: intf.y = intf.a;
      1: intf.y = intf.b;
    endcase
  end
endmodule
```

- **Step 3:** Testbench

```
module tb;
  logic clk;
  mux_if intf(clk); // interface instance

  // Connect DUT
  mux2x1 dut(intf);
  // Clock generation
  initial clk = 0;
  always #5 clk = ~clk;
  // Test process using clocking block
  initial begin
    intf.cb.sel <= 0; intf.cb.a <= 1; intf.cb.b <= 0;
    #10 $display("sel=%0d, y=%0d", intf.cb.sel, intf.cb.y);

    intf.cb.sel <= 1; intf.cb.a <= 1; intf.cb.b <= 1;
    #10 $display("sel=%0d, y=%0d", intf.cb.sel, intf.cb.y);
    $finish;
  end
endmodule
```

Step-by-Step Explanation

Step	Component	Description
1	Interface	Groups signals `a, b, sel, y` and defines how DUT and TB interact.
2	Clocking block	Synchronizes driving/sampling of signals at `posedge clk`.
3	Modport	Specifies direction: TB drives inputs, DUT drives outputs.

| 4 | DUT | Implements logic using signals from the interface. |

| 5 | Testbench | Drives inputs (`a, b, sel`) and reads output (`y`) via clocking block. |

Simulation Output Example

sel=0, y=1

sel=1, y=1

In short:

- **Interface groups signals**
- **Modport defines direction for DUT/TB**
- **Clocking block handles timing and avoids race conditions**

Program Block

In System Verilog, a program block is a special type of block used in testbenches to make sure that **testbench code runs after the design (DUT) in simulation.**

It helps to **avoid race conditions between the testbench and the design.**

Why do we need it

In Verilog, both testbench and design use procedural blocks like **`initial` or `always`**.

They might execute at the same simulation time, which can **cause race conditions** (unpredictable results).

- The **program block solves this problem** —

it executes after all design `always` or `assign` blocks finish in the current simulation time step.

Syntax

```
program program_name;
  // testbench code here
```

endprogram

- You can also connect a program to an interface for signal access:

```
program program_name (interface_name intf);
  // code using intf signals
endprogram
```

Simple Example

D flip-flop DUT.

DUT (Design)

```
module dff(input logic clk, d, output logic q);
  always_ff @(posedge clk)
    q <= d;
endmodule
```

Interface

```
interface dff_if;
  logic clk;
  logic d;
  logic q;
endinterface
```

Program Block (Testbench)

```
program test (dff_if intf);
  initial begin
    intf.d = 0;
```

```
    #5 intf.d = 1;
    #10 intf.d = 0;
    #10 $finish;
  end
endprogram
```

Top Module (Connecting Everything)

```
module top;
  dff_if intf();       // create interface instance
  dff dut(.clk(intf.clk), .d(intf.d), .q(intf.q));  // connect DUT
  test tb(intf);       // connect program block to interface

  // Clock generation
  initial begin
    intf.clk = 0;
    forever #5 intf.clk = ~intf.clk;
  end
endmodule
```

- **How It Works**

Simulation Order	Event
1️⃣	`always_ff` in DUT executes (design phase)
2️⃣	`program` block executes (testbench phase)
☑️	This ensures no race condition between DUT and TB

Key Points

Feature	Description
🔸 Purpose	Used for writing testbench code
🔸 Execution order	Runs after design blocks
🔸 Avoids	Race conditions
🔸 Can connect	Through interfaces
🔸 Common use	Verification environments (stimulus, checking, etc.)

In short:

- `module` = Design logic
- `program` = Testbench logic (runs later to avoid races)

Casting:

Casting means **converting one data type into another data type.**

System Verilog has two types of casting:

1. Static Casting → compile-time conversion

2. Dynamic Casting → run-time conversion (mainly for classes)

1. STATIC CASTING (Most common)

➞ done using type'(expression)

➞ Converts the value to the specified type immediately.

Example 1 — Convert int to byte

```
int a = 300;
byte b;

b = byte'(a); // cast int → byte
$display ("b = %0d", b);
```

- A byte can store only −128 to +127
- So 300 overflows → result becomes 44

Example 2 — Bit-vector casting

```
logic [7:0] x;
int y = 255;

x = logic'(y); // cast int → 8-bit logic
$display ("x = %0d", x);
```

Example 3 — Enum casting

```
enum {RED=0, GREEN=1, BLUE=2} color;

color = color'(1);  // integer → enum
$display ("color = %0d", color);
```

Output:

color = 1 (GREEN)

Why Static Casting is Useful?

✓ Prevents accidental mismatches

✓ Helps avoid warnings

✓ Converts between:

* int ↔ byte

* real ↔ int

* logic vectors ↔ int

* enum ↔ int

* class handles (up/down casting)

2. DYNAMIC CASTING (for classes)

Used in OOP.

Syntax:

```
child_class c;
parent_class p;

if (!$cast(c, p))
  $display("Cast failed");
```

➞ Checks at runtime whether casting is valid

➞ Prevents invalid pointer conversions

SIMPLE EXAMPLE OF DYNAMIC CASTING

```
class A;
endclass

class B extends A;
endclass

A a;
B b = new();

a = b;       // OK (upcasting)
```

```
if ($cast(b, a))   // try downcasting
   $display("Cast success");
else
   $display("Cast failed");
```

SUPER SIMPLE SUMMARY

Type of Cast	Syntax	When used
Static Cast	`type'(value)`	numbers, enums, bit vectors
Dynamic Cast	$cast(new_type, old_type)	classes (runtime check)

Final Example (Very Simple)

```
int num = 65;
byte b;

b = byte'(num);  // Static cast
$display("b = %0d", b);
```

Output:

```
b = 65
```

There are two types of casting in SV – static and dynamic casting.

Static casting – at compile time: divided into 3 types.

<type>`(<expression>)

<type> should not be variable

Example1:

```
longint a,y;
real r;
y= a + longint`(r**3);
```

Example2:

```
logic [15:0] a, b, c, sum;
logic carry;
sum =a+ 16`(5);
{carry, sum}= 17`(a+3);
Sum = a + 16` (b-2)/c;
```

Example3:

```
Sum = signed`(a) + signed`(a);
If (unsigned`(a-b) <=5)
…….
```

Dynamic casting – at run time

- Use $cast system function - $cast (destination variable, source variable);

Example:

```
longint a,y;
always @(posedge clk)
$cast(data, 3.154*2);
```

- Array vs structure – object must be in identical in size.
- An aggregate bit level object can be reshaped using casting.
- Packed ←→ unpacked, Array ←→ Structure

Example:

```
typedef struct {
```

```
bit [7:0] f1;
bit [7:0] f2;
bit [7:0] f3 [0:5];} unpacked_s;

typedef struct packed {
bit [0:2][15:0] f1;
bit [15:0] f2;} packed_s;
……
A = unpacked_s`(B);
B = packed_s`(A);
```

Complete example of above:

- Declaring packed and unpacked structs
- Casting unpacked → packed and packed → unpacked
- Printing the output values

1. FULL WORKING CODE

```
module struct_cast_example;
  // Unpacked Struct
  // ------------------------
  typedef struct {
    bit [7:0] f1;          // 8-bit
    bit [7:0] f2;          // 8-bit
    bit [7:0] f3 [0:5];    // unpacked array (6 elements)
  } unpacked_s;

  // Packed Struct
  typedef struct packed {
```

```
  bit [0:2][15:0] f1;      // 3 × 16-bit = 48 bits
  bit [15:0]       f2;      // 16 bits
} packed_s;

unpacked_s   A;
packed_s    B;

initial begin
  // Initialize packed struct B
  B.f1[0] = 16'hAAAA;
  B.f1[1] = 16'hBBBB;
  B.f1[2] = 16'hCCCC;

  B.f2 = 16'hDDDD;

  // CAST packed_s → unpacked_s
  A = unpacked_s'(B);

  $display("---- After casting B → A (packed → unpacked) ----");
  $display("A.f1 = %0h", A.f1);
  $display("A.f2 = %0h", A.f2);

  foreach (A.f3[i])
    $display("A.f3[%0d] = %0h", i, A.f3[i]);

  // Now modify unpacked struct A
  A.f1 = 8'h11;
```

```
    A.f2 = 8'h22;
    A.f3[0] = 8'h33;
    A.f3[1] = 8'h44;
    A.f3[2] = 8'h55;
    A.f3[3] = 8'h66;
    A.f3[4] = 8'h77;
    A.f3[5] = 8'h88;

    // ---------------------------------
    // CAST unpacked_s → packed_s
    // ---------------------------------
    B = packed_s'(A);

    $display("\n---- After casting A → B (unpacked → packed) ----");
    $display("B.f1[0] = %0h", B.f1[0]);
    $display("B.f1[1] = %0h", B.f1[1]);
    $display("B.f1[2] = %0h", B.f1[2]);
    $display("B.f2    = %0h", B.f2);

    $finish;
  end
endmodule
```

2. EXPECTED OUTPUT (Explained)

Output 1: After Casting B → A

(B was initialized manually)

---- After casting B → A (packed → unpacked) ----

A.f1 = AA

A.f2 = AA

A.f3[0] = AA

A.f3[1] = AA

A.f3[2] = BB

A.f3[3] = BB

A.f3[4] = CC

A.f3[5] = CC

Reason:

- The packed struct is flattened then assigned into the unpacked struct fields.
- 48+16 = 64 bits → same size as unpacked_s total bits.

Output 2: After Casting A → B

(A was filled with 1-byte values)

---- After casting A → B (unpacked → packed) ----

B.f1[0] = 1122

B.f1[1] = 3344

B.f1[2] = 5566

B.f2 = 7788

Explanation: The bytes in A get packed in order into the packed struct bits.

SIMPLE SUMMARY

- casting `(B → A)`

Packed fields flatten → fill unpacked fields.

- casting `(A → B)`

Unpacked bytes get packed tightly into the 64-bit packed struct.

Session-8 OOPS: -

OOPS: -

- Class
- Constructor- new()
- Class Object Creation
- Class Object Creation
- Array as Object
- Class Object Shallow Copy
- Class Object Deep Copy

Class: -

- Class is a user-defined data types that serves as a blueprint for creating an object.
- Class is in dynamic in nature, so for memory allocation we need to use **new** keyword.
- Class consists of properties, methods and constraints.
- Having memory address-oriented data.
- **Syntax:** class class_name;
  //properties
  // methods
  // constrains
  endclass
-

- **Class**- blueprint of a house.
- **Object**- A complete house → hardware
- **Handle**- memory location printout (pointer) or (handler)
- **Methods**- features- like- turn ON/OFF switches.
- **Properties**- Light switches (characteristics) → which type of hardware is present.

Class → properties → handle → object → methods

Object →← handle

- OOPS – Constructor- new() → in build constrctor → used to handle object create.

Pkt_a =new(); Objects creates using **new** constructors keyword.

Pkt_b = new();

Pkt → handle → **created address of house**

Pkt = new(20,18); // **memory overrides and updates**

Pkt =new(32,18);

- null → **delete / reset the memory / memory free.**

- Int and interger are 32-bit signed and 2-state and 4-state data type.

```
class packet;
int unsigned addr;
int data;

function new(int address, data_val);
    addr=address;
    data=data_val;
endfunction
endclass

module m;

packet pkt;
initial begin
    pkt = new(32, 15);
```

```
    $display("addr=%0d,data=%0d",pkt.addr,pkt.data);
end
endmodule
```

Simulation output:

```
addr=32,data=15
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

```
class packet;
int unsigned addr;
int data;

function new(int address, data_val);
    addr=address;
    data=data_val;
endfunction
endclass

module m;
packet pkt;

initial begin
    pkt = new(32, 15);
    $display("addr=%0d,data=%0d",pkt.addr,pkt.data);
    pkt = new(16, 20);
    $display("addr=%0d,data=%0d",pkt.addr,pkt.data);
    pkt = null;
```

```
    $display("addr=%0d,data=%0d",pkt.addr,pkt.data);
end
endmodule
```

Simulation Output:

```
addr=32,data=15
addr=16,data=20
Error-[NOA] Null object access
example.sv, 22
  The object at dereference depth 0 is being used before it was
  constructed/allocated.
  Please make sure that the object is allocated before using it.

  #0 in m at example.sv:22
  #1 in m
          V C S   S i m u l a t i o n   R e p o r t
Time: 0

class transaction;
    bit [31:0] data;
    int id;
endclass
module class_example;

transaction tr[5];
initial begin
    foreach(tr[i]) begin
        tr[i]=new();
```

```
        tr[i].data=i*i;
        tr[i].id=i+1;
    end

    foreach(tr[i])
        $display("tr[%0d].data=%0d,tr[%0d].id=%0d",i,tr[i].data,i,tr[i].id);
end
endmodule
```

Simulation Output:

```
tr[0].data=0,tr[0].id=1
tr[1].data=1,tr[1].id=2
tr[2].data=4,tr[2].id=3
tr[3].data=9,tr[3].id=4
tr[4].data=16,tr[4].id=5
        V C S  S i m u l a t i o n  R e p o r t
Time: 0

class transaction;
    bit [31:0] data;
    int id;
endclass

module class_example;

transaction tr[];

initial begin
```

```
    tr =new[5];
    foreach(tr[i]) begin
        tr[i]=new();
        tr[i].data=i*i;
        tr[i].id=i+1;
    end

    foreach(tr[i])
        $display("tr[%0d].data=%0d,tr[%0d].id=%0d",i,tr[i].data,i,tr[i].id);
end
endmodule
```

Simulation time:

```
tr[0].data=0,tr[0].id=1
tr[1].data=1,tr[1].id=2
tr[2].data=4,tr[2].id=3
tr[3].data=9,tr[3].id=4
tr[4].data=16,tr[4].id=5
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

OOPS an array of objects

Associate array of ojects

```
typedef enum {TRANS,RECEIVER,REPEATER} tr_type;
class transaction;
    bit [31:0] data;
    int id;
```

```
endclass

module class_example;
transaction tr[tr_type];
initial begin
    tr[TRANS] =new();
    tr[RECEIVER]=new();
    tr[REPEATER]=new();

    foreach(tr[i]) begin
        tr[i].data=i*i;
        tr[i].id=i+1;
    end
    foreach(tr[i])
        $display("tr[%0s].data=%0d,tr[%0s].id=%0d",i.name,tr[i].data,i.name,tr[i].id);
end
endmodule
```

Simulation Output:

```
tr[TRANS].data=0,tr[TRANS].id=1
tr[RECEIVER].data=1,tr[RECEIVER].id=2
tr[REPEATER].data=4,tr[REPEATER].id=3
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

By using the Dynamic function

```
module dynamic_array;
int data[];
```

```
int id[];

initial begin
    int size = 5;
    data = new[size];
    id   = new[size];

    foreach(data[i])begin
        data[i] = i * i;
        id[i]   = i + 1;
    end

    foreach(data[i])begin
        $display("tr[%0d].data = %0d, tr[%0d].id = %0d",i,data[i],i,id[i]);
    end
end
endmodule
```

Simulation Output:

```
tr[0].data = 0, tr[0].id = 1
tr[1].data = 1, tr[1].id = 2
tr[2].data = 4, tr[2].id = 3
tr[3].data = 9, tr[3].id = 4
tr[4].data = 16, tr[4].id = 5
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

OOPS – static properties and methods

```
class transaction;
```

```
    static int s_id;  //create a single memory address//update on same memory location
    int id;               // create a multiple memory address

    function new();
        s_id++;
        id++;
    endfunction
endclass

module class_example;

transaction tr[5];

initial begin
    foreach(tr[i])begin
        tr[i] =new();
        $display("value of s_id=%0h,id=%0h",tr[i].s_id,tr[i].id);
    end
end
endmodule
```

Simulation output:

```
value of s_id=1,id=1
value of s_id=2,id=1
value of s_id=3,id=1
value of s_id=4,id=1
value of s_id=5,id=1
```

V C S S i m u l a t i o n R e p o r t

Time: 0

- Astatic function can not access non-static (object) variables like id.
- Increments only static s_id

OOPS – STATIC METHODS EXAMPLES

```
class transaction;
        static int s_id;
        int id;

        static function void incr_s_id();
        s_id++;
        id++;
endfunction

function void incr_id();
        s_id++;
        id++;
endfunction
endclass

module class_example;
transaction tr[5], tr_new;

initial begin
```

```
        foreach(tr[i])begin
                tr[i] =new();
                tr[i].incr_s_id();
                $display("On calling incr_s_id: value of
s_id=%0h,id=%0h",tr[i].s_id,tr[i].id);
        end
        tr[0].s_id=0;
        foreach(tr[i])begin
                tr[i].incr_id();
                $display("On calling incr_id:value of s_id=%0h, id=%0h",tr[i].s_id,tr[i].id);
        end
end
endmodule
```

Simulation output:

```
Error-[SV-AMC] Non-static member access
example.sv, 8
$unit, "id"
  Illegal access of non-static member 'id' from static method
  'transaction::incr_s_id'.
1 warning
1 error
CPU time: .175 seconds to compile
```

After ignoring the statement in static function in id++

Simulation Output:

```
On calling incr_s_id: value of s_id=1,id=0
On calling incr_s_id: value of s_id=2,id=0
On calling incr_s_id: value of s_id=3,id=0
```

On calling incr_s_id: value of s_id=4,id=0

On calling incr_s_id: value of s_id=5,id=0

On calling incr_id:value of s_id=1, id=1

On calling incr_id:value of s_id=2, id=1

On calling incr_id:value of s_id=3, id=1

On calling incr_id:value of s_id=4, id=1

On calling incr_id:value of s_id=5, id=1

V C S S i m u l a t i o n R e p o r t

Time: 0

OOPS – EXAMPLE WITHOUT OBJECT CREATION

```
//
class transaction;
    static int s_id;
    int id;

    static function void incr_s_id();
    s_id++;
    //id++;
endfunction

function void incr_id();
    s_id++;
    id++;
endfunction
endclass

module class_example;
```

```
transaction tr[5];
initial begin
    transaction::incr_s_id();
    $display("static calling without object: s_id=%0d",transaction::s_id);

    foreach(tr[i])begin
        tr[i]=new();
        tr[i].incr_id();
        $display("After incr_id:value of s_id=%0d, id=%0d",tr[i].s_id,tr[i].id);
    end
end
endmodule
```

Simulation Output:

```
static calling without object: s_id=1
After incr_id:value of s_id=2, id=1
After incr_id:value of s_id=3, id=1
After incr_id:value of s_id=4, id=1
After incr_id:value of s_id=5, id=1
After incr_id:value of s_id=6, id=1
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

- **Static function = no object needed**
- **Non-static function = object required**

OOPS FUNCTION STATIC OR TASK STATIC METHOD CALL

//

```
class transaction;
        static int s_id;
        int id;

        function static int incr_id();
                int status;
                status = s_id++;
                id++;
                return status;
        endfunction
endclass

module class_example;
transaction tr[5];
int local_status;

initial begin
        foreach (tr[i]) begin
                tr[i] = new();
                local_status = tr[i].incr_id();
                $display("Value of s_id = %0h, id = %0h, local_status = %0h",
                 tr[i].s_id, tr[i].id, local_status);
     end
end
endmodule
```

Simulation error:

```
Value of s_id = 1, id = 1, local_status = 0
```

Value of s_id = 2, id = 1, local_status = 1

Value of s_id = 3, id = 1, local_status = 2

Value of s_id = 4, id = 1, local_status = 3

Value of s_id = 5, id = 1, local_status = 4

V C S S i m u l a t i o n R e p o r t

☑ SIMULATION OUTPUT

Value of s_id = 1, id = 1, local_status = 0

Value of s_id = 2, id = 1, local_status = 1

Value of s_id = 3, id = 1, local_status = 2

Value of s_id = 4, id = 1, local_status = 3

Value of s_id = 5, id = 1, local_status = 4

Final Summary (Very Easy)

`s_id` → static, shared by all objects

`id` → per-object

`status` inside function becomes static because of keyword static

Each call increments both `s_id` and `id`

OOPS – this keyword in classes

- this keyword is a predefined class handle that refers to the current instance of a class.

```
class MyClass;
  int value;
  function new(int value);
    this.value = value; // 'this.value' refers to the class property
                        // 'value' refers to the function argument
```

```
    endfunction
  endclass
```

Example without this keyword

```
class transaction;
  bit [31:0] data;
  int id;

  function new (bit [31:0] data, int id);
    data = data;
    id   = id;
  endfunction
endclass

module class_example;
  initial begin
    transaction tr = new(5, 1);
    $display("Value of data = %0h, id = %0h", tr.data, tr.id);
  end
endmodule
```

Simulation Output:

```
Value of data = 0, id = 0
          V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Example with this keyword

```
class transaction;
    bit [31:0] data;
```

```
    int id;

    function new (bit [31:0] data, int id);
        this.data = data;
        this.id   = id;
    endfunction
endclass

module class_example;

initial begin
    transaction tr = new(5, 1);
    $display("Value of data = %0h, id = %0h", tr.data, tr.id);
end
endmodule
```

Simulation Output:

```
Value of data = 5, id = 1
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

- this.data = data; → class variable
- data = data; → function argument
  - Assign argument data → class varible this.data
  - Assign argument id → class variable this.id
  - Object values gets updated correctly.

OOPS – Class Object Assignment

```
class A;
```

```
        int i;
endclass

class bustran;
        logic [31:0]data;
        logic [3:0] addr;
        int mem [15:0];

        A a=new();
        function new(int a,b,c[15:0]);
                logic [31:0]data;
                logic [31:0]addr;
                this.data=a;
                this.addr=b;

                foreach(mem[i])
                        mem[i]=c[i];
        endfunction
endclass

module m;
bustran b1,b2;

initial begin
        b1=new(10,15,'{3,4,5,7,6,2,3,45,8,54,23,23,43,45,65,75});
        b1.a.i=5;
        b2=b1;
```

```
        $display("b2=%p, b2.a.i=%0d",b2,b2.a.i);
        b2.data=20;
        b2.addr=35;
        b2.a.i=10;
        $display("b1=%p, b1.a.i=%0d",b1,b1.a.i);
end
endmodule
```

Simulation Output

b2='{data:'ha, addr:'hf, mem:'{3, 4, 5, 7, 6, 2, 3, 45, 8, 54, 23, 23, 43, 45, 65, 75} , a:{ ref to class A}}, b2.a.i=5

b1='{data:'h14, addr:'h3, mem:'{3, 4, 5, 7, 6, 2, 3, 45, 8, 54, 23, 23, 43, 45, 65, 75} , a:{ ref to class A}}, b1.a.i=10

V C S S i m u l a t i o n R e p o r t

Time: 0

- Since b2, b1 are the same object
- B2=b1 means both handles point to the same object.

OOPs – C opying Techniques in SV

OOPS – Shallow copy in System Verilog

```
//
class Header;
        int id;

function new (int id);
        this.id = id;
endfunction

function showId();
```

```
        $display ("id=%0d", id);
endfunction

endclass

class Packet;
        int address;
        int value;
        Header header;

        function new (int address, int value, int id);
                header = new (id);
                this.address = address;
            this.value = value;
        endfunction

        function display (string name);
                $display ("[%s] address=%0d value=%0d
id=%0d",name,address,value,header.id);
        endfunction

endclass

module tb;
Packet pkt1, pkt2;

initial begin
        pkt1 = new (100, 200, 10);
```

```
    pkt1.display("pkt1");

  pkt2 = new pkt1;
  pkt2.display ("pkt2");

  pkt1.address = 300;
  pkt1.value   = 400;
  pkt1.header.id = 5;
  pkt1.display("pkt1");

  pkt2.display("pkt2");
end
endmodule
```

Simulation Output:

[pkt1] address=100 value=200 id=10

[pkt2] address=100 value=200 id=10

[pkt1] address=300 value=400 id=5

[pkt2] address=100 value=200 id=5

V C S S i m u l a t i o n R e p o r t

Time: 0

OOPS – Shallow copy in System Verilog

```
class A;
  int i;
endclass

class bus;
```

```
    int k;
    A a=new();
endclass

module m;
bus b1,b2;
initial begin
    b1=new();
    b1.k=10;
    b1.a.i=15;
    $display("b1.a.i=%0d, b1.k=%0d",b1.a.i,b1.k);

    b2 = new b1;
    b2.k=20;
    b2.a.i=25;
    $display("b1.a.i=%0d, b1.k=%0d",b1.a.i,b1.k);

end
endmodule
```

Output:

b1.a.i=15, b1.k=10

b1.a.i=25, b1.k=10

V C S S i m u l a t i o n R e p o r t

Time: 0

Explanation:

k is a normal variable

Not an object.

- **In shallow copy:**

b2 = new b1;

Simple variables (like `k`) are copied separately.

Objects (like `a`) are shared.

Step 1:

b1.k = 10;

So b1.k = 10

Step 2:

b2 = new b1;

This makes:

b2.k = copy of b1.k = 10 (independent copy)

b2.a = same object as b1.a (shared)

Step 3:

b2.k = 20;

👉 This changes only b2.k

👉 It does NOT change b1.k

Because `k` is not shared.

☑ Final State:

- b1.k = 10 (unchanged)
- b2.k = 20 (modified)

⭐ SUPER SIMPLE ONE-LINE ANSWER:

`b1.k` does not become 20 because simple variables are NOT shared in a shallow copy — only objects are shared.

OOPs – DEEP COPY IN System verilog

//

class packet;

```
        bit [31:0]a;
        bit b;

        function new(bit [31:0]a,bit b);
                this.a=a;
                this.b=b;
        endfunction

endclass

class transaction;
        bit [31:0]data;
        int id;
        packet pkt;

        function new();
                data=100;
                id=1;
                pkt=new(10,1);
        endfunction

        function void display();
                $display("transaction:data=%0d,id=%0d",data,id);
                $display("packet: a=%0d, b=%0d",pkt.a,pkt.b);
        endfunction

        function deep_copy(transaction tr);
```

```
            this.data=tr.data;
            this.id=tr.id;
            this.pkt.a=tr.pkt.a;
            this.pkt.b=tr.pkt.b;
        endfunction
endclass

module deep_copy_example;
transaction tr1,tr2;

initial begin
        tr1=new();
        $display("Calling display method using tr1");

        tr1.display();

       tr2 = new();
        tr2.deep_copy(tr1);
       $display("After deep copy tr1 --> tr2");
        $display("calling display method using tr2");

        tr2.display();
        $display("-------------------------");

        tr1.data=200;
        tr1.id=2;
        tr1.pkt.a=32'd12;
```

```
        tr1.pkt.b=0;

        $display("calling display method using tr1");
        tr1.display();
        //tr2.deep_copy(tr1);
        $display("calling display method using tr2");
        tr2.display();
end
endmodule
```

Simulation Output:

```
Calling display method using tr1
transaction:data=100,id=1
packet: a=10, b=1
After deep copy tr1 --> tr2
calling display method using tr2
transaction:data=100,id=1
packet: a=10, b=1
------------------------
calling display method using tr1
transaction:data=200,id=2
packet: a=12, b=0
calling display method using tr2
transaction:data=100,id=1
packet: a=10, b=1
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

1. packet Class

```
class packet;
  bit [31:0] a;
  bit b;
  function new(bit [31:0] a, bit b);
    this.a = a;
    this.b = b;
  endfunction
endclass
```

This class has:

- 32-bit variable `a`
- 1-bit variable `b`
- Constructor initializes them.

2. transaction Class

Contains:

```
bit [31:0] data;
int id;
packet pkt;
#Constructor
function new();
  data = 100;
  id = 1;
  pkt = new(10,1);
endfunction
```

Creates:

data = 100

id = 1

pkt object with a=10, b=1

☑ 3. display()

Shows current values of transaction and its packet.

☑ 4. deep_copy(transaction tr)

```
function deep_copy(transaction tr);
  this.data = tr.data;
  this.id   = tr.id;
  this.pkt.a = tr.pkt.a;
  this.pkt.b = tr.pkt.b;
endfunction
```

This copies:

transaction data

id

packet.a

packet.b

Because packet fields are copied manually, this is called deep copy.

✓ pkt object is not shared

✓ Only values are copied

★ 5. Testbench Operation

Step 1: Create tr1

tr1.data = 100

tr1.id = 1

tr1.pkt.a = 10

tr1.pkt.b = 1

Step 2: Create tr2 and deep copy from tr1

tr2.deep_copy(tr1)

So tr2 gets same values:

tr2.data = 100

tr2.id = 1

tr2.pkt.a = 10

tr2.pkt.b = 1

But tr2.pkt is separate object, not shared with tr1.

★ 6. Modify tr1

tr1.data = 200;

tr1.id = 2;

tr1.pkt.a = 12;

tr1.pkt.b = 0;

tr1 becomes:

data = 200

id = 2

pkt.a = 12

pkt.b = 0

tr2 stays the same (because deep copy):

data = 100

id = 1

pkt.a = 10

pkt.b = 1

🎯 Final Meaning (VERY SIMPLE):

Action	Effect
tr2 = new tr1 (shallow copy)	packet object shared
deep_copy()	packet object NOT shared; only values copied

Because `deep_copy()` copies values manually:

✓ Changes in tr1 do NOT affect tr2.

OOPs – Deep copy in System verilog

OOPs – Class object Deep Copy

```
//
class A;
	int i;

	function A copy();
		copy=new();
		copy.i=this.i;
		return copy;
	endfunction
endclass

class bus;
	int k;
	A a=new();
	function bus copy();
		copy=new();
```

```
        copy.k=this.k;
        copy.a=a.copy;
        return copy;
    endfunction
endclass

module m;
bus b1,b2;

initial begin
    b1=new();
    b2=b1.copy;
    b1.a.i=10;
    $display(b1.a);
    b2.a.i=15;
    $display(b2.a);
end
endmodule
```

Simulation Output:

```
00:52 2025
'{i:10}
'{i:15}
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Explanation:

- Class A has one variable i.
- Deep copy function of class A

- A new object of A is created.
- The value of i from the current object (`this.i`) is copied to the new object.
- The new object is returned → this is deep copy.
- CLASS bus
- Class bus has:
- an integer k
- an object a of class A (created already using `new()`)
- Deep copy function of bus
- A new bus object is created
- `copy.k = this.k` → the value of k is copied
- `copy.a = a.copy` → creates a deep copy of inside object A
- Returns a completely separate bus object.

MODULE m

- Two handles (b1 and b2) created, but not objects yet.
- Start initial block
- Create b1
- Now b1 is a new object of bus.
- Inside it, `b1.a` also exists because we created `A a=new()` inside class.
- Deep copy to b2
- `b1.copy` creates a new bus object.
- b2 becomes that new object.
- b1 and b2 are two different objects.
- AND `b1.a` and `b2.a` are also different.
- Change b1.a.i
- Set value of i inside b1's A object.
- This will NOT affect b2, because deep copy created a separate object.
- Display b1.a
- Displays the memory address (handle) of `b1.a`.
- Change b2.a.i
- Modify i inside b2's A object.
- Again, this does NOT affect b1 because deep copy.
- Display b2.a
- Shows the handle (memory location) of `b2.a`.
- Final Understanding (Very Simple)

Object	i value	Description
b1.a	10	Separate A object
b2.a	15	New deep-copied A object

- b1 and b2 are totally independent
- Changing b1.a.i does NOT affect b2.a.i
- This is deep copy, not shallow copy

Shallow Copy and Deep Copy

1. Shallow Copy (Default Copy)

- **Copy only the handle (pointer), not the object data.**
- Both objects point to same memory.
- Changing one object will affect the other.

☑ **Example (Shallow Copy)**

```
class A;
  int x;
endclass

module test;
  A obj1 = new();
  A obj2;       // obj2 is only handle, not an object yet.

  initial begin
    obj1.x = 10;
```

```
  obj2 = obj1;   // SHALLOW COPY (handle copy)

  obj2.x = 20;   // change obj2  //since obj1 and obj2 share the same object

  $display("obj1.x = %0d", obj1.x); // 20 //both handle point to the same object
  $display("obj2.x = %0d", obj2.x); // 20
 end
endmodule
```

✓ What happened?

- obj2 = obj1 copied only the handle
- Both objects point to same memory
- Changing `obj2.x` changed `obj1.x`

This is shallow copy.

2. Deep Copy (Manual Clone)

Copy object data into a new object.

➞ Two separate memory locations

➞ Changing one object does NOT affect the other

How to do deep copy?

👉 Write a copy() function inside the class.

☑ Example (Deep Copy)

```
class A;
 int x;

 // Deep copy method
```

```
  function A copy();
    A temp = new();
    temp.x = this.x;
    return temp;
  endfunction
endclass

module test;
  A obj1 = new();
  A obj2;

  initial begin
    obj1.x = 10;

    obj2 = obj1.copy();   // DEEP COPY

    obj2.x = 20;          // change obj2

    $display("obj1.x = %0d", obj1.x); // 10
    $display("obj2.x = %0d", obj2.x); // 20
  end
endmodule
```

✓ What happened?

- copy()` created a new object
- Copied only the data (`x`)
- `obj1.x` and `obj2.x` are independent
- This is deep copy.

Simple Real-Life Example

- **Shallow Copy**
- Two people share one plate.
- If one person adds food, the other sees it → same plate.

- **Deep Copy**
- Each person gets separate plates.
- Adding food to your plate doesn't affect the other.

✓ Summary Table

Feature	Shallow Copy	Deep Copy
Memory	Same memory shared	New memory allocated
How?	`obj2 = obj1`	`obj2 = obj1.copy()`
Data independent?	❌ No	✓ Yes
Risk	Changing one affects other	Safe

Explanation

1. Class A with variable x

```
class A;
  int x;
```

✓ Class A contains one integer variable `x`.

2. Deep Copy Function

```
function A copy();
    A temp = new();
    temp.x = this.x;
    return temp;
```

endfunction

Simple meaning:

- A new object `temp` is created.
- The value of `x` from the current object (`this.x`) is copied into `temp.x`.
- The new object `temp` is returned.

⭐ This is a TRUE deep copy → a completely new object is made.

3. Creating objects in the testbench

A obj1 = new();

A obj2;

✓ **`obj1` becomes a new object.**

✓ **`obj2` is just a handle (empty, not created yet).**

4. Assign value to obj1.x

obj1.x = 10;

✓ You store 10 inside `obj1.x`.

5. Deep copy

obj2 = obj1.copy();

- A new object is created inside the `copy()` function.
- This new object gets x = 10.
- Now `obj1` and `obj2` are two different objects.

6. Modify obj2

obj2.x = 20;

✓ Only `obj2.x` changes to 20.

✓ `obj1.x` remains 10 because they are separate objects.

7. Display results

$display("obj1.x = %0d", obj1.x); // 10

$display("obj2.x = %0d", obj2.x); // 20

✓ **Output:**

obj1.x = 10

obj2.x = 20

⭐ Final Easy Meaning

Shallow copy → both variables share SAME object

Deep copy → creates NEW object

Here:

- `obj1` and `obj2` are different objects.
- So, changing one does not affect the other.

Session-9: -

- Abstract class
- Inheritance
- Polymorphism
- Scope Resolution Operator in SV
- Extern keyword
- Parameterized classes
- Data Encapsulation
- Local access Qualifier
- Protected access Qualifier
- Constant class properties
- Super keyword

- Virtual keyword

Abstract class

- Abstract class is also known as a virtual class.
- You can not create an object of an virtual class.

Without virtual keyword in the class

```
//
class parent_trans;
    bit [31:0] data;
    int id;

    function void display();
        $display("Base: Value of data = %0h and id = %0h", data, id);
    endfunction
endclass

module class_example;
initial begin
    parent_trans p_tr;
    p_tr = new();

    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
end
endmodule
```

Simulation Output:

Base: Value of data = 5 and id = 1

V C S S i m u l a t i o n R e p o r t

Time: 0

An Abstract class instantiation

```
virtual class parent_trans;
bit [31:0] data;
int id;

function void display();
    $display("Base: Value of data = %0h and id = %0h", data, id);
endfunction
endclass

module class_example;
initial begin
    parent_trans p_tr;
    p_tr = new();

    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
end
endmodule
```

Simulation Output:

Error-[SV-ACCNBI] An abstract class cannot be instantiated

example.sv, 15

class_example, "p_tr = new();"

Instantiation of the object 'p_tr' can not be done because its type

'parent_trans' is an abstract base class.

Perhaps there is a derived class that should be used.

1 warning

1 error

System Verilog Inheritance

A *child_trans* is an extended class from its *parent_trans* (base class). A *child_class* can access its base class properties (*data* variable) and methods (*disp_p* function).

Inheritance Example

```
class parent_trans;
	bit [31:0] data;

	function void disp_p();
		$display("Value of data = %0h", data);
	endfunction
endclass

class child_trans extends parent_trans;
	int id;

	function void disp_c();
		$display("Value of id = %0h", id);
	endfunction
endclass
```

```
module class_example;
initial begin
        child_trans c_tr;
        c_tr = new();
        c_tr.data = 5; // child class is updating property of its base class
        c_tr.id = 1;

        c_tr.disp_p(); // child class is accessing method of its base class
        c_tr.disp_c();
end
endmodule
```

Simulation Output:

Value of data = 5

Value of id = 1

V C S S i m u l a t i o n R e p o r t

Time: 0

Inheritance

Example Inheritance

```
//
class parent_trans;
        bit [31:0] data_p;
```

```
        function void disp_p();
                $display("parent_trans: Value of data = %0h", data_p);
        endfunction
endclass

class child1_trans extends parent_trans;
        bit [31:0] data_c1;

        function void disp_c1();
                $display("child1_trans: Value of data = %0h", data_c1);
        endfunction
endclass

class child2_trans extends parent_trans;
        bit [31:0] data_c2;

        function void disp_c2();
                $display("child2_trans: Value of data = %0h", data_c2);
        endfunction
endclass

class child_A_trans extends child1_trans;
        bit [31:0] data_cA;

        function void disp_cA();
                $display("child_A_trans: Value of data = %0h", data_cA);
        endfunction
```

```
endclass

class child_B_trans extends child_A_trans;
        bit [31:0] data_cB;

        function void disp_cB();
                $display("child1_2_trans: Value of data = %0h", data_cB);
  endfunction
endclass

module class_example;
initial begin
        child_B_trans cB_tr;
        cB_tr = new();
        cB_tr.data_p = 2;
        cB_tr.data_c1 = 4;
        cB_tr.data_cA = 6;
        cB_tr.data_cB = 8;
    //cB_tr.data_c2 = 3;  // Not possible as child_B_trans is not child class of child2_trans.

    cB_tr.disp_p();
    cB_tr.disp_c1();
    cB_tr.disp_cA();
    cB_tr.disp_cB();
  end
endmodule
```

Simulation Output:

parent_trans: Value of data = 2

child1_trans: Value of data = 4

child_A_trans: Value of data = 6

child1_2_trans: Value of data = 8

V C S S i m u l a t i o n R e p o r t

Time: 0

Overriding the Base class members

The child class properties and methods **override base class properties and methods when both the child and base class have the same naming** convention for class properties and method.

```
//
class parent_trans;
    bit [31:0] data = 100;
   int id = 1;

   function void display();
        $display("Base: Value of data = %0d and id = %0d", data, id);
    endfunction
endclass

class child_trans extends parent_trans;
    bit [31:0] data = 200;
    int id = 2;

    function void display();
        $display("Child: Value of data = %0d and id = %0d", data, id);
```

```
    endfunction
endclass

module class_example;
initial begin
    child_trans c_tr;
    c_tr = new();

    c_tr.display();
end
endmodule
```

Simulation Output:

```
Child: Value of data = 200 and id = 2
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Accessing class members in Inheritance

```
// Based on the handle of a class, the corresponding method will be called.
class parent_trans;
    bit [31:0] data;
    int id;

    function void display();
        $display("Base: Value of data = %0d and id = %0d", data, id);
    endfunction
endclass

class child_trans extends parent_trans;
```

```
    bit [31:0] data;
  int id;

    function void display();
        $display("Child: Value of data = %0d and id = %0d", data, id);
    endfunction
endclass

module class_example;
initial begin
    parent_trans p_tr;
  child_trans c_tr;

  p_tr = new();
  c_tr = new();

  p_tr.data = 100;
  p_tr.id = 1;

  c_tr.data = 200;
  c_tr.id = 2;

  p_tr.display();
  c_tr.display();
end
endmodule
```

Simulation Output:

Base: Value of data = 100 and id = 1

Child: Value of data = 200 and id = 2

V C S S i m u l a t i o n R e p o r t

super keyword in classes

Polymorphism

- **To use many forms of the method, the virtual keyword must be used in the method definition.**

Polymorphism Examples

```
//
class parent;
  bit [31:0] data;
  int id;

  virtual function void display();
     $display("Base: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

class child_A extends parent;
  function void display();
    $display("Child_A: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

class child_B extends parent;
  function void display();
```

```
    $display("Child_B: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

class child_C extends parent;
  function void display();
    $display("Child_C: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent p_A, p_B, p_C;
    child_A c_A = new();
    child_B c_B = new();
    child_C c_C = new();

    c_A.data = 200;
    c_A.id   = 2;

    c_B.data = 300;
    c_B.id   = 3;

    c_C.data = 400;
    c_C.id   = 4;

    p_A = c_A;
```

```
    p_B = c_B;
    p_C = c_C;

    p_A.data = 100;
    p_A.id   = 1;

    p_A.display();
    p_B.display();
    p_C.display();
  end
endmodule
```

simulation Output:

```
Child_A: Value of data = 100, id = 1
Child_B: Value of data = 300, id = 3
Child_C: Value of data = 400, id = 4
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

A Child class handle is assigned to the base class (base_class_handle = child_class_handle)

value assignment (data and id variables) using the **parent class handle is written first** and **then a child class handle is used to assign the values to variables**.

Simulation Output:

```
Base: Value of data = 10, id = 1
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Since variables are declared in both classes, **separate memory is allocated**. Hence, **base class values are not overridden by its child class.**

Polymorphism Examples

```
//
class parent_trans;
  bit [31:0] data;
  int id;

  function void display();
    $display("Base: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

class child_trans extends parent_trans;

  function void display();
    $display("Child: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();
```

```
  p_tr = c_tr;


  p_tr.data = 10;
  p_tr.id   = 1;


  c_tr.data = 5;
  c_tr.id   = 2;


  p_tr.display();
 end
endmodule
```

Simulation Output:

Base: Value of data = 5, id = 2

V C S S i m u l a t i o n R e p o r t

Time: 0

Since variables are only declared in the base class, both classes will have access to the same memory location. Hence, values changed by child class also reflect when it is accessed by the base class handle.

A base class handle is assigned to the child class (child_class_handle = base_class_handle)

Simulation Output: This will lead to a compilation error. `c_tr = p_tr;` (change in code)

how to resolve this problem using "Dynamic Casting"

Now, replace assignment = with $cast(,). The run time error is still expected because the base handle is not pointing to an object that is compatible with child handle.

Dynamic Casting

```
//

class parent_trans;

 bit [31:0] data;
```

```
  int id;

  function void display();
    $display("Base: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

class child_trans extends parent_trans;

  function void display();
    $display("Child: Value of data = %0d, id = %0d", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr; // or $cast(p_tr, c_tr);
    $cast(c_tr, p_tr);

    p_tr.data = 10;
    p_tr.id   = 1;

    c_tr.data = 5;
```

```
    c_tr.id   = 2;


    c_tr.display();
  end
endmodule
```

Solution Output:

```
Child: Value of data = 5, id = 2
            V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Scope Resolution Operator in SV

Scope Resolution Operator Example

```
//
class transaction;
        bit [31:0] data;
        static int id;


        static function disp(int id);
        $display("Value of id = %0h", id);
endfunction

function auto_disp(int id);
        $display("Value of id = %0h", id);
endfunction
endclass
```

```
module class_example;
initial begin
        transaction::id = 5;
        transaction::disp(transaction::id);

   //transaction::data = 2; // illegal
   //transaction::auto_disp(transaction::id); // illegal
end
endmodule
```

Simulation Output:

```
Value of id = 5
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Explanation:

You are declaring a class named transaction.

1. bit [31:0] data;

- This is a non-static (normal) variable.
- Belongs to the object, not the class.
- Must access through an object (like `t.data`).

2. static int id;

- This is a static variable.
- Belongs to the class, not the object.
- Can be accessed using:

```
transaction::id
```

3. static function disp(int id);

```
static function disp(int id);
  $display("Value of id = %0h", id);
```

endfunction

- This is a static function.
- Belongs to the class, not object.
- Can be called like:

```
transaction::disp( value );
```

✓ Static functions cannot use non-static variables inside them.

4. function auto_disp(int id);

```
function auto_disp(int id);
  $display("Value of id = %0h", id);
endfunction
```

- This is a non-static (normal) function.
- It belongs to an object.
- Must be called like:

```
transaction t;
t = new();
t.auto_disp(...);
```

❌ You cannot call it using `transaction::auto_disp()`

❌ Because it is NOT static.

⭐ MODULE: USING THE CLASS

```
module class_example;
  initial begin
```

Simulation starts here.

5. transaction::id = 5;

- **Access `id` using scope resolution (::) because `id` is static.**
- **You did NOT create any object → still OK.**

✓ Sets the class variable `id` to 5.

6. transaction::disp(transaction::id);

Call the static function `disp()` using `::`.

Pass `transaction::id` as argument.

It prints:

Value of id = 5

❌ 7. Illegal Statements (Correctly Commented)

a. transaction::data = 2;

`data` is non-static → belongs to object.

But here you are trying to access it using class name.

❌ Illegal

✓ Must be accessed:

t = new();

t.data = 2;

b. transaction::auto_disp(transaction::id);

`auto_disp()` is **non-static** (normal function)

Cannot be called using class name (`transaction::`)

❌ Illegal

✓ Must be called using an object:

t = new();

t.auto_disp(...);

⭐ SIMULATION OUTPUT

The only line printed:

Value of id = 5

🎯 Very Simple Understanding

Item	static?	Access using
`id`	Yes	`transaction::id`
`disp()`	Yes	`transaction::disp()`
`data`	No	`t.data` (object needed)
`auto_disp()`	No	`t.auto_disp()` (object needed)

Extern method

An *extern* method provides a facility for class methods to define them outside of the class body.

//

```
class transaction;
        bit [31:0] data;
        int id;

        extern function void display();
        extern task delay();
```

```
endclass

function void transaction::display();
        $display("data = %0d and id = %0d", data, id);
endfunction

task transaction::delay();
        #50;
        $display("Time = %0.0t, delayed data = %0d", $time, data);
endtask

module class_example;
transaction tr;
initial begin
        tr = new();
        tr.data = 100;
      tr.id = 1;
      tr.display();
      tr.delay();
end
endmodule
```

Simulation Output:

```
data = 100 and id = 1
Time = 50, delayed data = 100
        V C S  S i m u l a t i o n  R e p o r t
Time: 50
```

Explanation:

⭐CLASS DEFINITION

```
class transaction;
  bit [31:0] data;
  int id;
```

- You created a class named transaction.
- It has two non-static variables:
- `data` → 32-bit value
- `id` → integer

Extern Function & Task Declaration

```
  extern function void display();
  extern task delay();
endclass
```

- `extern` means:

✓ Function/task is declared here

❌ But defined outside the class

This makes the class definition cleaner.

⭐ FUNCTION DEFINITION OUTSIDE THE CLASS

```
function void transaction::display();
  $display("data = %0d and id = %0d", data, id);
endfunction
```

`transaction::display` means:

→ This function belongs to the transaction class

It prints the values of `data` and `id`.

⭐ TASK DEFINITION OUTSIDE THE CLASS

```
task transaction::delay();
  #50;
  $display("Time = %0.0t, delayed data = %0d", $time, data);
```

endtask

- `delay()` waits 50 time units
- After delay, it prints:
- Current simulation time (`$time`)
- The value of `data`

⭐ MODULE USING THE CLASS

module class_example;

transaction tr;

- Create a handle `tr` of class `transaction`.

Initial Block

initial begin

tr = new();

- Create object of class `transaction`

Assign Values

tr.data = 100;

tr.id = 1;

- Store values inside the object.

Call the Function

tr.display();

- Prints data and id.

Call the Task

tr.delay();

- Waits 50 time units
- Prints the time and data.

⭐ EXPECTED SIMULATION OUTPUT

data = 100 and id = 1

Time = 50, delayed data = 100

🎯 Very Simple Summary

- `extern` means function/task is declared inside class but defined outside.
- `display()` prints values immediately.
- `delay()` waits 50 time units and then prints time and data.
- `tr = new()` creates the object.

Extern method example

Parameterized classes

Parameterized classes are useful when the same class needs to be instantiated differently.

- These parameters can be overridden when it is instantiated.
- The parameter can be constant values and data types.

Parameterized classes

// to make code reusable and flexible of the data and variables.

```
class transaction #(parameter WIDTH = 2, type D_TYPE = bit [2:0]);
        bit [WIDTH-1:0] data;
        D_TYPE id;

        function void display();
                $display("data = %0d, id = %0d", data, id);
        endfunction
endclass

module class_example;
transaction tr1;
```

```
transaction #(3,int) tr2;


initial begin
    tr1 = new();
   tr2 = new();


    tr1.data = 7;
   tr1.id = 15;
   tr1.display();


   tr2.data = 7;
   tr2.id = 15;
   tr2.display();
end
endmodule
```

Simulation output:

```
data = 3, id = 7
data = 7, id = 15
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Data Encapsulation and Hiding

- Data encapsulation is a mechanism that combines class properties and methods.
- Data hiding is a mechanism to hide class members within the class. They are not accessible outside of class scope.
- By default, it is global variables.

Local Access variable Examples

Local Access Qualifier

If a class member is declared as a local, they will be available to that class alone. The child classes will not have access to a local class member of their parent class.

Access outside of the class scope

In the below example, the variable id is declared as a local variable. It is tried to access outside of class scope, which leads to a compilation error.

```
class transaction;
    bit [31:0] data;
    local int id;

    function void display();
        $display("data = %0d and id = %0d", data, id);
    endfunction
endclass

module class_example;
transaction tr;
initial begin
    tr = new();
    tr.data = 100;
    tr.id = 1;

    tr.display();
end
endmodule
```

Simulation Output:

```
Error-[SV-ICVA-L] Illegal class variable access
example.sv, 16
  Local member 'id' of class 'transaction' is not visible to scope
```

'class_example'.

Please make sure that the above member is accessed only from its own class methods as it is declared as local.

Please refer to the SystemVerilog LRM (1800-2012) Section 8.18 Data hiding and encapsulation.

1 warning

1 error

Access within the class scope

There is no compilation error expected if the local variable is accessed within the class scope.

```
class transaction;
    bit [31:0] data;
    local int id;

    function new();
        data = 100;
        id = 1;
    endfunction

    function void display();
        $display("data = %0d and id = %0d", data, id);
    endfunction
endclass

module class_example;
transaction tr;

initial begin
```

```
    tr = new();
    tr.display();
end
endmodule
```

Simulation Output:

```
data = 100 and id = 1
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Class methods as a local

Local Access method Examples

Access outside of the class scope

A class method *display()* is declared as local. So, calling the *display()* method outside of the class scope will lead to a compilation error.

```
class transaction;
  bit [31:0] data;
  int id;

  function new();
    data = 100;
    id = 1;
  endfunction

  local function void display();
    $display("data = %0d and id = %0d", data, id);
  endfunction
endclass

module class_example;
  transaction tr;

  initial begin
    tr = new();
    tr.display();
  end
endmodule
```

Simulation Output:

```
Error-[SV-ICMA] Illegal class method access
example.sv, 21
  Local method 'display' of class 'transaction' is not visible to scope
  'class_example'.
  Please make sure that the above method is called only from its own class
  properties as it is declared as local.
1 warning
1 error
```

Access within the class scope

There is no compilation error expected if the local *display()* method is accessed within the class scope.

```
class transaction;
  bit [31:0] data;
  int id;

  function new();
    data = 100;
    id = 1;
    display();
  endfunction

  local function void display();
    $display("data = %0d and id = %0d", data, id);
  endfunction
endclass

module class_example;
  transaction tr;
```

```
 initial begin
   tr = new();
 end
endmodule
```

Simulation Output:

data = 100 and id = 1

V C S S i m u l a t i o n R e p o r t

Time: 0

The child class access local member of the parent class

A child_trans is a derived class of base class *transaction*. A local method display() is defined in the base class and it is accessed by its child class child_trans. Since it is not allowed, a compilation error is expected.

Protected access Qualifier in SV

Protected access Qualifier Example

Protected method access in child class

The protected display method can be accessed in child class which is not possible in the case of local access qualifiers.

```
class transaction;
  bit [31:0] data;
  int id;

  function new();
    data = 100;
    id = 1;
  endfunction

  protected function void display();
```

```
    $display("data = %0d and id = %0d", data, id);
  endfunction
endclass

class child_trans extends transaction;
  bit [31:0] addr;

  task calc_addr;
    addr = data * id;
    display();  // child class is accessing protected access method of parent class
    $display("addr = %0d", addr);
  endtask
endclass

module class_example;
  child_trans tr;

  initial begin
    tr = new();
    tr.calc_addr();
  end
endmodule
```

Simulation Output:

```
data = 100 and id = 1
addr = 100
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Constant class properties in SV

Protected method access outside the class scope

A compilation error is expected when the protected *display()* method is accessed outside the class scope

- **can not accessed outside class scope**

Constant class properties in SV

To make class properties read-only, a const specifier is used. Since class objects are dynamic in nature, SystemVerilog provides two types of constants

1. Global constants
2. Instance constants

In both types, class properties have to specify with the *const* keyword.

The class methods can not use a *const* specifier.

Global constants

During variable declaration, **an initial value is assigned**, such class properties known as global constants. The value of a variable can not be changed after the declaration of the variable.

```
class transaction;
  bit [31:0] data;
  const int id = 1;

  function void display();
    $display("data = %0d and id = %0d", data, id);
  endfunction
endclass

module class_example;
  transaction tr;
```

```
  initial begin
    tr = new();
    tr.data = 100;
    //tr.id = 2;  // Invalid const usage
    tr.display();
  end
endmodule
```

Simulation Output:

```
data = 100 and id = 1
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Constant class properties in SV

Instance constants

During variable declaration, an initial value is not assigned, such class properties are known as instance constants. An instance constant allows a user to assign value in run time only once in the class constructor,

```
class transaction;
  bit [31:0] data;
  const int id;

  function new();
    data = 100;
    id = 1; // run time assignment only once
  endfunction

  function void display();
    $display("data = %0d and id = %0d", data, id);
```

```
  endfunction
endclass

module class_example;
  transaction tr;

  initial begin
    tr = new();
    tr.data = 200;
    //tr.id = 2; // Invalid const usage
    tr.display();
  end
endmodule
```

Simulation Output:

```
data = 200 and id = 1
```

Virtual keyword in classes

The virtual keyword is used while defining the class method (virtual methods) and the class itself (Abstract class)

virtual functions and tasks in SystemVerilog

A virtual method is a virtual function or task from the base class which can be overridden by a method of its child class having the same signature (same method name and arguments).

In simple words,

When a child class handle is assigned to its base class. On calling a method using a base class handle, the base class method will be executed. On declaring a method as a virtual method, a base class handle can call the method of its child class.

Note: Once a virtual keyword is used for a base class method, all corresponding methods in derived classes become virtual. It is not necessary to put the 'virtual' keyword for derived class methods.

Let's understand it with the below examples.

In both examples, a child class object is created, and then it is assigned to the base class handle.

```
parent_trans p_tr;

child_trans c_tr;

c_tr = new();

p_tr = c_tr;
```

virtual keyword in classes

Example without virtual keyword

Notice that in the below example base class *display()* method is called when it is not declared as virtual.

```
class parent_trans;
  bit [31:0] data;
  int id;


  function void display();
     $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  function void display();
    $display("Child: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
```

```
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

Simulation Output:

Base: Value of data = 5 and id = 1

Example with virtual keyword (child display executes)

Notice that in the below example child class *display()* method when it is declared as virtual.

```
class parent_trans;
  bit [31:0] data;
  int id;

  virtual function void display();
    $display("Base: Value of data = %0d and id = %0d", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  bit [31:0] data;
  int id;
  function void display();
    $display("Child: Value of data = %0d and id = %0d", data, id);
  endfunction
```

```
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    c_tr.data = 10;
    c_tr.id = 2;

    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

Simulation Output:

```
Child: Value of data = 10 and id = 2
```

Example with virtual keyword with different signature (Difference in argument list)

Both base and child class should have the same number of arguments otherwise, a compilation error is expected. Notice that the child class display method has only one argument whereas the base class display method has two arguments.

```
class parent_trans;
  bit [31:0] data;
  int id;
```

```
  virtual function void display(bit [31:0] data, int id);
     $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  function void display(bit [31:0] data);
    $display("Child: Value of data = %0h", data);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display(p_tr.data, p_tr.id);
  end
endmodule
```

Simulation Output:

```
Error-[SV-INACF] Invalid number of args to class function
  Too few arguments in class-method 'display' in derived class 'child_trans'.
```

```
  Base class-method declared at "testbench.sv", 7
  Derived class-method declared at "testbench.sv", 13
  Please make sure that correct number of arguments are specified.
```

Example with virtual keyword with the same signature

Notice that in both base and child classes, the display method has two arguments.

```
class parent_trans;
  bit [31:0] data;
  int id;

  virtual function void display(bit [31:0] data, int id);
    $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  function void display(bit [31:0] data, int id);
    $display("Child: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    p_tr.data = 5;
```

```
    p_tr.id = 1;
    p_tr.display(p_tr.data, p_tr.id);
  end
endmodule
```

super keyword

super keyword example

Example without super keyword

```
//
class parent_trans;
        bit [31:0] data;

        function void display();
                $display("Base: Value of data = %0h", data);
        endfunction
endclass

class child_trans extends parent_trans;
        bit [31:0] data;

        function void display();
                $display("Child: Value of data = %0h", data);
        endfunction
endclass

module class_example;
```

```
initial begin
        child_trans c_tr;
        c_tr = new();

        c_tr.data = 5; //data variable in base class will have default value 0.
        c_tr.display();
  end
endmodule
```

Simulation Output:

```
Child: Value of data = 5
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

In the first example, the value for the data variables in the child class is updated. The display method from the child class is called using the child class handle.

Example with super keyword

```
//
class parent_trans;
        bit [31:0] data;

        function void display();
                $display("Base: Value of data = %0h", data);
        endfunction
endclass

class child_trans extends parent_trans;
        bit [31:0] data;
```

```
    function void display();
        super.data = 3;
        super.display();
        $display("Child: Value of data = %0h", data);
    endfunction
endclass

module class_example;
initial begin
    child_trans c_tr;
  c_tr = new();

    c_tr.data = 5;
  c_tr.display();
end
endmodule
```

Simulation output:

Base: Value of data = 3

Child: Value of data = 5

V C S S i m u l a t i o n R e p o r t

Time: 0

in the second example, the *display()* method from child class is called using child class handle, but the super keyword makes a difference. **The *super.data* assignment assigns value to the *data* variable of the base class. Also, *super.display()* method calls *display()* method of its base class.**

- In the case of multilevel inheritance, class members are inherited one level up. The *super.super.<class_member>* is **not** allowed.

Usage of the super keyword in the constructor

By default, the SystemVerilog compiler calls *super.new()* function calls automatically from extended class. But if there are any arguments used in the *new()* function call, then the user has to call *super.new(<argument_list>)* explicitly. The *super.new()* function call shall be the first line in the derived class constructor because the parent class must be initialized before the derived class.

//

```
class parent_trans;
        bit [31:0] data;


        function new(bit [31:0] data);
                this.data = data;
                $display("Base: Value of data = %0h", data);
        endfunction
endclass


class child_trans extends parent_trans;
        bit [31:0] data;
        function new(bit [31:0] data_p, data_c);
                super.new(data_p);
                this.data = data_c;
                $display("Child: Value of data = %0h", data);
        endfunction
endclass
```

```
module class_example;

initial begin

        child_trans c_tr;

        c_tr = new(5, 7);

end

endmodule
```

Simulation Output:

```
Base: Value of data = 5

Child: Value of data = 7

        V C S   S i m u l a t i o n   R e p o r t

Time: 0
```

Example without arguments in the constructor

This example is a little tricky. Usually, we create an object for both base and child class and then do a method call and assign values to their class properties. To prove that SystemVerilog does *super.new()* function calls internally. An object is only created for the child class and then the child class handle is assigned to its base class. This is required to refer to the memory location for a base class that is created on *super.new()* call. Later base class data variable value is assigned. On calling *display()* method for the corresponding class handle, to print respective class properties.

```
//

class parent_trans;

        bit [31:0] data;

      int id;


        function void display();
```

```
            $display("Base: Value of data = %0d, id = %0d", data, id);
        endfunction
endclass

class child_trans extends parent_trans;
        bit [31:0] data;

    function void display();
            $display("Child: Value of data = %0d, id = %0d", data, id);
    endfunction
endclass

module class_example;
initial begin
        parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();
    c_tr.data = 5;
    c_tr.id   = 2;

    p_tr = c_tr;
```

```
    p_tr.data = 10;

    p_tr.id   = 1;


    p_tr.display();

    c_tr.display();
end

endmodule
```

Simulation Output:

Base: Value of data = 10, id = 1

Child: Value of data = 5, id = 1

V C S S i m u l a t i o n R e p o r t

Time: 0

Notice that the *data* variable has different values for base and child class since they are declared in both classes.

Pure virtual function keyword in classes

A pure virtual function:

✓ Has no body in the parent class

✓ Must be overridden in the child class

✓ Makes the class abstract

✓ You cannot create objects of that class

- It is used when you want a common template, but the child must implement the function.

Pure Virtual Syntax

virtual function void display(); pure virtual;

- `virtual` → allows overriding
- `pure virtual` → no body, must be implemented in derived class

Simple Example

1. Abstract Parent Class

```
class parent_trans;
  bit [31:0] data;
  int id;

  // Pure virtual function (no body)
  pure virtual function void display();
endclass
```

✓ This class contains a pure virtual function

✓ You cannot do: `parent_trans p = new();` → ERROR

2. Child Class MUST implement display()

```
class child_trans extends parent_trans;
  function void display();
    $display("Child: data = %0d, id = %0d", data, id);
  endfunction
endclass
```

✓ Child provides the actual code for `display()`

✓ Now child class is complete and can be created

⭐ Testbench Using Pure Virtual Class

```
module class_example;
  initial begin
    parent_trans p_tr;   // handle of parent class
    child_trans  c_tr;   // child object

    c_tr = new();        // allowed
    p_tr = c_tr;         // parent handle → child object

    c_tr.data = 10;
    c_tr.id   = 5;

    p_tr.display();      // calls child's implementation
  end
endmodule
```

⭐ Simulation Output

```
Child: data = 10, id = 5
```

⭐ Simple Explanation (in 3 lines)

1 Pure virtual means the function has no body in the base class.

2 Child class must override it.

3 Base class becomes abstract, cannot be instantiated.

Typedef class in SV

In certain cases, the **class handle of another class is required** even if it is not declared yet. In such cases, SystemVerilog provides a forward declaration of the class using the typedef keyword.

Syntax:

```
typedef class <class_name>
```

In the example, the tr_B class object is created inside the transaction_A class even though the transaction_B class definition is unknown. So, the compilation error is expected.

Without typedef keyword

In the example, the tr_B class object is created inside the transaction_A class even though the transaction_B class definition is unknown. So, the compilation error is expected.

```
class transaction_A;
  bit [31:0] data;
  int id;
  transaction_B tr_B = new();

  function void display();
    $display("transaction_A: data = %0d and id = %0d", data, id);
    $display("transaction_B: addr = %0d", tr_B.addr);
  endfunction
endclass

class transaction_B;
  bit [31:0] addr = 200;
endclass
```

```
module class_example;
  transaction_A tr_A;

  initial begin
    tr_A = new();

    tr_A.data = 100;
    tr_A.id = 1;
    tr_A.display();
  end
endmodule
```

Simulation output

```
Error-[SE] Syntax error
  Following verilog source has syntax error :
  "example.sv", 5: token is '='
    transaction_B tr_B = new();
                       ^
1 warning
1 error
```

With typedef keyword

The forward declaration is done with a typedef declaration as follows.

```
typedef class transaction_B;

class transaction_A;
  bit [31:0] data;
  int id;
```

```
  transaction_B tr_B = new();

  function void display();
    $display("transaction_A: data = %0d and id = %0d", data, id);
    $display("transaction_B: addr = %0d", tr_B.addr);
  endfunction
endclass

class transaction_B;
  bit [31:0] addr = 200;
endclass

module class_example;
  transaction_A tr_A;

  initial begin
    tr_A = new();

    tr_A.data = 100;
    tr_A.id = 1;
    tr_A.display();
  end
endmodule
```

Simulation output

```
transaction_A: data = 100 and id = 1
transaction_B: addr = 200
        V C S   S i m u l a t i o n   R e p o r t
```

Time: 0

Explanation:

1. Forward Declaration

typedef class transaction_B;

- This means we are telling SystemVerilog that a class named `transaction_B` will be defined later.
- This allows class A to use class B before class B is actually written.

2. Class transaction_A

```
class transaction_A;
  bit [31:0] data;
  int id;
  transaction_B tr_B = new();
```

Explanation:

This defines a class `transaction_A`.

It has:

`data` (32-bit)

`id` (integer)

It also creates an object of class B inside class A:

transaction_B tr_B = new();

✓ This means class A contains class B

✓ Automatically creates an object of `transaction_B`

display() function

```
function void display();
  $display("transaction_A: data = %0d and id = %0d", data, id);
  $display("transaction_B: addr = %0d", tr_B.addr);
endfunction
```

- Prints values stored in class A
- Then prints the `addr` variable from the `transaction_B` object `tr_B`

3. Class transaction_B

class transaction_B;

bit [31:0] addr = 200;

endclass

- Defines class B
- Contains one variable `addr` initialized to 200

4. Module class_example

transaction_A tr_A;

- Declare a handle of class A

Initial block

tr_A = new();

Create object of class A

tr_A.data = 100;

tr_A.id = 1;

Assign values to A's data and id

tr_A.display();

Calls display function of class A

This function also prints B's `addr`

SIMULATION OUTPUT

transaction_A: data = 100 and id = 1

transaction_B: addr = 200

Simple Summary

- `transaction_A` contains an object of `transaction_B`.
- `display()` prints values of class A and class B.

- Class B is forward-declared using `typedef class transaction_B;` so class A can use it before it is defined.

This is an example of composition:

> Class A contains Class B inside it.

Abstract Class

An abstract class is a class that cannot be instantiated (cannot create objects).

It is used only as a base class for other classes.

virtual class <class_name>

...

endclass

Abstract classes contain:

✓ Normal methods

✓ Virtual methods

✓ Pure virtual methods → **(mandatory to override in child class)**

☑ What is a Pure Virtual Method?

- A pure virtual method is a function/task that has no body.

pure virtual function void display();

This tells SystemVerilog:

👉 "Child class must implement this function."

❌ You cannot create object of abstract class

If a class has at least one pure virtual function, it becomes an abstract class.

⭐ Simple Example: Abstract Class in SystemVerilog

Abstract Base Class

```
class Animal;

  // Pure virtual method → makes this class abstract
  pure virtual function void sound();
endclass
```

- `Animal` is abstract
- You cannot do `Animal a = new();` → ERROR

⭐ **Child Class (Must Override pure virtual method)**

```
class Dog extends Animal;

  function void sound();
    $display("Dog barks!");
  endfunction
endclass

class Cat extends Animal;
  function void sound();
    $display("Cat meows!");
  endfunction
endclass
```

Child classes must write their own `sound()` method.

⭐ **Using the Classes**

```
module test;
  Animal a;    // handle of abstract class
  Dog d;
  Cat c;

  initial begin
    // a = new();  // ❌ ERROR → cannot create object of Animal

    d = new();
    c = new();

    a = d;      // dynamic polymorphism
    a.sound();  // Dog barks!

    a = c;
    a.sound();  // Cat meows!
  end
endmodule
```

🎯 **Why do we use Abstract Classes?**

1. To create a common template for child classes
2. To ensure child classes must implement some methods
3. For polymorphism in verification
4. Used heavily in UVM (drivers, sequences, monitors)

⭐ Real Verification Example

```
### Abstract transaction class
class transaction;
  rand bit [7:0] addr;

  pure virtual function void print();
endclass

### Child class must define print()
class write_tran extends transaction;

  bit [31:0] data;

  function void print();
    $display("Write addr=%0h data=%0h", addr, data);
  endfunction
endclass
```

🧠 Summary (Very Simple)

Concept	Meaning
Abstract Class	A class you cannot create an object of
Pure Virtual Method	A function without a body
Child must override	Yes, required
Why used?	Polymorphism + templates

❌virtual class is ALWAYS ABSTRACT.

That means:

👉 You cannot create an object of a virtual class

👉 Even if it has no pure virtual methods

👉 So this line causes the error:

p_tr = new(); // ❌ **ERROR — cannot instantiate virtual class**

☑ Why?

- A virtual class is an abstract class, and cannot be instantiated.

So the following is illegal:

virtual class parent_trans;

✓ How to solve the error?

Fix 1: Remove `virtual` keyword

If you want to create an object, then remove `virtual`.

Correct Code:

✓ Fix 2: Keep `virtual`, but use a child class

If you want `virtual class` (for inheritance), then create a child class and instantiate the child.

Example:

```
virtual class parent_trans;
  bit [31:0] data;
  int id;

  function void display();
```

```
    $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
endclass

module class_example;
  initial begin
    child_trans p_tr;  // instantiate child, NOT parent
    p_tr = new();

    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

Simulation output:

```
Base: Value of data = 5 and id = 1
```

Summary

Class Type	Can instantiate?	Rule
`class`	✓ Yes	Can create object
`virtual class`	❌ No	ALWAYS abstract
class with pure virtual method	❌ No	Also abstract

Your error comes from:

p_tr = new(); // instantiating virtual class → **ILLEGAL**

PURE VIRTUAL function

We will use:

- A pure virtual method
- A virtual (abstract) parent class
- A child class that overrides the pure virtual method
- A module to run the simulation

⭐ PURE VIRTUAL VERSION OF YOUR EXAMPLE

🔹 **1. Abstract Parent Class (cannot be instantiated)**

virtual class parent_trans;

  bit [31:0] data;

  int id;

  // Pure virtual function → must be overridden in child class

  pure virtual function void display();

endclass

Explanation:

- `virtual class parent_trans` → an abstract class
- Variables: `data` and `id`
- `pure virtual function void display();`
- No body
- Every child class must implement display()

🔹 2. Child Class (implements display())

class child_trans extends parent_trans;

```
  function void display();
    $display("Child: data = %0h and id = %0h", data, id);
  endfunction
endclass
```

Explanation:

- `child_trans` extends the parent class
- It implements the `display()` function

Now this class is instantiable

🔹 3. Module to Test the Classes

```
module class_example;
  initial begin
    parent_trans p_tr;   // handle of abstract class
    child_trans c_tr;    // actual object

    c_tr = new();        // create child object

    p_tr = c_tr;         // polymorphism

    p_tr.data = 5;
    p_tr.id = 1;

    p_tr.display();      // calls child version
  end
endmodule
```

⭐ FINAL SIMULATION OUTPUT

Child: data = 5 and id = 1

✓ Because `display()` is implemented in child class

✓ And `p_tr` is pointing to `c_tr` → (runtime polymorphism)

🎯 Why This Works

Concept	Meaning
Pure virtual function	No body, must override
Virtual class	Cannot create object
Child class	Implements pure virtual function
Polymorphism	Parent handle pointing to child object

An abstract class with a derived class

A derived class can be extended from an abstract class is the same as how it is extended normally using the extends keyword. The child class is instantiated as shown in the below example.

```
virtual class parent_trans;
  bit [31:0] data;
  int id;


  function void display();
     $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass
```

```
class child_trans extends parent_trans;
  function void display();
    $display("Child: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
  initial begin
    child_trans c_tr;
    c_tr = new();

    c_tr.data = 5;
    c_tr.id = 1;
    c_tr.display();
  end
endmodule
```

Simulation results:

Child: Value of data = 5 and id = 1

An abstract class with child class handle assignment

In the below example, an object for the child class is created and its handle is assigned to its parent class handle. This is to show that the parent class handle can access its method.

```
virtual class parent_trans;
  bit [31:0] data;
  int id;

  function void display();
```

```
    $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  function void display();
    $display("Child: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

Simulation Output:

Base: Value of data = 5 and id = 1

An abstract class with a virtual method

```
virtual class parent_trans;
```

```
  bit [31:0] data;
  int id;

  virtual function void display();
     $display("Base: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  function void display();
    $display("Child: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

Simulation Output:

Child: Value of data = 5 and id = 1

An abstract class with pure virtual methods

A method in an abstract class may or may not be overridden by its derived classes. Sometimes, if it is necessary to write a specific method for different derived classes, the pure virtual method is useful.

A pure virtual method makes it mandatory for methods to be implemented in derived classes whose prototype has been specified in an abstract class.

1. The pure virtual method has no implementation mentioned in the base class. It is just a prototype of the method.
2. A pure virtual method and its implementation in derived class shall use the same signature i.e. same arguments and return type if applicable.

Pure virtual method example

- Pure virtual methods include the *pure virtual function* and the *pure virtual task*.

```
virtual class parent_trans;
  bit [31:0] data;
  int id;

  pure virtual function void display();
endclass

class child_trans extends parent_trans;
  function void display();
    $display("Child: Value of data = %0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
```

```
 initial begin
   parent_trans p_tr;
   child_trans c_tr;
   c_tr = new();

   p_tr = c_tr;
   p_tr.data = 5;
   p_tr.id = 1;
   p_tr.display();
 end
endmodule
```

Simulation output:

Session-10: -

- **Interprocess Communications**
- **Events**
- **Semaphore**
- **Mailbox**

Interprocess communication is a way to communicate between processes or testbench components. SystemVerilog provides three mechanisms for communication.

1. Events
2. Semaphores
3. Mailbox

Types of events

- -> use for bocking statement. (Immediate or instantaneous operator)
- ->> use to trigger non-blocking events (small delta cycle time not immediately).
- @ event is triggered (edge sensitive operator).
- wait (level sensitive operator).

How two or more processes are synchronized?

For example, there are two processes A and B. The process_A task is used to trigger an event e1 and the process_B task is used to wait for the event using @ operator.

Type A: An event is triggered after waiting for the event trigger

```
module event_example();
event e1;
task process_A();
        #10;
        $display("@%0t: Before triggering event e1", $time);
        ->e1;
      $display("@%0t: After triggering event e1", $time);
endtask

task process_B();
        $display("@%0t: waiting for the event e1", $time);
        @e1;
      $display("@%0t: event e1 is triggered", $time);
endtask

initial begin
        fork
                process_A();
            process_B();
        join
end
endmodule
```

Simulation Output:

@0: waiting for the event e1

@10: Before triggering event e1

@10: After triggering event e1

@10: event e1 is triggered

V C S S i m u l a t i o n R e p o r t

Type B: an event is triggered before waiting for event trigger

```
module event_example();
event e1;

task process_A();
        $display("@%0t: Before triggering event e1", $time);
        ->e1;
      $display("@%0t: After triggering event e1", $time);
endtask

task process_B();
        #10;
        $display("@%0t: waiting for the event e1", $time);
        @e1;
      $display("@%0t: event e1 is triggered", $time);
endtask

initial begin
        fork
              process_A();
            process_B();
```

```
    join
end
endmodule
```

Simulation Output:

```
@0: Before triggering event e1
@0: After triggering event e1
@10: waiting for the event e1
        V C S  S i m u l a t i o n  R e p o r t
Time: 10
```

Type C: An event is triggered at the same time as waiting for the event trigger

```
module event_example();
event e1;

task process_A();
    $display("@%0t: Before triggering event e1", $time);
    ->e1;
    $display("@%0t: After triggering event e1", $time);
endtask

task process_B();
    $display("@%0t: waiting for the event e1", $time);
    @e1;
    $display("@%0t: event e1 is triggered", $time);
endtask

initial begin
```

```
    fork
        process_A();
        process_B();
    join
end
endmodule
```

Simulation Output:

```
@0: Before triggering event e1
@0: After triggering event e1
@10: waiting for the event e1
        V C S  S i m u l a t i o n  R e p o r t
Time: 10
```

II. Event is triggered using -> and waiting for System Verilog event to be triggered via wait() construct

For example, there are two processes A and B. The process_A task is used to trigger an event e1 and the process_B task is used to wait for the event using the wait() construct.

Type A: An event is triggered after waiting for the event trigger.

The process_A task has a 10ns delay which makes sure event e1 triggers after waiting for the event trigger. The wait of the event to be triggered via wait() construct will be unblocked once the e1 event is triggered.

```
module event_example();
event e1;

task process_A();
    #10;
    $display("@%0t: Before triggering event e1", $time);
```

```
        ->e1;
        $display("@%0t: After triggering event e1", $time);
endtask

task process_B();
        $display("@%0t: waiting for the event e1", $time);
       wait(e1.triggered);
       $display("@%0t: event e1 is triggered", $time);
endtask

initial begin
        fork
                process_A();
                process_B();
        join
end
endmodule
```

Simulation Output:

```
@0: waiting for the event e1
@10: Before triggering event e1
@10: After triggering event e1
@10: event e1 is triggered
        V C S  S i m u l a t i o n  R e p o r t
Time: 10
```

Type B: An event is triggered before waiting for event trigger

The process_B task has a 10ns delay which makes sure event e1 triggers before waiting for an event trigger. The wait of the event to be triggered via wait() construct will not be

unblocked since the e1 event is triggered before. Hence, statements after waiting for the trigger (with wait() construct) will not be executed.

```
module event_example();
event e1;

task process_A();
        $display("@%0t: Before triggering event e1", $time);
      ->e1;
      $display("@%0t: After triggering event e1", $time);
endtask

task process_B();
        #10;
      $display("@%0t: waiting for the event e1", $time);
      wait(e1.triggered);
      $display("@%0t: event e1 is triggered", $time);
endtask

initial begin
        fork
                process_A();
            process_B();
        join
end
endmodule
```

Simulation Output:

```
@0: Before triggering event e1
@0: After triggering event e1
@10: waiting for the event e1
```

V C S S i m u l a t i o n R e p o r t

Time: 10

Type C: An event is triggered at the same time as waiting for the event trigger.

The process_A and process_B have no delay involved to ensure triggering of an event and waiting for even triggers happens at the same time and wait () construct will detect an event triggering.

```
module event_example();
event e1;

task process_A();
    $display("@%0t: Before triggering event e1", $time);
  ->e1;
  $display("@%0t: After triggering event e1", $time);
endtask

task process_B();
  $display("@%0t: waiting for the event e1", $time);
  wait(e1.triggered);
  $display("@%0t: event e1 is triggered", $time);
endtask

initial begin
    fork
        process_A();
      process_B();
    join
end
endmodule
```

Simulation Output:

@0: Before triggering event e1

@0: After triggering event e1

@0: waiting for the event e1

@0: event e1 is triggered

V C S S i m u l a t i o n R e p o r t

Time: 0

Difference between @(event) and wait (event.triggered)

@(event): The waiting for an event using @ operator blocks the current process until an event is triggered. In a certain condition like waiting for an event and triggering an event can occur at the same time, this will cause race conditions between them and triggering an event will be missed if the wait for an event happens earlier.

wait(event.triggered): The waiting for an event using the wait() construct will unblock the waiting process even if an event is triggered at the time. Thus, the wait() construct eliminates race around condition between waiting for an event and triggering an event.

In short, wait() construct catches an event triggering at the same simulation whereas @ operator waiting for an event would lead to race conditions.

To explain using an example, three processes execute in the same simulation time.

process_A: Triggers at event e1;

process_B: wait for event e1 using @ operator

process_C: wait for event e1 using wait() construct

The process_C will be unblocked due to event e1 triggering whereas process B is blocked due to race around condition.

Race around condition

```
module event_example();

event e1;


task process_A();
        $display("@%0t: Process A: Before triggering event e1", $time);
        ->e1;
```

```
        $display("@%0t: Process A: After triggering event e1", $time);
endtask

task process_B();
          $display("@%0t: Process B: waiting for the event e1 using @", $time);
          @e1;
        $display("@%0t: Process B: event e1 is triggered using @", $time);
endtask

task process_C();
          $display("@%0t: Process C: waiting for the event e1 using wait(e1.triggered)",
$time);
          wait(e1.triggered);
        $display("@%0t: Process C: event e1 is triggered using wait(e1.triggered)", $time);
endtask
initial begin
          fork
                    process_A();
               process_B();
               process_C();
          join
end
endmodule
```

Simulation Output:

```
@0: Process A: Before triggering event e1
@0: Process A: After triggering event e1
@0: Process B: waiting for the event e1 using @
```

@0: Process C: waiting for the event e1 using wait(e1.triggered)

@0: Process C: event e1 is triggered using wait(e1.triggered)

V C S S i m u l a t i o n R e p o r t

Time: 0

Non-Blocking Events

The non-blocking event (using –>>) is triggered in the non-blocking region of the time slot. Ultimately, event triggering using ->> is a delayed version of the event triggering using ->. Hence, the process_B was waiting for the event using @ operator is completed.

```
module event_example();
event e1;

task process_A();
    $display("@%0t: process_A: Before triggering event e1 using ->>",$time);
    ->>e1;
    $display("@%0t: process_A: After triggering event e1 using ->>", $time);
endtask

task process_B();
    $display("@%0t: process_B: waiting for the event e1", $time);
    @(e1.triggered);
    $display("@%0t: process_B: event e1 is triggered", $time);
endtask

initial begin
    fork
        process_A();
        process_B();
```

```
    join
end
endmodule
```

Output:

```
@0: process_A: Before triggering event e1 using ->>
@0: process_A: After triggering event e1 using ->>
@0: process_B: waiting for the event e1
@0: process_B: event e1 is triggered
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Wait_order in SV events

In order event example

```
module event_example();
event e1, e2, e3;


task process_A();
    #5;
    ->e1;
    $display("@%0t: process_A: event e1 is triggered", $time);
endtask


task process_B();
    #15;
    ->e2;
    $display("@%0t: process_B: event e2 is triggered", $time);
endtask
```

```
task process_C();
        #10;
        ->e3;
        $display("@%0t: process_C: event e3 is triggered", $time);
endtask

  // wait for event triggering in order (e1, e3, e2)
task wait_process();
        $display("@%0t: waiting for the events e1, e2, e3", $time);
        wait_order(e1, e3, e2)
$display("Events are triggered in order");
else
        $display("Events are triggered out of order");
endtask

initial begin
        fork
                process_A();
            process_B();
            process_C();
            wait_process();
        join
end
endmodule
```

Output:

```
@0: waiting for the events e1, e2, e3
@5: process_A: event e1 is triggered
```

@10: process_C: event e3 is triggered

@15: process_B: event e2 is triggered

Events are triggered in order

V C S S i m u l a t i o n R e p o r t

Time: 15

Out of order event example

```
module event_example();
event e1, e2, e3;

task process_A();
        #5;
        ->e1;
        $display("@%0t: process_A: event e1 is triggered", $time);
endtask

task process_B();
        #15;
        ->e2;
        $display("@%0t: process_B: event e2 is triggered", $time);
endtask

task process_C();
        #10;
        ->e3;
        $display("@%0t: process_C: event e3 is triggered", $time);
endtask
```

```
  // wait for event triggering in out of order (e2, e1, e3)
task wait_process();
        $display("@%0t: waiting for the events e2, e1, e3", $time);
        wait_order(e2, e1, e3)
$display("Events are triggered in order");
else
        $display("Events are triggered out of order");
endtask

initial begin
        fork
              process_A();
            process_B();
            process_C();
            wait_process();
        join
end
endmodule
```

Output:

```
@0: waiting for the events e2, e1, e3
@5: process_A: event e1 is triggered
Events are triggered out of order
@10: process_C: event e3 is triggered
@15: process_B: event e2 is triggered
        V C S  S i m u l a t i o n  R e p o r t
Time: 15
```

Merging evens in SV

Merging events example

There are two events e1 and e2.

In process_A, an event e2 variable is assigned with the e1 variable and an event e2 is triggered.

In process_B, the wait() construct is waiting for event e1 to be triggered.

Due to event merging, even though process_B is unblocked due to e2 event triggering even though it is waiting for event e1.

```
module event_example();

event e1, e2;


task process_A();
        e2 = e1;
        #10;
        ->e2;
        $display("@%0t: process_A: event e2 is triggered", $time);
endtask


task process_B();
        $display("@%0t: process_B: waiting for the event e1", $time);
        wait(e1.triggered);
$display("@%0t: process_B: event e1 is received", $time);
endtask
initial begin
        fork
                process_A();
                process_B();
        join
end
```

```
endmodule
```

Output:

```
@0: process_B: waiting for the event e1
@10: process_A: event e2 is triggered
@10: process_B: event e1 is received
        V C S  S i m u l a t i o n  R e p o r t
Time: 10
```

Passing an event in SV

Passing an event in the class constructor

When a class object is created, an event is passed to the constructor as shown in the below example.

```
class transaction;
        event tr_e;

        function new(event e1);
                tr_e = e1;
        endfunction

        task process_A();
                #10;
                ->tr_e;
            $display("@%0t: process_A: tr_e is triggered", $time);
        endtask

        task process_B();
                $display("@%0t: process_B: waiting for the event tr_e", $time);
                wait(tr_e.triggered);
        $display("@%0t: process_B: event tr_e is received", $time);
```

```
endtask
endclass

module event_example();
transaction tr;
event ev;

initial begin
        tr = new(ev);
        fork
                tr.process_A();
                tr.process_B();
        join
end
endmodule
```

Output:

```
@0: process_B: waiting for the event tr_e
@10: process_A: tr_e is triggered
@10: process_B: event tr_e is received
        V C S  S i m u l a t i o n  R e p o r t
Time: 10
```

Passing an event as an argument in the task or function

Passing an event as an argument in the task or function

- **An event can also be passed to a function or task as below.**

```
module event_example();
```

```
function process_A(event e1);
        ->e1;
        $display("@%0t: process_A: e1 is triggered", $time);
endfunction

initial begin
        event e1;

        fork
                process_A(e1);
        begin
                $display("@%0t: process_B: waiting for the event e1", $time);
                wait(e1.triggered);
        $display("@%0t: process_B: event e1 is received", $time);
end
join
end
endmodule
```

Output:

```
@0: process_A: e1 is triggered
@0: process_B: waiting for the event e1
@0: process_B: event e1 is received
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

semaphores

System Verilog semaphores are used to control the access of shared resources. It is a built-in class in System Verilog used for synchronization which is a container that contains a fixed number of keys.

For example, the same memory location is accessed by two different cores. To avoid unexpected results when cores try to write or read from the same memory location, a semaphore can be used.

Semaphore Examples

Semaphore having a single key

To understand how semaphore provides controlled access to the shared resource (memory in the below example), compare the output of the below two codes.

In the below examples, the write_mem task takes 5ns to write into memory and the read_mem takes 4ns to read data from memory. Two different process wants to access memory which is not aware that another one is accessing at the same time, in such a cases semaphore is useful.

A.0 WITHOUT USING SEMAPHORES

```
module semaphore_example();

task write_mem();
    $display("Before writing into memory");
    #5ns;  // Assume 5ns is required to write into mem
    $display("Write completed into memory");
endtask


task read_mem();
    $display("Before reading from memory");
    #4ns;  // Assume 4ns is required to read from mem
    $display("Read completed from memory");
```

```
endtask

initial begin
    fork
        write_mem();
        read_mem();
    join
end
endmodule
```

Output:

```
Before writing into memory
Before reading from memory
Write completed into memory
Read completed from memory
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

B.0 Using Semaphore

```
module semaphore_example();
semaphore sem = new(1);

task write_mem();
      sem.get();
      $display("Before writing into memory");
    #5ns  // Assume 5ns is required to write into mem
    $display("Write completed into memory");
      sem.put();
endtask
```

```
task read_mem();
        sem.get();
        $display("Before reading from memory");
        #4ns  // Assume 4ns is required to read from mem
        $display("Read completed from memory");
        sem.put();
endtask

initial begin
        fork
                write_mem();
                read_mem();
        join
end
endmodule
```

Output:

```
Before writing into memory
Write completed into memory
Before reading from memory
Read completed from memory
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

How semaphores reduced complexity

Semaphore having multiple keys

The semaphore is created with 3 keys as shown in the below example.

process_A requires all 3 keys whereas process_B requires 2 keys to access the resource.

Semaphore having multiple keys

```
module semaphore_example();
semaphore sem = new(3);

task process_A();
        sem.get(3);
      $display("process_A started");
      #5ns;
      $display("process_A completed");
      sem.put(3);
endtask

task process_B();
        sem.get(2);
      $display("process_B started");
      #4ns;
      $display("process_B completed");
      sem.put(2);
endtask

initial begin
        fork
                process_A();
                process_B();
        join
end
```

endmodule

Output:

process_A started

process_A completed

process_B started

process_B completed

V C S S i m u l a t i o n R e p o r t

Time: 0

Put more keys back

Put more keys back

In the below example, 3 keys are obtained using the get method whereas 5 keys are put back. It is one of the ways to manage resources. Note that no error is observed in the example since keys allocation is initialized, it is not like fixed keys are assigned.

```
module semaphore_example();

semaphore sem = new(3);

task process();
    sem.get(3);
    $display("process is started");
    #5ns;
    $display("process is completed");
    sem.put(5);
endtask


initial begin
    process();
end
endmodule
```

Output:

process is started

process is completed

V C S S i m u l a t i o n R e p o r t

Time: 0

Put more keys back

Let's try to access more keys than initialized keys when process_A puts back extra keys i.e. 5 keys and same numbers keys are accessed with get() call in process_B. Thus, process_B will get executed. But process_C requires 6 keys that are no longer available which will block process_C execution.

```
module semaphore_example();
semaphore sem = new(3);


task process_A();
        sem.get(3);
      $display("process_A is started");
      #5ns;
      $display("process_A is completed");
      sem.put(5);
endtask


task process_B();
        sem.get(5);  // Accessing more keys than initialized
      $display("process_B is started");
      #5ns;
      $display("process_B is completed");
```

```
        sem.put(5);
endtask

task process_C();
          sem.get(6);  // Accessing more keys than available in the bucket
        $display("process_C is started");
        #5ns;
        $display("process_C is completed");
        sem.put(5);
endtask
initial begin
          fork
                  process_A();
              process_B();
              process_C();
          join
end
endmodule
```

Output:

```
process_A is started
process_A is completed
process_B is started
process_B is completed
          V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Try_get() example

try_get() example

try_get() is a non-blocking semaphore method where execution is not blocked if keys are not available. In the below example, process_A obtained a key at first and even if a key is not available, process_B execution is not blocked.

Output:

Before writing into memory

Before reading from memory

Write completed into memory

Read completed from memory

V C S S i m u l a t i o n R e p o r t

Time: 0

```
module semaphore_example();
semaphore sem = new(1);

task process_A();
        if(sem.try_get())
                $display("process_A: Key received");
        else
                $display("process_A: Key is not available");
        $display("process_A started");
        #5ns;
      $display("process_A completed");
      sem.put();
endtask

task process_B();
        if(sem.try_get())
                $display("process_B: Key received");
```

```
        else
                $display("process_B: Key is not available");
        $display("process_B started");
        #4ns;
      $display("process_B completed");
      sem.put();
endtask

initial begin
        fork
                process_A();
                process_B();
        join
end
endmodule
```

Output:

```
process_A: Key received
process_A started
process_B: Key is not available
process_B started
process_A completed
process_B completed
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Mailbox

A System Verilog mailbox is a way of communication between different processes to exchange data. One process can put data into a mailbox that stores data internally and can be retrieved by another process. Mailbox behaves as first-in, first-out (FIFO).

Mailbox Types

1. Generic mailbox
2. Parameterized mailbox

These mailboxes can be further categorized based on the size as

1. Bounded mailbox
2. Unbounded mailbox

Generic mailbox

The generic mailbox can be put or get data of any data_type like int, bit, byte, string, etc. By default, the mailbox is a typeless or generic mailbox.

Syntax: `mailbox <mailbox_name>`

Parameterized mailbox

The parameterized mailbox can be put or get data of particular data_type. The parameterized mailbox is useful when data_type needs to be fixed. For differences in data_type, a compilation error is expected.

Syntax: mailbox #(<type>) <mailbox_name>

Example: mailbox #(int) mb;

Bounded mailbox

If the size of the mailbox is defined then it is a bounded mailbox. When the mailbox is full, no further data can be put in the mailbox until an item or data is get from the mailbox.

Unbounded mailbox

The size is not defined. An unbounded mailbox has unlimited size.

Mailbox methods

Method name	Description
function new(int bound = 0)	Returns mailbox handle. An argument represents bounded mailbox size otherwise, it is an unbounded mailbox
task put(<data>)	Blocking method that stores data in the mailbox.
function int try_put(<data>)	The non-blocking method that stores data in the mailbox if it is not full and returns 1 else 0.

task get(ref <data>)	Blocking method to retrieve data from the mailbox
function int try_get(ref <data>)	The non-blocking method which returns data if a mailbox is non-empty else returns 0.
task peek(ref <data>)	Copies data from the mailbox without removing it from a mailbox
function int try_peek(ref <data>)	Tries to copy data from the mailbox without removing it from a mailbox
function int num()	Returns number of entries in the mailbox

Mailbox Types

Generic mailbox: Bounded mailbox example

Mailbox Examples

Bounded mailbox (Generic mailbox) example

```
module mailbox_example();
mailbox mb = new(3);

task process_A();
    int value = 5;
    string name = "SARIF";
  mb.put(value);
  $display("Put data = %0d", value);
  mb.put("SARIF");   // mb.put(name); correct way
  $display("Put data = %s", name);
endtask

task process_B();
    int value;
```

```
        string name;
        mb.get(value);
        $display("Retrieved data = %0d", value);
        mb.get(name);
        $display("Retrieved data = %s", name);
endtask

initial begin
        fork
                process_A();
                process_B();
        join
end
endmodule
```

Output:

```
Put data = 5
Put data = SARIF
Retrieved data = 5
Retrieved data = SARIF
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Generic mailbox: Unbounded mailbox example

Unbounded mailbox example

```
module mailbox_example();
mailbox mb = new();
```

```
task process_A();
        int value;
        repeat(10) begin
      value = $urandom_range(1, 50);
      mb.put(value);
      $display("Put data = %0d", value);
end
$display("----------------------");
endtask

task process_B();
        int value;
        repeat(10) begin
      mb.get(value);
        $display("Retrieved data = %0d", value);
end
endtask

initial begin
        fork
                process_A();
            process_B();
        join
end
endmodule
```

Output:

Put data = 40

Put data = 49

Put data = 20

Put data = 48

Put data = 7

Put data = 3

Put data = 20

Put data = 26

Put data = 19

Put data = 17

Retrieved data = 40

Retrieved data = 49

Retrieved data = 20

Retrieved data = 48

Retrieved data = 7

Retrieved data = 3

Retrieved data = 20

Retrieved data = 26

Retrieved data = 19

Retrieved data = 17

V C S S i m u l a t i o n R e p o r t

Time: 0

Parameterized mailbox example

```
module mailbox_example();
mailbox #(string) mb = new(3);
```

```
task process_A();
        string name = "Alex";
      mb.put(name);
      $display("Put data = %s", name);
      name = "Robin";
      mb.put(name);
      $display("Put data = %s", name);
endtask

task process_B();
        string name;
      mb.get(name);
      $display("Retrieved data = %s", name);
      mb.get(name);
      $display("Retrieved data = %s", name);
endtask

initial begin
        fork
              process_A();
           process_B();
        join
end
endmodule
```

Output:

Put data = Alex

Put data = Robin

Retrieved data = Alex

Retrieved data = Robin

V C S S i m u l a t i o n R e p o r t

Time: 0

Put and get method

```
module mailbox_blocking;
mailbox mb = new(2); // Mailbox with size 2

task producer();
    int i;
    for (i = 1; i <= 3; i++) begin
        $display("Producer: Trying to put %0d", i);
        mb.put(i); // Blocks if the mailbox is full
        $display("Producer: Successfully put %0d", i);
    end
endtask

task consumer();
    int value;
    repeat(3) begin
        #5; // Small delay before consuming
        $display("Consumer: Waiting for data...");
        mb.get(value); // Blocks if the mailbox is empty
        $display("Consumer: Got value = %0d", value);
```

```
        end
endtask
initial begin
        fork
                producer();
            consumer();
        join
end
endmodule
```

Output:

```
Producer: Trying to put 1
Producer: Successfully put 1
Producer: Trying to put 2
Producer: Successfully put 2
Producer: Trying to put 3
Consumer: Waiting for data...
Consumer: Got value = 1
Producer: Successfully put 3
Consumer: Waiting for data...
Consumer: Got value = 2
Consumer: Waiting for data...
Consumer: Got value = 3
        V C S  S i m u l a t i o n  R e p o r t
Time: 15
```

Non-Blocking Example (try_put and try_get)

Try_get , try_put example

```
module mailbox_example();
mailbox mb = new(3);

task process_A();
        int value;
        repeat(5) begin
                value = $urandom_range(1, 50);
                if(mb.try_put(value))
                        $display("successfully try_put data = %0d", value);
                else begin
                        $display("failed while try_put data = %0d", value);
                        $display("Number of messages in the mailbox = %0d", mb.num());
                end
        end
        $display("-------------------------------------");
endtask

task process_B();
        int value;
        repeat(5) begin
                if(mb.try_get(value))
                        $display("Successfully retrieved try_get data = %0d", value);
                else begin
                        $display("Failed in try_get data");
                   $display("Number of messages in the mailbox = %0d", mb.num());
                end
```

```
        end
endtask

initial begin
        fork
                process_A();
            process_B();
        join
end
endmodule
```

Output:

```
successfully try_put data = 40
successfully try_put data = 49
successfully try_put data = 20
failed while try_put data = 48
Number of messages in the mailbox = 3
failed while try_put data = 7
Number of messages in the mailbox = 3
---------------------------------------
Successfully retrieved try_get data = 40
Successfully retrieved try_get data = 49
Successfully retrieved try_get data = 20
Failed in try_get data
Number of messages in the mailbox = 0
Failed in try_get data
Number of messages in the mailbox = 0
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

Peek and try_peek example

```
module mailbox_example();
mailbox mb = new(3);

task process_A();
        int value;
        repeat(3) begin
                value = $urandom_range(1, 50);
                mb.put(value);
                $display("put data = %0d", value);
        end
        $display("--------------------------------");
endtask

task process_B();
        int value;
        mb.peek(value); // message is not removed
        $display("peek data = %0d", value);
        mb.peek(value); // message is not removed
        $display("peek data = %0d", value);

        if(mb.try_peek(value))
                $display("Successful try_peek data = %0d", value);
        else begin
```

```
            $display("Failed in try_peek");
        end
        $display("---------------------------------");

        repeat(3) begin
            mb.get(value);
            $display("get data = %0d", value);
        end
        $display("---------------------------------");

        if(mb.try_peek(value))
            $display("Successful try_peek data = %0d", value);
        else begin
            $display("Failed in try_peek");
        end
endtask

initial begin
        fork
            process_A();
          process_B();
        join
end
endmodule
```

Output:

```
put data = 40
put data = 49
```

put data = 20

---

peek data = 40

peek data = 40

Successful try_peek data = 40

---

get data = 40

get data = 49

get data = 20

Failed in try_peek

V C S S i m u l a t i o n R e p o r t

Time: 0

peek and try_peek.

- peek()

#Meaning:

Reads the first item in the mailbox without removing it.

#Key point:

✓ Data stays inside the mailbox.

Example:

mb.peek(value);

$display("peek = %0d", value);

If mailbox has: `[10, 20, 30]`

peek → returns 10 (but mailbox still remains `[10, 20, 30]`)

- try_peek()

#Meaning:

Checks mailbox and reads the first item only if available

(but does NOT remove it).

#Key point:

✓ It does NOT block

✓ Returns 1 if data exists

✓ Returns 0 if mailbox is empty

#Example:

```
if (mb.try_peek(value))
  $display("try_peek = %0d", value);
else
  $display("mailbox empty");
```

If mailbox is empty → prints "mailbox empty".

Summary (very short)

Method	Removes data?	Blocks if empty?	Return value
peek()	❌ No	✓ Yes (waits)	None
try_peek()	❌ No	❌ No (non-blocking)	1 or 0

Session-11: -

- Randomization in SV
- If else in constraints
- Implication operator in constraint
- Foreach loop in constraint
- Dist keyword in constraints
- Inheritance in constraints
- Function in constraint
- Disable randomization

Randomization in SV

Randomization is a process of producing random values of the mentioned data type.

As System Verilog also deals with objects, the *$random* method from Verilog is not sufficient for randomizing an object.

Need for Randomization

As per the increasing complexity of the design, there are high chances to have more bugs in the design when it is written for the first time. To verify DUT thoroughly, a verification engineer needs to provide many stimuli. There can be multiple cross combinations of variables in a real system. So, it is not possible practically to write directed cases to verify every possible combination. So, it is very much required to have randomization in the verification testbench.

Rand and randc keywords

rand and randc keywords

To randomize a class object, the following keywords are used while declaring class variables.

1. rand
2. randc

rand Keyword

On randomizing an object, the rand keyword provides uniformly distributed random values.

```
rand bit [4:0] value;
```

On randomizing, any values within 5'h0 to 5'h1F will be generated with equal probability

randc Keyword

On randomizing an object, the randc keyword provides random value without repeating the same value unless a complete range is covered. Once all values are covered, the value will repeat. This ensures that to have all possible values without repeating the same value unless every value is covered.

```
randc bit [1:0] value;   // Possible values = 0, 1, 2, 3
```

Possible random value generated: 2, 3, 1, 0, 3, 2, 0, 1..

Constraints in randomization

Constraints in randomization

In a design-specific scenario, DUT works only for a particular range of inputs as per feature. To satisfy this need, constraint becomes extremely important in randomization. Constraint block represents constraints for random variables.

Syntax for constraint block:

```
constraint <constraint_name> {<expression/ condition>; }
```

1. Constraints block must have a unique name within a class.
2. Constraint block can call a function to get constraint values as the return value from the function.
3. Constraints are written inside curly braces.
4. For a particular class, constraint blocks can be written inside a class or outside a class using *extern* keywords.
5. Expressions or conditions can be written inside a constraint block.

Inside keyword in constraints

Inside keyword in constraints

The inside keyword is helpful when randomized values have to be in the provided range.

The provided range in a bracket can be constant, parameter, define, or variable.

Syntax:

```
constraint <constraint_name> {<variable> inside {. . . .}; }
```

To specify range of values

```
constraint <constraint_name> {<variable> inside {[10:20]}; }
```

To specify set of values

```
constraint <constraint_name> {<variable> inside {40, 70, 80}; }
```

Combination of set of values and range

```
constraint <constraint_name> {<variable> inside {4, 7, 8, [10:20], 25, 30,
[40:70]}; }
```

Define based range in constraint

```
constraint <constraint_name> {<variable> inside {[`START_RANGE:`END_RANGE]}; }
```

Inside keyword in constraints

Variable based range in constraint

```
constraint <constraint_name> {<variable> inside {[<var1>:<var2>]}; }
```

Parameter based range in constraint

```
constraint <constraint_name> {<variable> inside {[<param1>:<param2>]}; }
```

Inverted inside constraint

```
constraint <constraint_name> { !(<variable> inside {[10:20]}); }
```

Inside constraint Examples

```
//
`define START_RANGE 35
`define END_RANGE 45


class seq_item #(parameter int p1 = 10, p2 = 20);
    rand bit [7:0] value1;
    rand bit [7:0] value2;
    rand bit [7:0] value3;
    rand bit [7:0] value4;
    rand bit [7:0] value5;
```

```
    rand bit [7:0] value6;
    rand bit [7:0] value7;

    constraint value1_c {value1 inside {[10:20]};} // constant value based range
    constraint value2_c {value2 inside {40,70, 80};} // Set of values
    constraint value3_c {value3 inside {[10:20], 21, 23, [25:30], 40, 70, 80};} // Mix
    constraint value4_c {!(value4 inside {[100:200]});} // Inverted Range
    constraint value5_c {value5 inside {[value1:value2]};} // range using variable
    constraint value6_c {value6 inside {[`START_RANGE:`END_RANGE]};} // Define based range
    constraint value7_c {value7 inside {[p1:p2]};} // parameter based range
endclass

module constraint_example;
seq_item #(100, 200) item;

initial begin
    item = new();

    repeat(3) begin
        item.randomize();
        $display("value1 = %0d, value2 = %0d, value3 = %0d, value4 = %0d, value5 = %0d, value6 = %0d, value7 = %0d", item.value1, item.value2, item.value3, item.value4, item.value5, item.value6,  item.value7);
    end
end
endmodule
```

Output:

value1 = 16, value2 = 40, value3 = 20, value4 = 208, value5 = 33, value6 = 43, value7 = 104

value1 = 15, value2 = 70, value3 = 10, value4 = 12, value5 = 54, value6 = 43, value7 = 113

value1 = 14, value2 = 70, value3 = 17, value4 = 240, value5 = 32, value6 = 40, value7 = 146

V C S S i m u l a t i o n R e p o r t

Time: 0

Inside constraint Examples

Implement 'randc' function in SystemVerilog

In the below implementation, the *my_randc* function is used to mimic randc behavior for 3 bits '*data*' variable. The '*mask*' having a width of 8 bits (= 2^3) is used to check whether the *data* value has been covered already or not.

```
module tb;

  bit [2:0] data;  // variable which provide random value

  bit [7:0] mask;


  function bit [2:0] my_randc;

    while(1) begin

      data = $random;

      if(!mask[data]) begin

        mask[data] = 1;

        return data;

      end

      else if(&mask) begin

        mask = 0;

        mask[data] = 1;

        break;

      end
```

```
    end
    return data;
  endfunction

  initial begin
    repeat(3) begin
      repeat(8)
        $display("data = %0d", my_randc());
      $display("------------");
    end
  end
endmodule
```

Output:

```
data = 4
data = 1
data = 3
data = 5
data = 2
data = 6
data = 7
data = 0
------------
data = 5
data = 4
data = 3
data = 2
data = 0
```

```
data = 6
data = 7
data = 1
------------
data = 0
data = 1
data = 3
data = 6
data = 4
data = 2
data = 5
data = 7
------------
```

If else in constraints

If else in constraints

Based on the If-else condition, values for the variable can be assigned. It is similar to a normal if-else condition.

Example for if-else

```
class seq_item;
    rand bit [7:0] value;
    rand enum {LOW, HIGH} scale;

    constraint scale_c { if(scale == LOW) value < 50;
                else value >= 50;
               }
endclass
```

```
module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        $display("scale = %s,value =%0d",item.scale.name(),item.value);
    end
end
endmodule
```

Output:

```
scale = HIGH,value =150
scale = HIGH,value =121
scale = HIGH,value =192
scale = HIGH,value =104
scale = LOW,value =34
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Example for if-else if-else

```
class seq_item;
    rand bit [7:0] value;
    rand enum {LOW, MID, HIGH} scale;

    constraint scale_c { if(scale == LOW) value < 30;
    else if(scale == MID) { value >= 30; value <= 70; }
```

```
        else value > 70;}
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();
    repeat(10) begin

        item.randomize();
        $display("scale = %s,value =%0d",item.scale.name(),item.value);
    end
end
endmodule
```

Output:

```
scale = HIGH,value =150
scale = HIGH,value =124
scale = HIGH,value =192
scale = MID,value =69
scale = MID,value =36
scale = HIGH,value =118
scale = LOW,value =18
scale = MID,value =47
scale = MID,value =67
scale = HIGH,value =150
```

V C S S i m u l a t i o n R e p o r t

Time: 0

Example for the conditional operator

```
class seq_item;
    rand bit [7:0] value;
    rand enum {LOW, HIGH} scale;

    constraint scale_c { value == ((scale == LOW) ? 20: 90); }
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();
    repeat(5) begin
        item.randomize();
        $display("scale = %s,value =%0d",item.scale.name(),item.value);
    end
end
endmodule
```

Output:

```
scale = HIGH,value =90
scale = LOW,value =20
scale = HIGH,value =90
scale = LOW,value =20
scale = LOW,value =20
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

Implicaton Operator in constraint

Implication operator in constraint

Implication operator (->) declares the relation between two variables. For an implication operator in constraint, it declares the relation between expression and constraint.

If the LHS expression of -> holds true, the RHS constraint is considered.

Syntax:

```
<expression> -> <constraint>
```

Implication operator in constraint example

```
class seq_item;
    rand bit [7:0] value;
    rand enum {LOW, HIGH} scale;

    constraint scale_c { (scale == LOW) -> value <50; }
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        $display("scale =%s,value = %0d",item.scale.name(),item.value);
    end
```

end

endmodule

Output:

scale =HIGH,value = 150

scale =HIGH,value = 121

scale =HIGH,value = 192

scale =HIGH,value = 104

scale =LOW,value = 33

V C S S i m u l a t i o n R e p o r t

Time: 0

foreach loop in constraint

Same as foreach loop, array constraints can also be implemented using foreach loop to iterate over array elements.

Syntax:

```
constraint <constraint_name> { foreach(variable[i]) variable[i] <condition>}
```

foreach loop in constraint example

foreach loop in constraints

typedef enum {LOW, MID1, MID2, HIGH} scale;

class seq_item;

rand bit [7:0] value_a[scale];

rand bit [3:0] array[];

constraint arr_size_c { value_a.size() == 4; array.size() inside {[2:5]};}

constraint array_c { foreach(array[i]) { array[i] > i*i; } }

constraint value_a_c {foreach(value_a[i]) {

```
                value_a[i] < 100;
                (i == LOW) -> value_a[i] < 30;
                (i == HIGH) -> value_a[i] > 70;
                (i == MID1) -> value_a[i] inside {[30:50]};
                (i == MID2) -> value_a[i] inside {[51:70]};
              }
            }
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        foreach(item.value_a[i]) $display("value[%s] = %0d", i.name(), item.value_a[i]);
        foreach(item.array[i]) $display("array[%0d] = %0d", i, item.array[i]);
    end
end
endmodule
```

Output:

```
value[LOW] = 15
value[MID1] = 37
value[MID2] = 54
```

value[HIGH] = 86

array[0] = 14

array[1] = 10

value[LOW] = 23

value[MID1] = 39

value[MID2] = 54

value[HIGH] = 96

array[0] = 15

array[1] = 3

value[LOW] = 17

value[MID1] = 47

value[MID2] = 59

value[HIGH] = 73

array[0] = 5

array[1] = 15

array[2] = 15

array[3] = 13

value[LOW] = 11

value[MID1] = 38

value[MID2] = 51

value[HIGH] = 76

array[0] = 4

array[1] = 11

array[2] = 11

array[3] = 15

value[LOW] = 22

value[MID1] = 43

value[MID2] = 58

value[HIGH] = 79

array[0] = 11

array[1] = 2

array[2] = 6

V C S S i m u l a t i o n R e p o r t

Time: 0

Dist keyword in constraints

dist is used to give **weighted randomization**.

“Some values should come more often than others.”

dist controls probability of random values.

Dist keyword in constraints

The dist keyword is helpful whenever to have weighed distribution during randomization. The probability random value occurrence can be controlled using the dist keyword.

How to allocate weighted distribution?

There are two ways to provide weightage for values.

1. Using :/ operator
2. Using := operator

Syntax for the operator:

value :/ weightage

value := weightage

//Value - Random value of the variable

//Weightage - weight for random variable i.e. occurrence of value can be specified.

Note: The sum of weightage is not necessary to be 100.

:/ operator

1. For specific value: Assign mentioned weight to that value
2. For range of values *([<range1>: <range2>])*: Assigns weight/(number of value) to each value in that range

:/ operator Example

```
constraint value_c {value dist {3:/4, [5:8] :/ 7}; }
```

//Elaboration:

value = 3. weightage = 4

value = 5. weightage = 7/4 = 1.75

value = 6. weightage = 7/4 = 1.75

value = 7. weightage = 7/4 = 1.75

value = 8. weightage = 7/4 = 1.75

:= operator

For a specific value or range of value, the mentioned weight is assigned.

:= operator Example

```
constraint value_c {value dist {3:=4, [5:8] := 7}; }
```

//Elaboration:

value = 3. weightage = 4

value = 5. weightage = 7

value = 6. weightage = 7

value = 7. weightage = 7

value = 8. weightage = 7

:/ and := Operator Example

In the below example, both operators are used. Notice how the occurrence of values is changed.

:/ operator and := operator Example

```
class seq_item;
    rand bit [7:0] value1;
    rand bit [7:0] value2;

    constraint value1_c {value1 dist {3:/4, [5:8] :/ 7}; }
    constraint value2_c {value2 dist {3:=4, [5:8] := 7}; }
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        $display("value1 (with :/) = %0d, value2 (with :=)= %0d", item.value1, item.value2);
    end
end
endmodule
```

Output:

```
value1 (with :/) = 3, value2 (with :=)= 8
value1 (with :/) = 5, value2 (with :=)= 5
value1 (with :/) = 3, value2 (with :=)= 7
value1 (with :/) = 3, value2 (with :=)= 6
```

value1 (with :/) = 3, value2 (with :=)= 6

V C S S i m u l a t i o n R e p o r t

Time: 0

Inheritance in constraint

Inheritance in constraint

Constraint blocks for a parent class can be overridden by its child class. Thus, the inherited class can modify constraints based on the requirement. To do the same, constraint block nomenclature must be the same.

Inheritance in constraint Example

```
class parent;
        rand bit [5:0] value;
        constraint value_c {value > 0; value < 10;}
endclass

class child extends parent;
        constraint value_c {value inside {[10:30]};}
endclass

module constraint_inh;
parent p;
child c;

initial begin
        p = new();
      c = new();

        repeat(3) begin
```

```
            p.randomize();
            $display("Parent class: value = %0d", p.value);
        end

        repeat(3) begin
            c.randomize();
            $display("Child class: value = %0d", c.value);
        end
end
endmodule
```

Output:

```
Parent class: value = 2
Parent class: value = 3
Parent class: value = 6
Child class: value = 11
Child class: value = 30
Child class: value = 14
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Function in constraint

- SystemVerilog allows you to **call a function from inside a constraint block**.
- You can call functions inside constraints.
- Function must be **pure**.
- Makes constraints cleaner and reusable.

```
class seq_item;
    rand bit [5:0] value;
    rand bit sel;
```

```
        constraint value_c {value == get_values(sel);}

        function bit [5:0] get_values(bit sel);
                return (sel? 'h10: 'h20);
        endfunction
endclass

module constraint_example;
seq_item item;

initial begin
        item = new();

        repeat(3) begin
                item.randomize();
                $display("constraint value = %0h", item.value);
        end
        $display("On functiopn call: value = %0h", item.get_values(1));
end
endmodule
```

Output:

```
constraint value = 10
constraint value = 20
constraint value = 10
On functiopn call: value = 10
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Disable randomization

Disable randomization

Randomization can be disabled using the rand_mode method.

1. rand_mode() is a function which returns 1 if randomization is enabled else returns 0.
2. By default, rand_mode is enabled. I.e. rand_mode(1)
3. To disable randomization, rand_mode(0) is used.
4. Randomization can be enabled once again for the previously disabled case.
5. Randomization needs to be disabled or enabled (if disabled before) before calling the randomize() method.

Methods of using rand_mode

Complete class randomization can be disabled

```
<object_handle>.rand_mode(0);
```

Particular variable randomization can be disabled

```
<object_handle>.<variable>.rand_mode(0);
```

Methods of using rand_mode

```
class seq_item;
        rand bit [7:0] value1;
        rand bit [7:0] value2;

        constraint value1_c {value1 inside {[10:30]};}
      constraint value2_c {value2 inside {40,70, 80};}
endclass

module constraint_example;
seq_item item;
```

```
initial begin
        item = new();


        item.randomize();
        $display("Before disabling randomization: value1 = %0d, value2 = %0d", item.value1, item.value2);
        item.rand_mode(0);  // To disable randomization for all class variables
        item.randomize();
        $display("After disabling randomization for all variables in a class (Retain old values): value1 = %0d, value2 = %0d", item.value1, item.value2);


        item.rand_mode(1);  // To enable randomization
      item.randomize();
        $display("After enabling randomization: value1 = %0d, value2 = %0d", item.value1, item.value2);


        item.value2.rand_mode(0);  // To disable randomization for value2 variable alone
      item.randomize();
      $display("After disabling randomization for value2 variables in a class: value1 = %0d, value2 = %0d", item.value1, item.value2);


      $display("rand_mode function returns for value1 = %0d, value2 = %0d", item.value1.rand_mode(), item.value2.rand_mode());
end
endmodule
```

Output:

```
Before disabling randomization: value1 = 14, value2 = 70
```

After disabling randomization for all variables in a class (Retain old values): value1 = 14, value2 = 70

After enabling randomization: value1 = 15, value2 = 80

After disabling randomization for value2 variables in a class: value1 = 22, value2 = 80

rand_mode function returns for value1 = 1, value2 = 0

V C S S i m u l a t i o n R e p o r t

Time: 0

Disable constraints

Disable Constraint

Similar to disabling randomization, the constraint can also be disabled.

1. constraint_mode() is a function which returns 1 if constraint is enabled and else returns 0.
2. By default, constraint mode is enabled i.e. constraint_mode(1)
3. To disable constraint, constraint_mode(0) is used.
4. A constraint can be enabled once again for the previously disabled case.
5. A Constraint needs to be disabled or enabled (if disabled before) before calling the randomize() method.

Disable Constraint Example

```
class seq_item;
  rand bit [7:0] value1;
  rand bit [7:0] value2;

  constraint value1_c {value1 inside {[10:30]};}
  constraint value2_c {value2 inside {40,70, 80};}

endclass

module constraint_example;
  seq_item item;
```

```
  initial begin
    item = new();


    item.randomize();
    $display("Before disabling constraint");
    $display("item: value1 = %0d, value2 = %0d", item.value1, item.value2);


    item.value2_c.constraint_mode(0);  // To disable constraint for value2 using handle item2
    item.randomize();
    $display("After disabling constraint for all value2 alone");
    $display("item: value1 = %0d, value2 = %0d", item.value1, item.value2);
    $display("constraint_mode function returns for value1 = %0d, value2 = %0d",item.value1_c.constraint_mode(), item.value2_c.constraint_mode());
  end
endmodule
```

Output:

```
Before disabling constraint
item: value1 = 14, value2 = 70
After disabling constraint for all value2 alone
item: value1 = 15, value2 = 63
constraint_mode function returns for value1 = 1, value2 = 0
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Session-12: -

- Static constraints

- Unique constraints
- Bidireactional constraints
- Solve before in constraints
- Inline constraints
- Soft constraints
- Randomization methods
- Randcase in sv

Static constraints

static constraint applies to ALL objects, not only one.

Constraints **cannot** be static.

This constraint is **shared by all objects**.

Both follow:

- value1: 10–30
- value2: must be **40, 70, or 80** (static constraint applies to both)

value1 → random between 10–30

value2 → either 40, 70, or 80

item2.value2_c.constraint_mode(0);

Normally this disables a constraint **for only one object**.

But because your constraint is **static**, meaning:

👉 **It belongs to the class, not the object.**
👉 Disabling through any object disables it for **all objects**.

So both item1 and item2 lose the constraint for value2.

Now:

- value1 → still 10–30 (normal constraint)
- value2 → **fully random (0–255)** because static constraint is disabled globally

item1: value1 = 18, value2 = 222

item2: value1 = 22, value2 = 89

Summary (Very Simple)

✓ value1 has a normal constraint → always 10–30
✓ value2 has a **static constraint** → applies to ALL objects
✓ Disabling static constraint removes the rule **from the entire class**, not just one object
✓ After disabling → both objects get **random** value2

```
class seq_item;
  rand bit [7:0] value1;
  rand bit [7:0] value2;

  constraint value1_c {value1 inside {[10:30]};}
  static constraint value2_c {value2 inside {40,70, 80};}

endclass

module constraint_example;
  seq_item item1, item2;

  initial begin
    item1 = new();
    item2 = new();

    item1.randomize();
    item2.randomize();
    $display("Before disabling constraint");
```

```
    $display("item1: value1 = %0d, value2 = %0d", item1.value1, item1.value2);
    $display("item2: value1 = %0d, value2 = %0d", item2.value1, item2.value2);


    item2.value2_c.constraint_mode(0);  // To disable constraint for value2 using handle item2
    item1.randomize();
    item2.randomize();
    $display("After disabling constraint for all value2 alone");
    $display("item1: value1 = %0d, value2 = %0d", item1.value1, item1.value2);
    $display("item2: value1 = %0d, value2 = %0d", item2.value1, item2.value2);
  end
endmodule
```

Output:

```
Before disabling constraint
item1: value1 = 14, value2 = 70
item2: value1 = 11, value2 = 70
After disabling constraint for all value2 alone
item1: value1 = 15, value2 = 63
item2: value1 = 30, value2 = 72
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Unique constraint

Unique constraint

A **unique constraint** ensures that **all random values in a set are different**.

In simple words:

No two variables can have the same value.

As the name suggests, a unique constraint is useful to generate unique values for variables and elements in an array (Fixed array, associative array, dynamic array, and queue).

Syntax:

```
constraint <constraint name> {unique {array or variable};}
```

Unique constraint example

```
typedef enum {M0, M1, M2, M3} scale_e;

class seq_item;

    rand bit [7:0] value_a[scale_e];

  rand bit [3:0] array[5];

  rand bit [2:0] val1, val2, val3, val4;

    scale_e scale;


    constraint array_c { unique {array}; }

  constraint value_a_c { unique {value_a};

            value_a.size == scale.num;

            foreach (value_a[i]) value_a[i] inside {[50:60]}; }

  constraint val_c {unique {val1, val2, val3, val4}; }

endclass


module constraint_example;

seq_item item;


initial begin

    item = new();


    repeat(5) begin

        item.randomize();
```

```
        $display("val1 = %0d, val2 = %0d, val3 = %0d, val4 = %0d", item.val1, item.val2, item.val3, item.val4);

        foreach(item.value_a[i]) $display("value_a[%s] = %0d", i.name(), item.value_a[i]);

        foreach(item.array[i]) $display("array[%0d] = %0d", i, item.array[i]);

    end

end

endmodule
```

Output:

```
val1 = 7, val2 = 4, val3 = 5, val4 = 3

value_a[M0] = 60

value_a[M1] = 53

value_a[M2] = 58

value_a[M3] = 56

array[0] = 14

array[1] = 3

array[2] = 15

array[3] = 6

array[4] = 9

val1 = 1, val2 = 2, val3 = 7, val4 = 3

value_a[M0] = 51

value_a[M1] = 58

value_a[M2] = 53

value_a[M3] = 52

array[0] = 2

array[1] = 3

array[2] = 13

array[3] = 9
```

array[4] = 12

val1 = 4, val2 = 7, val3 = 2, val4 = 1

value_a[M0] = 60

value_a[M1] = 56

value_a[M2] = 50

value_a[M3] = 53

array[0] = 2

array[1] = 4

array[2] = 10

array[3] = 14

array[4] = 7

val1 = 5, val2 = 2, val3 = 4, val4 = 3

value_a[M0] = 57

value_a[M1] = 55

value_a[M2] = 53

value_a[M3] = 58

array[0] = 2

array[1] = 15

array[2] = 10

array[3] = 13

array[4] = 5

val1 = 3, val2 = 1, val3 = 4, val4 = 7

value_a[M0] = 50

value_a[M1] = 57

value_a[M2] = 58

value_a[M3] = 54

array[0] = 13

```
array[1] = 12
array[2] = 9
array[3] = 11
array[4] = 2
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Example2

```
class packet;
  rand bit [3:0] a, b, c;

  constraint unique_c {
    unique {a, b, c};   // all must be different
  }
endclass

//packet p = new();
module unique_example;

initial begin
        packet p = new();

        repeat(5) begin
                p.randomize();
                $display("a=%0d b=%0d c=%0d", p.a, p.b, p.c);
        end
end
endmodule
```

Output:

a=6 b=7 c=11

a=9 b=1 c=10

a=12 b=1 c=10

a=8 b=5 c=4

a=4 b=10 c=8

V C S S i m u l a t i o n R e p o r t

Time: 0

Bidirectional constraint

A **bidirectional constraint** means:

Either variable can take the lead in solving the equation.
The solver is free to choose the direction.

In short:

✓ Both sides are **free variables**
✓ The constraint **does not force one variable to depend on the other**
✓ The solver can adjust **any** variable to satisfy the constraint

solves constraints parallelly for all random variables and makes sure no constraint fails.
While solving the constraint, the value of a variable can be impacted because of another variable.

```
class seq_item;
    rand bit [7:0] val1, val2, val3, val4;
    rand bit t1, t2;

    constraint val_c {val2 > val1;
        val3 == val2 - val1;
        val4 < val3;
```

```
        val4 == val1/val3; }

    constraint t_c { (t1 == 1) -> t2 == 0;}
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        $display("val1 = %0d, val2 = %0d, val3 = %0d, val4 = %0d", item.val1, item.val2, item.val3, item.val4);
        $display("t1 = %0h, t2 = %0h", item.t1, item.t2);
    end
end
endmodule
```

Output:

```
val1 = 117, val2 = 208, val3 = 91, val4 = 1
t1 = 0, t2 = 0
val1 = 116, val2 = 254, val3 = 138, val4 = 0
t1 = 1, t2 = 0
val1 = 14, val2 = 21, val3 = 7, val4 = 2
t1 = 0, t2 = 0
val1 = 46, val2 = 187, val3 = 141, val4 = 0
```

t1 = 0, t2 = 1

val1 = 97, val2 = 128, val3 = 31, val4 = 3

t1 = 0, t2 = 1

V C S S i m u l a t i o n R e p o r t

Time: 0

Example: Enum + Functions

```
module enum_example;

  enum {RED, GREEN, BLUE} color;

  initial begin
    color = RED;

    // Print Enum Value as int
    $display("Value = %0d", color);            // 0

    // Convert enum to string
    $display("Name = %s", color.name());       // "RED"

    // Get next value
    color = color.next();
    $display("Next = %s", color.name());       // "GREEN"

    // Get last value
    $display("Last = %s", color.last().name()); // "BLUE"

    // Number of enum items
```

```
    $display("Size = %0d", color.num());       // 3
  end
endmodule
```

Function	Meaning
`name()`	Returns string name of enum value
`next()`	Returns next enum value
`prev()`	Returns previous enum value
`first()`	Returns first enum item
`last()`	Returns last enum item
`num()`	Returns total enum items
`next(n)`	Next value after n steps
`prev(n)`	Previous value before n steps

Solve before in constraints

Solve Before in constraints

As discussed in the bidirectional constraint, there is a possibility that the value of a variable can impact another variable value due to the bidirectional nature.

By default, a constraint solver has an equal probability solving algorithm. In certain cases, there is a requirement to solve constraints in a certain order. This changes the probability of value occurrence.

Important things to note:

1. The dependency order of variables should not be bidirectional.
   For example: solve x before y; and solve y before x; // This is not allowed
2. Randc is not allowed.
3. Only integers are allowed.

Example without solve before

constraint en_c { if(en == 1) { val inside {[0:100]}; } }

This is a constraint using **if inside constraint**.

Meaning:

- If **en = 1**,
  then **val must be between 0 to 100**.
- If **en = 0**,
  there is **no restriction** → val can be 0–255.

This is called **conditional constraint**.

Example without solve before

```
class seq_item;
    rand bit [7:0] val;
    rand bit en;

    constraint en_c { if(en == 1) { val inside {[0:100]}; } }
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        $display("en = %0d, val = %0d", item.en, item.val);
    end
end
```

```
endmodule
```

Output:

```
en = 0, val = 150
en = 0, val = 115
en = 0, val = 64
en = 0, val = 209
en = 0, val = 18
        V C S   S i m u l a t i o n   R e p o r t
Time: 0
```

Example with solve before

- Set the order or preference of solving

```
class seq_item;
    rand bit [7:0] val;
    rand bit en;

    constraint en_c {solve en before val;
    if(en == 1) { val inside {[0:100]}; } }
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
```

```
        $display("en = %0d, val = %0d", item.en, item.val);
    end
end
endmodule
```

Output:

```
en = 1, val = 11
en = 0, val = 115
en = 1, val = 32
en = 0, val = 209
en = 0, val = 18
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Inline constraints

Inline constraints are constraints written **directly inside the randomize() call**, instead of writing them inside the class.

You use them when you want **temporary or one-time constraints** while randomizing.

✓ They do **not permanently change** the class constraint.
✓ They apply **only for that single randomize call**.

An inline constraint is written on calling a *randomize()* method using the *"with"* keyword.

Inline constraints Example

```
class seq_item;
    rand bit [7:0] val1, val2;

    constraint val1_c {val1 > 100; val1 < 200;}
   constraint val2_c {val2 > 5; val2 < 80;}
```

```
endclass

module constraint_example;
seq_item item;

initial begin
        item = new();

        repeat(5) begin
                item.randomize();
                $display("Before inline constraint: val1 = %0d, val2 = %0d", item.val1,
item.val2);
                item.randomize with {val1 > 150; val1 < 160;};
            item.randomize with {val2 inside {[10:15]};};
                $display("After inline constraint: val1 = %0d, val2 = %0d", item.val1,
item.val2);
        end
end
endmodule
```

Output:

```
Before inline constraint: val1 = 121, val2 = 75
After inline constraint: val1 = 161, val2 = 12
Before inline constraint: val1 = 176, val2 = 42
After inline constraint: val1 = 113, val2 = 13
Before inline constraint: val1 = 194, val2 = 76
After inline constraint: val1 = 161, val2 = 13
Before inline constraint: val1 = 128, val2 = 42
After inline constraint: val1 = 104, val2 = 14
```

Before inline constraint: val1 = 114, val2 = 79

After inline constraint: val1 = 179, val2 = 15

V C S S i m u l a t i o n R e p o r t

Time: 0

Explanation:

- No extra constraints
- Value2 = 0–255 (random)
- Value2 = 10–15 (random)

This adds temporary new rules:

- data must be **less than 10:15**

These constraints are used **only this one time**.

Inline constraints

soft constraint

By default, constraints are hard constraints in nature. To mention constraints as a soft, specifically *"soft"* keyword has to be used.

- A **soft constraint** is a *weak constraint*.
  SystemVerilog will try to follow it, **but it can be overridden** by:
- Soft constraints give a **default value**, but allow overriding when needed.
- Soft constraint is ignored.

Soft constraint

Example without soft constraint

class seq_item;

rand bit [7:0] val;

```
        constraint val_c {val inside {5, [10:15]};}
endclass

module constraint_example;
seq_item item;

initial begin
        item = new();

        repeat(5) begin
                item.randomize();
                $display("Before inline constraint: val = %0d", item.val);

                item.randomize with {val inside {[20:30]};};
                $display("After inline constraint: val = %0d", item.val);
        end
end
endmodule
```

Output:

```
Error-[CNST-CIF] Constraints inconsistency failure
example.sv, 18
  Constraints are inconsistent and cannot be solved.
  Please check the inconsistent constraints being printed above and rewrite
  them.

After inline constraint: val = 13
        V C S  S i m u l a t i o n  R e p o r t
```

Time: 0

Example with soft constraint

```
class seq_item;
    rand bit [7:0] val;

    constraint val_c {soft val inside {5, [10:15]};}
endclass

module constraint_example;
seq_item item;

initial begin
    item = new();

    repeat(5) begin
        item.randomize();
        $display("Before inline constraint: val = %0d", item.val);

        item.randomize with {val inside {[20:30]};};
        $display("After inline constraint: val = %0d", item.val);
    end
end
endmodule
```

Output:

```
Before inline constraint: val = 11
After inline constraint: val = 23
Before inline constraint: val = 12
```

After inline constraint: val = 21

Before inline constraint: val = 11

After inline constraint: val = 23

Before inline constraint: val = 10

After inline constraint: val = 22

Before inline constraint: val = 12

After inline constraint: val = 27

V C S S i m u l a t i o n R e p o r t

Time: 0

Randomization methods

Randomization methods

The *randomize()* method call is used to randomize class variables based on constraints if they are written. Along with *randomize()* method, SystemVerilog provides two callbacks

1. pre_randomize()
2. post_randomize()

A sequence of execution of methods:

```
pre_randomize() -> randomize() -> post_randomize()
```

pre_randomize method

It is used to do an activity just before randomization. This may involve disabling constraint for a particular variable (constraint_mode(0))or disabling randomization itself (rand_mode(0)). Refer disable randomization for more details.

post_randomize method

It is used to do an activity after randomization. This may involve printing randomized values of a class variable. Override the randomized value of a class variable.

pre_randomize and post_randomize methods Examples

A basic example

A basic example with pre_randomization and post_randomization methods which also include constraint_mode(0).

pre and post_randomize Examples

```
class seq_item;
        rand bit [7:0] val1;
        rand bit [7:0] val2;


          constraint val1_c {val1 > 100; val1 < 200;}
          constraint val2_c {val2 > 5; val2 < 8;}

          function void pre_randomize();
                  $display("Inside pre_randomize");
                  val2_c.constraint_mode(0);
          endfunction

          function void post_randomize();
                  $display("Inside post_randomize");
                  $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
          endfunction
endclass

module constraint_example;
seq_item item;

initial begin
```

```
        item = new();
        item.randomize();
  end
endmodule
```

Output:

```
Inside pre_randomize
Inside post_randomize
val1 = 121, val2 = 247
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Explanation:

You are creating a class named seq_item for randomization.

Random Variables

`val1` and `val2` are randomized 8-bit variables.

When you call `.randomize()`, these values will get random numbers.

Constraints

val1 must be between 101 and 199.

val2 must be 6 or 7.

pre_randomize()

```
   val2_c.constraint_mode(0);
```

This function runs automatically before randomization.

What happens here?

1. Prints message → `"Inside pre_randomize"`

2. `val2_c.constraint_mode(0);`

Disables the constraint for `val2`

Means `val2` is now free (no constraints applied)

`val1` constraint still active

So, during randomization:

val1 will follow constraint (100–200)

val2 will NOT follow constraint (6,7) → val2 can be any 8-bit random value.

post_randomize()

This runs after randomization.

It does:

1. Prints `"Inside post_randomize"`

2. Prints final values of `val1` and `val2`

☑ MODULE

Creating an object variable of the class

Initial Block

```
item = new();      // create object
item.randomize();  // start randomization
```

What happens during `item.randomize()`?

SystemVerilog performs steps:

1. Calls pre_randomize()

2. Applies constraints and randomizes

3. Calls post_randomize()

Explanation:

- val1 is within 100–200
- val2 is random (constraint disabled) → can be 0–255

Summary (Very Simple)

- `val1` follows constraint
- `val2` constraint disabled inside pre_randomize
- post_randomize prints final values

Example2

1. What is Randomization? (Simple Explanation)

Randomization means System Verilog automatically generates random values for variables marked with:

- rand
- randc

You use .randomize() to generate the random values.

2. What is pre_randomize()?

This function runs before randomization starts.

Used to initialize, reset, or prepare data.

3. What is post_randomize()?

This function runs after randomization finishes.

Used to print results, calculate extra values, or check conditions.

Example (Covers All Concepts)

```
class item;
  rand bit [7:0] value;   // random variable
  bit double_value;

  // Runs BEFORE randomization
  function void pre_randomize();
    $display("pre_randomize: Starting randomization...");
  endfunction

  // Runs AFTER randomization
  function void post_randomize();
    double_value = value * 2;  // extra calculation
```

```
    $display("post_randomize: value=%0d, double=%0d", value, double_value);
  endfunction
endclass

module tb;
  item i;

  initial begin
    i = new();
    // Randomize the class object
    i.randomize();
  end
endmodule
```

Explanation (Very Simple)

1rand bit [7:0] value;

SystemVerilog will generate a random 8-bit value.

2.0 i.randomize();

Randomization starts.

SystemVerilog automatically performs these steps:

#Step 1 → pre_randomize()

Runs first

pre_randomize: Starting randomization

Used to prepare data.

#Step 2 → Randomization happens

Example:

value = 37 (random)

#Step 3 → post_randomize()

Runs after randomization

post_randomize: value=37, double=74

Used to calculate extra values or print information.

Final Output Example

pre_randomize: Starting randomization...

post_randomize: value=37, double=74

Summary (Short & Simple)

Method	Purpose	When it Runs
randomize()	Creates random values	When called
pre_randomize()	Setup before randomizing	Before randomize
post_randomize()	Work after randomizing	After randomize

Example with rand_mode

In pre_randomize() method, rand_mode(0) is used to disable randomization. So, in post_randomization() initial values will be printed.

```
class seq_item;
        rand bit [7:0] val1 = 101; //Initialized to avoid constraint failure
        rand bit [7:0] val2 = 6;

      constraint val1_c {val1 > 100; val1 < 200;}
      constraint val2_c {val2 > 5; val2 < 8;}
```

```
    function void pre_randomize();
            $display("Inside pre_randomize");
            this.rand_mode(0);
      endfunction

      function void post_randomize();
            $display("Inside post_randomize");
            $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
    endfunction

endclass

module constraint_example;
seq_item item;

initial begin
      item = new();
      item.randomize();
end
endmodule
```

Output:

```
Inside pre_randomize
Inside post_randomize
val1 = 101, val2 = 6
      V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Example with a derived class

Part A: pre_randomization and post_randomization methods defined in a base and derived class

The base class and child class handle call respective pre_randomization and post_randomization methods as mentioned in the below example.

```
class seq_item;
      rand bit [7:0] val1;
     rand bit [7:0] val2;

     constraint val1_c {val1 > 100; val1 < 200;}
     constraint val2_c {val2 > 5; val2 < 8;}

     function void pre_randomize();
          $display("Inside pre_randomize");
     endfunction

     function void post_randomize();
          $display("Inside post_randomize");
          $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
     endfunction
endclass

class child_item extends seq_item;
      function void pre_randomize();
          $display("Inside pre_randomize of child_item class");
      endfunction

      function void post_randomize();
          $display("Inside post_randomize of child_item class");
```

```
        $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
    endfunction
endclass

module constraint_example;
seq_item item;
child_item c_item;

initial begin
    item = new();
    c_item = new();

    item.randomize();
    c_item.randomize();
end
endmodule
```

Output:

```
Inside pre_randomize
Inside post_randomize
val1 = 121, val2 = 7
Inside pre_randomize of child_item class
Inside post_randomize of child_item class
val1 = 107, val2 = 7
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Part B: pre_randomization and post_randomization methods defined in a base class only

The child class handle calls pre_randomization and post_randomization methods of the base class as mentioned in the below example.

```
class seq_item;
        rand bit [7:0] val1;
       rand bit [7:0] val2;

       constraint val1_c {val1 > 100; val1 < 200;}
       constraint val2_c {val2 > 5; val2 < 8;}

       function void pre_randomize();
                $display("Inside pre_randomize");
         endfunction

       function void post_randomize();
                $display("Inside post_randomize");
             $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
         endfunction
endclass

class child_item extends seq_item;
  // No method implemented.
endclass

module constraint_example;
seq_item item;
child_item c_item;

initial begin
        item = new();
```

```
    c_item = new();

      item.randomize();
      c_item.randomize();
end
endmodule
```

Output:

```
Inside pre_randomize
Inside post_randomize
val1 = 121, val2 = 7
Inside pre_randomize
Inside post_randomize
val1 = 107, val2 = 7
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

Note:

1. If randomization is failed post_randomization will not be called and the variable will retain previous values.
2. Pre and post-randomization functions can not be virtual functions.

Randcase in SystemVerilog

randcase

Randcase is a case statement that randomly selects one of its branch statements based on the probability of each statement.

Syntax:

```
randcase
  item 1: <statement 1>;
  item 2: <statement 2>;
  item 3: <statement 3>;
```

```
  ...
  …
  item n: <statement n>;
endcase
```

How to calculate the probability of each branch statement?

The probability of any item is calculated by the value of an item divided by the sum of all item values.

Ex: P(item1) = item1/sum(item 1..n)

randcase can also be written inside the module.

randcase Example

```
class seq_item;
        int cnt_arr[int];
        real i_sum;

        function void randcase_testing();
                repeat(10) begin
                        randcase
                                2: begin $display("Selected 2"); cnt_arr[2]++; end
                                3: begin $display("Selected 3"); cnt_arr[3]++; end
                                5: begin $display("Selected 5"); cnt_arr[5]++; end
                                7: begin $display("Selected 7"); cnt_arr[7]++; end
                        endcase
                end

                foreach(cnt_arr[i]) begin
                        i_sum += i;
```

```
            end

            foreach(cnt_arr[i]) begin
                $display("Probability for %0d = %0f in 1 iteration", i, i/i_sum);
                $display("cnt_arr[%0d] = %0d", i, cnt_arr[i]);
              $display("Probability for cnt_arr[%0d] = %0f", i, cnt_arr[i]/10.0);
                $display("--------------------------------");end
        endfunction
endclass
module constraint_example;
seq_item item;

initial begin
        item = new();
      item.randcase_testing();
end
endmodule
```

Output:

```
Selected 7
Selected 5
Selected 2
Selected 2
Selected 3
Selected 7
Selected 7
Selected 7
Selected 7
```

Selected 5

Probability for 2 = 0.117647 in 1 iteration

cnt_arr[2] = 2

Probability for cnt_arr[2] = 0.200000

Probability for 3 = 0.176471 in 1 iteration

cnt_arr[3] = 1

Probability for cnt_arr[3] = 0.100000

Probability for 5 = 0.294118 in 1 iteration

cnt_arr[5] = 2

Probability for cnt_arr[5] = 0.200000

Probability for 7 = 0.411765 in 1 iteration

cnt_arr[7] = 5

Probability for cnt_arr[7] = 0.500000

V C S S i m u l a t i o n R e p o r t

Time: 0

……..

randcase is like a random switch-case.

But instead of selecting a case based on a value…

it selects a case randomly based on weights.

Syntax

randcase

  weight1 : statement1;

```
  weight2 : statement2;
  weight3 : statement3;
endcase
```

Higher weight → higher chance of selection

Weight = probability

Example2

```
module tb;
  int x;

  initial begin
    randcase
      70 : x = 1;   // 70% chance
      20 : x = 2;   // 20% chance
      10 : x = 3;   // 10% chance
    endcase

    $display("x = %0d", x);
  end
endmodule
```

☑ Explanation

x = 1 has the highest probability (70%)

x = 2 medium probability (20%)

x = 3 lowest probability (10%)

Each time you run the program, one case is chosen randomly based on these weights.

Example Output (may vary)

x = 1

(But sometimes you may get 2 or 3 depending on probability)

Key Points (Very Short)

- randcase = random selection with weights
- Used when you want controlled randomness
- Weights decide probability

Session-13: -

- Coverage
- Code coverage
- Functional coverage

What is Coverage ?

Coverage is used in verification to check **how much of your design and test scenarios are tested**.

It helps you answer:
👉 *"Did my testcases really test everything?"*
👉 *"What parts of the design are still untested?"*

Need for Coverage

1. As you are aware that verification is a long-lasting process and we never know what type of input stimulus can capture the bug. Hence, it is essential to have a set of metrics that decide the endpoint for verification of the design once all metrics are satisfied.
2. It is also essential to see whether we verified all kinds of features supported by the design and cover every line from the design code.

There are **two main types of coverage:**

1. **Code Coverage** – Automatically measured by simulator

2. **Functional Coverage** – Written manually by verification engineer

Code Coverage

Code coverage deals with covering design code metrics. It tells how many lines of code have been exercised (tested) w.r.t. block, expression, FSM, signal toggling.

The code coverage is further divided as

1. Block coverage – To check how many lines of code have been covered.
2. Expression coverage – To check whether all combinations of inputs have been driven to cover expression completely.
3. FSM coverage – To check whether all state transitions are covered.
4. Toggle coverage – To check whether all bits in variables have changed their states.

Note:

1. Code coverage does not specify that the code behavior is correct or not. It is simply used to identify uncovered lines, expressions, state transitions, dead code, etc. in the design. Hence, it does not indicate design quality.
2. Verification engineers aim to achieve 100% code coverage.
3. There are industry tools available that show covered and missing code in code coverage.

Functional Coverage

Functional coverage deals with covering design functionality or feature metrics. It is a user-defined metric that tells about how much design specification or functionality has been exercised. The functional coverage can be classified into two types

1. Data intended coverage – To check the occurrence of data value combinations.
   Example: Writing different data patterns in a register.
2. Control intended coverage – To check the occurrence of sequences in the intended fashion.
   Example: Reading a register to retrieve reset values after releasing a system reset.

Note:

Since it is user-defined metrics, it is up to the verification engineer to consider all features because if some features are missed to add in functional coverage and remaining features are covered then functional coverage will show up 100% even though some cover points are missed to add.

Types of coverage

1. Code Coverage (AUTO by simulator)

Measures how much of your RTL code is executed.

Includes:

- **Line coverage** – Did each line of code run?
- **Toggle coverage** – Did each signal go 0→1 and 1→0?
- **Branch coverage** – Did all if-else paths run?
- **FSM coverage** – Did all states & transitions occur?

Example:

```
if(a > b)
  x = 1;
else
  x = 0;
```

To get 100% branch coverage:

- One test must make **a > b**
- One test must make **a <= b**

You don't write code coverage manually. Simulator gives it.

2.Functional Coverage (User-written)

Used to check **if all required scenarios were tested**.

You write **covergroups**, **coverpoints**, and **bins**.

Simple Example of Functional Coverage

Example 1: Covering a 2-bit signal

```
bit [1:0] mode;

covergroup cg;
  coverpoint mode;
endgroup
```

cg cov = new();

Whenever you do:

cov.sample();

Tool checks which values of mode were seen.

Values covered: 0,1,2,3.

Line Coverage

Line / Statement Coverage

Line coverage checks:
👉 Did every executable line of RTL code run at least once?

- It helps you know **which lines were executed and which lines were not tested**.
- This is **automatically measured** by the simulator (VCS, Questa, Incisive).
- You **do not write System Verilog code** for line coverage.

⭐ Simple Example 1

RTL Code:

```
module simple;
  reg a, b, c;

  always @(*) begin
    c = a & b;       // Line 1
    if(a)            // Line 2
      c = 1;         // Line 3
    else
      c = 0;         // Line 4
  end
endmodule
```

To get 100% line coverage:

You must execute **all lines**:

Line Code When it executes

Line	Code	When it executes
1	c = a & b;	Always executes
2	if(a)	Always executes
3	c = 1;	Executes when a = 1
4	c = 0;	Executes when a = 0

Testcases needed:

- Testcase 1: a = 1 → covers Line 3
- Testcase 2: a = 0 → covers Line 4

If you only test a = 1, then:

- Line 4 is **never executed** → line coverage < 100%

IMPORTANT POINTS

Line coverage is automatically measured — you don't code it in SV
✓ It only checks **execution of code**, NOT correctness
✓ High line coverage ≠ design is bug-free
✓ But low coverage means **missing testcases**

Summary (Easy Table)

Coverage Type	Meaning
Line / Statement Coverage	Checks if every line in RTL executed at least once

- 👉 Needs tests that trigger **every line** of RTL logic.

Block Coverage

What is Block Coverage?

Block Coverage checks:
👉 Did every block of code get executed at least once?

A **block** means:

- A group of statements inside { }

- A group of statements inside an if, else, case, or loop
- A sequence of code that behaves as one unit

Block coverage is **automatically provided by simulators** (like VCS, Questa).
You **don't have to write System Verilog code** for block coverage.

Difference Between Line & Block Coverage

Type	What it checks
Line Coverage	Each individual line executed or not
Block Coverage	Entire block of statements executed or not

Block coverage groups multiple lines into **one unit**.

★ Simple Example 1

RTL Code:

```
always @(*) begin
  if (a > b) begin
    x = a - b;       // Block 1
    y = a + b;
  end
  else begin
    x = 0;           // Block 2
    y = 0;
  end
end
```

Block 1:

```
x = a - b;
y = a + b;
```

Block 2:

```
x = 0;
y = 0;
```

To get 100% block coverage:

You must execute:

- Block 1 → happens when **a > b**
- Block 2 → happens when **a <= b**

If your test only does:

a = 5; b = 2; // a > b → block 1 executes

Then **block 2 never executes** → block coverage < 100%

Simple Example 2 (With case statement)

```
case(opcode)
  2'b00: begin          // Block A
    result = a + b;
  end

  2'b01: begin          // Block B
    result = a - b;
  end

  default: begin        // Block C
    result = 0;
  end
endcase
```

Here, we have **3 blocks**:

Block	When executes?
A	opcode = 00
B	opcode = 01
C	opcode = others

For **100% block coverage**, your tests must include:

- opcode = 00
- opcode = 01
- opcode = 10 or 11 (default block)

If you skip one case value → block coverage is incomplete.

Summary in Very Easy Words

👉 **Line coverage** = checks if each line ran
👉 **Block coverage** = checks if each *group of statements* ran

Block coverage helps ensure:
✓ all if blocks
✓ all else blocks
✓ all case branches
✓ all loop bodies
have been executed at least once.

Expression Coverage

What is Expression Coverage?

Expression Coverage checks:

👉 Did every Boolean expression evaluate to BOTH true and false during simulation?

In simple words:

✓ Did the expression become true at least once?

✓ Did the expression become false at least once?

Expression coverage is automatically calculated by the simulator.

You don't write any extra System Verilog code for it.

Why Expression Coverage Is Important?

Expressions are used in:

if conditions

while loops

case conditions

complex expressions like (a && b) || c

Expression coverage ensures all boolean combinations are tested.

Simple Example 1

RTL Code:

```
if (a > b)
    x = 1;
else
    x = 0;
```

Expression: a > b

To get 100% expression coverage:

1 test where a > b is true

1 test where a > b is false

Example tests:

a = 5; b = 3; // true

a = 2; b = 7; // false

If you only test one of these → expression coverage < 100%.

Simple Example 2 — Logical Expression

```
if (req && enable)
    grant = 1;
else
    grant = 0;
```

Expression: req && enable

To get 100% expression coverage:

Case where it becomes true

Case where it becomes false

Possible tests:

| req | enable | Expression (`req && enable`) |

| --- | ------ | -------------------------- |

| 1 | 1 | true ✓ |

| 1 | 0 | false ✓ |

| 0 | 1 | false ✓ |

| 0 | 0 | false ✓ |

Minimum needed:

- req=1, enable=1 → true
- req=1, enable=0 or `req=0, enable=1` or `req=0, enable=0` → false

Simple Example 3 — Complex Expression

```
if ((a == b) || (c > d))
   out = 1;
else
   out = 0;
```

Expression: (a == b) || (c > d)

You must test:

✓ expression becomes true

✓ expression becomes false

Example tests:

1. True case:

a = 4; b = 4; // true

OR

c = 9; d = 3; // true

2. False case:

a != b AND c <= d

Example:

a = 2; b = 6;

c = 3; d = 5; // false

Simple Example 4 — Expression in ternary operator

y = (sel) ? a : b;

Expression: sel

Coverage needs:

sel = 1 → select `a`

sel = 0 → select `b`

Summary (Very Easy)

Expression Coverage checks if every boolean expression:

- Becomes true at least once
- Becomes false at least once

It ensures your testcases properly test decisions in your RTL.

Expression coverage is important for:

✓ if conditions

✓ case blocks

✓ logical operations (AND, OR, NOT)

✓ comparisons (>, <, ==, !=)

Branch Coverage and Toggle Coverage

What is Branch Coverage?

Branch coverage checks:

👉 Did every possible branch (path) in your decision statements execute at least once

Branches come from:

- `if` / `else`
- `if` / `else if` / `else`
- `case` statements
- loops (`for`, `while`, `repeat`)

Branch coverage is automatically calculated by the simulator.

Why Branch Coverage Is Important?

Every `if` or `case` has multiple possible outcomes.

Branch coverage ensures your testcases execute all possible paths.

Simple Example 1 — if/else

RTL Code:

```
if (a > b)
   x = 1;
else
   x = 0;
```

This has two branches:

1. Branch 1: `a > b` is true

2. Branch 2: `a > b` is false

To get 100% branch coverage, you need:

- One test where `a > b`
- One test where `a <= b`

If your tests only use:

a = 10, b = 5

Then else branch never executes → branch coverage = 50%

Simple Example 2 — if / else if / else

```
if     (mode == 0)
   out = 1;
```

```
else if (mode == 1)
    out = 2;
else
    out = 3;
```

Branches:

1. `mode == 0`
2. `mode == 1`
3. `else` (mode = anything else)

To get 100% branch coverage, tests must include:

- `mode = 0`
- `mode = 1`
- `mode = 2` or `3` or any other value

If you don't test one branch → coverage reduces.

Simple Example 3 — case statement

```
case(op)
  2'b00: y = a + b;   // Branch A
  2'b01: y = a - b;   // Branch B
  default: y = 0;     // Branch C
endcase
```

Branches:

- op = 00 → Branch A
- op = 01 → Branch B
- op = others → Branch C

To cover all branches:

- Test op = 00

- Test op = 01
- Test op = 10 or 11

Summary (Very Easy)

- Branch coverage checks all decision paths:

Statement Type	Branch Examples
if/else	true branch, false branch
if/else if/else	each condition is one branch
case	each case branch + default
loops	loop entered, loop exited

To get 100% branch coverage, all branches must execute.

What is Toggle Coverage?

Toggle Coverage checks:

👉 Did every bit of every signal change from 0→1 and from 1→0 at least once?

In simple words:

✓ Did the signal toggle?

✓ Did each bit change in both directions?

This is automatically measured by the simulator

(No System Verilog coding is required for toggle coverage).

Why Toggle Coverage is Important?

It ensures:

Signals are not stuck at 0

Signals are not stuck at 1

Signals are actually switching and being tested

Especially important for:

Flip-flops

Buses

Control signals (enable, ready, etc.)

Simple Example 1 — 1-bit signal

always @(posedge clk)

    out <= in;

To get 100% toggle coverage on `out`, both must happen:

1. out goes 0 → 1

2. out goes 1 → 0

Example test:

in = 0 → 1 (0→1 toggle occurs)

in = 1 → 0 (1→0 toggle occurs)

If your test only makes `in = 0 always`,

toggle coverage = 0%.

Simple Example 2 — 4-bit bus

reg [3:0] data;

Toggle coverage checks each bit:

data[0]

data[1]

data[2]

data[3]

Each bit must:

toggle 0→1

toggle 1→0

Example:

If during simulation `data` changes like:

0000 → 1111 → 0000

Then all bits toggle in both directions → 100% toggle coverage.

Simple Example 3 — Toggle missing

If your test changes `data` like:

0000 → 0100 → 1100

Let's check:

Bit	0→1	1→0	Covered?
data[0]	❌	❌	Not toggled (always 0)
data[1]	❌	❌	Not toggled (always 0)
data[2]	✓	❌	Half toggled (no 1→0)
data[3]	✓	❌	Half toggled (no 1→0)

So toggle coverage < 100%.

Summary (Very Easy Words)

Toggle coverage checks:

- Did every bit of the signal go 0→1?
- Did every bit go 1→0?

It helps detect:

✓ stuck-at-0

✓ stuck-at-1

✓ unused signals

Toggle coverage is automatically generated by the simulator.

FSM coverage

What is FSM Coverage?

FSM Coverage means checking whether your testcases exercised:

1. All the states of the FSM

2. All the transitions between the states

FSM coverage helps ensure your design's state machine is fully tested.

Simulators automatically generate FSM coverage if the RTL uses:

- enum for states
- one-hot or binary encoded states
- case statements for next-state logic

FSM Coverage Has Two Parts

Type	What it checks
State Coverage	Did the FSM enter every state at least once?
Transition Coverage	Did the FSM move from every state to every valid next state?

Simple FSM Example

RTL Code (simple 3-state FSM)

```
typedef enum logic [1:0] {
  IDLE,
  READ,
  WRITE
```

} **state_t;**

```
state_t state, next_state;
always_ff @(posedge clk or negedge rstn) begin
  if (!rstn)
    state <= IDLE;
  else
    state <= next_state;
end
always_comb begin
  case(state)
    IDLE:  next_state = start ? READ : IDLE;
    READ:  next_state = done  ? WRITE : READ;
    WRITE: next_state = IDLE;
  endcase
end
```

#1. State Coverage (Simple Explanation)

This FSM has 3 states:

IDLE

READ

WRITE

To get 100% state coverage, your tests must enter all 3 states.

Example signals needed:

- Reset brings machine to IDLE
- Assert `start` → goes to READ
- Assert `done` → goes to WRITE

If you never generate a condition to reach WRITE,

then state coverage < 100%.

2. Transition Coverage (Simple Explanation)

Valid transitions:

1. IDLE → IDLE
2. IDLE → READ
3. READ → READ
4. READ → WRITE
5. WRITE → IDLE

To get full transition coverage, tests must trigger all these transitions.

Example test sequence:

Reset → IDLE

start = 0 → stays in IDLE (IDLE → IDLE)

start = 1 → goes to READ (IDLE → READ)

done = 0 → stays in READ (READ → READ)

done = 1 → goes to WRITE (READ → WRITE)

WRITE → IDLE (automatic transition)

This covers all transitions.

What If Some Transitions Don’t Happen?

Example:

If your test only does:

IDLE → READ → WRITE → IDLE

Then transitions like:

IDLE → IDLE

READ → READ

are not covered.

FSM transition coverage < 100%.

Very Simple Summary

Coverage Type	Meaning	What you must test
State Coverage	Did we visit all states?	Drive FSM into every state
Transition Coverage	Did we take all paths?	Trigger each state-to-state jump

FSM Coverage ensures:

✓ no dead states

✓ no unreachable branches

✓ all control behaviors are tested

Data Oriented Coverage

What is Data-Oriented Coverage?

Data-oriented coverage checks whether important values of signals or variables have been exercised in simulation.

In simple words:

👉 Did your test hit all important data values?

👉 Did it try minimum, maximum, normal, and illegal values?

👉 Did all combinations of data occur?

This is created using functional coverage (`covergroup`, `coverpoint`, `bins`).

Why Do We Need Data-Oriented Coverage?

RTL may behave differently for:

- Small values
- Large values
- Boundary values
- Specific protocol values (like address = 0, max, mid)

Data-oriented coverage ensures:

✓ No important value is missed

✓ Corner cases are tested

✓ Input space is fully covered

Example 1 — Data Value Coverage

Suppose your design uses an 8-bit `addr`.

You want to check whether:

- addr = 0
- addr = 255
- addr is in middle range

were tested.

#Covergroup:

```
covergroup cg;
  coverpoint addr {
    bins low  = {0};
    bins mid  = {[1:254]};
    bins high = {255};
  }
endgroup
```

This checks:

- Did test drive `addr = 0`?
- Did test drive `addr = 255`?
- Did test drive any middle value?

This is data-oriented coverage.

Example 2 — Data Ranges

Suppose your DUT processes packet sizes.

```
covergroup packet_cg;
  coverpoint packet_size {
    bins small  = {[0:63]};
    bins medium = {[64:511]};
    bins large  = {[512:1023]};
  }
endgroup
```

This checks:

- Was a small packet ever sent?
- Was a large packet tested?
- Did tests cover all ranges?

Example 3 — Checking Special Data Values

For an ALU input:

```
covergroup alu_cg;
  coverpoint operand {
    bins zero      = {0};
    bins one       = {1};
    bins max_val   = {32'hFFFF_FFFF};
    bins negatives = {[-100:-1]};
    bins positives = {[1:100]};
  }
endgroup
```

This verifies:

✓ Zero

✓ Max value

✓ Positive and negative values

are all tested.

Example 4 — Data Combination Coverage

Suppose a memory write has:

- addr
- data

We want to ensure combinations are tested.

```
covergroup mem_cg;
  coverpoint addr {
    bins low = {0};
    bins high = {1023};
  }
  coverpoint data {
    bins zero = {0};
    bins max  = {32'hFFFF_FFFF};
  }
  cross addr, data;
endgroup
```

Cross coverage checks:

- addr=0 with data=0

- addr=0 with data=max
- addr=1023 with data=0
- addr=1023 with data=max

This is data-oriented cross coverage.

Summary in Easy Words

Data-oriented coverage = checking important data values.

It checks:

- All important values hit
- Corner values tested
- Min/max covered
- Range tests
- Combination of values

Control Oriented Coverage

What is Control-Oriented Coverage?

Control-oriented coverage measures how well your testbench is exercising the control flow of the design, such as:

* IF / ELSE decisions

* CASE statements

* FSM transitions

* Loops

* Handshake signals

* Enable / Ready / Valid control paths

👉 It checks whether all control decisions your DUT can take have actually been taken in simulation.

This ensures no control path is left untested.

Why is Control-Oriented Coverage Important?

Because even if data is correct, the design may fail when:

- a specific branch of logic is taken,
- an exception condition occurs,
- an uncommon FSM transition happens,
- a handshake violates timing.

So, this coverage ensures all those situations are tested.

Types of Control-Oriented Coverage

#1.0 Branch Coverage (if/else)

Checks whether all branches of decision statements were executed.

2.0 Condition Coverage

Checks individual boolean conditions inside a branch.

3.0 State & Transition Coverage

Tracks FSM states and transitions.

4.0 Handshake / protocol coverage

Tracks VALID/READY, ENABLE/DONE sequences.

Example 1: Branch Coverage

DUT logic:

```
if (a > 10)
   y = 1;
else
   y = 0;
```

Control coverage:

```
covergroup cg_branch @(posedge clk);
  coverpoint (a > 10) {
    bins true_branch  = {1};
    bins false_branch = {0};
```

```
  }
endgroup
```

✓ This checks:

- Did test ever take a > 10 branch?
- Did test ever take else branch?

Example 2: Case Coverage

DUT code:

```
case (opcode)
  0: result = add;
  1: result = sub;
  2: result = mul;
  default: result = 0;
endcase
```

Covergroup:

```
covergroup cg_case @(posedge clk);
  coverpoint opcode {
    bins add_case     = {0};
    bins sub_case     = {1};
    bins mul_case     = {2};
    bins default_case = default;
  }
endgroup
```

✓ Ensures all CASE paths were exercised.

Example 3: FSM Transition Coverage

Suppose FSM:

IDLE → LOAD → EXEC → DONE → IDLE

Coverage:

```
covergroup cg_fsm @(posedge clk);
  // State coverage
  coverpoint state {
    bins IDLE = {0};
    bins LOAD = {1};
    bins EXEC = {2};
    bins DONE = {3};
  }
  // Transition coverage
  coverpoint state {
    bins idle_to_load = (0 => 1);
    bins load_to_exec = (1 => 2);
    bins exec_to_done = (2 => 3);
    bins done_to_idle = (3 => 0);
  }
endgroup
```

✓ Ensures all state moves (control decisions) happen during simulation.

Example 4: Handshake Control Coverage

DUT:

A transaction only happens when valid & ready are both 1.

Coverage:

```
covergroup cg_hs @(posedge clk);
  cp_valid: coverpoint valid;
```

```
  cp_ready: coverpoint ready;

  cross cp_valid, cp_ready;  // handshake coverage
endgroup
```

Checks:

valid	ready	Meaning
1	1	✓ Transaction occurred
1	0	Waiting for ready
0	1	Idle
0	0	No handshake

Summary (Very Simple)

Type	Checks	Why Needed?
Branch Coverage	if/else	ensure all decisions tested
Case Coverage	case statements	all instruction paths executed
FSM Coverage	states & transitions	verify control flow
Handshake Coverage	VALID/READY	protocol correctness
Loop Coverage	loop entry/exit	edge conditions

Final Simple Explanation

Control-oriented coverage = checking all decision-making paths in your design.

It ensures every if, else, case, state transition and protocol handshake is exercised so that your design works in all situations.

Control -oriented Coverage

Control-oriented Coverage

#Code

```
covergroup control_coverage;
  coverpoint state {
    bins all_states[] = {IDLE, READ, WRITE, ERROR};  // Cover all states
  }
  cross state, command;   // Cover the interaction of state and command
endgroup
```

Explanation (simple way)

covergroup control_coverage;

Creates a covergroup named control_coverage.

This group will measure how well control signals are exercised.

```
coverpoint state {
  bins all_states[] = {IDLE, READ, WRITE, ERROR};  // Cover all states
}
```

coverpoint state → We are monitoring the `state` signal.

bins all_states[] → Creates bins for each FSM state.

`{IDLE, READ, WRITE, ERROR}` → All states must be hit in simulation.

✓ If the design never enters a state (e.g., ERROR), coverage will show a hole.

```
cross state, command;   // Cover the interaction of state and command
```

cross coverage between:

`state`

`command`

It means you're checking every combination of state and command.

Example combinations it checks:

- IDLE + READ_CMD

- IDLE + WRITE_CMD
- READ + READ_CMD
- WRITE + WRITE_CMD

✓ Helps detect missing or illegal combinations

endgroup

End of the covergroup.

Branch Coverage Example

```
covergroup branch_coverage;
  coverpoint (control_signal) {
    bins branch_true  = {'b1};
    bins branch_false = {'b0};
  }
endgroup
```

Explanation

covergroup branch_coverage;

Creates a covergroup named branch_coverage.

Used to check if/else branch decisions.

coverpoint (control_signal) {

Monitoring the signal `control_signal` that controls a decision.

bins branch_true = {'b1};

Creates a bin for the condition being TRUE.

`'b1` means control_signal = 1.

bins branch_false = {'b0};

Creates a bin for the condition being FALSE.

'b0` means control_signal = 0.

✓ If your code has:

if (control_signal) ...

else ...

This coverage ensures both paths are tested.

endgroup

End of branch coverage group.

FSM Transition Coverage Example

```
covergroup fsm_transition_coverage;
  transition coverpoint fsm_state {
    bins idle_to_read = (IDLE => READ);
    bins read_to_write = (READ => WRITE);
  }
endgroup
```

covergroup fsm_transition_coverage;

Defines a new covergroup to check state transitions of FSM.

transition coverpoint fsm_state {

A special coverpoint type called transition coverpoint.

Used for state change coverage (not just the state value).

bins idle_to_read = (IDLE => READ);

Creates a bin for the transition:

from IDLE state

to READ state

✓ Ensures this transition actually happens.

bins read_to_write = (READ => WRITE);

Creates a bin for the transition:

from READ

to WRITE

✓ If the FSM never went from READ → WRITE, this bin remains uncovered.

endgroup

End of the transition coverage block.

Overall Simple Summary

Coverage Type	What It Checks	Example in Code
State Coverage	FSM entered all states	bins for IDLE, READ, WRITE, ERROR
Cross Coverage	Interaction of two control signals	state × command
Branch Coverage	If/else decisions	control_signal = 0/1
Transition Coverage	FSM moved from one state to another	IDLE→READ, READ→WRITE

covergroup, coverpoint, and bins

1.0 What is a covergroup

A covergroup is a container that holds all coverage items.

👉 It is like a box where you keep signals you want to monitor.

Example:

covergroup cg;

endgroup

This creates a covergroup named cg.

2. What is a coverpoint

A coverpoint tells which signal you want to measure coverage for.

👉 It is like saying "I want to monitor this signal's values."

Example:

coverpoint data;

This means:

➞ Cover how many times `data` gets different values.

3. What are bins?

Bins define the specific values or ranges you want to track.

👉 Bins = what values should be counted?

Example:

```
bins zero = {0};
bins high = {[10:20]};
```

Meaning:

✓ Count when value becomes 0

✓ Count when value is between 10 to 20

One Complete Short Example

```
covergroup cg;
  coverpoint data {
    bins zero = {0};        // Count when data = 0
    bins mid  = {[1:10]};   // Count when data is 1 to 10
    bins max  = {255};      // Count when data = 255
  }
endgroup
```

Simple Meaning:

covergroup cg → A box that holds coverage.

coverpoint data → We monitor the signal `data`.

bins → Define which data values we care about.

✓ This coverage checks:

- Did simulation hit data = 0?
- Did simulation hit values from 1 to 10?
- Did simulation hit data = 255?

Code Vs Functional Coverage

Session-14: -

- Coverpoint
- Covergroup
- Bins
- Covereg report

coverpoint

What is a Coverpoint

A coverpoint in SystemVerilog tells the simulator:

- Monitor this signal and record which values it takes.

It is used inside a covergroup to measure functional coverage.

Example 1: Coverpoint on a Signal

#Code:

```
covergroup cg1;
  coverpoint data; // monitor the 'data' signal
endgroup
```

Meaning (simple):

- coverpoint data; → checks how many times `data` takes different values during simulation.
- If `data` changes to 5, 10, 20 → coverage will record these events.

Example 2: Coverpoint with Bins

Code:

```
covergroup cg2;
  coverpoint opcode {
    bins read  = {0};
    bins write = {1};
  }
endgroup
```

Meaning (simple):

- We monitor the signal `opcode`.
- If `opcode = 0` → read bin gets covered
- If `opcode = 1` → write bin gets covered

✓ It checks if both read and write operations happened in simulation.

Simple Summary

- Coverpoint = what signal you want to check.
- It helps ensure important values of a signal occur during simulation.

Syntax:

```
covergroup <covergroup_name>;
  <coverpoint label>: coverpoint <coverpoint_name>;
  <coverpoint label>: coverpoint <coverpoint_name>;
  ...
endgroup
```

//Example:

```
covergroup cg @(posedge clk);
  cp1: coverpoint addr;
```

```
  cp2: coverpoint data;
endgroup : cg
```

Covergroup

A covergroup is a container that holds coverage items like:

- Coverpoints
- Bins
- cross coverage

👉 It is like a box where you put all the signals you want to measure for coverage.

#Example 1: Basic Covergroup

Code:

```
covergroup cg1;
  coverpoint data;
endgroup
```

Simple Meaning:

- covergroup cg1; → creates a coverage collector named `cg1`
- coverpoint data; → measures all values taken by the signal `data`

✓ This covergroup checks how many different values data gets during simulation.

Example 2: Covergroup with Bins

Code:

```
covergroup cg2;
  coverpoint addr {
    bins low  = {0};
    bins high = {255};
  }
endgroup
```

Simple Meaning:

- covergroup cg2; → creates a coverage group named `cg2`
- coverpoint addr → monitors the signal `addr`
- bins define which values we care about:
- `low` bin → checks if `addr = 0` ever happened
- `high` bin → checks if `addr = 255` ever happened

✓ This covergroup verifies if simulation used boundary address values.

Very Short Summary

- Covergroup = box that holds coverage checks.
- It contains coverpoints and bins.
- Used to measure functional coverage.

Bins

What are Bins

Bins are the specific values or ranges you want to track for coverage.

👉 Bins tell the simulator:

- Record how many times this value (or range) occurs.

They are always written inside a coverpoint.

Syntax:

covergroup <covergroup_name>;

<coverpoint label>: coverpoint <coverpoint_name> {bins <bin_name> = {<values>} };

...

endgroup

With explicit bins declaration

module func_coverage;

logic [3:0] addr;

logic [2:0] data;

```
  logic en;


  covergroup c_group;
    cp1: coverpoint addr {bins b1 = {1, 10, 12};
                          bins b2[] = {[2:9], 11};
                          bins b3[4] = {0:7};
                         }
    cp2: coverpoint data {bins b1 = {4,$};
                          bins b2[] = {2, 3, 6};
                         }
    cp3: coverpoint en {bins b1 = {1}; }
  endgroup


  c_group cg = new();
  ...
  ...
endmodule
```

Why Do We Need Bins?

Because instead of covering every single value, we focus on:

- important values
- boundary values
- ranges
- corner cases

Example 1: Single Value Bins

Code:

```
covergroup cg;
  coverpoint data {
    bins zero = {0};
```

```
  bins max  = {255};
 }
endgroup
```

Simple Meaning:

- `zero` bin → counts how many times `data = 0`
- `max` bin → counts when `data = 255`

✓ Checks if test stimulates `data` with boundary values.

Example 2: Range Bins

Code:

```
covergroup cg2;
 coverpoint addr {
  bins low_range  = {[0:10]};     // values 0 to 10
  bins high_range = {[100:200]};  // values 100 to 200
 }
endgroup
```

Simple Meaning:

- `low_range` bin → hits if `addr` is any value between 0 and 10
- `high_range` bin → hits if `addr` is between 100 and 200

✓ Records whether important ranges were exercised.

Example 3: Multiple Values in One Bin

Code:

```
bins special_values = {3, 5, 7};
```

Simple Meaning:

This bin hits if the signal becomes 3 OR 5 OR 7.

Very Short Summary

Bins = values or ranges you want to track.

Bins help check whether important values occurred in simulation.

Always used inside a coverpoint.

Automatic / Implicit bins

Automatic Bins

Automatic bins are bins that SystemVerilog creates automatically

— you don't manually write them.

👉 You tell SystemVerilog:

- Make bins for all values in this range.
- SV will split the range into multiple bins by itself.

Why Use Automatic Bins?

Because manually writing bins for many values is boring and time-consuming.

Example:

- If a signal is 0 to 255, writing 256 bins manually is impossible.
- Automatic bins solve this.

Example 1: Automatic Bins on a Small Range

Code:

```
covergroup cg;
  coverpoint a {
    bins auto_vals[] = {[0:3]};
  }
endgroup
```

Meaning:

SystemVerilog automatically creates 4 bins:

Bin Name	Value
auto_vals[0]	0
auto_vals[1]	1
auto_vals[2]	2
auto_vals[3]	3

✓ You wrote only 1 line

✓ SV created 4 bins

⭐ Example 2: Automatic Bins for a Large Range

Code:

```
covergroup cg2;
  coverpoint data {
    bins all_values[] = {[0:255]};
  }
endgroup
```

Meaning:

SystemVerilog creates 256 bins automatically:

- all_values[0] → for value 0
- all_values[1] → for value 1
- all_values[255] → for value 255

✓ No need to write 256 bins manually.

Very Short Summary

- Automatic bins = bins created automatically by SystemVerilog.
- Used when you want bins for a range of values.

- Saves time for large ranges.

Examples:

- `[0:3]` → 4 bins
- `[0:255]` → 256 bins

Explicit bins

Explicit Bins

Explicit bins are bins that you create manually by specifying exact values or ranges.

👉 You tell SystemVerilog exactly which values you want to track.

Unlike automatic bins (which SV creates),

explicit bins are fully controlled by you.

Why Use Explicit Bins?

- To track specific important values
- To check boundary conditions
- To cover corner cases
- To verify protocol-specific values

Example 1: Explicit Single Value Bins

Code:

```
covergroup cg1;
  coverpoint data {
    bins zero = {0};
    bins mid  = {128};
    bins max  = {255};
  }
endgroup
```

Meaning:

- `zero` bin → hits when `data == 0`
- `mid` bin → hits when `data == 128`
- `max` bin → hits when `data == 255`

✓ You manually defined the exact values to track.

This is explicit.

Example 2: Explicit Range Bins

Code:

```
covergroup cg2;
  coverpoint addr {
    bins low_range  = {[0:15]};     // from 0 to 15
    bins mid_range  = {[16:127]};   // from 16 to 127
    bins high_range = {[128:255]};  // from 128 to 255
  }
endgroup
```

Meaning:

You are explicitly telling SystemVerilog:

- Count how many times addr is in 0–15
- Count how many times addr is in 16–127
- Count how many times addr is in 128–255

✓ These ranges are explicitly defined by you.

Example 3: Explicit Multiple Values in One Bin

Code:

```
bins special_numbers = {3, 5, 7};
```

Meaning:

This bin hits when the signal equals 3 OR 5 OR 7.

✓ You explicitly listed the exact values.

Very Short Summary

- Explicit bins = bins you manually define.
- You specify exact values or ranges.
- Used when certain values are important to verify.

Examples:

- `{0}`, `{128}`, `{255}` → single values
- `{[0:15]}` → range
- `{3,5,7}` → multiple values

Bins for transition

transition bins

#What Are Transition Bins?

Transition bins are used to track changes from one value to another. (set of order)

👉 They check how a signal moves from previous value → new value.

Example:

- 0 → 1
- IDLE → READ
- 5 → 10 → 20

Transition bins are written using the symbol:

=>

#Why Use Transition Bins?

To verify:

- FSM state transitions
- Handshake signal changes
- Protocol sequences
- Value patterns over time

Example 1: Simple 1-Step Transition

Code:

```
covergroup cg1;
  coverpoint state {
    bins idle_to_read = (IDLE => READ);
  }
endgroup
```

Simple Meaning:

This bin hits when previous state = IDLE and next state = READ.

✓ Checks if FSM actually performed this transition.

Example 2: Multiple Transitions

Code:

```
covergroup cg2;
  coverpoint state {
    bins read_to_write = (READ => WRITE);
    bins write_to_idle = (WRITE => IDLE);
  }
endgroup
```

Meaning:

- First bin hits when state changes READ → WRITE
- Second bin hits when state changes WRITE → IDLE

✓ Used heavily in FSM coverage.

Example 3: 3-Step Transition (Sequence)

Code:

```
covergroup cg3;
```

```
  coverpoint data {
    bins pattern = (0 => 1 => 2);
  }
endgroup
```

Simple Meaning:

This bin hits only if `data` follows the exact sequence:

0 → 1 → 2

✓ Good for protocol sequences.

Example 4: Transition for Numerical Range

Code:

```
covergroup cg4;
  coverpoint value {
    bins up_range = ([0:10] => [11:20]);
  }
endgroup
```

Meaning:

Bin hits if:

- value moves from 0–10
- to 11–20

✓ Tracks movement between ranges.

Very Short Summary

- Transition bins track how values change over time.
- Use => to describe transitions:

* `(A => B)`

* `(A => B => C)`

* `([range1] => [range2])`

Examples:

* `(IDLE => READ)`

* `(0 => 1 => 2)`

* `([0:10] => [20:30])`

ignore bins

ignore bins

Ignore bins tell the simulator to ignore some values during coverage.

👉 It means:

“These values are not important — do NOT count them.”

Coverage will not track or show holes for these ignored values.

Why Use Ignore Bins?

To remove:

- invalid values
- unwanted cases
- don't-care values
- values that should not affect coverage goals

Example 1: Ignore Unwanted Value

Code:

```
covergroup cg1;
  coverpoint data {
    ignore_bins invalid = {255};
  }
```

endgroup

Simple Meaning:

Coverage will not count when `data == 255`.

255 is ignored because it may be:

- an invalid value
- a reserved value
- a reset/default value

✓ Only valid data values contribute to coverage.

Example 2: Ignore a Range of Values

Code:

```
covergroup cg2;
  coverpoint addr {
    bins valid = {[0:127]};
    ignore_bins unused = {[128:255]};
  }
endgroup
```

Simple Meaning:

- `0–127` → valid values to cover
- `128–255` → ignored (not counted in coverage)

✓ Useful when only a part of the address range is used in design.

Very Short Summary

- ignore_bins = values that should NOT count in coverage
- Used to skip invalid or unimportant cases
- Helps keep coverage report clean and meaningful

Examples:

- ignore_bins invalid = {255};` → ignore value 255
- ignore_bins unused = {[128:255]};` → ignore range 128–255

illegal bins

illegal bins

- Illegal bins are used in a covergroup to mark certain values or transitions as invalid or not allowed.
- If those values appear during simulation:
- The simulator reports an error or warning, because something illegal happened.

Think of illegal bins as:

❌ “These values should never occur. If they occur, inform me.”

Example 1: Illegal value for a signal

Suppose a packet `opcode` must be only `0,1,2`.

If opcode becomes `3`, it is illegal.

✓ Code:

```
covergroup cg_opcode;
  coverpoint opcode {
    bins valid_ops[] = {0, 1, 2};
    illegal_bins wrong_ops = {3};
  }
endgroup
```

✓ Meaning:

- valid_ops → legal values
- wrong_ops → if value 3 happens, simulator will show illegal bin hit

Example 2: Illegal transition

Suppose the state machine should not jump from IDLE (0) directly to DONE (3).

✓ Code:

```
covergroup cg_state;
  coverpoint state {
    illegal_bins bad_transition = (0 => 3);
  }
endgroup
```

✓ Meaning:

Transition 0 → 3 should never happen

If it occurs → simulator flags as illegal transition

Summary (Very Simple)

Feature	Meaning
illegal bins	Values or transitions that must NOT occur
If hit	Error/Warning → design or testbench bug
Usage	`illegal_bins name = {values};` or transitions

Cross coverage

Cross coverage

The cross-coverage allows having a cross product (i.e. cartesian product) between two or more variables or coverage points within the same covergroup. In simple words, cross-coverage is nothing but a set of cross-products of variables or coverage points.

Syntax:

<cross_coverage_label> : cross <coverpoint_1>, <coverpoint_2>,..., <coverpoint_n>

```
bit [7:0] addr, data;
bit [3:0] valid;
bit en;

covergroup c_group @(posedge clk);
  cp1: coverpoint addr && en; // labeled as cp1
  cp2: coverpoint data;       // labeled as cp2
  cp1_X_cp2: cross cp1, cp2; // cross coverage between two expressions
  valid_X_cp2: cross valid, cp2; // cross coverage between variable and expression
endgroup : c_group
```

Cross Coverage

Cross coverage means:

👉 Checking all possible combinations of two or more coverpoints.

It helps you verify that every combination of values occurred during simulation.

Think of it like a table:

A	B	Combination
0	0	(0,0)
0	1	(0,1)
1	0	(1,0)
1	1	(1,1)

Cross coverage checks whether all these combinations happened.

Example 1: Crossing two simple signals

Let's say you have:

- `mode` → can be 0 or 1
- `enable` → can be 0 or 1

✓ Code:

```
covergroup cg;
```

```
  coverpoint mode;
  coverpoint enable;
  cross mode, enable;
endgroup
```

Meaning:

This will check combinations:

mode	enable
0	0
0	1
1	0
1	1

The simulator reports which combinations were covered.

Example 2: Crossing bins

Here we define bins first and then cross them.

Code:

```
covergroup cg_transaction;
  coverpoint size {
    bins small = {1};
    bins medium = {2};
    bins large = {3};
  }
  coverpoint type {
    bins read  = {0};
    bins write = {1};
  }
  cross size, type;
endgroup
```

✓ Meaning:

Cross coverage will check these 6 cases:

size bin	type bin	Meaning
small	read	1-byte read
small	write	1-byte write
medium	read	2-byte read
medium	write	2-byte write
large	read	3-byte read
large	write	3-byte write

Why Do We Use Cross Coverage?

To ensure:

✓ All valid combinations happened

✓ No combination was missed

✓ Better functional coverage

Conditional coverage

Coverage constructs

iff construct

The expression within the iff construct provides feasibility to exclude the coverpoint in the coverage if an expression is evaluated as false.

Example:

```
module func_coverage;
  logic [3:0] addr;
  covergroup c_group;
```

```
  cp1: coverpoint addr iff(!reset_n) // coverpoint addr is covered when reset_n is low.
endgroup

c_group cg = new();
...
...
endmodule
```

Conditional Coverage

Conditional coverage means:

👉 You collect coverage only when a specific condition is true.

In other words:

> "Cover this only if some condition is satisfied."

This avoids capturing unwanted or invalid scenarios.

How do we write conditional coverage?

We use:

✓ iff (condition)

Example:

```
coverpoint data iff (enable);
```

This means:

- Cover `data` ONLY when `enable == 1`
- Ignore all other times

Example 1: Collect data coverage only when enable = 1

✓ Code:

```
covergroup cg;
```

```
  coverpoint data iff (enable);
endgroup
```

#Meaning:

- When `enable = 1`, `data` values are recorded.
- When `enable = 0`, coverage is not collected.

Useful when:

You want coverage only during valid cycles.

Example 2: Cover a signal only when reset is deasserted

You don't want to cover values during reset.

✓ Code:

```
covergroup cg2;
  coverpoint opcode iff (reset == 0);
endgroup
```

#Meaning:

- Coverage of `opcode` happens only when reset is OFF (0).
- During reset, `opcode` may be random or invalid, so coverage is ignored.

Why do we use Conditional Coverage?

✓ To avoid wrong or invalid coverage

✓ To ensure coverage counts only meaningful data

✓ To improve functional coverage accuracy

✓ To reduce noise in coverage reports

Conditional Coverage using start() and stop() -

Conditional Coverage (start/stop)

- In SystemVerilog, you can start and stop a covergroup manually.
- This lets you collect coverage only during a specific period.

Think of it like:

start() → Begin recording coverage

stop() → Pause/stop recording coverage

This is useful when:

- You want coverage only during valid transactions
- You want to ignore coverage during reset or idle time

Syntax

cg_inst.start();

cg_inst.stop();

Example

We want to collect coverage only when TEST is running, not during reset.

#Code

👉 Covergroup

```
covergroup cg;
  coverpoint data;
endgroup

cg cg_inst = new();
```

👉 Testbench Logic

```
initial begin
  reset = 1;
  #10 reset = 0;
  // Start coverage only after reset is complete
```

```
  cg_inst.start();
  repeat (5) begin
    data = $urandom_range(0, 15);
    #5;
  end
  // Stop coverage after test is done
  cg_inst.stop();
end
```

Explanation (Very Easy):

1. reset = 1 → coverage OFF

   We do not want to record junk values.

2. cg_inst.start();

   → Begin collecting coverage.

3. data gets random values

   → These values are recorded in coverage.

4. cg_inst.stop();

   → Stop collecting coverage when test is done.

Only the values generated between start() and stop() are included in coverage.

Why use start/stop?

✓ Control when coverage is collected

✓ Avoid invalid or unwanted coverage

✓ Make coverage reports meaningful

Coverage Lab

```
class monitor;
        rand bit [3:0] b1;
        rand bit [3:0] b2;
```

```
        covergroup cg;
                A: coverpoint b1 {
                        bins b1_0_7 = {[0:7]};
                bins b1_8 = {8};
                bins b1_dyne={[13:$]};
                bins reserve = default;}
                B: coverpoint b2 {
                        bins b2_0_7 ={[0:7]};
                bins b2_8 = {8};
                bins b1_dyne= {[13:$]};
                bins reserve = default;}
        endgroup

        function new();
                cg =new();
        endfunction;
endclass

module m;
bit clk;

always #5 clk=~clk;

initial begin
        monitor mon=new();
        repeat(2000) @(posedge clk)begin
```

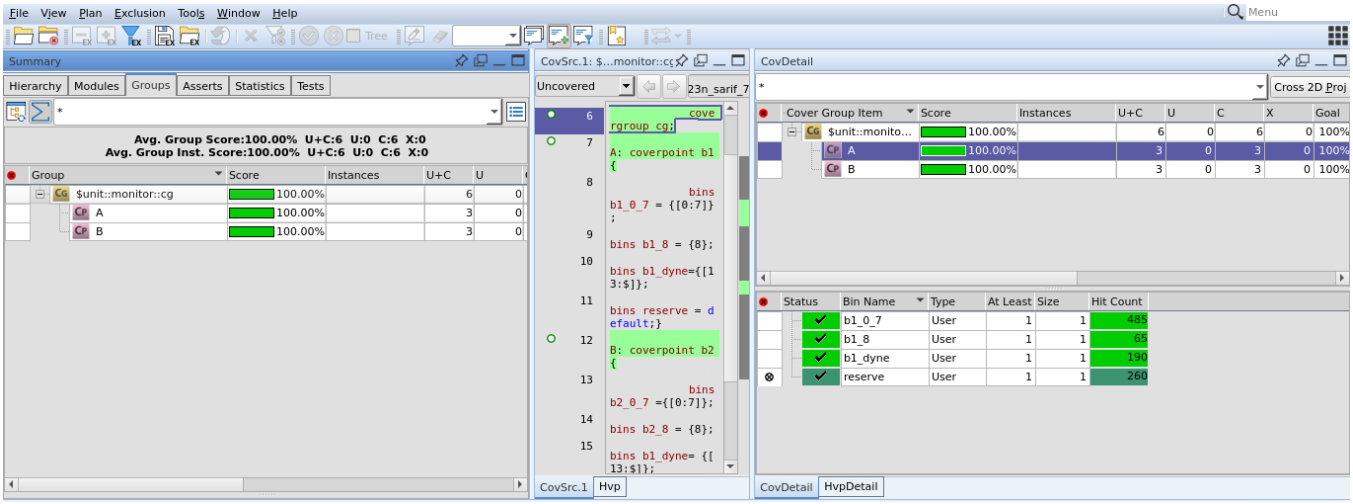
```
            mon.randomize();
            $display("%p",mon);
            mon.cg.sample();
        end
end
initial
        #10000 $finish;
endmodule
```

Command for run:

vcs -sverilog example.sv -kdb -lca -debug_access+all -full64

./simv

verdi -cov -covdir simv.vdb/



CovDetail

*

Cross 2D Proj

Cover Group Item	Score	Instances	U+C	U	C	X	Goal	Weight	AtLeast	PerInst	Overlap	AutoBinI	Missing	Comm
$unit::monito...	100.00%		6	0	6	0	100%	1	1	0	1	64	64	
A	100.00%		3	0	3	0	100%	1	1		1	0		
B	100.00%		3	0	3	0	100%	1	1		1	0		

Status	Bin Name	Type	At Least	Size	Hit Count
✓	b1_0_7	User	1	1	485
✓	b1_8	User	1	1	65
✓	b1_dyne	User	1	1	190
⊗ ✓	reserve	User	1	1	260

Commands to get coverage in Verdi

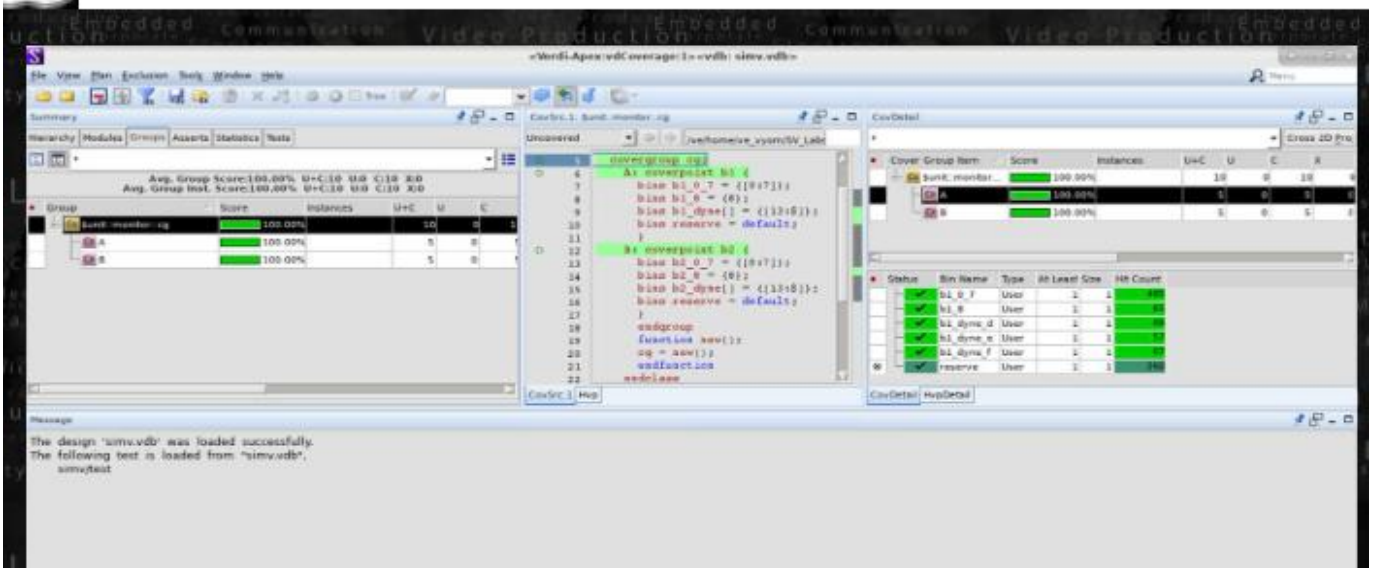
step 1:- vcs -sverilog adder.sv tb_adder.sv -kdb -lca -debug_access+all -full64

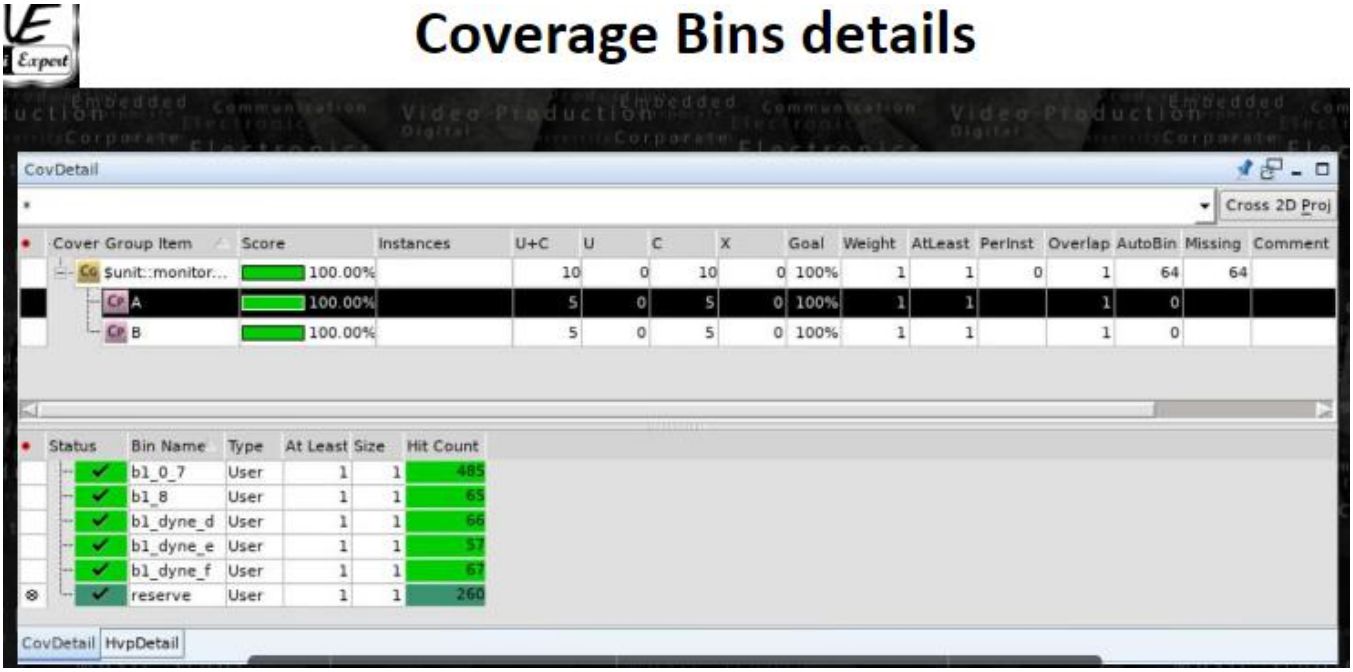
step 2:- ./simv

step 3:- verdi -cov -covdir simv.vdb/

www.vlsi-expert.com

Coverage in verdi Gui





Limitations of Code coverage

limitations of Code Coverage

Code coverage only checks which lines of code were executed.

It does NOT check correctness of the design.

1.0 Code Coverage cannot detect functional bugs

Even if all lines run, the output may still be wrong.

Example:

```
if (a > b)
   max = a;
else
   max = b;
```

If tests run both paths → 100% code coverage.

But what if the design mistakenly uses:

max = b; // wrong assignment

Coverage will still show 100%, but logic is wrong.

2.0 Code Coverage does NOT check all value combinations

It only checks execution, not data patterns.

#✓ Example:

```
if (mode == 3)
  start = 1;
```

If you test mode = 3 once, coverage = 100%.

But you never tested:

- mode = 0
- mode = 1
- mode = 2

Functional issue may remain hidden.

3.0 Unused or dead code lowers coverage unfairly

Some code can never execute due to architecture.

✓ Example:

```
if (0) begin
  a = 1;   // dead code
end
```

Coverage will always show 0%, but this code is useless anyway.

4. 0 Does not check timing or protocol rules

Code coverage cannot detect handshake/protocol violations.

✓ Example:

Coverage cannot tell if:

- "valid" was asserted before "ready"
- "write" happened before reset deasserted

5. 0Code Coverage cannot measure quality of test stimulus

Even hitting 100% coverage does not mean:

- All features tested
- Design is bug-free
- All corner cases covered

Why code coverage is limited?

Because it answers only:

> “Did this line run?”

NOT:

> “Did it behave correctly?”

Limitations of functional coverage

Limitations of Functional Coverage

Functional coverage checks what you planned to verify, but it depends on YOUR coverpoints.

If you miss something, coverage cannot catch it.

1.0 Functional Coverage only checks what you wrote

If you forgot to add a coverpoint, that scenario will never be checked.

Example:

```
coverpoint opcode {
  bins read = {0};
  bins write = {1};
```

}

You forgot to add a bin for:

opcode = 2 (erase)

Even if the erase operation is buggy, coverage will show 100%.

2.0 Incorrect or incomplete bins give false confidence

Badly written bins can hide bugs.

#Example:

bins small = {[0:10]};

If valid range is 0–20,

values 11–20 will never be covered.

Coverage looks complete, but test is incomplete.

3.0 Hard to define coverage for complex behaviour

Some behaviours are too complicated to express in simple coverpoints.

✓ Example:

Tracking a protocol interface to check:

- correct sequence
- correct timing
- multiple signals interaction

Functional coverage may not capture all corner cases.

4.0 Functional coverage does not check correctness

It checks occurrence, not correctness.

✓ Example:

coverpoint packet.size;

Packet size 4 may occur (coverage hit),

but the DUT could be producing wrong output for size 4.

Coverage cannot detect the bug.

5.0 Functional coverage needs good constraints/testbench

If stimulus does not generate cases, coverage stays low.

✓ Example:

- You want to cover values 0–15,
- but constraint limits random values to 0–7:

constraint c { val inside {[0:7]}; }

Coverage for 8–15 will always be 0%.

Key Takeaway

Functional coverage tells you:

> “Did this scenario happen?”

But it cannot tell:

> “Was the design correct?”

> “Did you forget to cover something?”

> “Was testbench written properly?”

Session-15: -

- Assertion: Part-1
- Introduction to Assertions
- What is an Assertions?
- Why assertion are important?
- Building block of SVA
- Types of assertions

- A simple action block
- Assertion severally levels
- Sequence with edge definitions
- Clock definition with SVA
- Forbidding a property

What Are Assertions in SystemVerilog?

Assertions are **statements that check whether a design behaves correctly** during simulation.

Think of assertions like:

✓ *Rules the design must follow*
✓ *Automatic checkers that catch bugs immediately*
✓ *Self-documentation for design behavior*

Assertions are used in **verification**, not synthesis.

Assertions are important because they:

✓ Automatically catch bugs during simulation

✓ Check design rules without manual debugging

✓ Find timing/protocol errors that are hard to see

✓ Make verification faster and more reliable

✓ Document the intended behavior of the design

In short:

👉 Assertions act like automatic guards that continuously check if the design is working correctly.

- Checks design specifications and reports errors or warnings in case of failure.
- It improves debugging time. For example, a bug due to an illegal state transition can propagate to the output. Writing an assertion helps out to improve debugging time.

- Can be used in formal verification.
- Can be re-used across verification testbench or design.
- Can be parameterized
- Can be turned on/off based on the requirement.

It compares:

❌ Traditional Verilog (manual checking)

vs

✓ SystemVerilog Assertion (automatic checking)

Both are checking the same rule:

🎯 RULE being checked

👉 When signal `a` rises (a goes from 0 → 1), signal `b` must rise within the next 1 to 3 clock cycles.

🟦 1. Verilog Procedural Code Explanation (manual checking)

```
always @(posedge a) begin
   repeat (1) @(posedge clk);       // wait 1 cycle
   fork: a_to_b

      begin
         @ (posedge b)
         $display("SUCCESS: b arrived in time\n", $time);
         disable a_to_b;
      end

      begin
         repeat (3) @(posedge clk);
         $display("ERROR: b did not arrive in time\n", $time);
```

```
        disable a_to_b;
      end

  join
end
```

✓ Simple explanation

1. Whenever a rises, the check starts.
2. The code waits 1 clock cycle before checking.
3. Then two blocks run in parallel:

   Block 1: Wait for b to rise

   → If it happens within 3 cycles → SUCCESS

   Block 2: Wait 3 clock cycles

   → If `b` does not rise → ERROR

4. `disable a_to_b` cancels the other block after the first finishes.

▨ 2. SystemVerilog Assertion version

```
a_to_b: assert property (@(posedge clk) $rose(a) |-> ##[1:3] $rose(b) );
```

✓ Simple explanation

- $rose(a) → a rising edge
- |-> → implies (if a rises then...)
- ##[1:3] → within 1 to 3 clock cycles
- $rose(b) → b must rise

Meaning:

👉 If `a` rises now, then `b` must rise anytime between 1 and 3 cycles later.

Very clean. No fork-join, no repeat loops.

📘 Waveform Explanation (Simple)

clk: ^ ^ ^ ^ ^ ^ ^ ^ ^ ...

a : ___ ___

| |

b : ____ __

From the waveform:

- At time slot 2, `a` rises.
- `b` rises at time slot 4.
- That is 2 cycles later → ✓ Valid (within 1–3 cycles)
- At time slot 11, `a` rises again.
- `b` rises at time slot 14.
- That is 3 cycles later → ✓ Valid (within 1–3 cycles)

Both satisfy the rule.

Final Output

👉 FOR THE VERILOG CODE

SUCCESS: b arrived in time. Time = <t1>

SUCCESS: b arrived in time. Time = <t2>

No ERROR messages.

👉 FOR THE SV ASSERTION

No failure message → assertion passes.

If it failed you would see:

Assertion a_to_b FAILED at time <t>

But here assertion passes for both events.

⭐ FINAL SUMMARY (Very Simple)

- The code checks that b must rise 1–3 cycles after a rises.
- Both waveform cases satisfy the rule.
- Output = SUCCESS messages only
- Assertion passes (no failures).

☑ Waveform Explanation (Very Simple)

The graph shows 3 signals:

1. clk → clock (continuous pulses)

2. a → input signal

3. b → another signal that must follow `a`

🔍 What the graph is showing

- When `a` goes HIGH (rising edge)
- → `b` must go HIGH within 1 to 3 clock cycles

This timing relationship is what the assertion is checking.

▧ Detailed Explanation of the Graph

Cycle Numbers:

Across the top you see 1 2 3 4 … 18

These are the clock cycles.

▧ 1st Event (Left side of graph)

At clock cycle 2

👉 `a` has a rising edge (goes from 0 → 1)

What happens to `b`?

- `b` rises at cycle 4.

Time difference:

From cycle 2 → cycle 4 = 2 cycles

✓ This is valid because 2 is within allowed range (1 to 3 cycles)

▨ 2nd Event (Right side of graph)

At clock cycle 11

👉 `a` has another rising edge.

What happens to `b`?

`b` rises at cycle 14.

Time difference:

- From cycle 11 → cycle 14 = 3 cycles

✓ This is valid because 3 is also in the allowed range (1 to 3 cycles)

★ Final Meaning of the Graph

✓ Every time `a` rises

✓ `b` rises within 1–3 clock cycles

✓ Therefore, the timing requirement is satisfied

✓ The assertion PASS for both events

SystemVerilog Assertions (SVA) are built using 4 building blocks:

1. Boolean Expression

A simple condition like `a==1`, `req && !grant`, `$rose(a)`.

2. Sequence

A sequence describes events over time.

sequence seq_name;

  <boolean / timing expression>;

endsequence

Example:

"After `a` rises, `b` should rise 2 cycles later"

3. Property

A property describes a rule using sequences.

property prop_name;

  <sequence or condition>;

endproperty

Example:

"If `a` rises → `b` must rise within 1–3 cycles"

4. Assert Property

Checks the property during simulation.

assert property(prop_name);

⭐ Simple Example to Explain All Steps

🎯 Requirement (Rule)

👉 If `req` becomes 1, then `ack` must become 1 within 2 to 4 cycles.

We will build this requirement using the 4 steps.

▧ Step 1: Create Boolean Expression

Boolean conditions:

$rose(req) // req rises

$rose(ack) // ack rises

▨ Step 2: Create Sequence Expression

Sequence describing timing:

```
sequence req_to_ack_seq;
  $rose(req) ##[2:4] $rose(ack);
endsequence
```

Meaning:

- When `req` rises
- After 2 to 4 clock cycles
- `ack` must rise

▨ Step 3: Create Property

```
property req_ack_property;
  req_to_ack_seq;
endproperty
```

▧ Step 4: Assert the Property

```
assert property (req_ack_property)
  else $error("ACK did not arrive in 2–4 cycles after REQ");
```

🎉 Final Complete SVA Code

```
sequence req_to_ack_seq;
  $rose(req) ##[2:4] $rose(ack);
```

endsequence

property req_ack_property;

req_to_ack_seq;

endproperty

assert property(req_ack_property)

else $error("ACK did not arrive in 2–4 cycles after REQ");

🟥 Explanation in One Sentence

- Sequence → defines “when req rises, ack must rise after 2–4 cycles”
- Property → holds this rule
- Assert → checks the rule automatically during simulation

✨ SUPER SIMPLE SUMMARY

Step	Meaning	What you write
Step 1	Boolean condition	`$rose(req)`
Step 2	Sequence	`$rose(req) ##[2:4] $rose(ack)`
Step 3	Property	Wrap the sequence
Step 4	Assert	Check the rule

Condition: Question

Example: Read and Write should NEVER be HIGH at the same time

🎯 Requirement:

👉 `read` and `write` cannot be 1 together (illegal condition).

This is a very common rule.

◇ Step 1: Boolean Expression

Illegal condition:

read && write

Legal condition:

!(read && write)

◇ Step 2: Sequence Expression

This is a simple rule checked every clock.

Since it does not have time sequencing, sequence is simple:

```
sequence rw_seq;
  !(read && write);
endsequence
```

Meaning:

✓ At every clock, read and write must not be 1 together.

◇ Step 3: Create a Property

```
property rw_property;
  rw_seq;
endproperty
```

◇ Step 4: Assert the Property

```
assert property(rw_property)
  else $error("ERROR: read and write are HIGH together!");
```

🎉 Complete SVA Example

```
sequence rw_seq;
  !(read && write);
endsequence

property rw_property;
  rw_seq;
endproperty

assert property(rw_property)
  else $error("ERROR: read and write are HIGH together!");
```

⭐ Even Simpler Version (Without sequence block)

If you don’t want to create sequence/property blocks:

```
assert property(@(posedge clk)
  !(read && write)
) else $error("Read and write HIGH together!");
```

🧠 VERY SIMPLE EXPLANATION

- This assertion checks at every clock:
- “Are read and write both HIGH?”
- If yes → ERROR
- If no → OK

Data must stay stable when `valid = 1`”

☑ One-line (Most Common & Clean)

```
assert property (@(posedge clk) valid |-> $stable(data))
```

else $error("Data changed while valid was HIGH");

🔹 **Meaning (very simple):**

- On every clock edge
- If `valid` is 1
- `data` must not change in that cycle.

☑ Using non-overlapping implication (next-cycle stability):-

If you mean data must remain stable starting from the next cycle:

assert property (@(posedge clk) valid |=> $stable(data))

else $error("Data changed after valid was asserted");

☑ Three-line version (property style)

property data_stable_when_valid;

@(posedge clk) valid |-> $stable(data);

endproperty

assert property (data_stable_when_valid);

📘 Small Example (Cycle-based)

Cycle	valid	data	Result
1	0	8'hA	OK
2	1	8'h5	OK
3	1	8'h5	OK
4	1	8'h7	❌ FAIL (data changed)

⭐ **Interview Tip: -**

> `$stable(data)` checks that the value of `data` did not change from the previous sampled clock.

“Request must be followed by Grant”

☑ Most Common (Simple & Clean)

assert property (@(posedge clk) req |-> ##[1:3] grant)

else $error("Grant did not follow Request");

🔹 Meaning (very simple):

- When `req` becomes 1
- `grant` must become 1
- within 1 to 3 clock cycles

☑ If Grant must come in the NEXT cycle only

assert property (@(posedge clk) req |=> grant)

else $error("Grant did not come in next cycle");

☑ **Three-line (Property Style)**

property req_followed_by_grant;

@(posedge clk) req |-> ##[1:3] grant;

endproperty

assert property (req_followed_by_grant);

📘 Cycle Example

Cycle	req	grant	Result
1	0	0	OK
2	1	0	Waiting
3	0	1	✓ PASS
4	0	0	Done

⭐ Interview Tip

- Use `|=>` → exact next cycle
- Use `##[ ]` → range of cycles

☑ One-line SystemVerilog Assertion

assert property (@(posedge clk) !(read && write));

☑ Two-line Version (Recommended – with message)

assert property (@(posedge clk) !(read && write))

else $error("ERROR: read and write both HIGH");

🧠 Simple Meaning

At every rising edge of clock:

- Check: `read` and `write` are not 1 at the same time.
- If they are → assertion fails.

⭐ Interview Tip

Most real projects prefer this short form unless the rule is complex.

Yes ☑ — we can write it cleanly in exactly THREE lines using `property`.

☑ Three-line SystemVerilog Assertion

property rw_prop; @(posedge clk) !(read && write); endproperty

assert property (rw_prop);

🧠 Simple Explanation

1. Line 1: Define the rule

   → “At every clock edge, read and write must NOT be high together”

2. Line 2: Assert (check) that rule during simulation

⭐ Why this form is useful

✓ More readable than one-line

✓ Easy to reuse the property

✓ Commonly used in real projects

✓ Optional (still 3 lines, with message)

property rw_prop; @(posedge clk) !(read && write); endproperty

assert property (rw_prop) else $error("Read & Write both HIGH");

Simple and short form (ONE line) without explicitly writing sequence and property blocks.

☑ Short & Simple One-Line Code

assert property (@(posedge clk) $rose(req) |-> ##[2:4] $rose(ack))

else $error("ACK did not arrive in 2–4 cycles after REQ");

🤓 One-Line Meaning (Very Simple)

👉 Whenever `req` rises, `ack` must rise within 2 to 4 clock cycles.

⭐ Why this works

- `$rose(req)` → detects request
- `|->` → implies (if req happens, then…)
- `##[2:4]` → wait 2 to 4 cycles
- `$rose(ack)` → acknowledge must happen

This single assertion line replaces all 4 steps (boolean → sequence → property → assert).

✓ Interview Tip

You can say:

> “For simple checks, we usually write SVA in one-line `assert property` form to keep the code compact and readable.”

◇ 1. When `req` rises

- `req` changes from 0 → 1
- This is detected using:

$rose(req)

- It means a request has been made and the check should start now.

◇ 2. After 2 to 4 clock cycles (define the range like [min:max])

The system waits at least 2 cycles and at most 4 cycles

This is written as:

##[2:4]

It defines the allowed time window for the response.

◇ 3. `ack` must rise

- `ack` must change from 0 → 1
- This is detected using:

$rose(ack)

- It means the request is successfully acknowledged.

🧠 Putting all three together (Plain English)

> “When `req` goes high, then within the next 2 to 4 clock cycles, `ack` must go high.”

★ Why this is important-

- Ensures correct handshake timing

- Catches late or missing acknowledgments
- Commonly used in bus protocols and interfaces

If you want, I can also explain this with a **cycle-by-cycle table or waveform**.

Got it 👍

Difference between `|->` and `|=>` in SystemVerilog Assertions

Both are implication operators, but they differ in WHEN the checking starts.

◇ `|->` → Overlapping implication

👉 The result can start in the SAME clock cycle.

Meaning in simple words:

> "If the left condition happens, the right condition may start now or later."

◇ `|=>` → Non-overlapping implication

👉 The result starts from the NEXT clock cycle

Meaning in simple words:

> "If the left condition happens, the right condition must happen after this cycle."

📘 Simple Example (Best Way to Understand)

🎯 Requirement:

When `req` becomes high, `ack` must become high one cycle later.

❌ Wrong way (using `|->`)

```
assert property (@(posedge clk)
  $rose(req) |-> $rose(ack)
```

);

❓ What happens?

- If `req` and `ack` rise in the same clock
- Assertion PASSES

❌ But we wanted ack next cycle, not same cycle

☑ Correct way (using `|=>`)

```
assert property (@(posedge clk)
  $rose(req) |=> $rose(ack)
);
```

✓ What happens?

- `req` rises at cycle N
- `ack` must rise at cycle N+1
- Correct behavior enforced

📊 Waveform Comparison

Case 1: `|->` (Overlapping)

clk : ↑ ↑

req : 0 → 1

ack : 0 → 1 ← same cycle allowed

✓ PASS

Case 2: `|=>` (Non-overlapping)

clk : ↑ ↑ ↑

req : 0 → 1

ack : 0 → 1 ← next cycle required

✓ PASS

🧠 One-Line Memory Trick (Interview Gold)

> ◇ `|->` → Same cycle allowed

> ◇ `|=>` → Next cycle only

⭐ Another Common Example

Requirement:

- If `reset` is asserted, output must be zero in the same cycle.

```
assert property (@(posedge clk)
  reset |-> (out == 0)
);
```

✓ Use `|->` because it is same cycle

Requirement:

If `reset` is asserted, output must be zero from next cycle.

```
assert property (@(posedge clk)
  reset |=> (out == 0)
);
```

✓ Use `|=>` because it is next cycle

🎯 Final Summary Table

Operator	Name	When RHS starts	
`	->`	Overlapping	Same cycle or later
`	=>`	Non-overlapping	Next cycle only

##1 vs |=> (Very Simple Explanation)

Both are used to say "next clock cycle", but they are used in different places and for different purposes.

🔹 `##1` → Explicit cycle delay

Means: wait exactly 1 clock cycle

Used inside sequences

Simple meaning:

> "Do this, then after 1 clock, do that"

🔹 `|=>` → Non-overlapping implication

- Means: RHS starts from the next clock cycle
- Used between condition and consequence

Simple meaning:

> "If this happens now, then from the next cycle onward, that must happen"

📘 Simple Example (Same Requirement)

🎯 Requirement:

When `req` rises, `ack` must rise in the next clock cycle.

☑ Using `##1`

```
assert property (@(posedge clk)
  $rose(req) |-> ##1 $rose(ack)
);
```

How it works:

- `$rose(req)` at cycle N

- `##1` → wait one cycle
- `$rose(ack)` must occur at N+1

☑ Using `|=>`

```
assert property (@(posedge clk)
  $rose(req) |=> $rose(ack)
);
```

How it works:

- `$rose(req)` at cycle N
- `|=>` automatically shifts checking to cycle N+1
- `$rose(ack)` must occur at N+1

📊 Cycle-by-Cycle View

Cycle	req	ack	Result
N	↑	0	req detected
N+1	0	↑	✓ PASS

Both assertions PASS.

🧠 Key Difference (Very Important)

Feature	`##1`	`|=>`
What it does	Explicit delay	Implicit next cycle
Used in	Sequences	Implication
Flexibility	Can do `##[1:3]`,`##2`	Always next cycle

| Readability | More control | More concise |

⭐ Interview One-Line Answer

> `##1` explicitly delays the sequence by one clock, while `|=>` automatically starts checking the consequence from the next clock cycle.

🎯 When to use what-

✓ Use `##1` when:

- You want exact control over delay (`##2`, `##[1:3]`)
- Writing complex sequences

✓ Use `|=>` when:

- You simply mean “next cycle”
- You want short, clean code

Types of assertions

Immediate Assertions

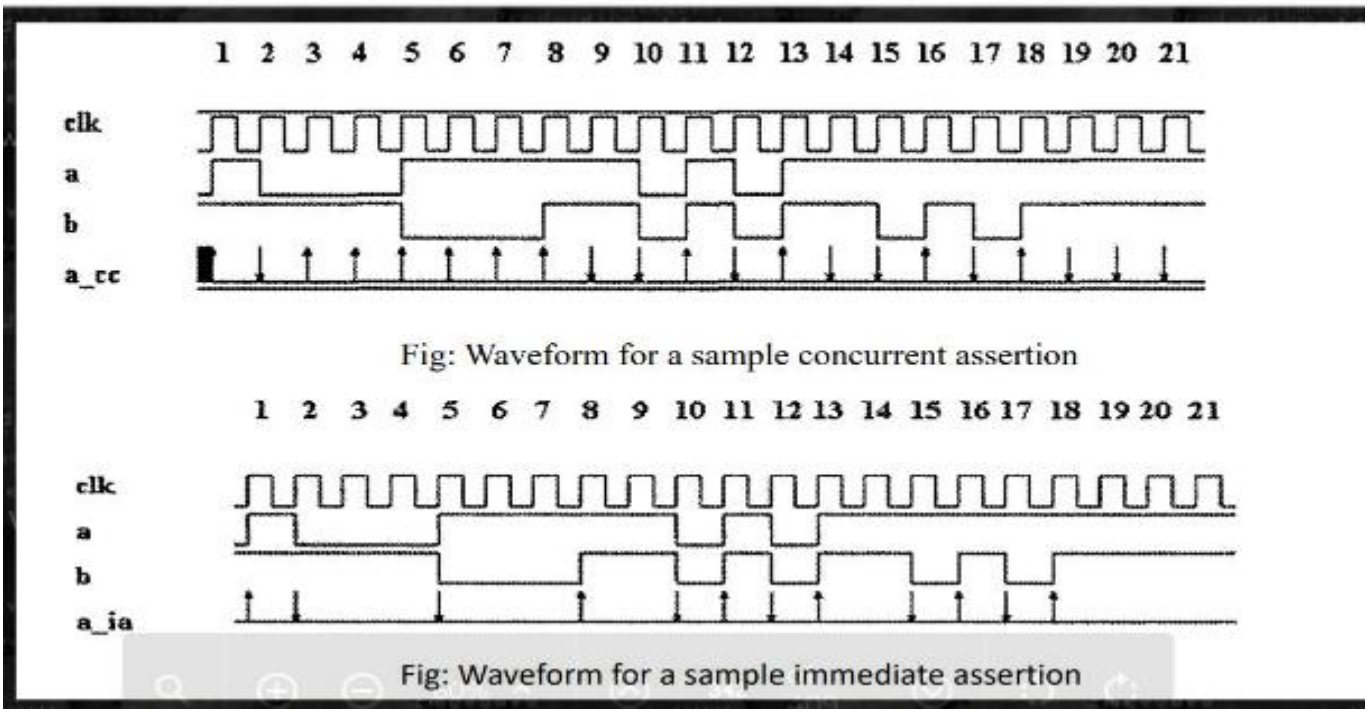
An assertion that checks a condition at the current simulation time is called immediate assertions. They are executed like procedural statements like the if-else statements.

Syntax:

<label>: assert (expression) <pass_statement> else <fail_statement>

1. If an expression evaluates true then pass_statement is executed, else the fail_statement is executed and said to be assertion failure. An expression evaluates to be 0, X, Z then an assertion is said to be failed.
2. Both pass_statement and fail_statement in the above syntax are optional.

Types of assertions



▨ Top waveform: Concurrent Assertion

What is happening in the graph?

- The assertion samples signals only at clock edges
- Each arrow on `a_cc` shows the assertion being checked at every posedge of `clk`
- The assertion observes a and b over multiple clock cycles

Simple meaning:

👉 Concurrent assertion checks the relationship between signals across time.

Example idea:

> “If `a` goes high, `b` must go high after some clock cycles.”

▧ Bottom waveform: Immediate Assertion

What is happening in the graph?

- The assertion is checked whenever the procedural code executes
- Arrows on `a_ia` show instant checks
- It does not wait for future clock cycles

Simple meaning:

👉 Immediate assertion checks the condition instantly at that moment.

Example idea:

> “At this time, `a` must be equal to `b`.”

🔑 Key Difference Shown by the Image

Concurrent Assertion	Immediate Assertion
Checked on clock edges	Checked immediately
Looks across cycles	Checks only current values
Used for timing rules	Used for simple checks

🧠 One-line Summary

- Top graph: Concurrent assertion → checks behavior over time
- Bottom graph: Immediate assertion → checks value right now

That’s exactly what the image is explaining 👍

Yes 👍

I’ll explain it using only the numbers (clock cycles) shown in the image.

Concurrent Assertion (Top waveform)

👉 In the image, concurrent assertion (`a_cc`) is checked at EVERY clock edge.

So the assertion occurs at ALL clock numbers:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21

📌 Meaning:

At every posedge of clk, the concurrent assertion samples signals.

That’s why arrows appear continuously.

🟦 Immediate Assertion (Bottom waveform)

👉 In the image, immediate assertion (`a_ia`) occurs ONLY when the code executes, not every clock.

From the waveform arrows, it occurs only at specific cycles:

1, 3, 6, 9, 11, 12, 13, 15, 17, 18

📌 Meaning:

Immediate assertion runs only when triggered by procedural code (like `always`, `if`, etc.).

So arrows appear only at those numbers, not continuously.

🔑 Final Simple Comparison

| Assertion Type | Cycle Numbers Where It Occurs |

| ---------- | ---------- |

| Concurrent | 1 → 21 (every clock) |

| Immediate | 1, 3, 6, 9, 11, 12, 13, 15, 17, 18 |

🧠 One-line takeaway

Concurrent assertion → checked at every clock number

Immediate assertion → checked at selected clock numbers only

Types of Assertions:-

There are two main types:

1. Immediate Assertions

2. Concurrent Assertions

🟦 1. Immediate Assertions

- Checked immediately when the statement is executed

- Used inside procedural blocks (`initial`, `always`)
- Similar to an if-else check

🔹 **When to use?**

✓ Checking values at a specific time

✓ Simple conditions

✓ Combinational checks

🧪 Example 1: Check division by zero

always @(posedge clk) begin

  assert (b != 0)

    else $error("Division by zero");

end

🧠 Meaning:

At every clock edge, check that `b` is not zero.

🧪 Example 2: Signal should not be X

assert (data !== 1'bx)

  else $error("DATA is X");

🧠 **Meaning:**

Immediately checks if `data` is unknown.

⭐ Key Point

> Immediate assertions check right now, not over time.

▨ 2. Concurrent Assertions

- Checked over multiple clock cycles
- Used outside procedural blocks
- Best for timing and protocol rules

🔹 When to use?

✓ Handshake protocols

✓ Timing relationships

✓ Sequence checking

🧪 Example 1: Request must be followed by Grant

```
assert property (@(posedge clk)
  req |-> ##[1:3] grant
);
```

🧠 Meaning:

If `req` becomes high, `grant` must become high within 1–3 cycles.

🧪 Example 2: Data must stay stable when valid = 1

```
assert property (@(posedge clk)
  valid |-> $stable(data)
);
```

🧠 **Meaning:**

When `valid` is 1, `data` must not change.

⭐ Key Point

> Concurrent assertions check behavior across time, not instantly.

📊 Immediate vs Concurrent (Very Simple Table)

Feature	Immediate	Concurrent
Checked	Immediately	Over time
Location	Inside `always` / `initial`	Outside blocks
Best for	Simple checks	Timing & protocols
Uses clock	Optional	Usually required

🎯 One-Line Memory Trick (Interview Ready)

> ◇ Immediate assertions are like `if` statements

> ◇ Concurrent assertions are like timing rules

☑ Immediate Assertion

Checks instantly at that moment

No timing involved

Example:

assert (a == b);

Waveform idea:

time → t0

a : 1

b : 1 ✓ checked now

👉 “Check this value right now.”

☑ Concurrent Assertion

Checks over clock cycles

Timing matters

Example:

assert property (@(posedge clk) req |=> grant);

Waveform idea:

clk : ↑ ↑

req : 1

grant : 1 ✓ next cycle

👉 “If this happens now, that must happen later.”

⭐ One-Line Difference (Easy to Remember)

> Immediate assertion → checks now

> Concurrent assertion → checks over time

A simple action block

What is an Action Block in SV Assertion?

An action block is the code that runs when an assertion passes or fails.

There are two action blocks:

Pass action

Fail action

🧱 Basic Syntax

assert property (property_name)

  pass_action

  else fail_action;

👉 `pass_action` → runs when assertion passes

👉 `fail_action` → runs when assertion fails

Example

Rule: Request must be followed by Grant

```
assert property (@(posedge clk) req |=> grant)
  $display("PASS: Grant received")
  else $error("FAIL: Grant missing");
```

🧠 Simple Explanation

If `req` is followed by `grant` → PASS message printed

If not → ERROR message printed

🟩 Another Example

Rule: Read and Write must not be high together

```
assert property (@(posedge clk) !(read && write))
  $display("OK")
  else $fatal("ERROR: read & write both HIGH");
```

⭐ Key Points (Very Short)

- Action block = what to do on pass/fail
- `else` part is fail action
- Pass action is optional
- Used for debugging and reporting

😵 One-line Summary

> Action blocks tell the simulator what to do when an assertion passes or fails.

Assertions Severity Levels

What are Assertion Severity Levels?

Severity levels decide what action the simulator takes when an assertion fails.

They tell:

👉 Should we just print a message or stop the simulation?

🟦 Main Severity Levels in System Verilog

Severity	What it does	Simulation
`$info`	Just information	Continues
`$warning`	Warning message	Continues
`$error`	Error reported	Continues
`$fatal`	Fatal error	Stops

1. `$info` – Informational

Used when failure is not serious, only for tracking.

```
assert property (@(posedge clk) a == b)
  else $info("INFO: a and b are different");
```

✓ Prints message

✓ Simulation continues

2. `$warning` – Warning

Used when something is not ideal but acceptable.

```
assert property (@(posedge clk) data < 100)
  else $warning("WARNING: data is high");
```

✓ Shows warning

✓ Simulation continues

3. `$error` – Error

Used for real functional bugs.

```
assert property (@(posedge clk) !(read && write))
  else $error("ERROR: Read & Write both HIGH");
```

✓ Reports error

✓ Simulation continues (useful to catch more errors)

🟥🟥 4. `$fatal` – Fatal Error

Used for critical bugs.

```
assert property (@(posedge clk) reset |-> (out == 0))
  else $fatal("FATAL: Reset failed");
```

❌ Simulation stops immediately

🎯 Simple Example Showing All Levels

```
assert (sig1 == sig2)
  else begin
    $info("Info message");
    $warning("Warning message");
    $error("Error message");
    //$fatal("Fatal message");
  end
```

⭐ When to Use Which? (Very Simple)

- `$info` → debug / monitoring
- `$warning` → suspicious but OK
- `$error` → bug, but continue testing
- `$fatal` → serious bug, stop simulation

🧠 One-line Memory Trick (Interview Ready)

- `$info` → tell
- `$warning` → caution

- `$error` → bug
- `$fatal` → stop

Sequence in Assertions

- The series or expressions spread over one or more clock cycles is called a sequence. A complex property can be phrased with multiple sequence blocks. Sequences are constructed over System Verilog boolean expressions. If an expression is evaluated correctly, a sequence is said to be matched. These expressions evaluate over a period of time that may involve one or more clock cycles.
- The sequence feature provides the capability to manipulate and build sequential behavior.
- The property checks for one or more System Verilog sequences. The evaluation of sequence involves a search for a match of the sequence beginning at a particular clock tick.
  The clock cycle delay is specified using ## from the end of the first sequence till the start of the second sequence.

sequence <sequence_name>;

<expression>;

endsequence

sequence with edge definitions

Sequence with edge definition

☑ What is a Sequence in SVA?

A sequence describes how signals change over time (cycle by cycle).

👉 It answers:

“What happens first, next, and later?”

☑ What are Edge Definitions?

Edge definitions detect signal transitions:

| Edge | Meaning (Simple) |

| ---------- | ---------- |

| `$rose(sig)` | sig goes 0 → 1 |

| `$fell(sig)` | sig goes 1 → 0 |

| `$stable(sig)` | sig does not change |

| `$changed(sig)` | sig changes value |

These are commonly used inside sequences.

🟩 Basic Sequence Syntax

sequence seq_name;

<edge / boolean expressions with timing>;

endsequence

📘 Example 1: Request followed by Grant (using edges)

🎯 Requirement:

👉 When `req` rises, `grant` must rise within 1–3 clock cycles.

sequence req_to_grant;

$rose(req) ##[1:3] $rose(grant);

endsequence

assert property (@(posedge clk) req_to_grant);

🧠 Simple meaning:

- Detect rising edge of `req`
- Wait 1 to 3 clock cycles
- Detect rising edge of `grant`

🟩 Example 2: Reset goes low, output must become stable

🎯 Requirement:

👉 After reset falls, output must not change.

```
sequence reset_release;
  $fell(reset) ##1 $stable(out);
endsequence

assert property (@(posedge clk) reset_release);
```

🧠 Simple meaning:

- Detect reset falling edge
- After 1 cycle
- Output must remain same

▧ Example 3: Valid high → Data must remain stable

```
sequence valid_data_stable;
  valid ##1 $stable(data);
endsequence

assert property (@(posedge clk) valid_data_stable);
```

🧠 Meaning:

- When valid is high
- In the next cycle
- Data should not change

📘 Example 4: Enable rising → Busy must go high

```
sequence en_to_busy;
  $rose(enable) ##1 $rose(busy);
endsequence

assert property (@(posedge clk) en_to_busy);
```

⭐ Why Use Edge-Based Sequences?

✓ Clearly detect events

✓ Avoid repeated checks

✓ Best for handshake protocols

✓ Makes assertions readable and accurate

🤓 Very Short Summary

> A sequence with edge definitions describes how signal edges must occur over time.

Sequence with edge definitions

👉 It explains how `$past` with a gating signal is used in System Verilog assertions to check past values only when a condition is valid.

Syntax

$past(signal_name, number_of_clock_cycles, gating_signal)

Simple meaning:

> "Give me the value of `signal_name` from N cycles ago,

> but only if `gating_signal` was TRUE at that time."

🔹 Code shown in the image

```
property p;
  @(posedge clk) b |-> ($past(a, 2, c) == 1);
endproperty
assert property (p);
```

🔹 Explain this code in VERY SIMPLE words

Step-by-step meaning:

1. `@(posedge clk)`

Check everything only at rising edges of clock

2. `b |->`

If `b` is HIGH now, then the condition on the right must be true

3. `$past(a, 2, c)`

- Look at signal `a`
- 2 clock cycles ago
- But only if `c` was HIGH at that time

3. `== 1`

- `a` must have been HIGH (1)

🧠 Plain English Rule

> “Whenever `b` is HIGH, signal `a` must have been HIGH two cycles earlier,

> but only if `c` was HIGH at that time.”

🟦 Simple Timing Example (Cycle by Cycle)

Cycle	a	b	c	Check?
1	0	0	1	—
2	1	0	1	—
3	0	0	0	—
4	0	1	1	Check happens

2 cycles before (cycle 2)	a=1		c=1	✓ PASS

Why PASS?

- `b = 1` at cycle 4
- Go back 2 cycles → cycle 2
- At cycle 2:
- `a = 1`
- `c = 1` (gating signal valid)
- Condition satisfied ✓

❌ Failure Example

| Cycle | a | b | c |

| ----- | - | - | - |

| 2 | 0 | 0 | 1 |

| 4 | 1 | 1 | 1 |

- `b` is HIGH at cycle 4
- 2 cycles ago (cycle 2):
- `a = 0`
- `c = 1`

* ❌ Assertion FAILS

⭐ Why use gating signal (`c`)?

✓ Avoid checking invalid cycles

✓ Ignore garbage / reset cycles

✓ Apply assertion only when condition is meaningful

🧠 One-line Summary (Interview Ready)

> `$past(a,2,c)` checks the value of `a` two cycles ago only when `c` was true.

🔹 Short explanation

`$past(a,2)`

👉 Checks the value of `a` 2 clock cycles ago, always (no condition).

`$past(a,2,c)`

👉 Checks the value of `a` 2 clock cycles ago, only if `c` was HIGH at that time.

(`c` is a gate or enable signal)

🔹 Simple diagram (clock-by-clock)

Case 1: `$past(a,2)` (NO gating)

clk : ↑ ↑ ↑ ↑ ↑

cycle: 1 2 3 4 5

a : 0 1 0 1 0

b : 0 0 0 1 0

^

|-- check here

looks at a @ cycle 3

✓ At cycle 5, `$past(a,2)` → value of `a` at cycle 3

Case 2: `$past(a,2,c)` (WITH gating)

clk : ↑ ↑ ↑ ↑ ↑

cycle: 1 2 3 4 5

a : 0 1 0 1 0

c : 1 0 1 1 1

b : 0 0 0 1 0

^

|-- check here

looks at a @ cycle 3

ONLY if c @ cycle 3 = 1

✓ If `c` at cycle 3 = 1 → check is valid

❌ If `c` at cycle 3 = 0 → check is ignored

🔑 One-line summary (very important)

> `$past(a,2)` = unconditional past value

> `$past(a,2,c)` = past value only when `c` was true

This is commonly used to ignore reset or invalid cycles in assertions.

☑ What is `$past`?

`$past` is used in assertions to get the value of a signal from a previous clock cycle.

👉 Think of it as:

“What was the value earlier?”

🧠 **Basic Syntax-**

$past(signal)

$past(signal, N)

- `$past(signal)` → value 1 cycle ago
- `$past(signal, N)` → value N cycles ago

🟦 Example 1: Check signal did not change

🎯 Requirement:

Data should remain the same as last cycle.

```
assert property (@(posedge clk)
  data == $past(data)
);
```

Simple meaning:

- Compare current data
- With previous cycle data
- If different → assertion fails

🟦 Example 2: Detect rising edge (without `$rose`)

🎯 Requirement:

Detect when `a` goes from 0 → 1.

```
assert property (@(posedge clk)
  (a == 1) && ($past(a) == 0)
);
```

Simple meaning:

- Now `a` is 1
- Previously `a` was 0
- So a rising edge occurred

🟦 Example 3: Reset behavior

🎯 Requirement:

If reset was high last cycle, output must be 0 now.

assert property (@(posedge clk)

  $past(reset) |-> (out == 0)

);

Simple meaning:

- Look at previous reset
- If it was high
- Output must be zero now

🟦 Example 4: Multi-cycle past value

🎯 Requirement:

Counter must increase by 1 every cycle.

assert property (@(posedge clk)

  count == $past(count) + 1

);

🟦 Example 5: Using `$past` with delay

🎯 Requirement:

Signal `b` must equal `a` from 2 cycles ago.

assert property (@(posedge clk)

  b == $past(a, 2)

);

Important Rule (Very Important!)

`$past` is undefined in the first cycle.

☑ Safe way (with reset):

assert property (@(posedge clk)

  disable iff (reset)

  data == $past(data)

);

⭐ When to use `$past`?

✓ Compare current vs previous values

✓ Check counters, pipelines

✓ Detect edges manually

✓ Debug timing issues

🧠 One-Line Memory Trick

> `$past(signal)` = signal value from previous clock.

Sequence with logical relationship

sequence h;

  @(posedge clk) a || b;

endsequence

🔹 Meaning of the code (in very simple words)

- The sequence `h` is checked at every positive edge of the clock.
- At each clock ↑ :
- Either `a` must be HIGH
- OR `b` must be HIGH
- If both `a` and `b` are LOW, the assertion FAILS

👉 This is a logical OR (||) relationship.

◇ One-line explanation (easy to remember)

> On every clock edge, at least one of `a` or `b` must be 1.

🔹 Understanding the waveform

Signals shown:

- `clk` → clock
- `a` → signal a
- `b` → signal b
- `h` → assertion check result (arrow means “checked here”)

✓ PASS condition

At a clock edge:

a = 1 , b = 0 → PASS

a = 0 , b = 1 → PASS

a = 1 , b = 1 → PASS

❌ FAIL condition

At a clock edge:

a = 0 , b = 0 → FAIL ❌

This is exactly what the circled points (1 and 2) in the waveform show.

🔹 Simple timing example

clk ↑ ↑ ↑ ↑

a 1 0 0 1

b 0 1 0 0

✓ ✓ ✗ ✓

3rd clock edge fails because:

- `a = 0`
- `b = 0`

🔹 Why this assertion is useful?

✓ Ensures at least one valid signal is active

✓ Used in mutual-activity checks

✓ Common in enable / request / valid logic

◇ Very short summary

- `a || b` → logical OR
- Checked every clock
- Both LOW → ✗ FAIL
- Any one HIGH → ✓ PASS

◇ FAIL cycles (circled in the graph)

✗ FAIL at point ①

Clock edge at cycle ~12

Values:

Copy code

a = 0

b = 0

✗ Both are LOW → assertion FAIL

✗ **FAIL at point ②**

- Clock edge at **cycle ~17**

Values:

a = 0

b = 0

❌ Again, both LOW → assertion FAIL

🔹 **Simple rule to remember**

a || b = 1 → PASS
a || b = 0 → FAIL

🔹 One-line summary

The circled numbers in the waveform show clock cycles where **both a and b are 0**, so the assertion fails there; all other numbered cycles pass.

Clock definitions in SVA

👉Clock should be written in the PROPERTY, not inside the SEQUENCE

- Makes sequences reusable
- Keeps timing clean and flexible

Sequence (NO clock here)

```
sequence ha;
  a ##2 b;
endsequence
```

**Property (clock defined here)

```
property p;
  @(posedge clk) ha;
endproperty
```

Assertion

```
c: assert property(p);
```

🔹 Simple explanation of each part

1 Sequence `ha`

a ##2 b;

👉 Means:

- First `a` becomes 1
- After 2 clock cycles
- `b` must become 1

⚠️ No clock mentioned here → clock-independent

2Property `p`

@(posedge clk) ha;

👉 Means:

- Check sequence `ha`
- Only on rising edge of `clk`

3Assertion

assert property(p);

👉 **Actively checks the rule during simulation**

🔹 Simple timing example

Signal behavior

clk : ↑ ↑ ↑ ↑ ↑

cycle:1 2 3 4 5

a :0 1 0 0 0

b :0 0 0 1 0

What happens?

- At cycle 2, `a = 1`
- After 2 cycles → check at cycle 4
- `b = 1` ✓ → PASS

❌ FAIL case

a = 1 at cycle 2

b = 0 at cycle 4

❌ Assertion FAILS

🔹 One-line summary (exam friendly)

Sequences describe behavior, properties define clocking, assertions check them.

Forbidding a property

What “Forbidding a property” means

👉 Sometimes we want to ensure that something NEVER happens.

👉 If that thing does happen, the assertion must FAIL.

🔹 Code from the image (simplified)

```
sequence ha;
  @(posedge clk) a ##2 b;
endsequence

property v;
  not ha;
endproperty
vh: assert property(v);
```

🔹 Simple explanation (line by line)

1Sequence `ha`

a ##2 b;

➞ Means:

- `a` becomes HIGH
- After 2 clock cycles
- `b` becomes HIGH

2Property `v`

not ha;

➞ Means:

This sequence must NEVER occur

3 Assertion

assert property(v);

➞ Checks that `a ##2 b` does NOT happen

🔹 How the waveform shows PASS / FAIL

✓ PASS

- `a` occurs
- But `b` does not occur after 2 cycles

✓ Safe → assertion passes

❌ FAIL (circled ① and ② in waveform)

- `a` goes HIGH
- Exactly 2 cycles later, `b` goes HIGH

❌ Forbidden behavior detected → assertion FAILS

🔹 Simple timing example

clk ↑ ↑ ↑ ↑ ↑

a 0 1 0 0 0

b 0 0 0 1 0

- `a` at cycle 2
- `b` at cycle 4

❌ Forbidden → FAIL

◇ One-line summary (exam ready)

Forbidding a property ensures that a specific sequence never occurs; if it occurs, the assertion fails.

Session-16: -

- Assertion: Part-2
- Implication operator
- Overlapped implication
- Non-overlapped implication
- Timing windows
- Repetition operators
- Consecutive repetition operator
- Go to repetition operator
- Non- Consecutive repetition operator

Implication Operator

The implication operator does a property check conditionally if the sequential antecedent is matched.

Syntax:

```
sequence_exp |-> property_exp

sequence_exp |=> property_exp
```

The LHS operand sequence_exp is called an antecedent

The RHS operand property_exp is called a consequent

Type of Implication

1. Overlapped implication
2. Non-overlapped implication

Overlapped implication

The overlapped implication operator is denoted by the |-> symbol.

The evaluation of the consequent starts immediately on the same clock cycle if the antecedent holds true.

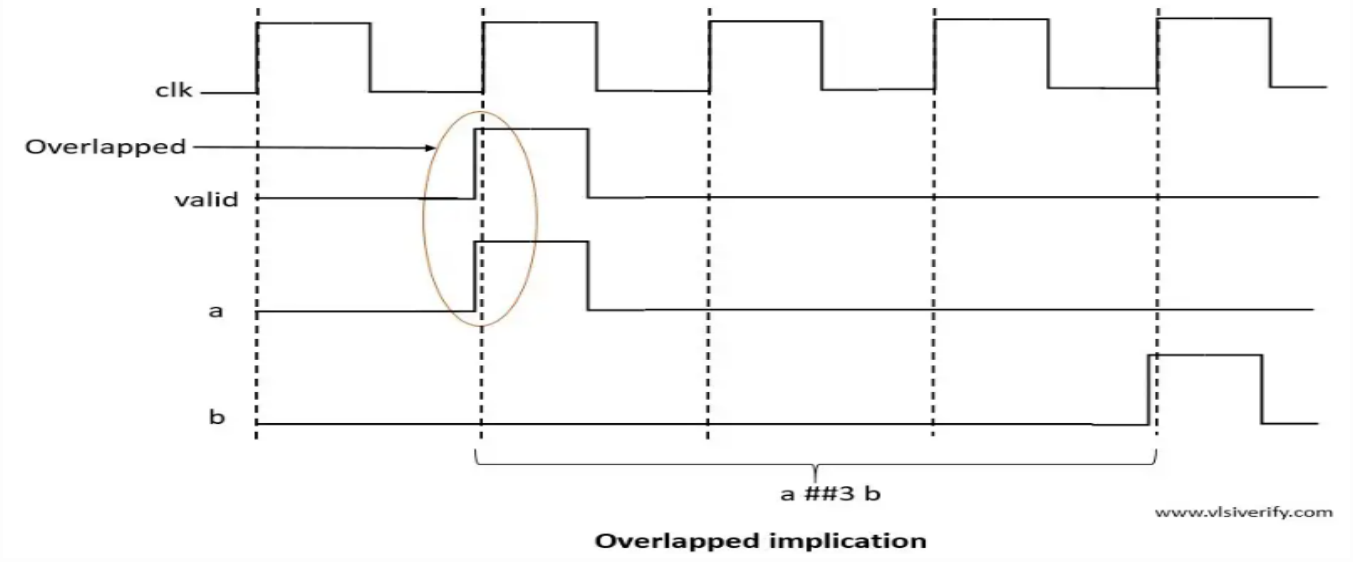
The consequent is not evaluated if the antecedent is not true.

Example:

```
property prop;

  @(posedge clk) valid |-> (a ##3 b);

endproperty
```



Overlapped implication

Non-overlapped implication

The non-overlapped implication operator is denoted by the |=> symbol.

The evaluation of the consequent starts in the next clock cycle if the antecedent holds true.

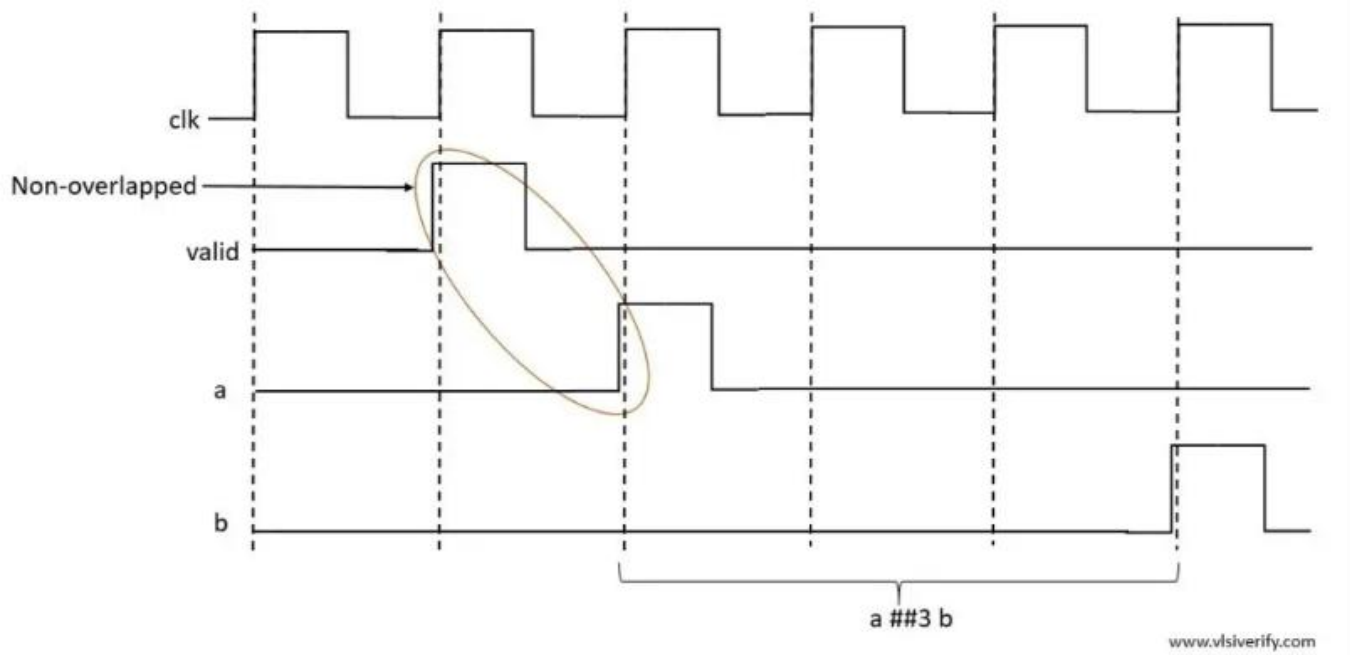
The consequent is not evaluated if the antecedent is not true.

Example:

```
property prop;

  @(posedge clk) valid |=> (a ##3 b);
```

```
endproperty
```



Non-overlapped implication

implication operator (`|->`, `|=>`)

What is an implication operator?

👉 It expresses “IF something happens, THEN something must follow.”

IF (antecedent) → THEN (consequent)

In SVA, this is written using implication operators.

◇ Types of implication operators

1Overlapping implication — |->

- Consequent can start in the SAME clock cycle

2 Non-overlapping implication — |=>

- Consequent starts from the NEXT clock cycle

◇ 1 Overlapping implication (|->)

Example

assert property (@(posedge clk) req |-> ack);

Meaning (simple)

- If `req` is HIGH at a clock edge
- Then `ack` must be HIGH at the same clock edge

Timing

clk ↑ ↑ ↑

req 0 1 0

ack 0 1 0 ✓ PASS

❌ Fail if `req=1` and `ack=0` at same edge.

◇ 2 Non-overlapping implication (`|=>`)

Example

assert property (@(posedge clk) req |=> ack);

Meaning (simple)

- If `req` is HIGH at this clock edge
- Then `ack` must be HIGH at the next clock edge

Timing

clk ↑ ↑ ↑

req 0 1 0

ack 0 0 1 ✓ PASS

❌ Fail if `ack` is not HIGH in the next cycle.

◇ With delay (very common)

assert property (@(posedge clk) req |=> ##2 ack);

👉 If `req` occurs

👉 `ack` must occur 2 cycles later

◇ Very important rule (easy exam point)

❗ **If the IF part never happens, the assertion passes automatically.**

◇ Real-life simple example

“Door rule”

- IF door button is pressed
- THEN door should open

button |=> door_open

◇ One-line summary

| Operator | Meaning | |

| --- | --- | --- |

| `overlapped | ->` | Same cycle |

| `non-overlapped | =>` | Next cycle |

◇ Short exam-friendly line

Implication operator checks that when a condition happens, a required response follows in the same or next clock cycle.

waveform diagrams

1.0 Overlapping implication `|->`

Rule:

> IF `req` is HIGH, THEN `ack` must be HIGH in the SAME clock cycle

Assertion

@(posedge clk) req |-> ack;

Waveform (PASS case)

clk : ↑ ↑ ↑

req : 0 1 0

ack : 0 1 0

✓

✓ At the same clock edge where `req = 1`, `ack = 1` → PASS

Waveform (FAIL case)

clk : ↑ ↑ ↑

req : 0 1 0

ack : 0 0 1

✗

✗ `ack` not HIGH in same cycle → FAIL

2.0 Non-overlapping implication `|=>`

Rule:

> IF `req` is HIGH, THEN `ack` must be HIGH **in the NEXT clock cycle**

Assertion

@(posedge clk) req |=> ack;

Waveform (PASS case)

clk : ↑ ↑ ↑

req : 0 1 0

ack : 0 0 1

✓

✓ `ack` goes HIGH one cycle after `req` → PASS

Waveform (FAIL case)

clk : ↑ ↑ ↑

req : 0 1 0

ack : 0 0 0

❌

❌ `ack` did not occur next cycle → FAIL

3.0 Non-overlapping with delay

Assertion

@(posedge clk) req |=> ##2 ack;

Meaning:

> `ack` must be HIGH 2 cycles after `req`

Waveform

clk : ↑ ↑ ↑ ↑ ↑

req : 0 1 0 0 0

ack : 0 0 0 1 0

✓

◇ Easy memory trick

|-> = same cycle

|=> = next cycle

☑ Final one-line summary

Implication operators describe cause–effect behavior across clock cycles using simple timing rules.

Overlapped implication

Non- Overlapped implication

Timing windows in SVA Checkers

Timing Windows

property vs;

@(posedge clk) (a && b) |-> ##[1:3] c;

endproperty

is: assert property(vs);

🔹 Simple meaning (plain words)

- When both `a` and `b` are HIGH,
- then `c` must become HIGH sometime between 1 and 3 clock cycles later.
- This 1–3 cycle range is called a timing window.

🔹 Understanding ##[1:3] (very important)

##[1:3] c

Means:

- Check `c` after 1 OR 2 OR 3 clock cycles
- If `c` is HIGH in any one of these cycles → PASS
- If not → FAIL

🔹 Waveform explanation (from the image)

- Arrows on `is` show assertion checking points
- Assertion starts when `(a && b)` becomes true
- Then a window opens for 3 cycles
- If `c` goes HIGH inside that window → ✓ PASS
- If `c` stays LOW → ❌ FAIL

🔹 Example (simple timing)

clk ↑ ↑ ↑ ↑ ↑ ↑

a 0 1 0 0 0 0

b 0 1 0 0 0 0

c 0 0 1 0 0 0

- `(a && b)` at cycle 2
- `c` becomes HIGH at cycle 4 (2 cycles later)

✓ PASS (inside 1–3 window)

🔹 **Other timing window types (mentioned in image)**

1. 0 **Overlapping window**

##[0:3] c

- `c` may be HIGH in the same cycle
- `0` allows overlap

2. 0 **Indefinite window**

##[1:$] c

- `c` must occur sometime after 1 cycle
- No upper time limit

◇ One-line summary (exam friendly)

Timing windows allow checking that an event occurs within a specific range of clock cycles after a condition.

Repetition operators

What are repetition operators?

👉 Repetition operators specify how many times a condition or sequence must occur across clock cycles.

Think of them as “how long / how many cycles”.

🔹 Types of repetition operators

Operator	Meaning
[*]	Consecutive repetition
[=]	Non-consecutive repetition
[->]	Go-to repetition

1.0 Consecutive repetition `[*]`

Meaning (simple)

- Condition must hold continuously for N cycles

Example

assert property (@(posedge clk) a[*3]);

✓ `a` must be HIGH for 3 continuous clock cycles

Timing

clk ↑ ↑ ↑ ↑

a 1 1 1 0 ✓ PASS

❌ FAIL if `a` drops LOW in between

2.0 Non-consecutive repetition `[=]`

Meaning (simple)

- Condition must occur N times, not necessarily back-to-back

Example

assert property (@(posedge clk) a[=3]);

✓ `a` must be HIGH 3 times anywhere

Timing

clk ↑ ↑ ↑ ↑ ↑

a 1 0 1 0 1 ✓ PASS

3.0 Go-to repetition `[->]`

Meaning (simple)

- Skip cycles until the condition happens N times,
- then continue checking next sequence.

Example

assert property (@(posedge clk) a[->2] ##1 b);

✓ After `a` becomes HIGH twice,

✓ `b` must be HIGH in the next cycle

Timing

clk ↑ ↑ ↑ ↑ ↑

a 0 1 0 1 0

b 0 0 0 0 1 ✓ PASS

🔹 Common ranges

a[*2:4] // 2 to 4 consecutive cycles

a[=1:3] // 1 to 3 times (anywhere)

🔹 Very short memory trick

[*] → continuous

[=] → count only

[->] → jump then continue

🔷 One-line summary (exam ready)

Repetition operators define how many times and in what manner a condition must occur over clock cycles.

Operators in Assertions

Clock delays

1. ## : represents cycle delay

2. ##n – represents “n” clock cycles

3. ##0 – represents same clock cycle

4. ## [min:max] represents a range of clock cycles. Where min and max must be 0 or greater than 0.

```
sequence seq;

  @(posedge clk) req1 ## [2:6] req2;

endsequence
```

The sequence will be matched if req1 is true then within clock cycle delay of range 2 to 6, req2 should be high.

5. $ represents an infinite number of clock cycles i.e. till the end of the simulation.

```
sequence seq;

  @(posedge clk) req1 ## [2:$] req2;

endsequence
```

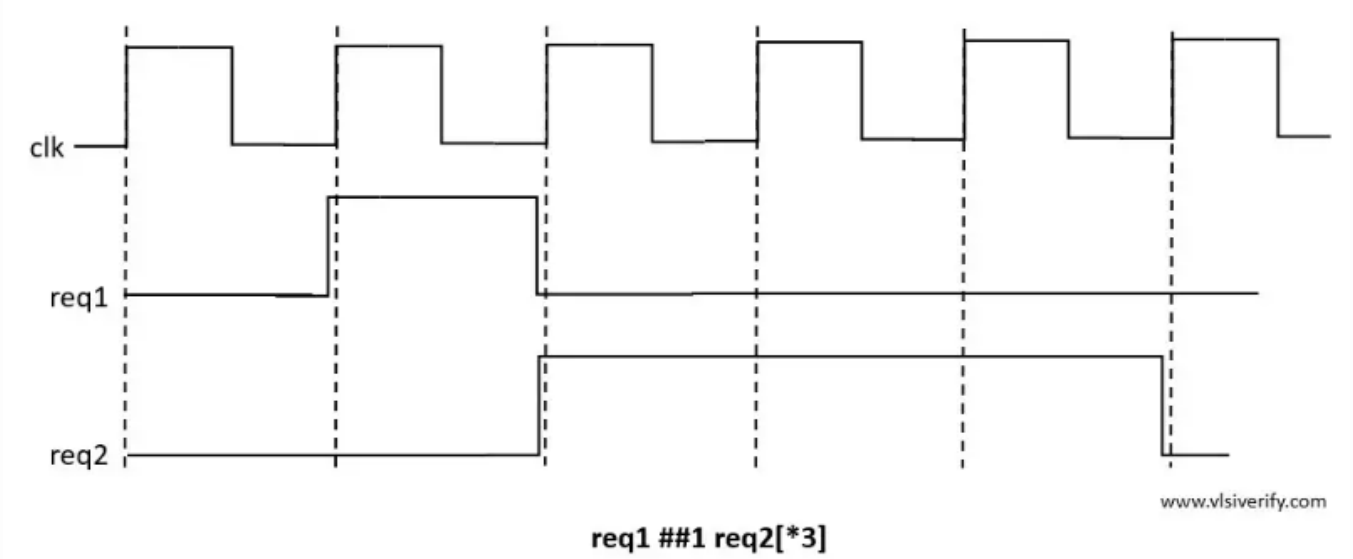
The sequence will be matched if req1 is true then req2 can be high at any time before simulation ends.

Repetition Operator

If the sequence of events happens repeatedly for n times and it is represented as [*n] where “n” > 0 and “n” can not be $.

Example 1:

```
sequence seq;
  @(posedge clk) req1 ##1 req2[*3];
endsequence
```



req1 ##1 req2[*3]

In this example, if req1 is true then after 1 clock cycle, req2 must be true for 3 consecutive clock cycles.

req1 ##1 req2[*3] is same as req1 ##1 req2 ##1 req2 ##1 req2.

Example 2:

The repetition operator can also be used in a certain range using [*m:n] where both “m” and “n” > 0 and n can not be $.

```
sequence seq;
  @(posedge clk) req1 ##1 req2[*2:4];
endsequence
```

In this example, if req1 is true then after 1 clock cycle, req2 must be true for a minimum of 2 and a maximum of 4 consecutive clock cycles.

req1 ##1 req2[*2:4] is same as

req1 ##1 req2 ##1 req2 or

req1 ##1 req2 ##1 req2 ##1 req2 or

req1 ##1 req2 ##1 req2 ##1 req2 ##1 req2;

Non-consecutive repetitive operator

If sequence event repetition has to be detected for n non-consecutive clock cycles then [=n] can be used where n > 0 and n can not be $.

sequence seq;

@(posedge clk) req1 ##1 req2[=4];

endsequence

In the above example, once req1 holds true after one clock cycle req2 must be true for 4 clock cycles but it is not mandatory to be consecutive clock cycles.

The non-consecutive repetitive operator with a range

The non-consecutive repetition can be mentioned in a range as [=m:n] where minimum “m” and maximum “n” non-consecutive repetition.

sequence seq;

@(posedge clk) req1 ##1 req2[=2:4];

endsequence

In the above example, once req1 holds true after one clock cycle, req2 must be true for a minimum of 2 and a maximum of 4 clock cycles but it is not mandatory to be consecutive clock cycles.

Consecutive repetition operator[*]

Go to repetition operator [->]

Go-to repetition operator `[->]`

◆ Code in the image (simplified)

property hr;

@(posedge clk) $rose(start) |-> ##2 (a[->3]) ##1 stop;

endproperty

assert property (hr);

🔹 **Simple meaning (plain English)**

- When `start` rises,
- 2 clock cycles later,
- signal `a` must occur 3 times (not necessarily continuously),
- and 1 cycle after the 3rd `a`,
- `stop` must be HIGH.

🔹 **Key part: `a[->3]` (Go-to repetition)**

- `a[->3]` means:
- Count 3 occurrences of `a`
- They do NOT have to be consecutive
- After the 3rd occurrence, move to the next part of the sequence

🔹 How this matches the waveform

Step-by-step in waveform

1.0 $rose(start)

✓ Start goes HIGH → assertion begins

2.0 ##2

✓ Wait 2 clock cycles

3.0 a[->3]

✓ `a` goes HIGH three times (can have gaps)

4.0 ##1 stop

✓ `stop` must go HIGH 1 cycle after the 3rd `a`.

🔹 Simple timing example

clk ↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

start 0 1 0 0 0 0 0 0

a 0 0 1 0 1 0 1 0

stop 0 0 0 0 0 0 0 1

✓ `a` occurs 3 times

✓ `stop` occurs after 3rd `a`

✓ Assertion PASSES

🔹 ❌ Fail case (simple)

- `a` occurs only 2 times
- OR `stop` does not come after 3rd `a`

→ Assertion FAILS

🔹 Why `[->]` is useful?

✓ Counting events

✓ Protocol sequences

✓ Handshake tracking

✓ Burst detection

🔹 One-line summary (exam ready)

Go-to repetition `[->]` waits for a signal to occur N times (with gaps allowed) before continuing to the next sequence.

Difference between Non-Consecutive `[=]` and Go-to `[->]` repetition operators

1.0 Non-consecutive repetition `[=]`

Meaning (simple)

> Count how many times a signal becomes TRUE

> Do NOT move forward to next sequence

Key point

✓ Counts occurrences

❌ Does not trigger the next event

Example

assert property (@(posedge clk) a[=3]);

Explanation

- `a` must be HIGH 3 times
- They can be anywhere
- No next condition after counting

Waveform

clk ↑ ↑ ↑ ↑ ↑

a 1 0 1 0 1

✓ ✓ ✓

✓ PASS after 3 occurrences

(no next step)

2.0 Go-to repetition `[->]`

Meaning (simple)

- Count how many times a signal becomes TRUE
- Then MOVE to the next condition

Key point

✓ Counts occurrences

✓ Triggers the next sequence

Example

assert property (@(posedge clk) a[->3] ##1 b);

Explanation

- Wait for `a` to occur 3 times
- After the 3rd occurrence
- `b` must be HIGH next cycle

Waveform

clk ↑ ↑ ↑ ↑ ↑ ↑

a 1 0 1 0 1 0

✓ ✓ ✓

b 0 0 0 0 0 1

✓

✓ PASS because `b` follows

◇ ❌ Failure case for `[->]`

a occurs 3 times

b does NOT occur

→ FAIL

◇ Side-by-side difference table

Feature	`[=]` Non-Consecutive	`[->]` Go-to
Counts occurrences	✓	✓
Requires continuous	❌	❌
Moves to next step	❌	✓
Used for	Counting only	Counting + action

| Common use | Coverage-like checks | Protocol sequences |

◇ Easy memory trick

[=] → Count ONLY

[->] → Count and GO

◇ One-line exam answer

Non-consecutive repetition counts occurrences without advancing the sequence, while Go-to repetition counts occurrences and then continues to the next part of the assertion.

Non-consecutive repetition operator [=]

Repetition Operators Comparison

1.0 Consecutive repetition `[*]`

Meaning

- Condition must be TRUE continuously for N cycles

Example

@(posedge clk) a[*3]

Waveform idea

a : 1 1 1 ✓

Use case

✓ Stability checks

✓ Hold conditions

2.0 Non-consecutive repetition `[=]`

Meaning

- Condition must be TRUE N times, anywhere
- ❌ Does NOT move to next sequence

Example

@(posedge clk) a[=3]

Waveform idea

a : 1 0 1 0 1 ✓

Use case

✓ Counting events

✓ Occurrence checks

3.0 Go-to repetition `[->]`

Meaning

- Condition must be TRUE N times, anywhere
- ✓ THEN continue to next sequence

Example

@(posedge clk) a[->3] ##1 b

Waveform idea

a : 1 0 1 0 1

b : 1 ✓

Use case

✓ Protocol sequencing

✓ Handshake / burst checks

🔹 **Side-by-Side Comparison Table**

Feature	`[*]`	`[=]`	`[->]`
Continuous required	✓	❌	❌
Counts occurrences	✓	✓	✓

Allows gaps	❌	✓	✓
Moves to next step	✓	❌	✓
Used for	Hold time	Event count	Event-then-action

◇ Easy memory trick

[*] → Hold

[=] → Count

[->] → Count + Go

◇ One-line exam answer

[*] checks continuous repetition, `[=]` counts occurrences without advancing, and `[->]` counts occurrences and then continues to the next part of the assertion.

Session-17: -

- Assertion: Part-3
- The “and” , “or” constrctors
- The “intersect” constrctors
- The “firstmatch” constrctors
- The “throughout” constrctors
- The “within” constrctors
- The “disable iff” constrctors
- Built-in system function
- Connecting SVA to the design
- SVA for the functional coverage

The “and , or” constrcuctrs

and operator (Conjunction) (hint:union)

The binary and operator are used when two sequence operands are expected to match, but the **end time of both operand sequences can be different.**

Syntax:

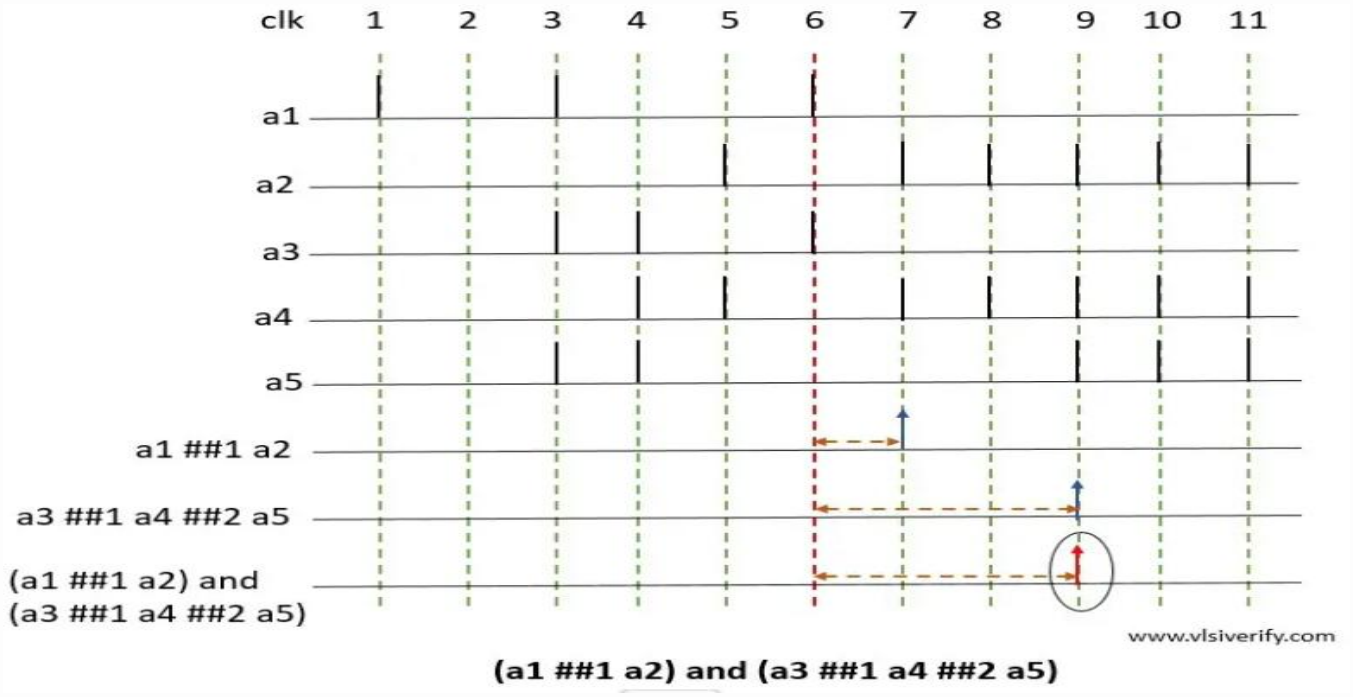
```
<seq_exp> and <seq_exp>
```

The requirement for and operation match

1. Both operand sequences should start at the same time.
2. Both operands must match. In case one operand matches, it waits for another operand to match.
3. The end time of the resultant or composite sequence is the end time of the operand that finishes last.

Example for AND of two sequences

```
(a1 ##1 a2) and (a3 ##1 a4 ##2 a5)
```



Explanation

a1 ##1 a2 as operand A and a3 ##1 a4 ##2 a5 as operand B.operand B

a1 ##1 a2 (Operand A): The a1 evaluates to true followed by a2 one clock tick matches at clock tick 7

a3 ##1 a4 ##2 a5 (operand B): The operand a3 evaluates to true followed by a4 one clock tick later, followed by a5 two clock ticks later i.e. at clock tick 9.

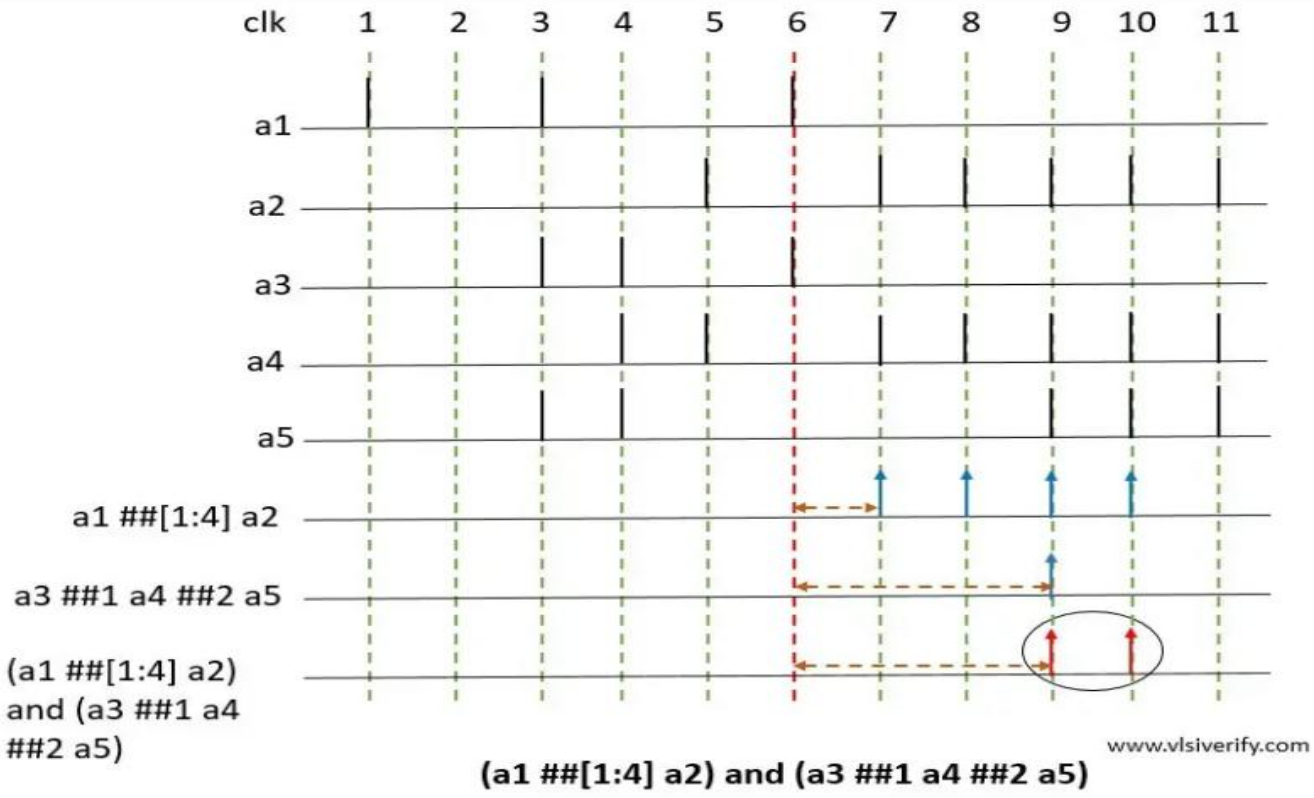
(a1 ##1 a2) and (a3 ##1 a4 ##2 a5): The resultant sequence is said to be matched when both of the above operand sequence matches. The end times of matches for individual operand sequences A and B are clock ticks 7 and 9 respectively.

The resultant sequence match should follow the conditions like individual operand sequences should start at the same clock tick (clock tick 6) and the end time of the last finishing operand sequence will be the end time of the resultant sequence (clock tick 9).

An important point to note here: The two operand sequences need not be matched at the same number of clock ticks.

Example for AND of two sequences including a time range

```
(a1 ##[1:4] a2) and (a3 ##1 a4 ##2 a5)
```



Explanation

Let's consider a1 ##[1:4] a2 as operand A and a3 ##1 a4 ##2 a5 as operand B.operand B

a1 ##[1:4] a2: The first operand a1 evaluates to true followed by a2 which evaluates to true for 1,2,3, or 4 clock ticks later. So, sequence matches at clock ticks 7,8,9, and 10 respectively.

a3 ##1 a4 ##2 a5: The operand a3 evaluates to true followed by a4 one clock tick later, followed by a5 two clock ticks later i.e. at clock tick 9.

(a1 ##1 a2) and (a3 ##1 a4 ##2 a5): The resultant sequence is said to be matched when both of the above operand sequence matches. The end times of matches for individual operand sequences are clock ticks 10 and 9.

The resultant sequence match should follow the conditions like individual operand sequences should start at the same clock tick (clock tick 6) and end time of resultant sequence can have two matches possible at clock tick 9 and 10 since if operand A match with operand B at clock tick 9 and operand A has end time at clock tick 10 as a last finishing operand sequence.

or operator (Disjunction) (hint: intersection)

The binary or operator is used when at least one of the two sequence operands is expected to match.

Syntax:

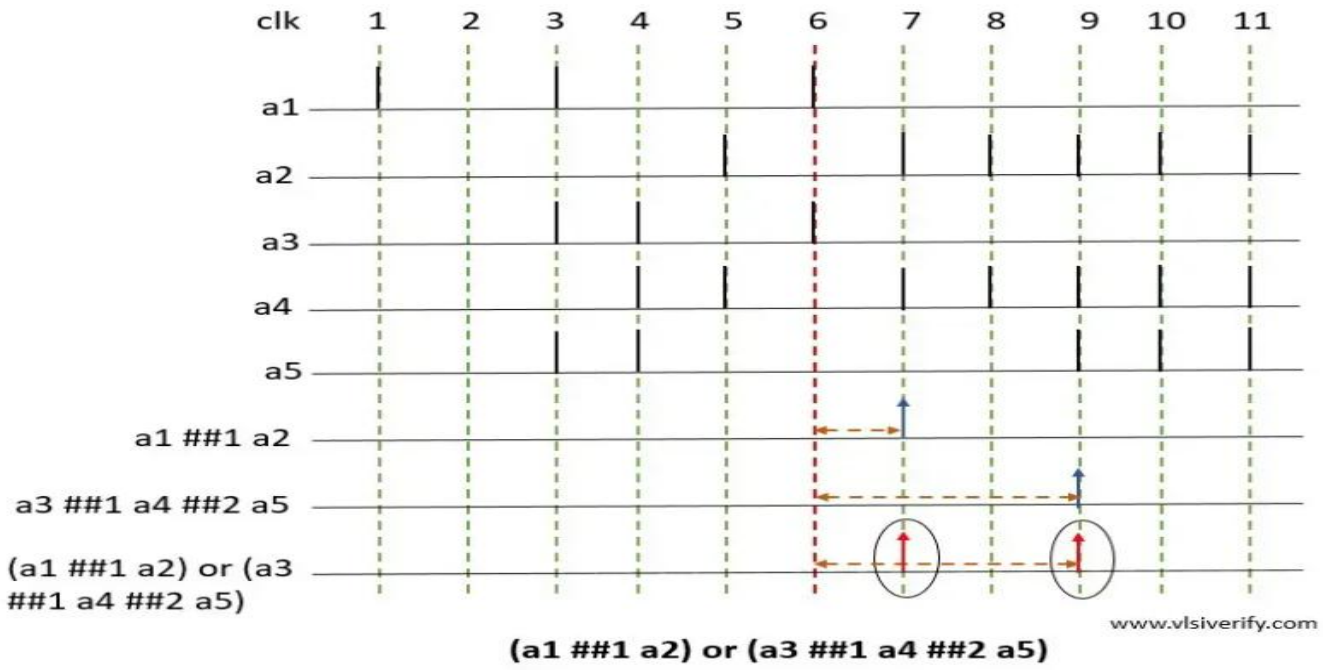
```
<seq_exp> or <seq_exp>
```

The requirement for or operation match

1. Both operand sequences should start at the same time.
2. The resultant or composite sequence match when either seq1 or seq2 matches
3. The end time of the resultant or composite sequence is the end time of the operand that finishes last.

Example for OR of two sequences

```
(a1 ##1 a2) or (a3 ##1 a4 ##2 a5)
```



(a1 ##1 a2) or (a3 ##1 a4 ##2 a5)

Explanation

Let's consider a1 ##1 a2 as operand A and a3 ##1 a4 ##2 a5 as operand B.operand B

a1 ##1 a2 (Operand A): The a1 evaluates to true followed by a2 one clock tick matches at clock tick 7

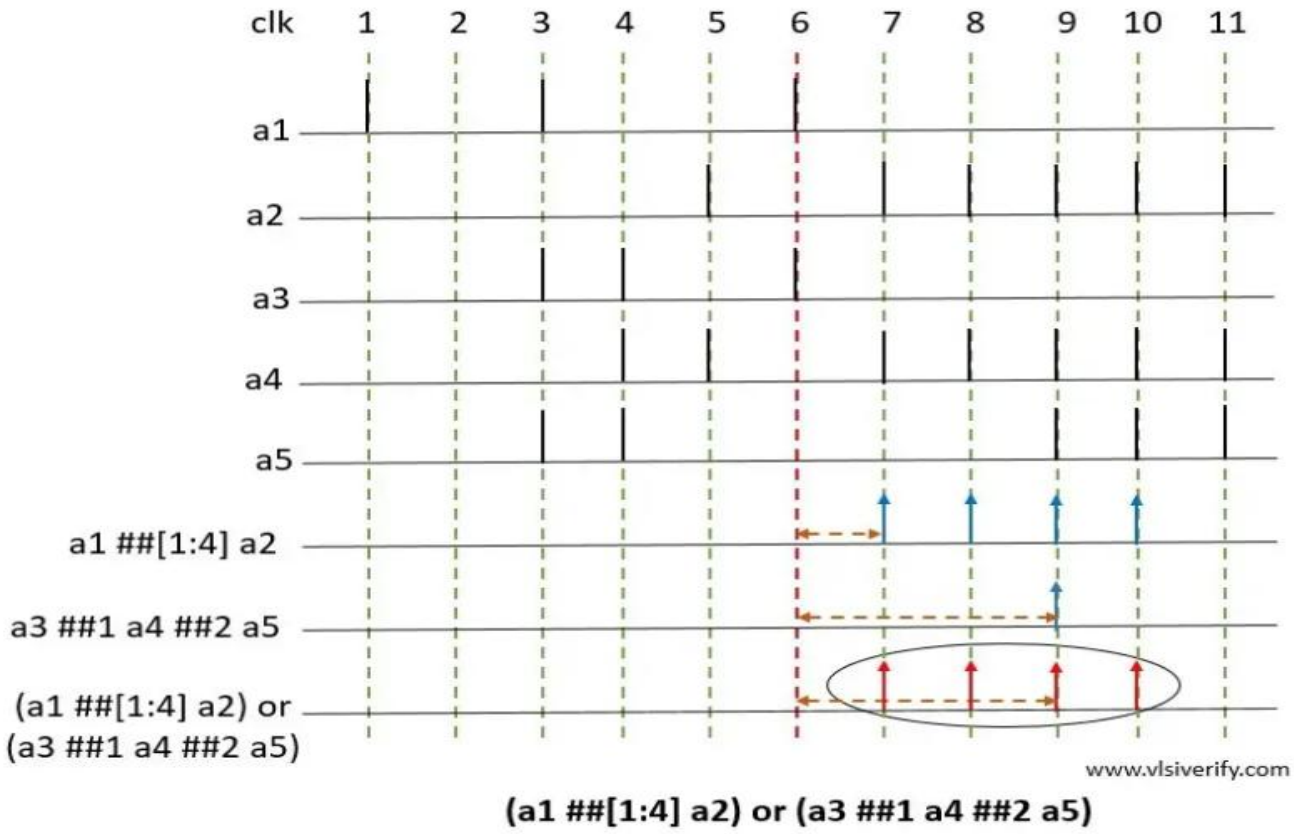
a3 ##1 a4 ##2 a5 (operand B): The operand a3 evaluates to true followed by a4 one clock tick later, followed by a5 two clock ticks later i.e. at clock tick 9.

(a1 ##1 a2) and (a3 ##1 a4 ##2 a5): The resultant sequence is said to be matched when both of the above operand sequence matches. The end times of matches for individual operand sequences A and B are clock ticks 7 and 9 respectively.

The resultant sequence match should follow the conditions like individual operand sequences should start at the same clock tick (clock tick 6) and either of two operand sequences should match (clock ticks 7 and 9).

Example for OR of two sequences including a time range

```
(a1 ##[1:4] a2) or (a3 ##1 a4 ##2 a5)
```



(a1 ##[1:4] a2) or (a3 ##1 a4 ##2 a5)

Explanation

Let's consider a1 ##[1:4] a2 as operand A and a3 ##1 a4 ##2 a5 as operand B.operand B

a1 ##[1:4] a2: The first operand a1 evaluates to true followed by a2 which evaluates to true for 1,2,3, or 4 clock ticks later. So, sequence matches at clock ticks 7,8,9, and 10 respectively.

a3 ##1 a4 ##2 a5: The operand a3 evaluates to true followed by a4 one clock tick later, followed by a5 two clock ticks later i.e. at clock tick 9.

(a1 ##1 a2) and (a3 ##1 a4 ##2 a5): The resultant sequence is said to be matched when both of the above operand sequence matches. The end times of matches for individual operand sequences are clock ticks 10 and 9.

The resultant sequence match should follow the conditions like individual operand sequences should start at the same clock tick (clock tick 6) and either of two operand sequences should match (clock ticks 7,8,9 and 10).

The “intersect” construct

Intersection operator (length size match)

The binary *intersect* operator is used when two sequence operands are expected to match, and the end time of both operand sequences must be the same.

Syntax:

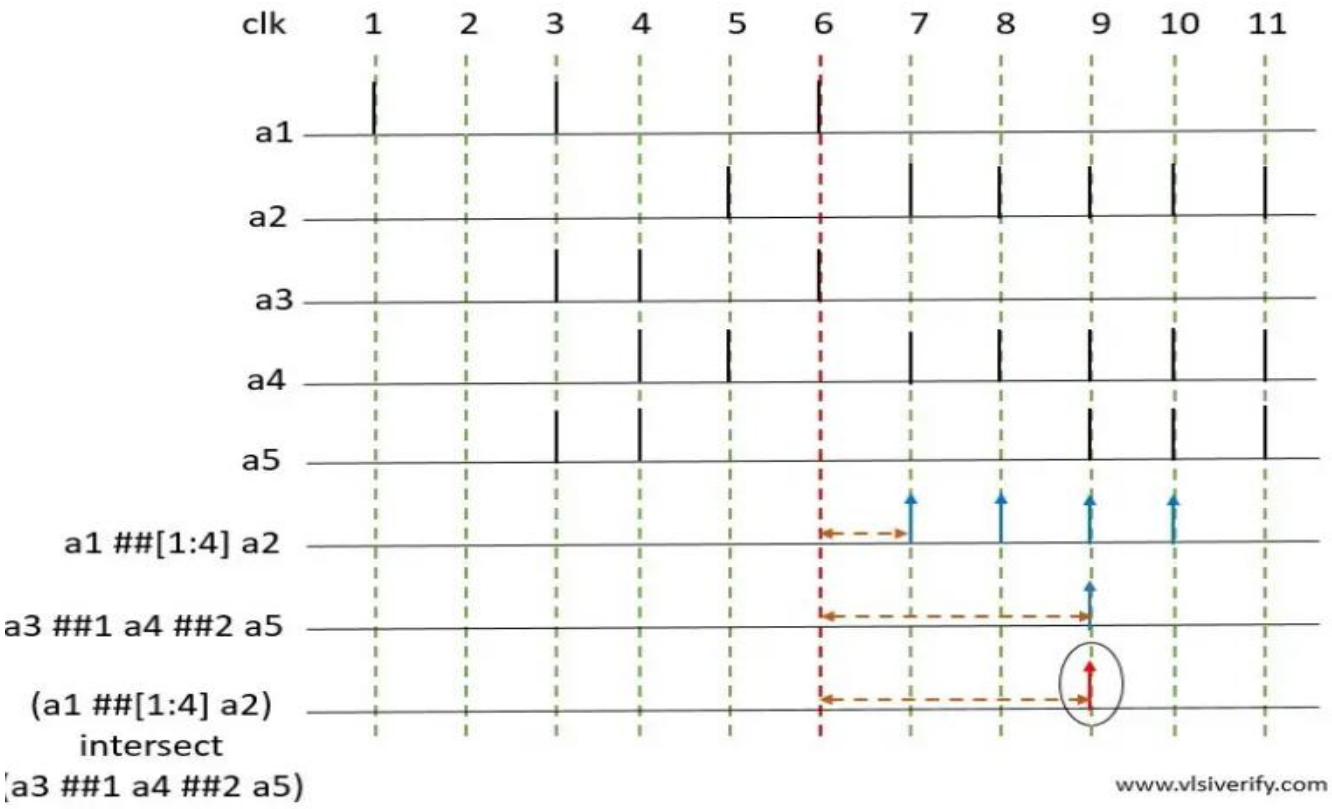
```
<seq_exp> intersect <seq_exp>
```

The requirement for *intersect* operation match

1. Both operand sequence lengths must be the same.
2. Both operands must match. In case one operand matches, it waits for another operand to match.
3. The end time of the resultant or composite sequence is equal to the end time of the matched operand sequences.

Example for Intersection of two sequences

```
(a1 ##[1:4] a2) intersect (a3 ##1 a4 ##2 a5)
```



(a1 ##[1:4] a2) intersect (a3 ##1 a4 ##2 a5)

Explanation

a1 ##[1:4] a2: The first operand a1 evaluates to true followed by a2 which evaluates to true for 1,2,3, or 4 clock ticks later. So, sequence matches at clock ticks 7,8,9, and 10 respectively.

a3 ##1 a4 ##2 a5: The operand a3 evaluates to true followed by a4 one clock tick later, followed by a5 two clock ticks later i.e. at clock tick 9.

(a1 ##1 a2) intersect (a3 ##1 a4 ##2 a5): The resultant sequence is matched at clock tick 9 as both of individual operand sequence has matched.

The “firstmatch” constrcutor

What is `first_match`?

👉 `first_match` keeps only the earliest (first) matching sequence

👉 All later matches are ignored

Used when multiple matches are possible, but you want only the first one.

◇ Code from the image (simplified)

sequence s1;

```
  @(posedge clk) a ##[1:3] b;
endsequence

sequence s2;
  @(posedge clk) c ##[2:3] d;
endsequence

property pr;
  @(posedge clk) first_match(s1) |-> s2;
endproperty
assert property (pr);
```

🔹 Simple explanation line-by-line

1.0 Sequence `s1`

a ##[1:3] b

✓ `a` goes high

✓ `b` must go high 1 to 3 cycles later

2.0 Sequence `s2`

c ##[2:3] d

✓ `c` goes high

✓ `d` must go high 2 to 3 cycles later

3.0 Property with `first_match`

first_match(s1) |-> s2;

✓ Detect only the first occurrence of `s1`

✓ When that first `s1` happens → `s2` must follow

🔹 Why `first_match` is needed?

Without `first_match`:

- Every possible `a→b` match starts a new check
- Can cause multiple overlapping checks

With `first_match`:

- ✓ Only earliest valid `a→b` is used
- ✓ Clean, predictable behavior

🔹 Simple timing example

Suppose waveform:

a: ↑ ↑

b: ↑ ↑

- Two possible `a→b` matches
- `first_match(s1)` picks the first one only
- Assertion checks `s2` once

◇ Easy memory line

- `first_match` = use the first valid sequence, ignore the rest

◇ One-line exam answer

- `first_match` ensures that only the earliest matching instance of a sequence is used for property evaluation and all subsequent matches are discarded.

first_match Operator (remain discard match)

The first_match operator matches only for the first match out of all possible multiple matches.

Syntax:

```
first_match(<seq_exp>)
```

The evaluation attempt for seq_expression happens on the same clock cycle when the first_match operator is evaluated.

The first_match(<seq_exp>) produces no match when <seq_exp> produces no match The first_match(<seq_exp>) produces a match when the earliest match happens for <seq_exp> for the same clock tick.

first_match operator example

```
sequence seq;

  first_match(req1 ## [1:4] req2);

endsequence
```

This is equivalent to

```
req1 ##1 req2

req1 ##2 req2

req1 ##3 req2

req1 ##4 req2
```

Out of the above 4 expressions, whichever matches first is said to be matched for the sequence seq.

```
module assertion_example;

  bit clk, req1, req2;


  always #2 clk = ~clk;


  sequence seq;

    first_match(req1 ##[1:4] req2);

    //req1 ##[1:4] req2;

  endsequence


  property prop;
```

```
    @(posedge clk) seq;
  endproperty

  label_a: assert property(prop) $info("passed at %0t",$time);

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars;

    req1 = 0;
    req2 = 0;

    #2 req1 = 1;
    #4 req2 = 1;
    #4 req1 = 0;
    #4 req2 = 0;
    #15 req1 = 0;
    #15 req2 = 0;

    #20; $finish;
  end
endmodule
```

Output:

```
"testbench.sv", 17: assertion_example.label_a: started at 2ns failed at 2ns
       Offending 'req1'
Info: "testbench.sv", 17: assertion_example.label_a: at time 10 ns
passed at 10
```

```
"testbench.sv", 17: assertion_example.label_a: started at 14ns failed at 14ns
       Offending 'req1'
Info: "testbench.sv", 17: assertion_example.label_a: at time 14 ns
passed at 14
"testbench.sv", 17: assertion_example.label_a: started at 18ns failed at 18ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 22ns failed at 22ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 26ns failed at 26ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 30ns failed at 30ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 34ns failed at 34ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 38ns failed at 38ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 42ns failed at 42ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 46ns failed at 46ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 50ns failed at 50ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 54ns failed at 54ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 58ns failed at 58ns
       Offending 'req1'
"testbench.sv", 17: assertion_example.label_a: started at 62ns failed at 62ns
```

```
       Offending 'req1'
$finish called from file "testbench.sv", line 33.
```

The “throughout” constrctors

throughout Operator (check a1 true then do remaining work otherwise not)

The throughout operator is used under circumstances where the occurrence of certain values is prohibited while processing a transaction.

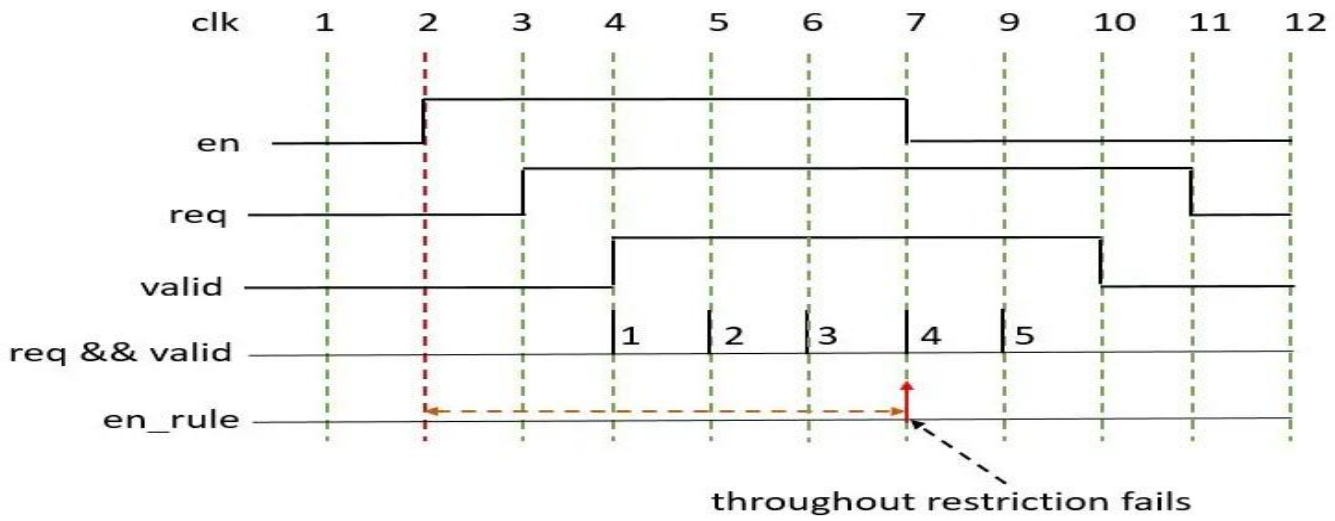
The construct exp throughout seq is an abbreviation for

```
exp [*0:$] intersect seq;
```

The “exp throughout seq” matches along with finite consecutive clock ticks when seq matches along with the interval and exp also evaluates to true for each clock tick of the interval.

throughout operator Example

```
sequence en_rule;
  @(posedge clk) $rose (en) ##0
  (en) throughout (##2 (req && valid)) [*5]);
endsequence
```



Explanation

The signal en is low at clock tick 1 and high at clock tick 2, $rose(en) is true at clock tick 2.

To complete match of en_rule, the en signal is required to be high throughout a match of the subsequence (##1 (req && valid)) [*5]). In the above diagram, the en signal goes low before consecutive 5 clock cycles when req and valid signals are high then it is a violation of the en_rule.

What is `throughout`?

👉 `throughout` means: a condition must stay TRUE for the entire duration of a sequence

📌 Think of it as:

- "While this sequence is happening, this signal must always be valid."

🔹 Code from the image (simplified)

```
property pr;
  @(posedge clk)
  $fell(start) |-> (start) throughout
                (##1 (a && !b) ##1 (c[*3]) ##1 (a && b));
endproperty
assert property (pr);
```

◇ Simple line-by-line explanation

#1.0 $fell(start)

✓ Triggers when `start` goes from 1 → 0

#2.0 Sequence after `|->`

##1 (a && !b) ##1 (c[*3]) ##1 (a && b)

This means:

- After 1 cycle → `a=1`, `b=0`
- Then `c` must be high for 3 cycles
- Then `a=1`, `b=1`

#3.0 start throughout (sequence)

✓ `start` must remain HIGH during the entire sequence

✓ If `start` goes LOW at any point → ❌ assertion FAIL

◇ Easy English meaning

- When `start` falls, then while the sequence is executing, `start` must stay HIGH the whole time.

🔹 Simple timing example

✓ PASS case

start: ————————————————

a: ————————

b: ————┐——

c: ■■■

✓ `start` stays HIGH during full sequence

✓ Assertion PASSES

❌ FAIL case

start: ————————┐____

↑ start drops early

❌ `start` drops before sequence finishes

❌ Assertion FAILS

◇ One-line exam definition

- `throughout` ensures that a condition remains true for the complete duration of a sequence.

🔹 Memory trick 🧠

throughout = “always during”

Here are only 3 simple differences

| Point | `throughout` | `first_match`

| Purpose | Ensures a signal stays true for the entire sequence | Keeps only the first matching sequence |

| Checks | Condition validity during sequence | Which match to choose

| Failure reason | Fails if signal goes false any time during sequence | Other matches are ignored, not failures |

In short:

👉 `throughout` = “stay true during whole time”

👉 `first_match` = “use the earliest match only”

The “within” constrctors

What `within` means

👉 `within` checks that one sequence happens completely inside another sequence

(start after the outer sequence starts and finish before it ends).

Code from image (simplified)

```
sequence my_seq1;
  read[=8] within (rdy ##[9:15] done);
endsequence
```

Simple explanation

- `rdy` → start of the outer window
- `done` → end of the outer window (after 9–15 cycles)

- `read[=8]` → `read` must occur exactly 8 times
- All 8 reads must happen between `rdy` and `done`

✓ If reads happen outside → FAIL

✓ If reads are fewer/more than 8 → FAIL

Easy real-life example

🚌 Bus analogy

- `rdy` = bus starts
- `done` = bus stops
- `read[=8]` = 8 passengers must board

👉 All 8 passengers must board while the bus is moving, not before or after.

In one line

`within` = "This sequence must fully occur inside another sequence's time window."

within Operator

The within operator is used to express the containment of a sequence within another sequence.

Syntax:

```
<seq_exp> within <seq_exp>
```

The construct seq1 within seq2 is an abbreviation for

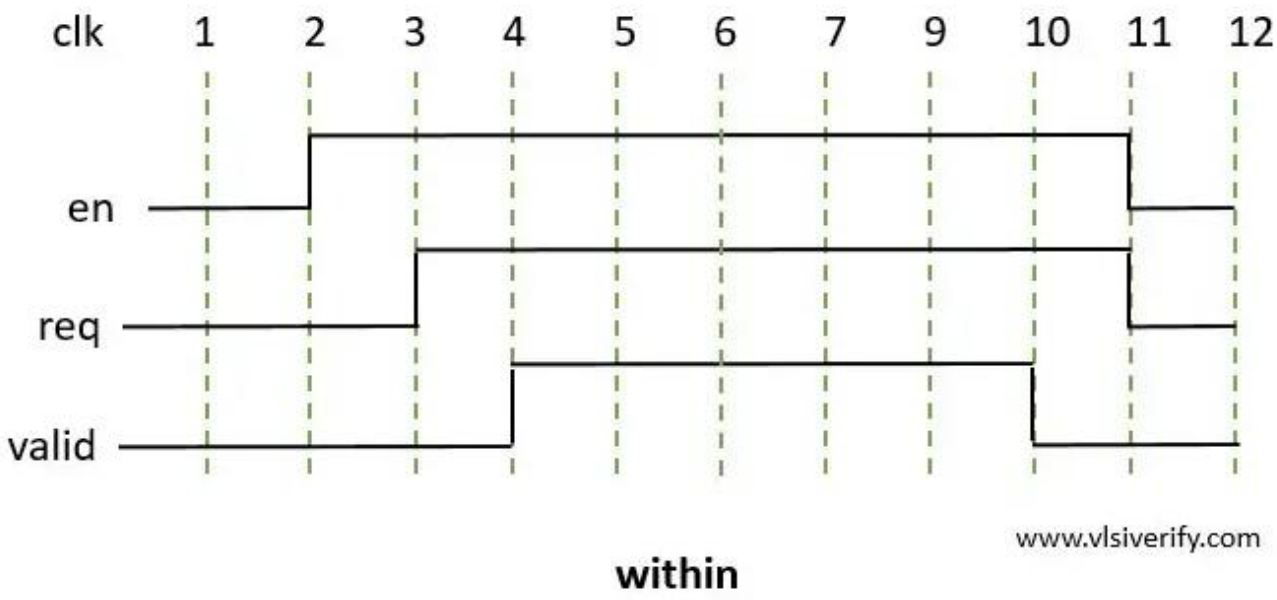
```
[1[*0:$] ##1 seq1 ##1 q[*0:$]] intersect seq2
```

The "seq1 within seq2" matches for a finite interval of consecutive clock ticks if seq2 matches along with the complete interval and seq1 matches for sub-interval of consecutive clock tick and must satisfy the below conditions

1. The start point of the match of seq1 must be greater than or equal to the start point of the match of seq2.
2. The endpoint of the match of seq1 must be less than or equal to the endpoint of the match of seq2.

within operator example

```
valid [*5] within (($rose req) ##1 req[*6])
```



Explanation

When valid is low consecutively for 5 cycles within req goes high and followed by 1 clock cycle req signal stays high for 6 cycles. The above sequence matches if req remains set for 6 clock cycles and valid is set for 5 consecutive clock cycles within the same window.

not Operator in an assertion

The not operator is used to express the non-occurrence of the sequence.

Syntax:

```
not (<seq_exp>);
```

not operator example

```
sequence seq;

  req1 ## valid;

endsequence


property prop;

  @(posedge clk) !en |-> (not (seq));

endproperty
```

```
assert property prop;
```

The “disable iff” constrctors

What is `disable iff`?

👉 Temporarily turns OFF an assertion when a condition is TRUE

👉 Commonly used with reset

🔹 **Basic syntax**

assert property (@(posedge clk) disable iff (reset) <property>);

🔹 Simple example

Requirement

✓ Check that `req` is followed by `ack`

❌ But do NOT check during reset

assert property (
  @(posedge clk)
  disable iff (rst)
  req |=> ack
);

🔹 Simple explanation

On every clock edge:

- If `rst = 1` → assertion is disabled
- If `rst = 0` → assertion is checked
- If `req` happens:
- `ack` must happen next cycle

🔹 Waveform idea

clk : ↑ ↑ ↑ ↑

rst : 1 1 0 0 → disabled first, enabled later

req : 0 1 1 0

ack : 0 0 0 1 ✓

🔹 Why `disable iff` is important?

✓ Avoids false failures during reset

✓ Skips invalid cycles

✓ Cleaner assertions

🔹 Easy memory line

- `disable iff` = "Don't check when this condition is true"

☑ One-line exam answer

- `disable iff` disables an assertion whenever the specified condition becomes true, commonly used to mask reset cycles.

Built-in system functions

Connecting SVA to the design

Binding with Assertions

The assertion can be written as a part of the design code. A designer or verification engineer can plan for adding an assertion. It is not a flexible and recommended way to have assertions in the design files since designers do want to change their code by the verification team. SystemVerilog provides flexibility to write assertions in separate files in the testbench and then bind the same design file.

The 'bind' directive allows binding assertions with

1. Design module
2. Interface

Binding is possible with

1. A specific instance of a module or interface

2. All instances of a module or interface

Syntax:

To bind with specific instance

```
bind <dut_specific_instance_path> <assertion_module> <instance>
```

To bind with all instances

```
bind <dut_module> <assertion_module> <instance>
```

bind example

assertion.sv

```
module assertion_dff (
  input clk, rst_n, d, q
);

  sequence seq1;
    d ##1 q;
  endsequence

  property prop;
    @(posedge clk) disable iff(rst_n)
    d |=> seq1;
  endproperty

  dff_assert: assert property (prop) else $display("Assertion failed at time = %0t", $time);
endmodule
```

```
`include "assertion.sv"
```

```
module tb;
  reg clk, rst_n;
  reg d;
  wire q;

  D_flipflop dff1(clk, rst_n, d, q);
  D_flipflop dff2(clk, rst_n, d, q);

  // To bind with all instances of DUT
  bind D_flipflop assertion_dff all_inst(clk, rst_n, d, q);

  // To bind with single instance of DUT
  bind tb.dff2 assertion_dff single_inst(clk, rst_n, d, q);

  always #2 clk = ~clk;
  initial begin
    clk = 0; rst_n = 0;
    d = 0;
    #3 rst_n = 1;
    repeat(6) begin
      d = $urandom_range(0, 1);
      #3 rst_n = $urandom_range(0, 1);
    end
    $finish;
  end
  initial begin
    $monitor("At time = %0t: rst_n = %b, d = %b, q = %b", $time, rst_n, d, q);
```

```
  end

endmodule
```

Output:

```
At time = 0: rst_n = 0, d = 0, q = x

At time = 2: rst_n = 0, d = 0, q = 0

At time = 3: rst_n = 1, d = 1, q = 0

At time = 6: rst_n = 0, d = 1, q = 0

At time = 9: rst_n = 1, d = 0, q = 0

At time = 15: rst_n = 0, d = 0, q = 0

At time = 18: rst_n = 1, d = 1, q = 1
```

SVA for functional coverage

What is SVA functional coverage?

👉 Assertion-based coverage tells you whether a specific behavior ever happened, not whether it was correct or not.

- Assertion → checks correctness
- Cover property → checks occurrence (coverage)

🔹 **Basic syntax**

cover property (<property>);

✓ No PASS/FAIL

✓ Only records “this happened”

◇ Simple example 1: Event occurred

Requirement

✓ Did `req` ever happen?

cover property (@(posedge clk) req);

📌 Coverage hit when `req = 1`

🔹 Simple example 2: Sequence coverage

Requirement

✓ Did `req` followed by `ack` happen?

cover property (@(posedge clk) req ##1 ack);

📌 Coverage hit when `ack` comes 1 cycle after `req`

🔹 Simple example 3: Timing window coverage

Requirement

✓ Did `ack` come within 2–4 cycles after `req`?

cover property (@(posedge clk) req ##[2:4] ack);

🔹 Difference: Assert vs Cover

Feature	assert property	cover property
Checks correctness	✓	❌
Reports failure	✓	❌
Tracks occurrence	❌	✓

🔹 Where SVA coverage is useful?

✓ Protocol handshakes

✓ Rare corner cases

✓ Timing scenarios

✓ FSM transitions

🔹 **Easy memory line**

- Assert = “Is it correct?”
- Cover = “Did it ever happen?”

☑ One-line exam answer

SVA functional coverage uses `cover property` to record whether a specified sequence or behavior occurred during simulation.

Session-18: -

- Virtual interface in OOPs
- Mailbox in OOPs
- System Verilog Environment
- Packet
- Generator
- Driver
- Monitor
- Scoreboared

Overview of RTL Verification

##1. What is RTL Verification?

RTL Verification is the process of checking whether the RTL design (written in Verilog/SystemVerilog) works according to the specification.

Goal 👉 Find bugs before silicon fabrication

2. Why System Verilog for Verification?

System Verilog extends Verilog with powerful verification features:

- Object-Oriented Programming (classes)
- Randomization & constraints
- Assertions (SVA)
- Functional coverage
- Interfaces & modports
- Mailbox, semaphore, events (IPC)

👉 Makes testbench smarter, reusable, and scalable.

3. Typical RTL Verification Flow

Specification → RTL Design → Testbench → Simulation → Debug → Coverage

4. Key Components of RTL Verification

1 DUT (Design Under Test)

- The RTL module being verified

```
module adder(input logic a,b, output logic sum);
  assign sum = a ^ b;
endmodule
```

2 Testbench (TB)

Non-synthesizable code used to verify DUT

Contains:

- Stimulus generation
- DUT connection
- Checking logic

3 Interface

Groups DUT signals and avoids long port lists

```
interface adder_if;
  logic a, b, sum;
endinterface
```

4 Driver

Drives inputs to DUT

```
dut_if.a = 1;
dut_if.b = 0;
```

5 Monitor

Observes DUT signals (no driving)

$display("a=%0b b=%0b sum=%0b", a, b, sum);

6 Scoreboard

Compares expected vs actual output

if (exp_sum != act_sum)

  $error("Mismatch!");

5. Randomization & Constraints

Used to create multiple test scenarios automatically

class trans;

  rand bit a, b;

endclass

6. Assertions (SVA)

Checks protocol and timing behavior

assert property (@(posedge clk) a |-> b);

👉 Finds bugs immediately when violation occurs

7. Coverage

🔹 Code Coverage

- Line
- Branch
- Toggle
- FSM

👉 **“Have we exercised RTL code?”**

🔸 Functional Coverage

Checks specification scenarios

```
covergroup cg;
  coverpoint a;
  coverpoint b;
endgroup
```

👉 **“Have we tested all required cases?”**

8. Debugging Tools

- VCS for simulation
- Verdi for waveform & debug
- Logs, assertions, coverage reports

9. Verification Methodologies

- Directed Testing – fixed inputs
- Constrained Random Verification (CRV) – random + constraints
- UVM – industry-standard methodology

10. Final Goal of RTL Verification

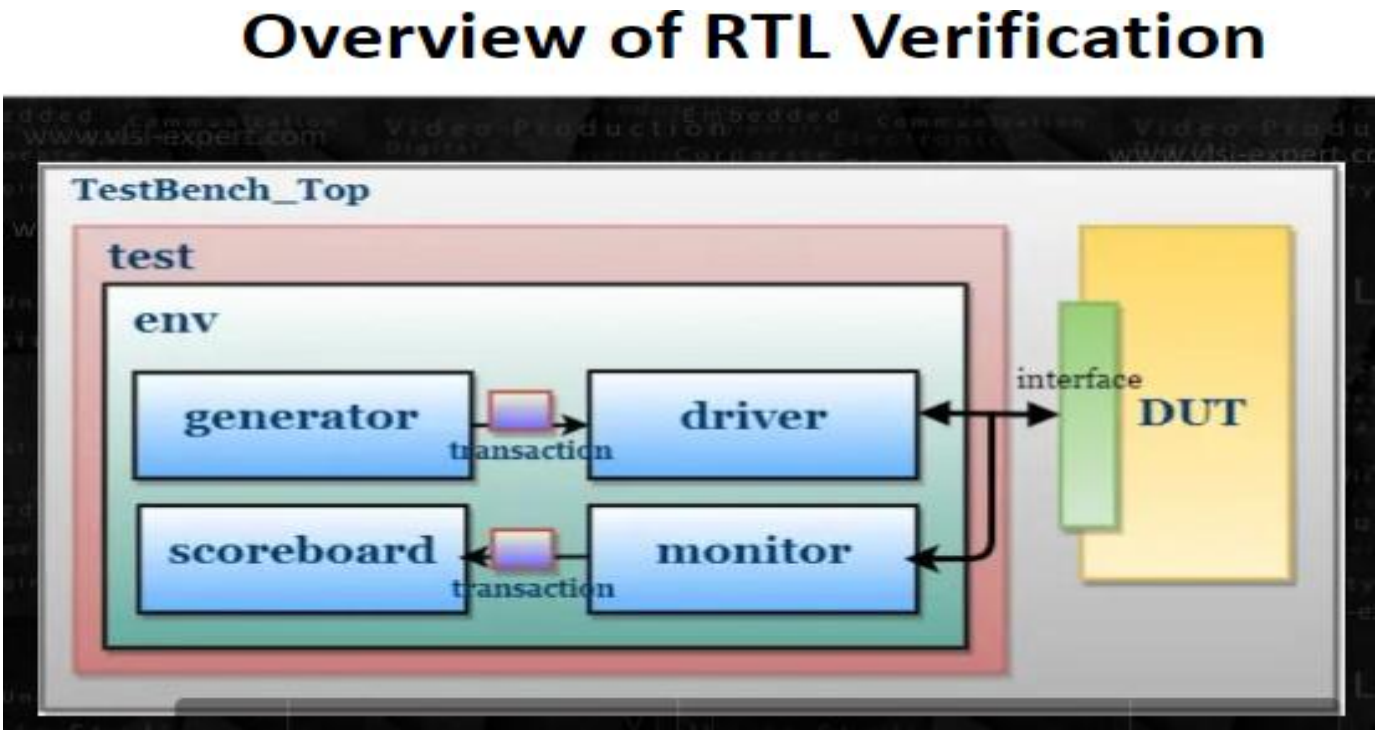
✓ Bug-free RTL

✓ High code & functional coverage

✓ Confidence before tape-out

💡 One-Line Summary (Interview Ready):

RTL Verification ensures that the RTL design meets its specification using SystemVerilog-based testbenches, assertions, and coverage before fabrication.



Virtual interface

1 What is an Interface?

An interface bundles signals and related logic together.

```
interface bus_if;
  logic clk;
  logic rst;
  logic [7:0] data;
endinterface
```

👉 Used to connect testbench ↔ DUT

2 Why Virtual Interface?

Classes cannot directly connect to modules/interfaces using ports.

So we use a virtual interface:

- It is a handle (pointer) that allows a class to access a real interface.

3 Simple Definition (Interview Ready)

A virtual interface is a reference to a physical interface that allows class-based testbench components to drive and monitor DUT signals.

4 Without Virtual Interface (Problem)

❌ Classes cannot access interface signals directly.

```
class driver;
  // Cannot do this
  bus_if bif;   // ERROR
endclass
```

5 With Virtual Interface (Solution)

Step-1: Define Interface

```
interface bus_if;
  logic clk;
  logic [7:0] data;
endinterface
```

Step-2: Declare Virtual Interface in Class

```
class driver;
  virtual bus_if vif;

  task drive();
    vif.data = 8'hAA;
  endtask
endclass
```

Step-3: Connect Interface in Top Module

```
module tb;
  bus_if bif();
  driver d;

  initial begin
    d = new();
    d.vif = bif;   // binding virtual interface
    d.drive();
  end
endmodule
```

6 How It Works (In Simple Words)

- bus_if bif` → real interface (actual wires)
- virtual bus_if vif` → reference to those wires
- Class uses `vif` to access signals

👉 Like a remote control for the interface

7 Why Virtual Interface is Important?

✓ Enables OOP-based testbenches

✓ Required in UVM drivers & monitors

✓ Makes testbench reusable and scalable

8 Real-Life Analogy

- Interface = TV
- Virtual Interface = Remote control
- Driver class = Person using remote

9 Interview One-Liners

Why virtual interface?

➜ Classes can’t have ports, so they use virtual interfaces.

Where used?

➜ Driver, Monitor, UVM components.

🔑 Key Point to Remember

- Module → Interface (physical)
- Class → Virtual Interface (reference)

Virtual interface

```
module and1(input a, b, output sum, carry);
assign {carry, sum} = a + b;
endmodule

interface intf();
        logic a, b, sum, carry;
endinterface

class driver;
        virtual intf vif;

        function new(virtual intf inf_arg);
                this.vif = inf_arg;
        endfunction

        task run;
```

```
            vif.a <= 0; vif.b <= 0;
            $display("vif.sum = %b, vif.carry = %b", vif.sum, vif.carry);

            #5;
            vif.a <= 0; vif.b <= 1;
            $display("vif.sum = %b, vif.carry = %b", vif.sum, vif.carry);

            #5;
            vif.a <= 1; vif.b <= 0;
        $display("vif.sum = %b, vif.carry = %b", vif.sum, vif.carry);

            #5;
        vif.a <= 1; vif.b <= 1;
        $display("vif.sum = %b, vif.carry = %b", vif.sum, vif.carry);
    endtask
endclass

module top;
intf inf();

and1 a1(.a(inf.a),.b(inf.b),.sum(inf.sum),.carry(inf.carry));

driver drv;
initial begin
    drv = new(inf);
    drv.run;
end
```

```
initial begin
    $monitor("inf.sum = %b, inf.carry = %b", inf.sum, inf.carry);
end
endmodule
```

Simulation output:

```
vif.sum = x, vif.carry = x
inf.sum = 0, inf.carry = 0
vif.sum = 0, vif.carry = 0
inf.sum = 1, inf.carry = 0
vif.sum = 1, vif.carry = 0
vif.sum = 1, vif.carry = 0
inf.sum = 0, inf.carry = 1
        V C S  S i m u l a t i o n  R e p o r t
Time: 15
```

1.0 SIMPLE & SHORT EXPLANATION (LINE BY LINE IDEA)

module and1`

- This is the DUT
- It adds `a + b`
- Outputs: sum, carry

interface intf`

- Groups signals: `a, b, sum, carry`
- Acts as a connection bridge between DUT and testbench

virtual intf vif`

- Declared inside class
- A reference (handle) to the real interface
- Allows class to access DUT signals

👉 Classes cannot connect directly → virtual interface solves this

function new(virtual intf inf_arg)

- Constructor
- Connects real interface (`inf`) to virtual interface (`vif`)

this.vif = inf_arg; //used to passing the argument

🔹 `task run`

- Drives inputs using `vif`
- Applies different values of `a` and `b`
- Prints `sum` and `carry`

🔹 **`module top`**

Creates:

- Interface instance `inf`
- DUT instance
- Driver object

drv = new(inf); // bind interface to virtual interface

drv.run; // start driving

🔹 `$monitor`

Continuously prints output whenever it changes

3 VERY SIMPLE EXAMPLE (EASY TO REMEMBER)

Think like this 👇

- Interface (`intf`) = Electrical wires
- Virtual interface (`vif`) = Remote control
- Driver class = Person using remote
- DUT = Device (adder)

- Driver does not touch DUT directly
- Driver → `vif` → `interface` → DUT

Example Operation

| a | b | sum | carry |

| - | - | --- | ----- |

| 0 | 0 | 0 | 0 |

| 0 | 1 | 1 | 0 |

| 1 | 0 | 1 | 0 |

| 1 | 1 | 0 | 1 |

- The driver drives `a` and `b` through `vif`, and DUT produces `sum` and `carry`.

🔑 ONE-LINE INTERVIEW ANSWER

A virtual interface allows class-based components to access and control DUT signals through an interface.

……

function new(virtual intf inf_arg)`

Line:

function new(virtual intf inf_arg);

1.0 function

- Used to define a function
- Executes in zero simulation time
- Returns a value (constructor returns object handle)

👉 Here, it defines a constructor

2.0 new

- Special constructor name in System Verilog
- Automatically called when object is created

drv = new(inf);

👉 Used to initialize class variables

3.0 ( ) — Parentheses

- Used to pass arguments to the constructor

4.0 virtual

- Keyword meaning reference / handle
- Used because:
- Classes cannot connect directly to interfaces
- Allows class to access real interface signals

👉 Think: pointer to interface

5.0 intf`

- Name of the interface type
- Tells compiler:
- This virtual handle refers to an `intf` interface

6.0 inf_arg

- Argument name
- Holds the actual interface instance passed from `top`

Example:

drv = new(inf); // inf → inf_arg

What This Line Means (In One Sentence)

- This constructor receives a reference to an interface and stores it so the class can use DUT signals.

How It Connects (Flow)

top module

|

| inf (real interface)

↓

new(inf)

|

| inf_arg → vif

↓

driver class accesses signals

Why This Line Is IMPORTANT?

✓ Connects module world → class world

✓ Enables driver to drive DUT signals

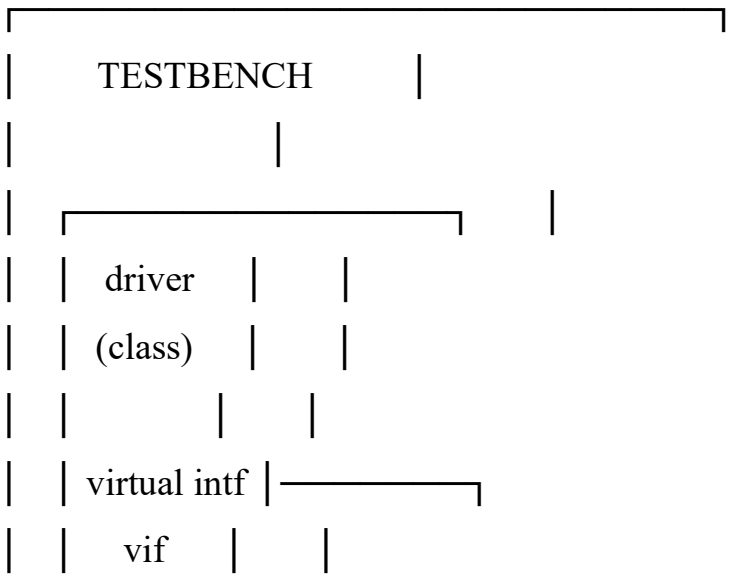
✓ Mandatory in UVM drivers/monitors

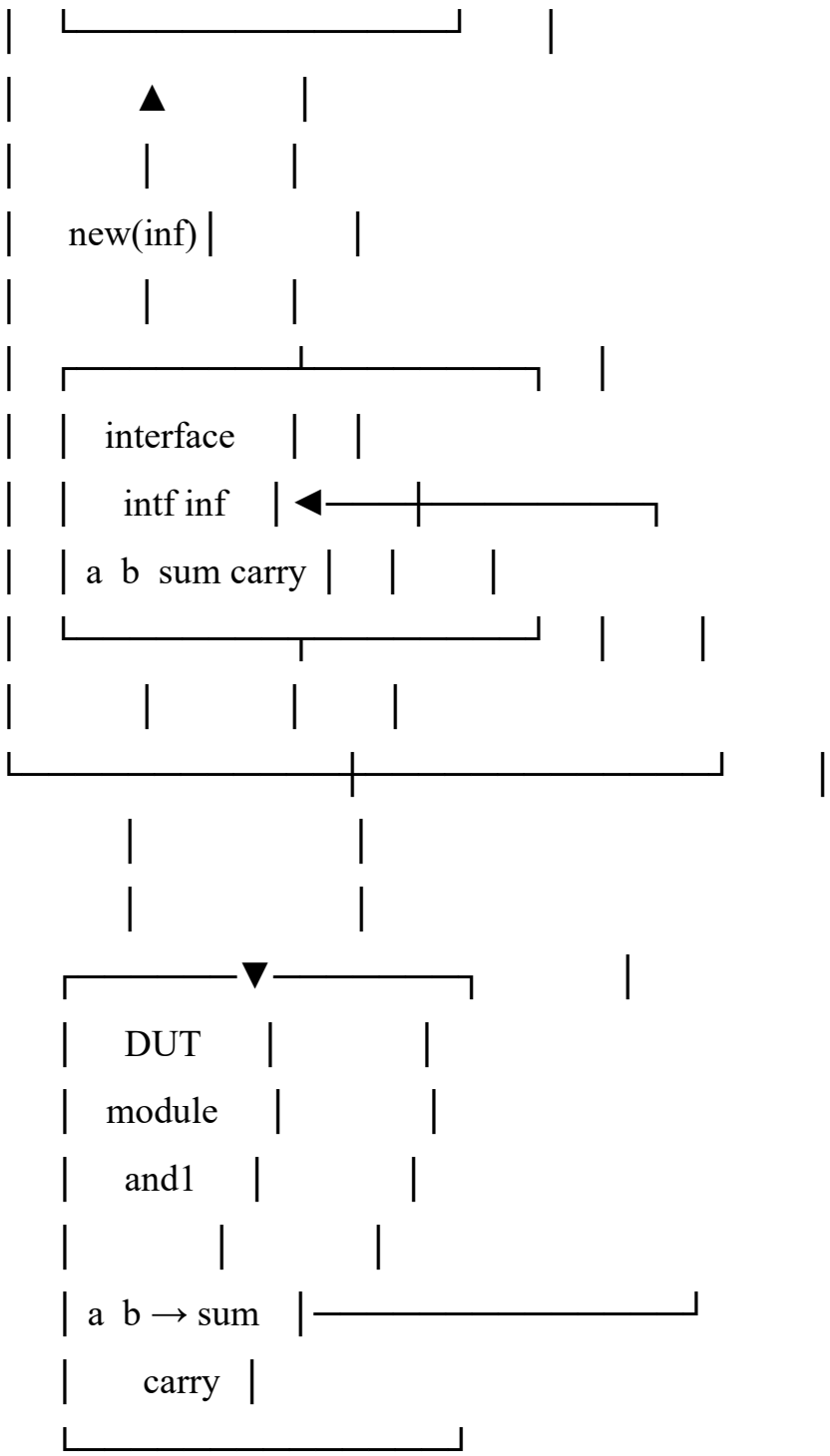
Interview One-Liner

> `function new(virtual intf inf_arg)` is a constructor that accepts a virtual interface handle to connect class-based components with the DUT.

connection diagram showing how DUT and Testbench are connected in your code using interface + virtual interface

🔷 CONNECTION DIAGRAM (TEXT / BLOCK DIAGRAM)





🔷 SIGNAL FLOW (VERY SIMPLE)

Driver (class)

↓ via virtual interface (vif)

Interface (intf)

↓ physical wires

DUT (and1)

↑

Interface

↑

Monitor / $display / $monitor

🔷 **STEP-BY-STEP CONNECTION EXPLANATION**

1.0 DUT (RTL)

module and1(input a, b, output sum, carry);

- Actual hardware logic
- Does `a + b`

2.0 Interface (Bridge)

interface intf;

logic a, b, sum, carry;

endinterface

- Holds signals
- Physically connects TB ↔ DUT

3.0 DUT Connected to Interface

and1 a1(.a(inf.a),.b(inf.b),.sum(inf.sum), .carry(inf.carry));

👉 DUT pins are wired to interface signals

4.0 Driver Uses Virtual Interface

class driver;

virtual intf vif;

👉 `vif` points to `inf`

5.0 Binding Happens Here

drv = new(inf);

- `inf` → real interface
- Passed to constructor
- Stored in `vif`

6.0 Driver Drives DUT Indirectly

vif.a <= 1;

vif.b <= 1;

👉 Driver → Interface → DUT

🔷 **REAL-LIFE ANALOGY (EASY)**

Item	Meaning
DUT	TV
Interface	TV wires
Virtual interface	Remote control
Driver	Person using remote

Person never touches TV wires directly

🔑 INTERVIEW ONE-LINER

> The driver class accesses DUT signals through a virtual interface that references the real interface connected to the DUT.

Mailbox in Verification

```
class packet;
  int data;
  int addr;
```

```
endclass

class producer;
  packet pkt_send, pkt2, pkt3;
  mailbox mbx_send;

  function new(mailbox mbx);
    this.mbx_send = mbx;
  endfunction

  task send();
    pkt_send = new();
    pkt2 = new();
    pkt3 = new();

    pkt_send.data = 100;
    pkt_send.addr = 99;

    pkt2.data = 10;
    pkt2.addr = 9;

    pkt3.data = 20;
    pkt3.addr = 19;

    mbx_send.put(pkt_send);
    mbx_send.put(pkt2);
    mbx_send.put(pkt3);
```

```
  endtask
endclass


class consumer;
  packet pkt_recv;
  packet pkt2;
  packet pkt3;
  mailbox mbx_recv;

  function new(mailbox mbx);
    this.mbx_recv = mbx;
  endfunction

  task recv();
    mbx_recv.get(pkt_recv);
    mbx_recv.get(pkt2);
    mbx_recv.get(pkt3);

    $display("pkt_recv = %p", pkt_recv);
    $display("pkt2 = %p", pkt2);
    $display("pkt3 = %p", pkt3);
  endtask
endclass
```

2.0 SIMPLE & SHORT EXPLANATION (BLOCK BY BLOCK)

🔹 `class packet`

class packet;

int data;

int addr;

endclass

- A data container
- Holds `data` and `address`
- Sent from producer → consumer

🔹 `class producer`

Purpose: Creates packets and sends them

Key parts:

- `mailbox mbx_send` → communication channel
- `new(mailbox mbx)` → connects mailbox
- `send()` task:

- Creates 3 packets
- Assigns values
- Sends them using `put()`

mbx_send.put(pkt_send);

👉 Producer puts data into mailbox

🔹 `class consumer`

Purpose: Receives packets

Key parts:

- `mailbox mbx_recv` → same mailbox
- `recv()` task:
- Receives packets using `get()`
- Prints packet contents

mbx_recv.get(pkt_recv);

👉 Consumer waits until data arrives

3 HOW MAILBOX WORKS (VERY SIMPLE)

🔑 Definition

> A mailbox is a synchronization mechanism used to safely pass data between processes.

4 FLOW OF THIS CODE (STEP-BY-STEP)

Producer Mailbox Consumer

--- --- ---

pkt_send ——— put ——▶ mbx ——— get ——▶ pkt_recv

pkt2 ——— put ——▶ ——— get ——▶ pkt2

pkt3 ——— put ——▶ ——— get ——▶ pkt3

5 REAL-LIFE EXAMPLE (EASY TO REMEMBER)

📬 Mailbox Analogy

- Producer = Person sending letters
- Mailbox = Postbox
- Consumer = Person receiving letters
- put() = Drop letter
- get() = Take letter (waits if empty)

6 IMPORTANT INTERVIEW POINTS 🔗

- `put()` → blocking if mailbox is full
- `get()` → blocking if mailbox is empty
- Order is FIFO
- Used for inter-process communication (IPC)

☑ ONE-LINE INTERVIEW ANSWER

> A mailbox in System Verilog is used for safe and synchronized communication between producer and consumer processes.

Mailbox in verification

```
module m;
  mailbox mbx;
  producer p;
  consumer c;

  initial begin
    mbx = new(2);
    p = new(mbx);
    c = new(mbx);

    p.send();
    c.recv();

    $display("INSIDE MODULE : pkt_recv = %p", c.pkt_recv);
  end
endmodule
```

2 SIMPLE & SHORT EXPLANATION (LINE BY LINE)

◇ `module m;`

- Top-level test module
- Connects producer and consumer

◇ Declarations

mailbox mbx;

producer p;

consumer c;

- `mbx` → mailbox used for communication
- `p` → producer object
- `c` → consumer object

◇ `mbx = new(2);`

- Creates a mailbox of size 2
- Can store 2 packet objects at a time
- If full → `put()` blocks
- If empty → `get()` blocks

◇ Object creation

p = new(mbx);

c = new(mbx);

- Same mailbox passed to both
- Enables producer ↔ consumer communication

◇ Function calls

p.send();

c.recv();

- `p.send()` → producer puts packets into mailbox
- `c.recv()` → consumer gets packets from mailbox

◇ Display statement

$display("INSIDE MODULE : pkt_recv = %p", c.pkt_recv);

- Prints received packet from inside module
- `%p` prints full object contents

3 HOW EVERYTHING WORKS (FLOW)

Producer (p) ── put() ──▶ Mailbox (mbx) ── get() ──▶ Consumer (c)

- Producer sends packets
- Mailbox temporarily stores them
- Consumer receives them in FIFO order

4 VERY SIMPLE REAL-LIFE EXAMPLE 📦

📪 Postbox Example

- Producer → Person posting letters
- Mailbox → Postbox (capacity = 2 letters)
- Consumer → Person collecting letters

If postbox is:

Full → sender waits

Empty → receiver waits

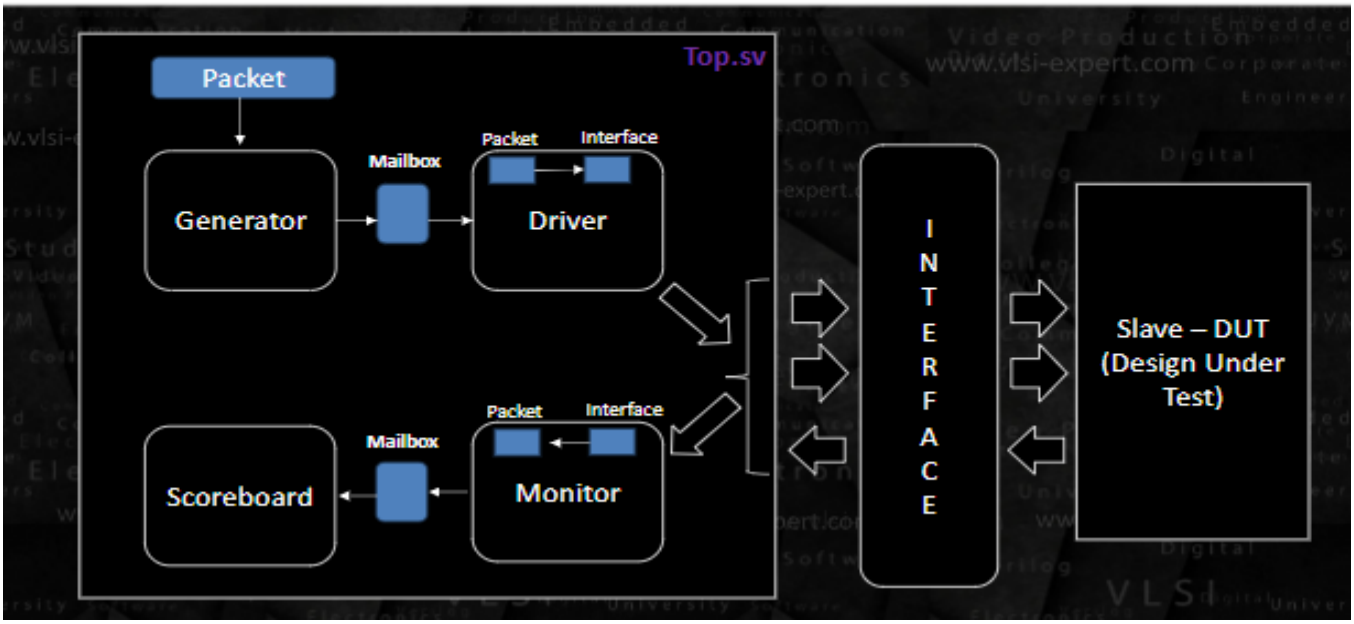
5 IMPORTANT INTERVIEW POINTS 🔑

- `new(2)` → mailbox size = 2
- Mailbox ensures safe synchronization
- Same mailbox must be shared
- Order of transfer is FIFO
- `%p` prints object structure

☑ ONE-LINE SUMMARY

> This module connects producer and consumer using a mailbox to safely exchange packet objects in a synchronized manner.

System Verilog Verification Environment



1.0 WHAT THE IMAGE SHOWS

It shows how testbench components are connected to verify a DUT (Design Under Test).

2. 0 EXPLANATION IN THE SAME ORDER AS IMAGE

◇ **1. Packet**

- A data object
- Contains fields like address, data, control bits
- Used to transfer information between blocks

📦 Example: `{addr=5, data=20}`

◇ 2. Generator

- Creates random or directed packets
- Acts as a stimulus source
- Does NOT talk to DUT directly

➞ Sends packets to mailbox

◇ 3. Mailbox (Generator → Driver)

- Used for safe communication

- Holds packets until driver is ready

📮 Like a buffer between generator and driver

🔹 4. Driver

- Converts packet-level data to pin-level signals
- Drives signals to DUT via interface

➡ Packet → signals (clk, data, valid)

🔸 5. Interface

- Bundle of DUT signals
- Connects testbench ↔ DUT
- Shared by driver and monitor

📌 **Example: `clk, data, valid, ready`**

🔸 6. DUT (Slave – Design Under Test)

- Actual RTL design
- Receives inputs and produces outputs

🧠 **Example: ALU, FIFO, adder**

🔸 7. Monitor

- Passively observes DUT signals
- Converts signals back into packets
- Does NOT drive anything

➡ **Signals → packet**

🔹 8. Mailbox (Monitor → Scoreboard)

- Transfers monitored packets
- Keeps data safe and ordered

◇ 9. Scoreboard

- Compares expected vs actual output
- Declares PASS / FAIL

✓ **Verification decision happens here**

3 COMPLETE VERIFICATION FLOW (ONE LINE)

Generator → Mailbox → Driver → Interface → DUT → Interface → Monitor → Mailbox → Scoreboard

4 WHY THIS IS CALLED VERIFICATION ENVIRONMENT?

Because it:

- Generates inputs
- Applies them to DUT
- Observes outputs
- Checks correctness automatically

🎯 **Goal: Find bugs in RTL**

5 SIMPLE REAL-LIFE EXAMPLE 🚗

🚦 Traffic Light Controller DUT

Generator

- Generates traffic scenarios

(car present / no car)

Driver

- Converts scenario to signals

(`sensor = 1`)

DUT

- Decides signal color (RED/GREEN)

Monitor

- Reads output color

Scoreboard

- Checks:
- Car present → GREEN ☑
- No car → RED ☑

6 VERY SHORT SUMMARY 🔑

Component	Purpose
Packet	Data container
Generator	Creates stimulus
Mailbox	Safe communication
Driver	Drives DUT
Interface	Signal bridge
DUT	Design being tested
Monitor	Observes DUT
Scoreboard	Checks correctness

7 INTERVIEW ONE-LINE ANSWER

> A System Verilog verification environment uses generator, driver, monitor, mailbox, and scoreboard to apply stimulus to the DUT, observe outputs, and verify correctness automatically.

Verification environment (Generator → Driver → Interface → DUT → Monitor → Scoreboard).

◇ Example DUT (2-input Adder)

```
module adder (
  input  logic [3:0] a, b,
  output logic [4:0] sum
);
  assign sum = a + b;
endmodule
```

🔹 Interface

```
interface adder_if;
  logic [3:0] a, b;
  logic [4:0] sum;
endinterface
```

🔹 Packet (Transaction)

```
class packet;
  rand bit [3:0] a, b;
  bit  [4:0] sum;
endclass
```

🔹 Generator

```
class generator;
  mailbox mbx;

  function new(mailbox mbx);
```

```
    this.mbx = mbx;
  endfunction

  task run();
    packet pkt = new();
    pkt.randomize();
    mbx.put(pkt);
  endtask
endclass
```

🔹 **Driver**

```
class driver;
  mailbox mbx;
  virtual adder_if vif;

  function new(mailbox mbx, virtual adder_if vif);
    this.mbx = mbx;
    this.vif = vif;
  endfunction

  task run();
    packet pkt;
    mbx.get(pkt);
    vif.a = pkt.a;
    vif.b = pkt.b;
  endtask
endclass
```

🔹 **Monitor**

```
class monitor;
  mailbox mbx;
  virtual adder_if vif;

  function new(mailbox mbx, virtual adder_if vif);
    this.mbx = mbx;
    this.vif = vif;
  endfunction

  task run();
    packet pkt = new();
    #1;
    pkt.sum = vif.sum;
    mbx.put(pkt);
  endtask
endclass
```

🔹 **Scoreboard**

```
class scoreboard;
  mailbox mbx;

  function new(mailbox mbx);
    this.mbx = mbx;
  endfunction

  task run();
```

```
    packet pkt;
    mbx.get(pkt);
    $display("Observed Sum = %0d", pkt.sum);
  endtask
endclass
```

🔹 Top Testbench

```
module tb;
  adder_if vif();
  adder dut (.a(vif.a), .b(vif.b), .sum(vif.sum));

  mailbox g2d = new();
  mailbox m2s = new();

  generator  gen;
  driver     drv;
  monitor    mon;
  scoreboard sb;

  initial begin
    gen = new(g2d);
    drv = new(g2d, vif);
    mon = new(m2s, vif);
    sb  = new(m2s);

    gen.run();
    drv.run();
```

```
    mon.run();
    sb.run();
  end
endmodule
```

◇ HOW THIS MATCHES THE DIAGRAM

Diagram Block	Code
Packet	`class packet`
Generator	`class generator`
Mailbox	`mailbox g2d, m2s`
Driver	`class driver`
Interface	`adder_if`
DUT	`adder`
Monitor	`class monitor`
Scoreboard	`class scoreboard`

◆ ONE-LINE FLOW

Generator → Mailbox → Driver → Interface → DUT → Interface → Monitor → Mailbox → Scoreboard

◇ VERY SHORT INTERVIEW EXPLANATION

> Generator creates transactions, driver drives DUT through interface, monitor observes DUT output, and scoreboard checks correctness.

◆ 1. Clocked DUT (Registered Adder)

```
module adder (
```

```
  input  logic        clk,
  input  logic [3:0]  a, b,
  output logic [4:0]  sum
);
  always_ff @(posedge clk)
    sum <= a + b;
endmodule
```

✓ Output updates only on clock edge

🔹 **2. Clocked Interface**

```
interface adder_if (input logic clk);
  logic [3:0] a, b;
  logic [4:0] sum;
endinterface
```

✓ Clock shared between driver, monitor, DUT

🔹 **3. Packet (Transaction)**

```
class packet;
  rand bit [3:0] a, b;
  bit  [4:0] sum;
endclass
```

🔹 **4. Generator (No Clock Needed)**

```
class generator;
  mailbox mbx;

  function new(mailbox mbx);
```

```
  this.mbx = mbx;
 endfunction

 task run();
  packet pkt = new();
  pkt.randomize();
  mbx.put(pkt);
 endtask
endclass
```

🔹 5. Clocked Driver

```
class driver;
 mailbox mbx;
 virtual adder_if vif;

 function new(mailbox mbx, virtual adder_if vif);
  this.mbx = mbx;
  this.vif = vif;
 endfunction

 task run();
  packet pkt;
  mbx.get(pkt);
  @(posedge vif.clk);
  vif.a <= pkt.a;
  vif.b <= pkt.b;
 endtask
```

```
endclass
```

✓ Drives inputs only at posedge

🔹 **6. Clocked Monitor**

```
class monitor;
  mailbox mbx;
  virtual adder_if vif;

  function new(mailbox mbx, virtual adder_if vif);
    this.mbx = mbx;
    this.vif = vif;
  endfunction

  task run();
    packet pkt = new();
    @(posedge vif.clk);
    pkt.sum = vif.sum;
    mbx.put(pkt);
  endtask
endclass
```

✓ Samples output after clock edge

🔹 **7. Scoreboard**

```
class scoreboard;
  mailbox mbx;
```

```
  function new(mailbox mbx);
    this.mbx = mbx;
  endfunction

  task run();
    packet pkt;
    mbx.get(pkt);
    $display("Clocked Output Sum = %0d", pkt.sum);
  endtask
endclass
```

🔹 8. Top Testbench (Clock Generation)

```
module tb;
  logic clk = 0;
  always #5 clk = ~clk;   // 10 time-unit clock

  adder_if vif(clk);
  adder dut (.clk(clk), .a(vif.a), .b(vif.b), .sum(vif.sum));

  mailbox g2d = new();
  mailbox m2s = new();

  generator  gen;
  driver     drv;
  monitor    mon;
  scoreboard sb;
```

```
  initial begin
    gen = new(g2d);
    drv = new(g2d, vif);
    mon = new(m2s, vif);
    sb  = new(m2s);

    gen.run();
    drv.run();
    mon.run();
    sb.run();

    #20 $finish;
  end
endmodule
```

🔹 CLOCKED FLOW (VERY IMPORTANT)

Generator → Mailbox → Driver @(posedge clk)
→ DUT @(posedge clk)
→ Monitor @(posedge clk)
→ Mailbox → Scoreboard

◇ KEY DIFFERENCE (Unclocked vs Clocked)

Unclocked	Clocked
Immediate	Posedge based
No timing control	Realistic RTL behavior

| Simple | Industry-style |

🔹 INTERVIEW ONE-LINER

> In a clocked verification environment, the driver drives inputs and the monitor samples outputs synchronously with the DUT clock.

Generator vs Driver (Simple Difference)

🔸 Generator

- Creates data
- Works at transaction level
- Does NOT know about signals or pins

👉 "What data to send?"

Example:

Generates `a = 3, b = 5`

🔸 Driver

- Applies data to DUT
- Works at signal / pin level
- Uses interface / clocking block

👉 "How to send data to DUT pins?"

Example:

Drives `a=3, b=5` on `posedge clk`

🔸 One-Line Difference (Interview Ready)

> Generator creates stimulus, Driver converts that stimulus into DUT signals.

🔸 Quick Analogy -

- Generator → prepares the package

- Driver → delivers the package to the destination

◇ Monitor vs Scoreboard

◇ **Monitor**

- Observes DUT signals
- Does not check correctness
- Converts signals → data (packet)

👉 "What did the DUT output?"

Example:

Reads `sum = 8` from DUT

◇ **Scoreboard**

- Checks correctness
- Compares expected vs actual
- Decides PASS / FAIL

👉 *"Is the output correct?"*

Example:

Expected `3 + 5 = 8` → PASS ☑

◇ **One-Line Difference (Interview Ready)**

> Monitor collects data, Scoreboard verifies data.

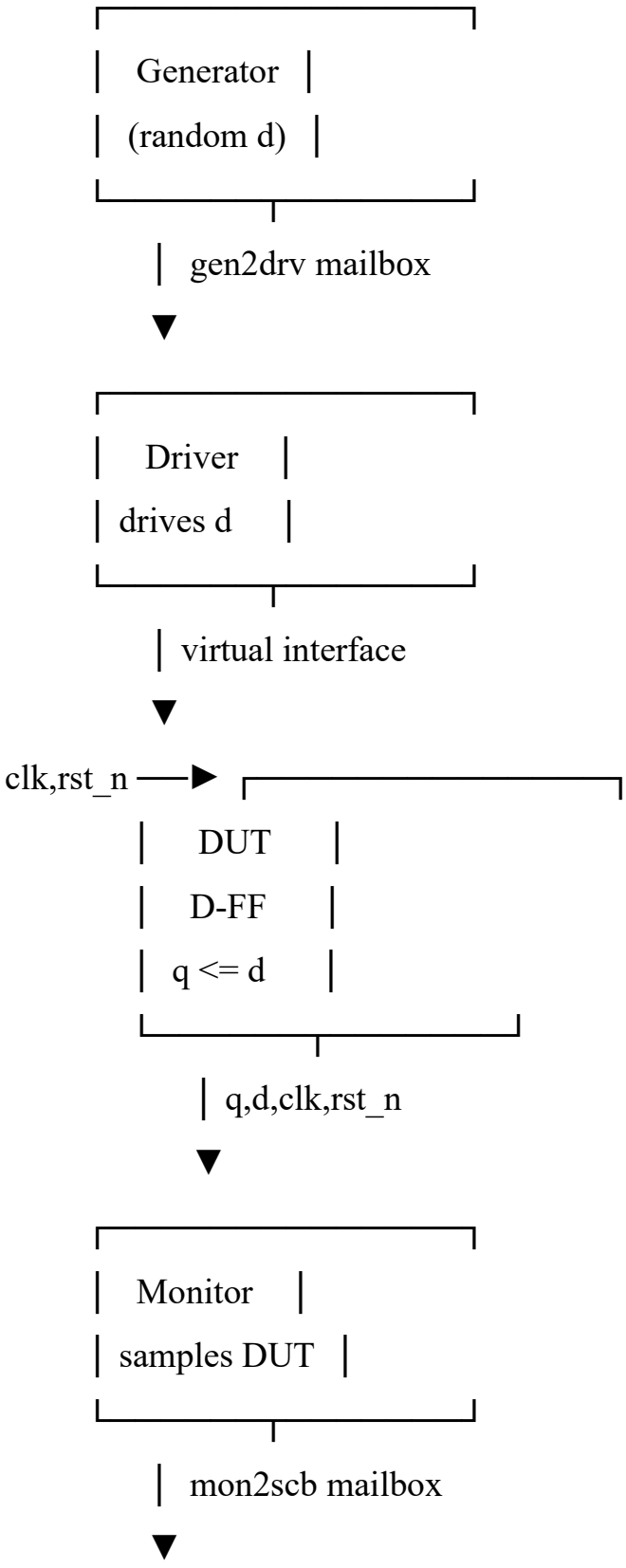
◇ Simple Analogy-

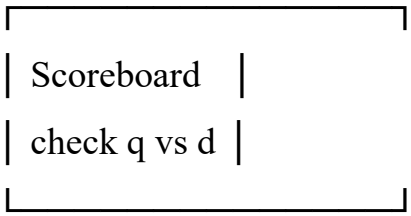
Monitor → note taker

Scoreboard → examiner

block diagram of your D-Flip-Flop testbench architecture (generator–driver–monitor–scoreboard flow).

📌 Block Diagram (Text / ASCII)





🧠 **How data flows (very short)**

Generator → Driver → DUT → Monitor → Scoreboard

- Generator → creates random `d`
- Driver → drives `d` to DUT using interface
- DUT → D Flip-Flop
- Monitor → captures `d`, `q`, `clk`, `rst_n`
- Scoreboard → checks correctness

🕒 Clock & Reset (side flow)

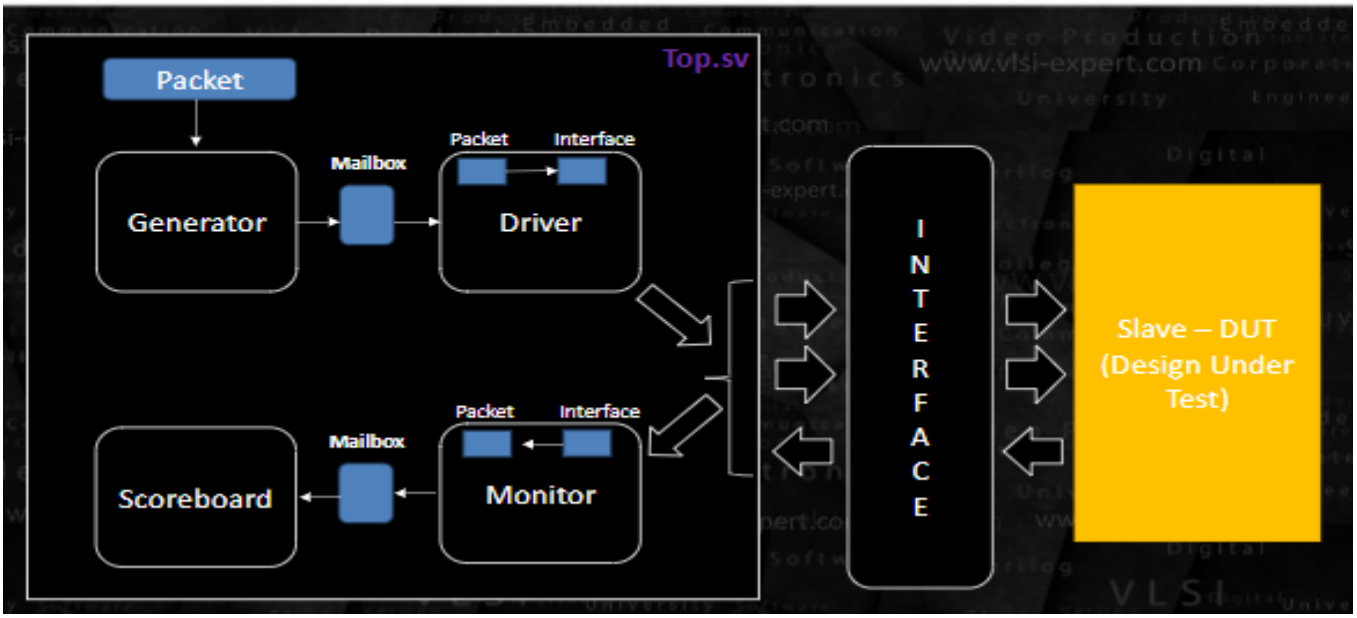
Clock Generator ————▶ DUT, Interface

Reset Logic ————▶ DUT, Interface

☑ One-line summary

👉 This block diagram shows a classic System Verilog verification flow where stimulus is generated, applied to DUT, observed, and checked.

Design Under Test (DUT)



[D-flip flop as DUT]

[Interface]

[Packet]

[Generator]

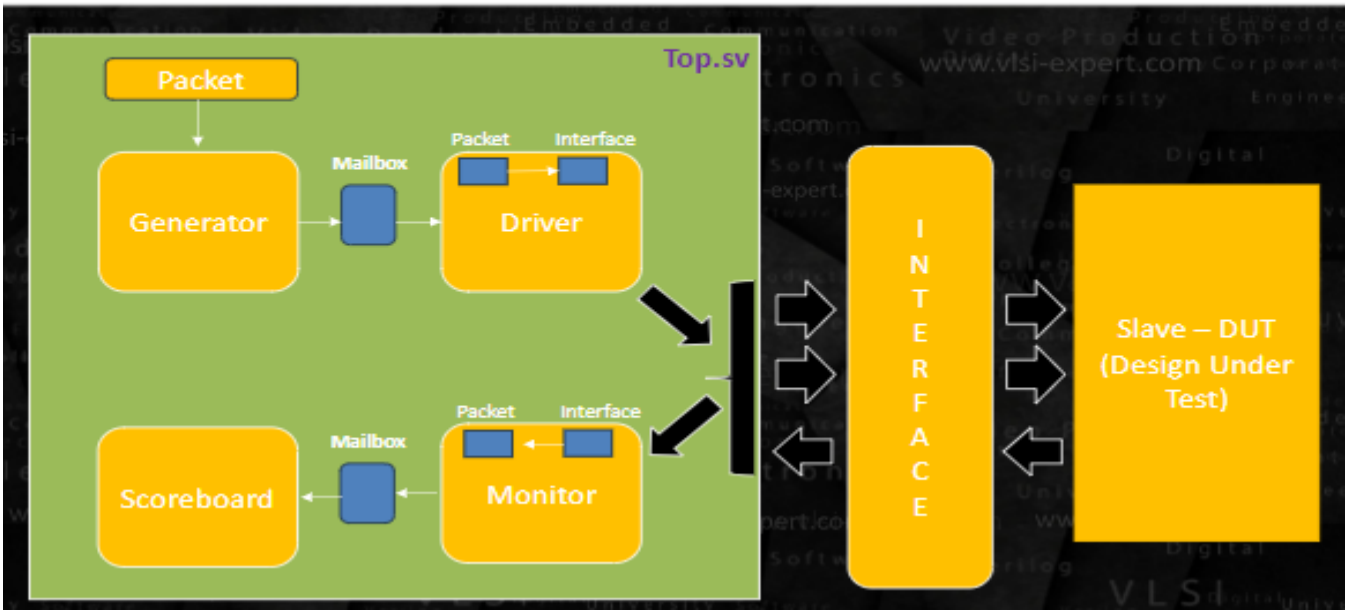
[Driver]

[Monitor]

[Scoreboared]

[Top module]

Scoreboard



File Name:- design.sv

```
//Desugn of D-FF using negdge reset
module d_ff(input logic d,clk,rst_n,output logic q);

always_ff @(posedge clk)begin
        if (!rst_n)
                q <=1'b0;
        else
                q <=d;
end
endmodule
```

Explanation:

This module implements a D-Flip-Flop with active-low reset

```
module d_ff(input logic d, clk, rst_n, output logic q);
```

- `d` → Data input
- `clk` → Clock signal
- `rst_n` → Reset (active LOW, `_n` means negative)
- `q` → Output of flip-flop

always_ff @(posedge clk)

- Flip-flop triggers only on rising edge of clock
- `always_ff` ensures this block models sequential logic

if (!rst_n)

q <= 1'b0;

- If `rst_n = 0` → reset is active
- Output `q` is cleared to 0

else

q <= d;

- If reset is inactive (`rst_n = 1`)
- Output `q` stores the value of `d` at clock edge

Simple example

| clk ↑ | rst_n | d | q (next) |

| ----- | ----- | - | -------- |

| ↑ | 0 | X | 0 |

| ↑ | 1 | 1 | 1 |

| ↑ | 1 | 0 | 0 |

✓ When reset is LOW → `q = 0`

✓ When reset is HIGH → `q` follows `d` on clock edge

One-line summary

👉 On every rising clock edge, `q` stores `d`, unless reset is LOW, then `q` becomes 0.

File Name:- interface.sv

```
//interface of D-FF


interface intf(input logic clk,input logic rst_n);
        logic d;
        logic q;
endinterface
```

Explanation:

interface intf(input logic clk, input logic rst_n);

- Creates an interface named `intf`
- `clk` and `rst_n` are inputs to the interface
- These signals usually come from the testbench (clock/reset generator)

logic d;

logic q;

- `d` → Data signal (driven by driver)
- `q` → Output signal (monitored from DUT)
- These signals are shared by DUT, driver, and monitor

endinterface

- Ends the interface definition

✓ Fewer connections

✓ Cleaner code

✓ Easy to use in UVM (virtual interface)

One-line summary

👉 Interface bundles related signals (`clk`, `rst_n`, `d`, `q`) into a single object for easy connection and reuse.

File Name:- packet.sv

```
//packect/transaction of D-FF


class packet;
    bit clk,rst_n;
    rand bit d;
    bit q;
endclass
```

Explanation:

This class defines a transaction (packet) used in a testbench to represent one data transfer for a D flip-flop.

Code explanation

class packet;

- Defines a class named `packet`
- Used to store stimulus and response data

bit clk, rst_n;

- `clk` → Clock value for the transaction
- `rst_n` → Reset value
- Stored for reference/debug, not usually randomized

rand bit d;

- `d` → Data input
- `rand` means this value can be randomized
- Used to generate different test cases automatically

bit q;

- `q` → Output captured from DUT
- Filled by monitor, not randomized

endclass

- Ends the class definition

Important note (very common interview point)

👉 Clock is usually NOT included in packet, because:

- Clock is a timing signal, not data
- Driver uses clock from interface, not from transaction

One-line summary

👉 Packet class represents one flip-flop operation: input `d` and output `q`, used between generator, driver, and monitor.

File Name:- generator.sv

```
// generator


class generator;
	packet pkt;
	mailbox mbx;

	function new(mailbox mbx);
		this.mbx=mbx;
	endfunction

	task run();
		repeat(10)begin
			pkt=new();
			pkt.randomize();
			this.mbx.put(pkt);
			#10;
		end
	endtask
```

endclass

Explanation:

The generator creates random packets (transactions) and sends them to the driver using a mailbox.

Code explanation

class generator;

    packet pkt;

    mailbox mbx;

- `pkt` → handle of packet (transaction)
- `mbx` → mailbox used to send packets to driver

function new(mailbox mbx);

    this.mbx = mbx;

endfunction

- Constructor
- Receives mailbox from testbench
- Stores it inside generator

task run();

- Main task of generator
- Runs during simulation

repeat(10) begin

- Generates 10 transactions

pkt = new();

pkt.randomize();

- Creates a new packet
- Randomizes `d` value

this.mbx.put(pkt);

- Sends packet to mailbox
- Driver will receive it using `get()`

#10;

- Waits 10 time units before next packet

Simple example (what actually happens)

Assume random values generated:

pkt.d = 1

pkt.d = 0

pkt.d = 1

...

```

Flow:

Generator → Mailbox → Driver → DUT

### One-line summary

👉 Generator creates random input data (`d`) and sends it to the driver using a mailbox.

**File Name:- driver.sv**

// driver of D-FF

```
class driver;
	packet pkt;
	mailbox mbx;
	virtual intf vif;

	function new(mailbox mbx,virtual intf vif);
		this.mbx=mbx;
		this.vif=vif;
```
```

```
    endfunction


    task run();
        repeat(10)begin
            this.mbx.get(pkt);
            vif.d=pkt.d;
            pkt.q=vif.q;
            #10;
        end
    endtask
endclass
```

Explanation:

The driver takes packets from the generator and drives the DUT signals through the interface.

Code explanation

```
class driver;
    packet pkt;
    mailbox mbx;
    virtual intf vif;
```

- `pkt` → transaction received from generator
- `mbx` → mailbox to receive packets
- `vif` → virtual interface used to drive DUT signals

```
function new(mailbox mbx, virtual intf vif);
    this.mbx = mbx;
    this.vif = vif;
endfunction
```

- Constructor
- Gets mailbox and interface from testbench

task run();

- Main driving task

this.mbx.get(pkt);

- Receives one packet from mailbox
- Blocks until packet is available

vif.d = pkt.d;

- Drives DUT input `d` using interface

pkt.q = vif.q;

- Samples DUT output `q`

#10;

- Waits before driving next packet

Simple example (data flow)

Assume:

pkt.d = 1

Driver action:

vif.d = 1 → DUT sees d = 1

After clock edge:

```

vif.q = 1 → pkt.q = 1

### One-line summary

👉 Driver receives transactions and applies input `d` to the DUT using the virtual interface.

📌 Interview note:

Usually, driver drives inputs only, and monitor samples outputs (`q`). Your code samples `q` inside driver, which is okay for learning.
```

File Name:- monitor.sv

```
// monitor of D-FF
class monitor;
	mailbox mbx;
	virtual intf vif;
	packet pkt;

	function new(mailbox mbx,virtual intf vif);
		this.mbx=mbx;
		this.vif=vif;
	endfunction

	task run();
		repeat(10)begin
			pkt=new();
			pkt.clk=vif.clk;
			pkt.rst_n=vif.rst_n;
			pkt.d=vif.d;
			pkt.q=vif.q;

			mbx.put(pkt);
			#10;
		end
	endtask
endclass
```

Explanation:

The monitor observes the DUT signals through the interface and collects them as packets for checking or coverage.

Code explanation

class monitor;

    mailbox mbx;

    virtual intf vif;

    packet pkt;

- `mbx` → mailbox to send observed data
- `vif` → virtual interface to read DUT signals
- `pkt` → packet to store sampled values

function new(mailbox mbx, virtual intf vif);

    this.mbx = mbx;

    this.vif = vif;

endfunction

- Constructor
- Receives mailbox and interface from testbench

task run();

- Main monitoring task

pkt = new();

- Creates a new packet for captured data

pkt.clk   = vif.clk;

pkt.rst_n = vif.rst_n;

pkt.d   = vif.d;

pkt.q   = vif.q;

- Samples DUT signals from interface
- Stores them inside the packet

mbx.put(pkt);

- Sends the captured packet to scoreboard / checker

#10;

- Waits before capturing next values

Simple example (what monitor captures)

At a clock edge:

```
clk = 1

rst_n = 1

d = 1

q = 1
```

Packet created by monitor:

pkt = {clk=1, rst_n=1, d=1, q=1}

One-line summary

👉 Monitor watches DUT signals and converts them into packets for verification.

📌 Important note (good practice):

Monitor usually samples signals on clock edge:

@(posedge vif.clk);

File Name:- scoreboard.sv

```
//scoreboard of D-ff
class scoreboard;
	mailbox mbx;
	packet pkt;

	function new(mailbox mbx);
		this.mbx=mbx;
	endfunction
```

```
    task run();
        repeat(10)begin
            mbx.get(pkt);

            if (!pkt.rst_n)
                if (!pkt.q)
                    $display("reset condition is passed!!!");
                else
                    $display("reset condition is failing!!!");
            else
                if (pkt.q == pkt.d)
                    $display("q is matched with d succesfully");
                else
                    $display("q is not matching with d !!!");
            #10;
        end
    endtask
endclass
```

Explanation:

The scoreboard checks correctness of DUT output by comparing expected vs actual values.

Code explanation

```
class scoreboard;
    mailbox mbx;
    packet pkt;
```

- `mbx` → mailbox to receive packets from monitor
- `pkt` → packet containing sampled DUT data

function new(mailbox mbx);

this.mbx = mbx;

endfunction

- Constructor
- Gets mailbox from testbench

task run();

Main checking task

mbx.get(pkt);

- Receives one packet from monitor
- Blocks until data is available

Checking logic

◇ **Reset condition**

if (!pkt.rst_n)

- If reset is active (`rst_n = 0`)

if (!pkt.q)

"reset condition is passed"

else

"reset condition is failing"

✓ During reset, output must be 0

◇ **Normal operation**

else

if (pkt.q == pkt.d)

- When reset is inactive
- Output `q` should match input `d`

$display("q is matched with d successfully");

✓ Flip-flop works correctly

$display("q is not matching with d !!!");

❌ Functional error detected

#10;

- Wait before checking next packet

Simple example

| rst_n | d | q | Result |

| ----- | - | - | ---------- |

| 0 | X | 0 | ☑ Reset passed |

| 1 | 1 | 1 | ☑ Match |

| 1 | 0 | 1 | ❌ Fail |

One-line summary

👉 Scoreboard verifies DUT behavior: during reset `q=0`, otherwise `q` must equal `d`.

File Name:- top.sv

//testbench for D-FF

`include "design.sv"

`include "interface.sv"

`include "packet.sv"

`include "generator.sv"

`include "driver.sv"

`include "monitor.sv"

`include "scoreboard.sv"

module testbench;

```
bit clk;
bit rst_n;

intf inf(clk,rst_n);

d_ff dut(.clk(clk),.d(inf.d),.q(inf.q),.rst_n(rst_n));

always #5 clk=~clk;

mailbox gen2drv;
mailbox mon2scb;
generator gen;
driver drv;
monitor mon;
scoreboard scb;

initial begin
        gen2drv=new();
        mon2scb=new();
        gen =new(gen2drv);
        drv =new(gen2drv,inf);
        mon =new(mon2scb,inf);
        scb =new(mon2scb);

        rst_n=1'b0;
        #50 rst_n =1'b1;
```

```
        fork
                gen.run();
                drv.run();
                mon.run();
                scb.run();
        join
        #2000 $finish();
end


initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,testbench);
end
endmodule
```

Explanation:

This is the top testbench that:

- Connects D-FF DUT
- Creates clock & reset
- Instantiates generator, driver, monitor, scoreboard
- Runs the complete verification flow

Include files

```
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"
```

- Includes all required files
- Similar to importing classes/modules

Testbench module

module testbench;

bit clk;

bit rst_n;

- Declares clock and reset

Interface instance

intf inf(clk,rst_n);

- Creates interface
- Shares signals (`d`, `q`, `clk`, `rst_n`) between TB and DUT

DUT instantiation

d_ff dut(.clk(clk),.d(inf.d),.q(inf.q),.rst_n(rst_n));

- Connects D-FF to interface and clock/reset

Clock generation

always #5 clk = ~clk;

- Generates clock with 10 time-unit period

Mailboxes

mailbox gen2drv;

mailbox mon2scb;

- `gen2drv` → Generator → Driver
- `mon2scb` → Monitor → Scoreboard

Component handles

generator gen;

driver drv;

monitor mon;

scoreboard scb;

- Declares testbench components

Create objects & connect

gen2drv = new();

mon2scb = new();

gen = new(gen2drv);

drv = new(gen2drv, inf);

mon = new(mon2scb, inf);

scb = new(mon2scb);

- Creates mailboxes and passes them to components
- Passes interface to driver & monitor

Reset logic

rst_n = 1'b0;

#50 rst_n = 1'b1;

- Reset active for first 50 time units
- Then normal operation starts

Parallel execution

fork

    gen.run();

    drv.run();

    mon.run();

    scb.run();

join

- **All components run in parallel:**

Generator → Driver → DUT → Monitor → Scoreboard

Finish simulation

#2000 $finish();

- Stops simulation after enough time

Waveform dump

$fsdbDumpfile("dump.fsdb");

$fsdbDumpvars(0,testbench);

- Generates waveform file for Verdi / Debussy

Simple flow example

1. Generator creates `d = 1`
2. Driver drives `d` to DUT
3. DUT updates `q` on clock
4. Monitor captures `{d,q}`
5. Scoreboard checks:
   - reset → `q=0`
   - normal → `q == d`

One-line summary

👉 This top testbench connects all verification components and checks whether the D flip-flop works correctly using generator-driver-monitor-scoreboard flow.

……

📦 **Include files**

`include "design.sv"

`include "interface.sv"

`include "packet.sv"

`include "generator.sv"

`include "driver.sv"

`include "monitor.sv"

`include "scoreboard.sv"

➞ Includes DUT and all testbench components

➞ Similar to importing files so the compiler knows these definitions

🧱 **Testbench module**

module testbench;

➞ Start of top-level testbench

⏱️ Clock & reset

bit clk;

bit rst_n;

➞ `clk` → clock signal

➞ `rst_n` → active-low reset

🔗 **Interface instance**

intf inf(clk,rst_n);

➞ Creates interface object

➞ Shares `clk`, `rst_n`, `d`, `q` between TB and DUT

🏵️ **DUT instantiation**

d_ff dut(.clk(clk),.d(inf.d),.q(inf.q),.rst_n(rst_n));

➞ Instantiates D-Flip-Flop

➞ Connects:

- `d` & `q` → through interface
- `clk`, `rst_n` → from testbench

🕒 **Clock generation**

always #5 clk = ~clk;

➞ Toggles clock every 5 time units

➞ Clock period = 10 time units

Example

0 → 5 → 10 → 15 ...

clk: 0 1 0 1

📬 **Mailboxes**

mailbox gen2drv;

mailbox mon2scb;

➞ `gen2drv` → Generator → Driver

➞ `mon2scb` → Monitor → Scoreboard

🏵 **Component handles**

generator gen;

driver drv;

monitor mon;

scoreboard scb;

➞ Declares testbench components

🚀 **Main initial block**

initial begin

➞ Runs once at start of simulation

Create mailboxes

gen2drv = new();

mon2scb = new();

➞ Allocates memory for mailboxes

Create components

gen = new(gen2drv);

drv = new(gen2drv,inf);

mon = new(mon2scb,inf);

scb = new(mon2scb);

➞ Passes:

- Mailbox → for communication
- Interface → for signal access

🔄 **Reset sequence**

rst_n = 1'b0;

#50 rst_n = 1'b1;

➞ Apply reset for first 50 time units

➞ Then release reset

Example

rst_n = 0 → q = 0

rst_n = 1 → normal operation

🔀 **Parallel execution**

fork

gen.run();

```
        drv.run();
        mon.run();
        scb.run();
join
```

➞ All components run in parallel

Flow

Generator → Driver → DUT → Monitor → Scoreboard

⏹ **End simulation**

#2000 $finish();

➞ Stops simulation after enough cycles

📊 Waveform dump

➞ Creates waveform file for Verdi

➞ Helps debug signals visually

☑ One-line summary

👉 This file connects all verification components, drives clock & reset, runs generator-driver-monitor-scoreboard in parallel, and checks the D-Flip-Flop functionality.

Run command:

vcs -sverilog top.sv -kdb -lca -debug_access+all -full64

./simv

File Name:- in Directory for including in testbench

driver.sv generator.sv monitor.sv packet.sv top.sv design.sv interface.sv scoreboard.sv

Simulation output:

q is matched with d succesfully

q is not matching with d !!!

```
q is not matching with d !!!
q is not matching with d !!!
q is not matching with d !!!
q is not matching with d !!!
q is matched with d succesfully
q is not matching with d !!!
q is matched with d succesfully
q is matched with d succesfully
$finish called from file "top.sv", line 44.
$finish at simulation time                2150
           V C S   S i m u l a t i o n   R e p o r t
Time: 2150
```

New updated code of above code:-

```
//Desugn of D-FF using negdge reset

module d_ff(input logic d,
        input logic clk,
        input logic rst_n,
        output logic q
);

always_ff @(posedge clk)begin
        if (!rst_n)
                q <=1'b0;
        else
                q <=d;
```

```
end
endmodule
```

//interface of D-FF

```
interface intf(input logic clk,input logic rst_n);
        logic d;
        logic q;
endinterface
```

//packect/transaction of D-FF

```
class packet;
        rand bit d;
        bit q;
endclass
```

// generator

```
class generator;
        mailbox mbx;
        packet pkt;

        function new(mailbox mbx);
                this.mbx=mbx;
        endfunction
```

```
        task run();
                repeat(10)begin
                        pkt=new();
                        pkt.randomize();
                        mbx.put(pkt);
                end
        endtask
endclass
```

// driver of D-FF

```
class driver;
        mailbox mbx;
        virtual intf vif;
        packet pkt;

        function new(mailbox mbx,virtual intf vif);
                this.mbx=mbx;
                this.vif=vif;
        endfunction

        task run();
                repeat(10)begin
                        mbx.get(pkt);
                        @(posedge vif.clk);
                        vif.d <= pkt.d;
                end
```

```
        endtask
endclass

// monitor of D-FF
class monitor;
        mailbox mbx;
        virtual intf vif;
        packet pkt;

        function new(mailbox mbx,virtual intf vif);
                this.mbx=mbx;
                this.vif=vif;
        endfunction

        task run();
                repeat(10)begin
                        @(posedge vif.clk);
                        pkt=new();
                        pkt.d=vif.d;
                        pkt.q=vif.q;
                        mbx.put(pkt);
                end
        endtask
endclass

//scoreboard of D-ff
class scoreboard;
```

```
        mailbox mbx;
        packet pkt;
        bit prev_d;

        function new(mailbox mbx);
                this.mbx=mbx;
        endfunction

        task run();
                prev_d =0;
                repeat(10)begin
                        mbx.get(pkt);
                        if (pkt.q == prev_d)
                                $display("[PASS] q=%0b matches with succesfully!!! d=%0b",
pkt.q, prev_d);
                        else
                                $display("[FAIL] q=%0b does NOT match with!!! d=%0b",
pkt.q, prev_d);
                        prev_d = pkt.d;
                end
        endtask
endclass
```

//top.sv file for testbench for D-FF

```
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
```

```
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module testbench;
bit clk;
bit rst_n;

intf inf(clk,rst_n);

d_ff dut(.clk(clk),.d(inf.d),.q(inf.q),.rst_n(rst_n));

always #5 clk=~clk;

mailbox gen2drv;
mailbox mon2scb;

generator gen;
driver drv;
monitor mon;
scoreboard scb;

initial begin
        gen2drv=new();
        mon2scb=new();

        gen =new(gen2drv);
```

```
        drv =new(gen2drv,inf);
        mon =new(mon2scb,inf);
        scb =new(mon2scb);

        rst_n=1'b0;
        #20 rst_n =1'b1;

        fork
                gen.run();
                drv.run();
                mon.run();
                scb.run();
        join
        #200 $finish();
end

initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,testbench);
end
endmodule
```

Simulation output:

```
[PASS] q=0 matches with succesfully!!! d=0
[PASS] q=0 matches with succesfully!!! d=0
[PASS] q=0 matches with succesfully!!! d=0
[PASS] q=1 matches with succesfully!!! d=1
[PASS] q=0 matches with succesfully!!! d=0
```

[PASS] q=1 matches with succesfully!!! d=1

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=1 matches with succesfully!!! d=1

[PASS] q=1 matches with succesfully!!! d=1

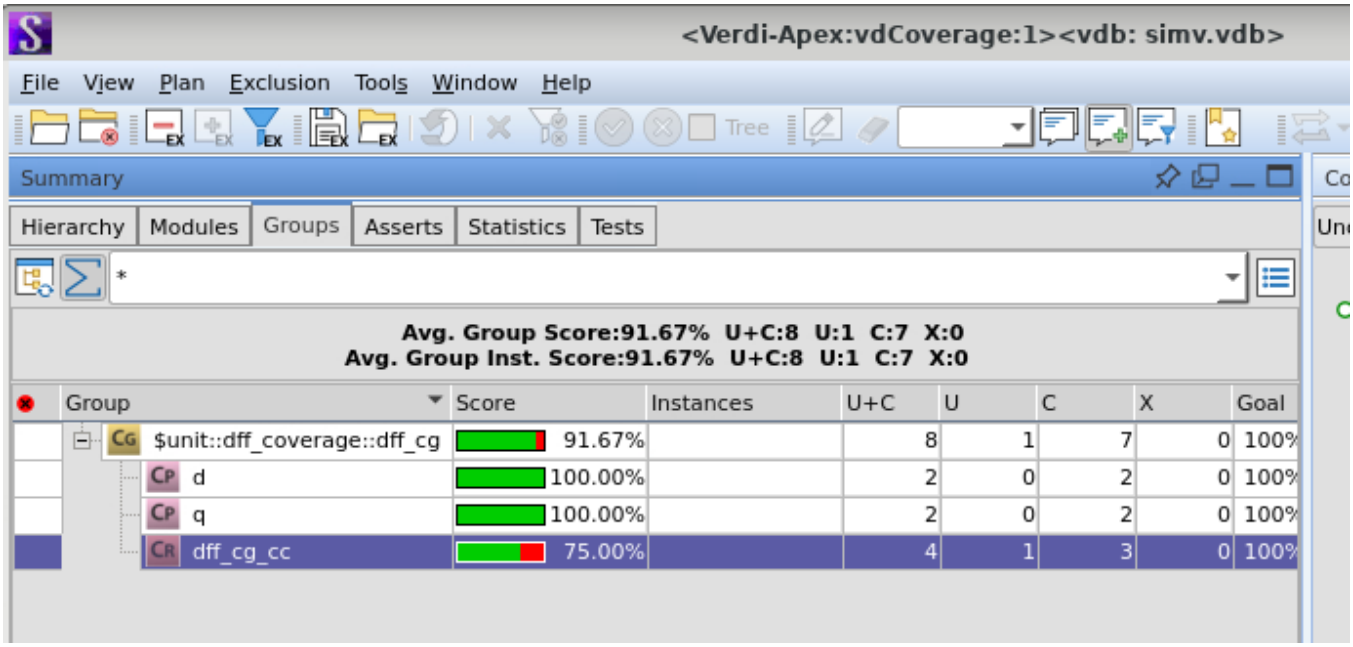
[PASS] q=0 matches with succesfully!!! d=0

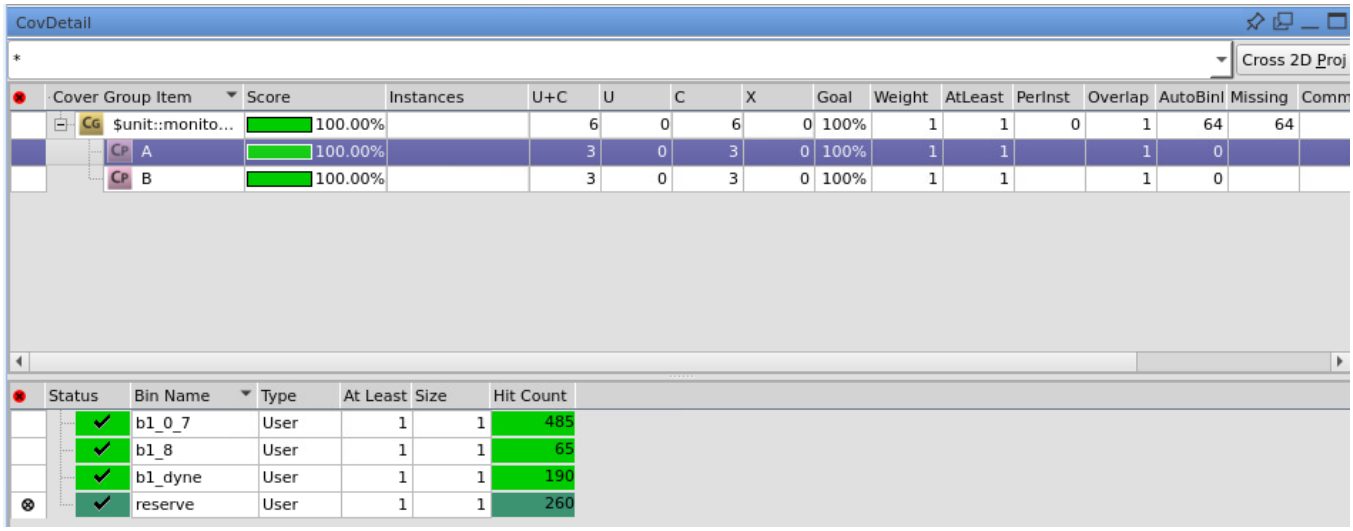
$finish called from file "top.sv", line 46.

$finish at simulation time 315

V C S S i m u l a t i o n R e p o r t

Time: 315





New updated code of above code:-

After adding constraint in packet + functional coverage → create class + modified monitor only + add coverage file in tup.sv (`include “coverage.sv”)

Packet.sv

```
//packect/transaction of D-FF


class packet;
	rand bit d;
	bit q;


	constraint d_dist { d dist { 0 := 50 , 1 := 50};}
endclass
```

coverage.sv

```
//coverage


class dff_coverage;
	bit d;
	bit q;


	covergroup dff_cg;
		coverpoint d {
			bins zero = {0};
		bins one = {1};
	}


	coverpoint q {
```

```
            bins zero = {0};
        bins one = {1};}

        cross d,q;
endgroup

function new();
        dff_cg = new();
endfunction

function void sample(bit d_i, bit q_i);
        d=d_i;
        q=q_i;
        dff_cg.sample();
endfunction
endclass
```

monitor.sv

```
// monitor of D-FF
class monitor;
        mailbox mbx;
        virtual intf vif;
        packet pkt;
        dff_coverage cov;

        function new(mailbox mbx,virtual intf vif);
                this.mbx=mbx;
                this.vif=vif;
```

```
            cov = new();
        endfunction

        task run();
            repeat(10)begin
                @(posedge vif.clk);
                pkt=new();
                pkt.d=vif.d;
                pkt.q=vif.q;

                cov.sample(pkt.d,pkt.q);
                mbx.put(pkt);
            end
        endtask
endclass
```

top.sv

```
//testbench for D-FF
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "coverage.sv"
`include "monitor.sv"
`include "scoreboard.sv"
```

Simulation output:

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=1 matches with succesfully!!! d=1

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=1 matches with succesfully!!! d=1

[PASS] q=0 matches with succesfully!!! d=0

[PASS] q=1 matches with succesfully!!! d=1

$finish called from file "top.sv", line 47.

$finish at simulation time 315

2-input 4-bit adder (sum & carry using without using clock):-

// Design of 2-input 4-bit adder

```
module adder_4bit (
        input  logic [3:0] a, b,
        output logic [3:0] sum,
        output logic carry
);

assign {carry, sum} = a + b;
endmodule

interface adder_if;
        logic [3:0] a, b;
```

```
        logic [3:0] sum;
        logic carry;
endinterface

class adder_packet;
        rand bit [3:0] a, b;
        bit [3:0] sum;
        bit carry;
endclass

class generator;
        mailbox mbx;
        adder_packet pkt;

        function new(mailbox mbx);
                this.mbx=mbx;
        endfunction

        task run();
                repeat(15)begin
                        pkt=new();
                        pkt.randomize();
                        mbx.put(pkt);
                end
        endtask
endclass
```

```
class driver;
	mailbox mbx;
	virtual adder_if vif;
	adder_packet pkt;

	function new(mailbox mbx,virtual adder_if vif);
		this.mbx=mbx;
		this.vif=vif;
	endfunction

	task run();
		repeat(15)begin
			mbx.get(pkt);
			vif.a <=pkt.a;
			vif.b <=pkt.b;
			#1;
		end
	endtask
endclass

class monitor;
	mailbox mbx2;
	virtual adder_if vif;
	adder_packet pkt;

	function new(mailbox mbx2,virtual adder_if vif);
		this.mbx2=mbx2;
```

```
        this.vif=vif;
    endfunction

    task run();
        repeat(15)begin
            #1;
            pkt=new();
            pkt.a=vif.a;
            pkt.b=vif.b;
            pkt.sum=vif.sum;
            pkt.carry=vif.carry;
            mbx2.put(pkt);
        end
    endtask
endclass

class scoreboard;
    mailbox mbx2;
    adder_packet pkt;

    function new(mailbox mbx2);
        this.mbx2=mbx2;
    endfunction

    task run();
        repeat(15)begin
            mbx2.get(pkt);
```

```
                if ({pkt.carry, pkt.sum} == pkt.a + pkt.b)
                        $display("PASS : a=%0d b=%0d sum=%0d carry=%0d",pkt.a, pkt.b, pkt.sum, pkt.carry);
                else
                        $display("FAIL : a=%0d b=%0d sum=%0d carry=%0d",pkt.a, pkt.b, pkt.sum, pkt.carry);
            end
        endtask
endclass

// testbench
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module tb;
adder_if vif();

adder_4bit dut (.a(vif.a),.b(vif.b),.sum(vif.sum),.carry(vif.carry));

mailbox mbx = new();
mailbox mbx2= new();
```

```
generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;

initial begin
        gen = new(mbx);
      drv = new(mbx, vif);
      mon = new(mbx2, vif);
      sb  = new(mbx2);

        fork
              gen.run();
              drv.run();
           mon.run();
           sb.run();
        join

        #20 $finish;
end
initial begin
      $fsdbDumpfile("dump.fsdb");
      $fsdbDumpvars(0,tb);
end
endmodule
```

Simulation output:

```
PASS : a=8 b=9 sum=1 carry=1
```

PASS : a=15 b=8 sum=7 carry=1

PASS : a=8 b=14 sum=6 carry=1

PASS : a=1 b=10 sum=11 carry=0

PASS : a=12 b=11 sum=7 carry=1

PASS : a=2 b=4 sum=6 carry=0

PASS : a=7 b=9 sum=0 carry=1

PASS : a=9 b=15 sum=8 carry=1

PASS : a=2 b=6 sum=8 carry=0

PASS : a=1 b=15 sum=0 carry=1

PASS : a=2 b=5 sum=7 carry=0

PASS : a=14 b=5 sum=3 carry=1

PASS : a=5 b=11 sum=0 carry=1

PASS : a=14 b=3 sum=1 carry=1

PASS : a=9 b=13 sum=6 carry=1

$finish called from file "top.sv", line 36.

$finish at simulation time 35

V C S S i m u l a t i o n R e p o r t

Time: 35

2-input 4-bit adder (sum & carry using using clock):-

```
module adder_4bit (
        input logic clk,
        input  logic [3:0] a, b,
        output logic [3:0] sum,
        output logic carry
);

always_ff @(posedge clk)begin
```

```
        {carry, sum} = a + b;
end
endmodule

interface adder_if(input logic clk);
        logic [3:0] a, b;
        logic [3:0] sum;
        logic carry;
endinterface

class adder_packet;
        rand bit [3:0] a, b;
        bit [3:0] sum;
        bit carry;
endclass

class generator;
        mailbox mbx;
        adder_packet pkt;

        function new(mailbox mbx);
                this.mbx=mbx;
        endfunction

        task run();
                repeat(10)begin
                        pkt=new();
```

```
                pkt.randomize();
                mbx.put(pkt);
            end
        endtask
endclass

class driver;
        mailbox mbx;
        virtual adder_if vif;
        adder_packet pkt;

        function new(mailbox mbx,virtual adder_if vif);
            this.mbx=mbx;
            this.vif=vif;
        endfunction

        task run();
            repeat(10)begin
                mbx.get(pkt);
                @(posedge vif.clk);
                vif.a <=pkt.a;
                vif.b <=pkt.b;
            end
        endtask
endclass

class monitor;
```

```
        mailbox mbx2;
        virtual adder_if vif;
        adder_packet pkt;

        function new(mailbox mbx2,virtual adder_if vif);
                this.mbx2=mbx2;
                this.vif=vif;
        endfunction

        task run();
                repeat(10)begin
                        @(posedge vif.clk);
                        pkt=new();
                        pkt.a=vif.a;
                        pkt.b=vif.b;
                        @(posedge vif.clk);
                        pkt.sum=vif.sum;
                        pkt.carry=vif.carry;
                        mbx2.put(pkt);
                end
        endtask
endclass

class scoreboard;
        mailbox mbx2;
        adder_packet pkt;
```

```
	function new(mailbox mbx2);
		this.mbx2=mbx2;
	endfunction

	task run();
		repeat(10)begin
			mbx2.get(pkt);

			if ({pkt.carry, pkt.sum} == pkt.a + pkt.b)
				$display("PASS : a=%0d b=%0d sum=%0d carry=%0d",pkt.a, pkt.b, pkt.sum, pkt.carry);
			else
				$display("FAIL : a=%0d b=%0d sum=%0d carry=%0d",pkt.a, pkt.b, pkt.sum, pkt.carry);
		end
	endtask
endclass

// testbench
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module tb;
```

```
logic clk;

initial clk=0;
always #5 clk=~clk;

adder_if vif(clk);

adder_4bit dut (.clk(clk),.a(vif.a),.b(vif.b),.sum(vif.sum),.carry(vif.carry));

mailbox mbx = new();
mailbox mbx2= new();

generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;

initial begin
        gen = new(mbx);
      drv = new(mbx, vif);
      mon = new(mbx2, vif);
      sb  = new(mbx2);

        fork
              gen.run();
              drv.run();
           mon.run();
```

```
            sb.run();
        join

        #20 $finish;
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb);
end
endmodule
```

Simulation output:

```
PASS : a=0 b=0 sum=0 carry=0
PASS : a=9 b=3 sum=12 carry=0
PASS : a=9 b=6 sum=15 carry=0
PASS : a=9 b=6 sum=15 carry=0
PASS : a=8 b=15 sum=7 carry=1
PASS : a=11 b=9 sum=4 carry=1
PASS : a=11 b=9 sum=4 carry=1
PASS : a=11 b=9 sum=4 carry=1
PASS : a=11 b=9 sum=4 carry=1
PASS : a=11 b=9 sum=4 carry=1
$finish called from file "top.sv", line 41.
$finish at simulation time                 215
          V C S   S i m u l a t i o n   R e p o r t
Time: 215
```

2x1 mux (without using clock):-

```
module mux_2x1 (
        input  logic a, b,
        input logic sel,
        output logic y
);

assign y =sel? b : a;

endmodule

interface mux_if;
        logic a, b;
        logic sel;
        logic y;
endinterface

class mux_packet;
        rand bit a, b;
        rand bit sel;
        bit y;
endclass

class generator;
        mailbox mbx;
        mux_packet pkt;

        function new(mailbox mbx);
```

```
        this.mbx=mbx;
    endfunction

    task run();
        repeat(5)begin
            pkt=new();
            pkt.randomize();
            mbx.put(pkt);
        end
    endtask
endclass

class driver;
    mailbox mbx;
    virtual mux_if vif;
    mux_packet pkt;

    function new(mailbox mbx,virtual mux_if vif);
        this.mbx=mbx;
        this.vif=vif;
    endfunction

    task run();
        repeat(5)begin
            mbx.get(pkt);
            vif.a <=pkt.a;
            vif.b <=pkt.b;
```

```
                vif.sel<=pkt.sel;
                #1;
            end
        endtask
endclass

class monitor;
        mailbox mbx2;
        virtual mux_if vif;
        mux_packet pkt;

        function new(mailbox mbx2,virtual mux_if vif);
            this.mbx2=mbx2;
            this.vif=vif;
        endfunction

        task run();
            repeat(5)begin
                #1;
                pkt=new();
                pkt.a=vif.a;
                pkt.b=vif.b;
                pkt.sel=vif.sel;
                pkt.y=vif.y;
                mbx2.put(pkt);
            end
        endtask
```

```
endclass

class scoreboard;
	mailbox mbx2;
	mux_packet pkt;

	function new(mailbox mbx2);
		this.mbx2=mbx2;
	endfunction

	task run();
		repeat(5)begin
			mbx2.get(pkt);

			if (pkt.y == (pkt.sel ? pkt.b : pkt.a))
				$display("PASS : a=%0d b=%0d sel=%0d y=%0d",pkt.a, pkt.b, pkt.sel, pkt.y);
			else
				$display("FAIL : a=%0d b=%0d sel=%0d y=%0d",pkt.a, pkt.b, pkt.sel, pkt.y);
		end
	endtask
endclass

// testbench
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
```

```
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module tb;
mux_if vif();

mux_2x1 dut (.a(vif.a),.b(vif.b),.sel(vif.sel),.y(vif.y));

mailbox mbx = new();
mailbox mbx2= new();

generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;

initial begin
        gen = new(mbx);
      drv = new(mbx, vif);
      mon = new(mbx2, vif);
      sb  = new(mbx2);

        fork
              gen.run();
              drv.run();
```

```
            mon.run();
            sb.run();
        join

        #20 $finish;
end
initial begin
    $fsdbDumpfile("dump.fsdb");
    $fsdbDumpvars(0,tb);
end
endmodule
```

Simulation output:

```
PASS : a=0 b=1 sel=1 y=1
PASS : a=1 b=0 sel=0 y=1
PASS : a=0 b=0 sel=1 y=0
PASS : a=1 b=0 sel=0 y=1
PASS : a=0 b=1 sel=0 y=0
$finish called from file "top.sv", line 36.
$finish at simulation time                   25
          V C S   S i m u l a t i o n   R e p o r t
Time: 25
```

2x1 mux (using clock):-

```
module mux_2x1 (
        input logic clk,
        input  logic a, b,
```

```
        input logic sel,
        output logic y
);

always_ff @(posedge clk)begin
        y <=sel? b : a;
end
endmodule

interface mux_if(input logic clk);
        logic a, b;
        logic sel;
        logic y;
endinterface

class mux_packet;
        rand bit a, b;
        rand bit sel;
        bit y;
endclass

class generator;
        mailbox mbx;
        mux_packet pkt;

        function new(mailbox mbx);
                this.mbx=mbx;
```

```
	endfunction

	task run();
		repeat(5)begin
			pkt=new();
			pkt.randomize();
			mbx.put(pkt);
		end
	endtask
endclass

class driver;
	mailbox mbx;
	virtual mux_if vif;
	mux_packet pkt;

	function new(mailbox mbx,virtual mux_if vif);
		this.mbx=mbx;
		this.vif=vif;
	endfunction

	task run();
		repeat(5)begin
			mbx.get(pkt);
			@(posedge vif.clk);
			vif.a <=pkt.a;
			vif.b <=pkt.b;
```

```
                vif.sel<=pkt.sel;
            end
        endtask
endclass

class monitor;
        mailbox mbx2;
        virtual mux_if vif;
        mux_packet pkt;

        function new(mailbox mbx2,virtual mux_if vif);
            this.mbx2=mbx2;
            this.vif=vif;
        endfunction

        task run();
            repeat(5)begin
                @(posedge vif.clk);
                pkt=new();
                pkt.a=vif.a;
                pkt.b=vif.b;
                pkt.sel=vif.sel;
                @(posedge vif.clk);
                pkt.y=vif.y;
                mbx2.put(pkt);
            end
        endtask
```

```
endclass

class scoreboard;
	mailbox mbx2;
	mux_packet pkt;

	function new(mailbox mbx2);
		this.mbx2=mbx2;
	endfunction

	task run();
		repeat(5)begin
			mbx2.get(pkt);

			if (pkt.y == (pkt.sel ? pkt.b : pkt.a))
				$display("PASS : a=%0d b=%0d sel=%0d y=%0d",pkt.a, pkt.b,
pkt.sel, pkt.y);
			else
				$display("FAIL : a=%0d b=%0d sel=%0d y=%0d",pkt.a, pkt.b,
pkt.sel, pkt.y);
		end
	endtask
endclass

// testbench
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
```

```
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module tb;
logic clk;

initial clk=0;
always #5 clk=~clk;

mux_if vif(clk);

mux_2x1 dut (.clk(clk),.a(vif.a),.b(vif.b),.sel(vif.sel),.y(vif.y));

mailbox mbx = new();
mailbox mbx2= new();

generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;

initial begin
          gen = new(mbx);
        drv = new(mbx, vif);
        mon = new(mbx2, vif);
```

```
        sb  = new(mbx2);

          fork
                gen.run();
                drv.run();
              mon.run();
              sb.run();
          join

          #20 $finish;
end
initial begin
        $fsdbDumpfile("dump.fsdb");
        $fsdbDumpvars(0,tb);
end
endmodule
```

Simulation output:

```
PASS : a=0 b=0 sel=0 y=0
PASS : a=1 b=1 sel=0 y=1
PASS : a=1 b=0 sel=1 y=0
PASS : a=1 b=0 sel=1 y=0
PASS : a=1 b=0 sel=1 y=0
$finish called from file "top.sv", line 41.
$finish at simulation time                 215
          V C S   S i m u l a t i o n   R e p o r t
Time: 115
```

```
……
// generator of FIFO

class generator;
        mailbox #(fifo_packet) gen2drv;

        function new(mailbox #(fifo_packet) mbx);
                this.gen2drv = mbx;
        endfunction

        task run();

                fifo_packet pkt;

                repeat (20) begin
                        pkt = new();
                        pkt.wr_en = 1;
                   pkt.rd_en = 0;
                   pkt.data_in  = $urandom_range(1,255);
                        gen2drv.put(pkt);
                end

                repeat (20) begin
                        pkt = new();
                        pkt.wr_en = 0;
                        pkt.rd_en = 1;
```

```
                gen2drv.put(pkt);
            end

            repeat (10) begin
                pkt = new();
              pkt.wr_en = 1;
                pkt.rd_en = 1;
                pkt.data_in  = $urandom_range(1,255);
              gen2drv.put(pkt);
            end
        endtask
endclass

//monitor of FIFO
class monitor;
  virtual fifo_if vif;
  mailbox #(fifo_packet) mon2sb;

  function new(mailbox #(fifo_packet) mbx, virtual fifo_if vif);
    this.mon2sb = mbx;
    this.vif    = vif;
  endfunction

  task run();
    fifo_packet pkt;

    forever begin
```

```
    @(posedge vif.clk);
    if (vif.wr_en || vif.rd_en) begin
            pkt = new();
          pkt.wr_en     = vif.wr_en;
          pkt.rd_en     = vif.rd_en;
          pkt.data_in   = vif.data_in;
          pkt.full      = vif.full;
          pkt.empty     = vif.empty;
          pkt.overflow  = vif.overflow;
          pkt.underflow = vif.underflow;

            if (vif.rd_en && !vif.empty) begin
                  @(posedge vif.clk);
                  pkt.data_out = vif.data_out;
            end
            mon2sb.put(pkt);end
    end
  endtask
endclass

class scoreboard;

  mailbox #(fifo_packet) mon2sb;
  logic [7:0] ref_q[$];

  function new(mailbox #(fifo_packet) mbx);
    this.mon2sb = mbx;
```

```
  endfunction

  task run();
        fifo_packet pkt;
        logic [7:0] exp_data;

        forever begin
              mon2sb.get(pkt);

              if (pkt.wr_en && !pkt.full && !pkt.overflow)
                    ref_q.push_back(pkt.data_in);

              if (pkt.wr_en && pkt.full && !pkt.overflow)
                    $error("SB ERROR: Overflow not asserted");
              if (pkt.rd_en && !pkt.empty && !pkt.underflow) begin
                    exp_data = ref_q.pop_front();
                    if (pkt.data_out !== exp_data)
                          $error("SB MISMATCH: Exp=%0d Got=%0d", exp_data,
pkt.data_out);
              end

              if (pkt.rd_en && pkt.empty && !pkt.underflow)
                    $error("SB ERROR: Underflow not asserted");
        end
  endtask
endclass
```

……..

```
// DUT of synchronous FIFO

module Sync_fifo #(parameter DATA_WIDTH=8,
        parameter FIFO_DEPTH=16,
        parameter ADDR_WIDTH=4)(

        input logic clk,
        input logic rst_n,
        input logic rd_en,
        input logic wr_en,
        input logic [DATA_WIDTH-1:0] data_in,
        output logic [DATA_WIDTH-1:0] data_out,
       output logic underflow,
       output logic overflow,
       output logic full,
       output logic empty
);

logic [DATA_WIDTH-1:0] memory [FIFO_DEPTH];
logic [ADDR_WIDTH-1:0] wr_ptr, rd_ptr;
logic [ADDR_WIDTH:0] count;


always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
                wr_ptr   <= '0;
                rd_ptr   <= '0;
```

```
    count    <= '0;
    data_out <= '0;
    overflow <= 1'b0;
    underflow<= 1'b0;
end
else begin
        overflow  <= 1'b0;
        underflow <= 1'b0;


        if (wr_en && !full) begin
                memory[wr_ptr] <= data_in;
                wr_ptr <= (wr_ptr == FIFO_DEPTH-1) ? '0 : wr_ptr + 1'b1;
        end
        else if (wr_en && full) begin
                overflow <= 1'b1;
        end

        if (rd_en && !empty) begin
                data_out <= memory[rd_ptr];
                rd_ptr <= (rd_ptr == FIFO_DEPTH-1) ? '0 : rd_ptr + 1'b1;
        end
        else if (rd_en && empty) begin
                underflow <= 1'b1;
        end

        case ({(wr_en && !full), (rd_en && !empty)})
```

```
                2'b10: count <= count + 1'b1;
                2'b01: count <= count - 1'b1;
                2'b11: count <= count;
                default: count <= count;
            endcase
        end
end

always_comb begin
        full  = (count == FIFO_DEPTH);
        empty = (count == 0);
end

endmodule
// interface of FIFO
interface fifo_if #(parameter DATA_WIDTH = 8)(input logic clk);

        logic rst_n;
        logic rd_en;
        logic wr_en;
        logic [DATA_WIDTH-1:0] data_in;
        logic [DATA_WIDTH-1:0] data_out;
        logic full;
        logic empty;
        logic overflow;
        logic underflow;
```

```
endinterface
// packet of FIFO

class fifo_packet;
        bit wr_en;
      bit rd_en;
      rand bit [7:0] data_in;
        bit [7:0] data_out;
        bit full;
      bit empty;
      bit overflow;
      bit underflow;

endclass

// generator of FIFO

class generator;
        mailbox #(fifo_packet) gen2drv;

        function new(mailbox #(fifo_packet) mbx);
                this.gen2drv = mbx;
        endfunction

        task run();

                fifo_packet pkt;
```

```
            repeat (50) begin
                pkt = new();
                pkt.randomize();
                gen2drv.put(pkt);
            end
        endtask
endclass
// driver for FIFO

class driver;
        mailbox #(fifo_packet) gen2drv;
        virtual fifo_if vif;

        function new(mailbox #(fifo_packet) mbx, virtual fifo_if vif);
            this.gen2drv = mbx;
            this.vif = vif;
        endfunction

        task reset();
            vif.rst_n <= 0;
            vif.wr_en <= 0;
            vif.rd_en <= 0;
            vif.data_in <= 0;
            repeat (5) @(posedge vif.clk);
            vif.rst_n <= 1;
            $display("DRV] : DUT RESET DONE");
            @(posedge vif.clk);
```

```
endtask

task write();
        @(posedge vif.clk);
        vif.wr_en   <= 1;
        vif.rd_en   <= 0;
        vif.data_in <= $urandom_range(1,255);
        @(posedge vif.clk);
        vif.wr_en   <= 0;
        $display("[DRV] : DATA Write data : %0d",vif.data_in);
        @(posedge vif.clk);
endtask

task read();
        @(posedge vif.clk);
        vif.rd_en <= 1;
        vif.wr_en <= 0;
        @(posedge vif.clk);
        vif.rd_en <= 0;
        $display("[DRV] : DATA READ ");
        @(posedge vif.clk);
endtask

task write_read();
        @(posedge vif.clk);
        vif.wr_en   <= 1;
        vif.rd_en   <= 1;
```

```
        vif.data_in <= $urandom_range(1,255);
        @(posedge vif.clk);
        vif.wr_en   <= 0;
        vif.rd_en   <= 0;
        $display("[DRV] : DATA Write data : %0d",vif.data_in);
        @(posedge vif.clk);
endtask

task run();
        int i;
        fifo_packet pkt;
        reset();

        for (i = 0; i < 20; i++) begin
                pkt = new();
                pkt.randomize();
                write();
        end

        for (i = 0; i < 20; i++) begin
                read();
        end

        for (i = 0; i < 10; i++) begin
                pkt = new();
                pkt.randomize();
                write_read();
```

```
            end
            $display("DRIVER: 20 WRITE, 20 READ, 10 WRITE-READ DONE");
        endtask
endclass
//monitor of FIFO
class monitor;
  virtual fifo_if vif;
  mailbox #(fifo_packet) mon2sb;

  function new(mailbox #(fifo_packet) mbx, virtual fifo_if vif);
    this.mon2sb = mbx;
    this.vif    = vif;
  endfunction

  task run();
        fifo_packet pkt;
        forever begin
            repeat(2) @(posedge vif.clk);
            pkt = new();
          pkt.wr_en     = vif.wr_en;
          pkt.rd_en     = vif.rd_en;
          pkt.data_in   = vif.data_in;
          pkt.full      = vif.full;
          pkt.empty     = vif.empty;
          pkt.overflow  = vif.overflow;
          pkt.underflow = vif.underflow;
            @(posedge vif.clk);
```

```
            pkt.data_out  = vif.data_out;
        mon2sb.put(pkt);
            $display("[MON] : wr:%0d rd:%0d din:%0d dout:%0d full:%0b
empty:%0b overflow:%0b
underflow:%0b",pkt.wr_en,pkt.rd_en,pkt.data_in,pkt.data_out,pkt.full,pkt.empty,pkt.over
flow,pkt.underflow);
      end
  endtask
endclass

// scoreboard of FIFO

class scoreboard;
    mailbox #(fifo_packet) mon2sb;
    byte ref_q[$];   // Reference FIFO queue

    function new(mailbox #(fifo_packet) mbx);
        this.mon2sb = mbx;
    endfunction



    task run();
        fifo_packet pkt;
        byte exp_data;

        forever begin
            mon2sb.get(pkt);
```

```
                if (pkt.wr_en && !pkt.full) begin
                        ref_q.push_back(pkt.data_in);
                        $display("[SCO] : Data stored in Queue :%0d",pkt.data_in);
                end

                if (pkt.overflow) begin
                        $display("[%0t] SCOREBOARD: OVERFLOW detected",
$time);
                end

                if (pkt.rd_en && !pkt.empty) begin
                        if (ref_q.size() == 0) begin
                                $error("[%0t] SB ERROR: Read when reference FIFO
empty", $time);
                        end
                        else begin
                                exp_data = ref_q.pop_front();
                                if (pkt.data_out !== exp_data) begin
                                        $error("[%0t] SB MISMATCH: Exp=%0h
Got=%0h",$time, exp_data, pkt.data_out);
                                end
                        end
                end

                if (pkt.underflow) begin
                        $display("[%0t] SCOREBOARD: UNDERFLOW detected",
$time);
                end
        end
```

```
        endtask
endclass
// testbench of FIFO
`include "design.sv"
`include "interface.sv"
`include "packet.sv"
`include "generator.sv"
`include "driver.sv"
`include "monitor.sv"
`include "scoreboard.sv"

module tb;
parameter DATA_WIDTH = 8;
parameter FIFO_DEPTH = 16;
parameter ADDR_WIDTH = 4;

logic clk;

initial clk = 0;
always #5 clk = ~clk;

fifo_if #(DATA_WIDTH) vif (clk);

Sync_fifo #(.DATA_WIDTH(8),.FIFO_DEPTH(16),.ADDR_WIDTH(4)) dut
(.clk(clk),.rst_n(vif.rst_n),.wr_en(vif.wr_en),.rd_en(vif.rd_en),.data_in(vif.data_in),.data_
out (vif.data_out),.full(vif.full),.empty(vif.empty),.overflow(vif.overflow),.underflow
(vif.underflow));
```

```
mailbox #(fifo_packet) gen2drv;
mailbox #(fifo_packet) mon2sb;

generator  gen;
driver     drv;
monitor    mon;
scoreboard sb;

initial begin
        clk = 0;

        gen2drv = new();
        mon2sb  = new();

        gen = new(gen2drv);
      drv = new(gen2drv, vif);
      mon = new(mon2sb,  vif);
      sb  = new(mon2sb);

        fork
                gen.run();
                drv.run();
            mon.run();
            sb.run();
        join_none

        #2000;
```

```
        $display("Test has been completed");
        $finish;
end

initial begin
        $fsdbDumpfile("fifo.fsdb");
        $fsdbDumpvars(0,tb);
end
endmodule

// driver for FIFO

class driver;
        mailbox #(fifo_packet) gen2drv;
        virtual fifo_if vif;

        function new(mailbox #(fifo_packet) mbx, virtual fifo_if vif);
                this.gen2drv = mbx;
                this.vif = vif;
        endfunction

        task reset();
                vif.rst_n <= 0;
                vif.wr_en <= 0;
                vif.rd_en <= 0;
                vif.data_in <= 0;
                repeat (5) @(posedge vif.clk);
```

```
        vif.rst_n <= 1;
        $display("DRV] : DUT RESET DONE");
        @(posedge vif.clk);
endtask

task write();
        @(posedge vif.clk);
        vif.wr_en   <= 1;
        vif.rd_en   <= 0;
        vif.data_in <= $urandom_range(1,255);
        @(posedge vif.clk);
        vif.wr_en   <= 1;
        $display("[DRV] : DATA Write data : %0d",vif.data_in);
        @(posedge vif.clk);
endtask

task read();
        @(posedge vif.clk);
        vif.rd_en <= 1;
        vif.wr_en <= 0;
        @(posedge vif.clk);
        vif.rd_en <= 0;
        $display("[DRV] : DATA READ ");
        @(posedge vif.clk);
endtask

task write_read();
```

```
        @(posedge vif.clk);
        vif.wr_en   <= 1;
        vif.rd_en   <= 1;
        vif.data_in <= $urandom_range(1,255);
        @(posedge vif.clk);
        vif.wr_en   <= 0;
        vif.rd_en   <= 0;
        $display("[DRV] : DATA Write data : %0d",vif.data_in);
        @(posedge vif.clk);
endtask

task run();
        int i;
        fifo_packet pkt;
        reset();

        for (i = 0; i < 20; i++) begin
                pkt = new();
                pkt.randomize();
                write();
        end

        for (i = 0; i < 20; i++) begin
                read();
        end

        for (i = 0; i < 10; i++) begin
```

```
            pkt = new();
            pkt.randomize();
            write_read();
        end
        $display("DRIVER: 20 WRITE, 20 READ, 10 WRITE-READ DONE");
    endtask
endclass
```

System Verilog scheduling Semantics

- System Verilog event regions
- Use of event scheduling
- Preponed region
- Active region
- Inactive region
- Non-blocking Assignment events region (NBA)
- Post-NBA region
- Observed region
- Post-observed region
- Reactive region
- Re-inactive events region
- Postponed region

Timing region

- Pre-poned region
- Active region
- Inactive region
- NBA
- Observed region
- Reactive region
- Post-poned region

Use of Events scheduling

Think of timing regions as ordered time slots inside the same simulation time ⏱️.

🕒 **System Verilog Event Scheduling Regions (Simple)**

Order (easy to remember)

Preponed

→ Active

→ Inactive

→ NBA

→ Observed

→ Reactive

→ Postponed

1.0 Preponed Region

- What happens here?
  - Signals are sampled
  - No updates allowed
- Used for-
  - Assertions sampling
  - `$past()` values
- Simple meaning

> “Take a snapshot before anything changes”

- **Example**

```
// Assertion samples signals here
assert property (@(posedge clk) q == $past(d));
```

2.0 Active Region

- What happens here?
  - Blocking assignments (`=`)
  - `always`, `initial` blocks start here
- Simple meaning

> “Main execution starts”

- **Example**

```
always @(posedge clk)
   a = b;   // blocking assignment
```

3.0 Inactive Region

- What happens here?
  - `#0` delayed statements
  - Events postponed inside same time
- Simple meaning

> “Do it later, but at the same time”

- **Example**

```
#0 c = a;
```

4.0 NBA (Non-Blocking Assignment) Region

- What happens here?
  - Non-blocking assignments (`<=`) update
- Used for
  - Sequential logic (flip-flops)
- Simple meaning

> “Update registers together”

- **Example**

```
always @(posedge clk)
   q <= d;   // updates in NBA
```

5.0 Observed Region

- What happens here?
  - Assertions check conditions
  - No signal updates allowed
- Simple meaning

> “Check design behavior”

- **Example**

```
assert property (@(posedge clk) q == d);
```

6.0 Reactive Region

- What happens here?
  - Testbench reacts to DUT
  - Drivers update inputs
- Used in
  - Clocking blocks
  - Testbench stimulus
- Simple meaning

> “Testbench responds to DUT”

- **Example**

clocking cb @(posedge clk);

output d;

endclocking

cb.d <= 1;

7.0 Postponed Region

- What happens here?
  - `$display`, `$monitor`
  - Waveform dump
- Simple meaning

> “Show final results”

- **Example**

$display("q = %0b", q);

🧠 One Complete Mini Example

always @(posedge clk) begin

a = b; // Active

q <= a; // NBA

end

assert property (@(posedge clk) q == $past(a)); // Preponed → Observed

$display("q=%0b", q); // Postponed

🧩 Why Timing Regions Matter?

✓ Avoid race conditions

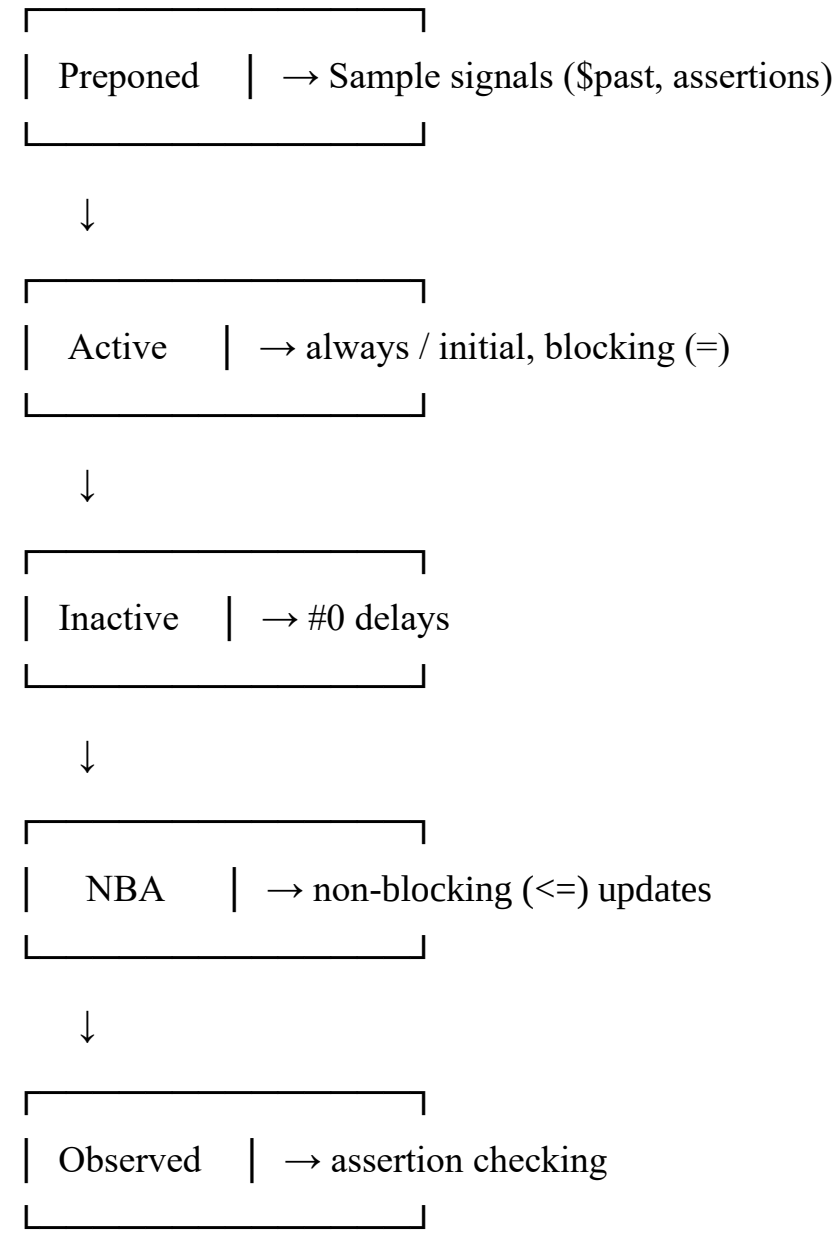
✓ Proper TB–DUT synchronization

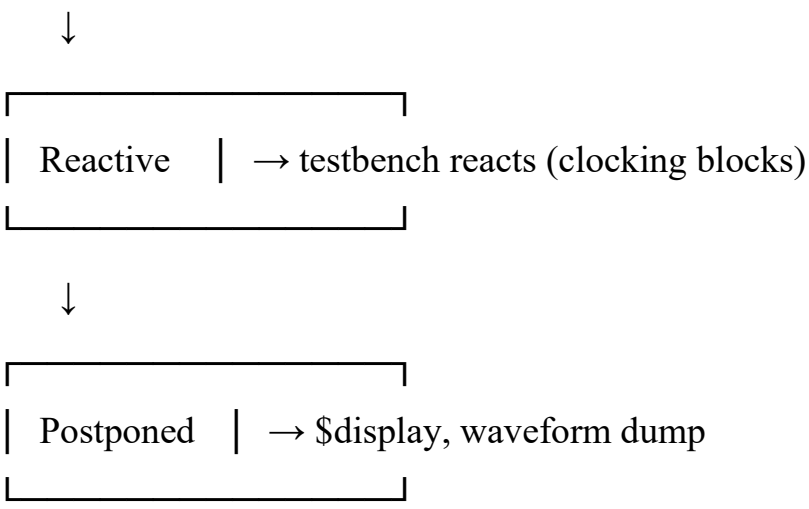
✓ Correct assertion behavior

⭐ Interview One-Liner

> “SystemVerilog timing regions define when signals are sampled, updated, checked, and displayed within the same simulation time.”

Timing Regions Diagram (Same Simulation Time) & Block Diagram:-





🧠 Simple Meaning of Each Region

Region	Very Simple Meaning
Preponed	Take snapshot before anything changes
Active	Execute design & TB code
Inactive	Same time, but slightly later
NBA	Update flip-flops together
Observed	Check correctness (assertions)
Reactive	Testbench responds to DUT
Postponed	Print results / dump waves

◇ **One Small Example**

```
always @(posedge clk)
   q <= d;     // NBA region
assert property (@(posedge clk) q == $past(d));  // Preponed → Observed
$display("q=%0b", q);   // Postponed
```

⭐ One-line Interview Answer

> “Timing regions define the order of sampling, execution, updating, checking, and displaying signals within the same simulation time.”

Session-17: -

```
//
module tb;
class pattern_generator;
	rand int arr[9];

	constraint c_fixed {
		foreach(arr[i]) {
			if (i < 4)
				arr[i] == i + 1;
			else if (i == 4)
				arr[i] == 0;
			else
				arr[i] == arr[8 - i];
		}
	}
endclass

initial begin
	pattern_generator p;
	p = new();
```

```
        p.randomize();
        $display("Pattern = %p", p.arr);
end

endmodule
```

Simulation Result

```
Pattern = '{1, 2, 3, 4, 0, 4, 3, 2, 1}
        V C S  S i m u l a t i o n  R e p o r t
Time: 0
```

```
module wildcard_example;
  logic [3:0] a = 4'b10??;
  logic [3:0] b = 4'b1010;

  initial begin
    //$display("Wildcard Equality: %b ==? %b => %b", a, b, (a ==? b));
    $display("Wildcard Equality: %b =?= %b => %b", a, b, (a =?= b));
    //$display("Wildcard Inequality: %b !=? %b => %b", a, b, (a !=? b));
  end
endmodule
```